

AngularJS to Angular 21 — Enterprise Migration Guide

Sudeep Parchure | AI Assisted | May 2026 | Angular Solutions Architect

This is a living document. It is structured as a series of parts. Each part is complete on its own and builds on the previous. Source: github.com/sudeep31/angularTodo — documents folder

Document Parts (Build Order)

Part	Title	Status
1	AI-Assisted Discovery — Q&A	✔ Complete
2	High-Level Design (HLD)	✔ Complete
3	Low-Level Design (LLD)	✔ Complete
4	Nx Workspace — Folder Structure & Setup	✔ Complete
5	AI Prompts, GitHub MCP, agents.md, skills.md	✔ Complete
6	Architectural Decisions & Risk Register	✔ Complete
7	Testing Guide — Vitest + Playwright with AI	✔ Complete
8	Deployment Architecture — Azure, GitHub Actions, Jenkins	✔ Complete
9	Master Prompt Library & Prompt Engineering Techniques	✔ Complete

Part 1 — AI-Assisted Discovery: Q&A

How to use this section: Before any line of code is written, a solution architect must answer these questions with the business, engineering leads, and product owners. This Q&A is designed to be used in an AI-assisted workshop — paste each question into GitHub Copilot Chat or your preferred LLM with your specific codebase context. The answers drive every decision in Parts 2 through 7.

Section 1: Understanding the Legacy Codebase

Q1 — What version of AngularJS are we running, and what does the codebase look like?

Why it matters: AngularJS 1.2, 1.6, and 1.8 each have different levels of ES5/ES6 support, component model maturity, and compatibility with the `ngUpgrade` bridge. AngularJS 1.8 reached End of Life on 31 December 2021. Every day after that is unpatched security risk.

Answer template (fill in for your project):

Attribute	Your Value	Impact on Migration
AngularJS version	e.g. 1.8.3	1.5+ supports <code>.component()</code> API — easier to migrate
Number of controllers	e.g. 120	Each controller → one standalone Angular 21 component
Number of services/factories	e.g. 80	Each → one <code>@Injectable</code> service
Number of custom directives	e.g. 40	Structural → <code>@if / @for</code> ; attribute → Angular Directive/Pipe
Number of filters	e.g. 25	Each → Angular Pipe or <code>computed()</code>
Number of routes (<code>\$route</code>)	e.g. 35	Each → Angular Router path
<code>\$scope</code> usage density	e.g. heavy/medium/low	Heavy = more <code>signal()</code> conversions
<code>\$rootScope</code> usage	e.g. 15 broadcasts	Each <code>\$rootScope.\$broadcast</code> → Signal-based event bus
Third-party AngularJS modules	e.g. ui-router, ui-grid	Affects migration complexity; some have no Angular equivalents
Test coverage	e.g. 35%	< 40% = high risk; use AI to generate tests before migration
Line count (JS + HTML)	e.g. 85,000 lines	Divides into sprint capacity to estimate duration

AI audit prompt — paste into Copilot Chat with your repo open:

```
You are an Angular migration expert. Analyse all .js files in the /app folder of this AngularJS
project and produce a migration backlog in this exact format:

| File | Type (controller/service/directive/filter) | $scope vars | $http calls | $watch count | Priority (H/M/L) |

Identify any patterns that do not have a direct Angular 21 equivalent and flag them.
Sort by Priority descending.
```

Q2 — Is the AngularJS app still receiving active feature development?

Why it matters: This single answer is the primary input into strategy selection. If the AngularJS app is frozen, you can plan a clean cutover. If it is still receiving features, you need a coexistence strategy for 12–18 months.

Scenarios and their consequences:

Scenario	Strategy Implication
Frozen — bug fixes only	Plan a clean migration; route-by-route or AI-assisted full rewrite
Active — 1–2 features per quarter	Strangler Fig or Nx Shell + MFE; new features go directly into Angular 21
Active — weekly releases	Nx Shell + MFE is mandatory; two separate CI/CD pipelines run in parallel
Active — multiple teams, multiple sprints	Polyrepo Nx workspace; each team owns its own migration slice

Answer: __

Q3 — What is our test coverage baseline, and where are the gaps?

Why it matters: Migration without tests is a leap of faith. Every AngularJS file you convert needs a corresponding Vitest unit test before you can safely delete the original. Low coverage means the AI must generate tests, not just convert code.

Coverage gap analysis prompt:

```
Run vitest --coverage on the current project and show me:
1. Files with 0% coverage
2. Files with < 40% coverage
3. Functions with no tests at all

For each uncovered file, generate a Vitest spec file that achieves >= 80% branch coverage.
Use describe/it blocks, mock all HTTP calls with vi.fn(), and test both happy path and error states.
```

Coverage thresholds for migration:

Coverage Level	Migration Risk	Recommended Action
> 80%	Low	Proceed with migration; AI conversion is safe
60%–80%	Medium	Generate missing tests with AI before migrating each file
40%–60%	High	AI test generation sprint BEFORE migration begins
< 40%	Very High	2-sprint test-writing phase; do not start migration until 60%

Q4 — What are the critical business paths that cannot break?

Why it matters: Not all code is equal. A bug in the login flow is a P0 incident. A bug in the FAQ page is a P3. Before migration, map your critical paths and assign rollback triggers to each one.

Critical path mapping worksheet:

User Journey	AngularJS Routes	Business Impact if Broken	Rollback Trigger
e.g. User authentication	/login , /mfa	CRITICAL — full outage	Error rate > 0.1% above baseline
e.g. Core transaction flow	/transfer , /payment	CRITICAL — revenue loss	Error rate > 0.5% above baseline
e.g. Account management	/accounts , /profile	HIGH — support calls	Error rate > 1% above baseline
e.g. Reporting / statements	/statements	MEDIUM	Error rate > 2% above baseline
e.g. Settings / preferences	/settings	LOW	Error rate > 5% above baseline

AI prompt to generate Playwright E2E tests for critical paths:

You are a Playwright expert. Generate a complete E2E test suite for this critical user journey:
[PASTE USER JOURNEY DESCRIPTION]

Requirements:

- Use Playwright's page object model pattern
- Assert on both visible UI state and network responses using page.route()
- Include accessibility checks using @axe-core/playwright
- Cover: happy path, validation errors, server error (500), timeout (network slow)
- Use test.describe and test.step for clear reporting
- Output a .spec.ts file ready to run with `npx playwright test`

Q5 — What third-party libraries are we using in AngularJS and do they have Angular equivalents?

Why it matters: AngularJS-specific libraries like ui-grid , angular-material , ui-router , and angular-translate have no direct drop-in Angular 21 equivalent. Their replacement must be scoped as separate work items, not bundled into controller conversion.

Library audit table:

AngularJS Library	Purpose	Angular 21 Equivalent	Migration Effort	Notes
ui-router	SPA routing	@angular/router	Low — 1:1 mapping	\$stateParams → ActivatedRoute
ui-grid	Data grid	AG Grid, CDK Table, or custom	High	No drop-in replacement; rewrite
angular-material	UI components	Angular Material 21 / CDK	Medium	New component API; visual parity
angular-translate	i18n	@angular/localize or ngx-translate	Medium	Key format compatible
restangular	REST client	HttpClient + interceptors	Low	Direct functional equivalent
ng-file-upload	File uploads	Angular CDK or browser File API	Medium	Must retest CORS + multipart
angular-animate	CSS animations	Angular Animations / CSS classes	Low	Replace with @keyframes + Tailwind
\$ocLazyLoad	Code splitting	loadComponent() + @defer	Low	Angular Router handles natively
Custom directives	Business logic UI	Standalone components / Directives	Variable	Audit each one individually

Section 1b: Third-Party Library Deep-Dive Questions

These questions expand on Q5. They apply specifically to enterprise apps where third-party libraries are load-bearing — not cosmetic. Answer each one before writing a single migration ticket.

Q5a — Do any third-party libraries wrap jQuery plugins, and is jQuery still in the dependency tree?

Why it matters: Many AngularJS apps were built on top of jQuery because AngularJS itself shipped with `jqLite` — a subset. Libraries like `angular-ui-bootstrap`, `angular-datatables`, and `select2` are thin wrappers over full jQuery plugins. Angular 21 has no jQuery dependency. Carrying jQuery into the Angular 21 bundle adds 30–87 KB gzipped and introduces direct DOM manipulation patterns that conflict with Angular's change detection model.

Audit command — find jQuery usage:

```
# Find all jQuery $ references and .jQuery calls in your source
grep -r --include="*.js" --include="*.html" '\$\' src/ | grep -v '\.spec\.' | wc -l
grep -r --include="*.js" 'angular.element\|jqLite\|jQuery\b' src/

# Check package.json for jQuery
npm list jquery
```

jQuery dependency decision matrix:

jQuery Usage Level	Recommended Action
0 references (jqLite only)	No action — remove jQuery from package.json after migration
1–20 references (utility use)	Replace with native DOM APIs during migration; no Angular 21 risk
20–100 references (spread use)	Dedicate 1 sprint to jQuery removal before migration; isolate in a service
100+ references (pervasive)	Wrap jQuery calls behind an abstraction service; migrate wrapper last
jQuery plugin wrappers	Replace each plugin with Angular CDK or a jQuery-free alternative

AI prompt — jQuery removal plan:

```
I have [COUNT] jQuery references in my AngularJS codebase.
The most common patterns are: [LIST: e.g. $.ajax, $(selector).hide(), $.extend]

For each pattern, give me:
1. The native browser API / TypeScript equivalent in Angular 21
2. Whether it can be automated with a codemod or needs manual review
3. If it is inside a third-party library wrapper, what Angular 21 library replaces it

Format as a migration table with effort estimates.
```

Q5b — Are we using AngularJS libraries for authentication or SSO (angular-oauth2-oidc, angular-jwt, Okta AngularJS SDK)?

Why it matters: Auth libraries have security implications beyond the migration. Swapping an auth library is not a refactor — it is a security-sensitive change that requires separate penetration testing, compliance sign-off, and rollout gates. It must be scoped as its own workstream, not bundled with UI migration.

Common AngularJS auth libraries and their Angular 21 replacements:

AngularJS Auth Library	Angular 21 Equivalent	Breaking Changes	Security Review Required?
angular-oauth2-oidc	angular-oauth2-oidc (supports Angular)	Config API changed; token storage strategy differs	✔ Yes
angular-jwt (Auth0)	@auth0/auth0-angular	New redirect flow; localStorage vs memory storage	✔ Yes
Okta AngularJS SDK	@okta/okta-angular	New OktaAuthModule replaced by standalone provider	✔ Yes
Custom \$http interceptor	HttpInterceptorFn	Functional interceptors; no \$q promises	✔ Yes
ngCookies for session	HttpClient with withCredentials	SameSite cookie policy changes since 2020	✔ Yes
AWS Cognito custom service	aws-amplify + custom inject() service	Amplify v6 API completely rewritten	✔ Yes

Auth migration rules — non-negotiable:

- Auth library migration is a SEPARATE workstream – never bundled with screen migration
- New auth flow tested in a dedicated staging environment with production-like IDP config
- Token storage decision documented as an ADR (memory vs localStorage vs cookie)
- PKCE flow verified for all OIDC/OAuth2 integrations
- Session timeout behaviour validated across old + new framework during coexistence period
- Security team / pen-test sign-off obtained before any auth change goes to production

AI prompt — auth migration ADR:

We are migrating from AngularJS to Angular 21. We currently use [LIBRARY] for authentication.
Our auth flow is: [OIDC / OAuth2 / SAML / Custom JWT / Session Cookie]
Our IDP is: [Okta / Auth0 / Azure AD / Keycloak / Custom]

Generate an Architecture Decision Record (ADR) for replacing [LIBRARY] with [NEW_LIBRARY].
Include:

- Context (why change is needed)
- Decision options with pros/cons
- Security implications for each option
- Token storage recommendation (memory / localStorage / HttpOnly cookie)
- Session synchronisation plan during hybrid coexistence period
- Rollback plan
- Consequences

Q5c — Are we using AngularJS form libraries (angular-formly, ngMessages, custom form validation)?

Why it matters: Enterprise apps with 100+ screens typically have 50–80 forms. If those forms are driven by a schema-based library like angular-formly, the migration is not just a template rewrite — it is a form engine replacement. This can easily be the largest single workstream in the entire migration.

Form library audit and migration path:

AngularJS Form Pattern	Angular 21 Equivalent	Effort per Form	Notes
angular-formly (schema-driven)	@ngx-formly/core (Angular-compatible)	Low-Medium	Formly has an Angular port; config format same
ng-messages validation display	Signal-based errors = computed()	Low	Remove ng-messages module entirely
ngModel two-way binding	Signal form with [value] + (input) binding	Medium per field	No direct equivalent; manual conversion
\$validators / \$asyncValidators	AbstractControl validators or signal guards	Medium	1:1 mapping to Angular validator functions
Custom form directive (typeahead)	CDK Combobox or Angular Material Autocomplete	High	Full rewrite; functional parity must be tested
Dynamic form fields (runtime-added)	Signal array + @for + dynamic validators	High	Complex; scope as separate epic
Multi-step wizard forms	Angular Router with step routes + FormGroup	High	State persistence across steps is the hard part

Form migration strategy — 100+ screen enterprise:

Phase A – Form Inventory (Sprint 0):

- Catalogue all forms: field count, validation rules, submit actions
- Classify by complexity: Simple (< 5 fields) / Medium (5-15) / Complex (15+)
- Identify shared form components reused across screens

Phase B – Shared Form Components First:

- Migrate address-input, date-picker, currency-field components once
- Publish to shared-ui Nx library
- All screens import from shared-ui – reduces total form migration effort by 30-40%

Phase C – Screen-by-Screen Form Migration:

- Simple forms: AI-assisted conversion, 1-2 h per form
- Medium forms: AI draft + dev review, 3-5 h per form
- Complex forms: Manual design, AI generates tests only, 1-3 days per form

AI prompt — form migration:

Convert this AngularJS form (using ngModel + \$validators) to Angular 21.

Rules:

- Use signal() for each form field value
- Use computed() for all validation error messages
- Use a single isValid = computed() that checks all fields
- Use @if (errors.fieldName()) to show validation messages
- Do NOT use ReactiveFormsModule or FormGroup
- The submit button must be [disabled]="!isValid()"
- All form fields must have aria-describedby pointing to their error element

[PASTE ANGULARJS FORM CODE HERE]

Q5d — Are we using data visualisation libraries (Highcharts, D3, Chart.js, ngx-charts wrappers)?

Why it matters: Charting libraries with AngularJS wrappers (highcharts-ng , angular-chart.js) are wrappers that manipulate the DOM directly. In Angular 21 with OnPush, chart re-renders must be triggered explicitly. Wrong handling causes charts that never update or update on every keystroke — both are common post-migration bugs.

Visualisation library migration paths:

AngularJS Chart Wrapper	Angular 21 Approach	Key Challenge	Effort
highcharts-ng	highcharts-angular (official wrapper)	Config format is identical; swap wrapper only	Low
angular-chart.js	ng2-charts or direct Chart.js with effect()	API changed; config migration needed	Medium
nvd3 (AngularJS D3)	Direct D3 v7 with Angular signal integration	No Angular 21 wrapper; write a thin component	High
angular-google-charts	angular-google-charts (Angular port)	Module config changed; mostly compatible	Low
Custom SVG directives	Angular component with SVG template	Often easiest migration — pure template rewrite	Medium
angular-echarts	ngx-echarts	Config API nearly identical; swap imports	Low

Signal-safe chart update pattern — OnPush safe:

```
// chart-wrapper.component.ts — signal drives chart data, effect triggers redraw
@Component({
  selector: 'app-chart',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<div #chartContainer class="chart-host"></div>`,
})
export class ChartComponent {
  readonly data = input.required<ChartDataPoint[]>();
  // ✅ Angular 21: use viewChild.required() (signal-based) instead of @ViewChild decorator
  // See Section 3.11.2 for the full signal query migration pattern
  protected readonly containerRef = viewChild.required<ElementRef>('chartContainer');
  private chart: Highcharts.Chart | undefined;

  constructor() {
    // afterNextRender: SSR-safe; runs once after first browser render
    afterNextRender(() => {
      this.chart = Highcharts.chart(this.containerRef().nativeElement, {
        series: [{ data: this.data() }],
      } as Highcharts.Options);
    });

    // effect() runs when data() signal changes — re-renders chart safely
    effect(() => {
      const series = this.data(); // read signal inside effect — tracked
      if (this.chart) {
        this.chart.series[0].setData(series, true, { duration: 300 });
      }
    });
  }

  // ngAfterViewInit no longer needed — replaced by afterNextRender above
  // Keeping placeholder for reference only:
  // ngAfterViewInit(): void {
  //   this.chart = Highcharts.chart(this.containerRef().nativeElement, {
  //     series: [{ data: this.data() }],
  //   });
  // }

  ngOnDestroy(): void {
    this.chart?.destroy();
  }
}
```

Q5e — Are we using AngularJS UI component libraries (Kendo UI for AngularJS, PrimeNG AngularJS, Angular-Bootstrap)?

Why it matters: UI component libraries are the single biggest visual regression risk. Users notice when a dropdown behaves differently, a modal animates differently, or a grid loses a column feature. Each component swap needs visual comparison testing in addition to unit tests.

Component library migration paths:

AngularJS UI Library	Angular 21 Equivalent	Visual Parity Risk	Effort
angular-ui-bootstrap	@ng-bootstrap/ng-bootstrap or Angular CDK	Medium	Medium
Kendo UI for AngularJS	@progress/kendo-angular-*	Low	Low-Med
PrimeNG (AngularJS port)	primeng (Angular 21 native)	Low	Low
Angular Material (AJS)	@angular/material v17+	Medium	Medium
ui-select	Angular Material Autocomplete or CDK Listbox	High	High
angular-chosen	@ng-select/ng-select	Medium	Medium
Custom component library	Nx shared-ui lib with Angular 21 components	High	High

Visual regression testing — mandatory for UI library swaps:

```
# Use Playwright visual comparison to catch UI regressions
npx playwright test --reporter=html --project=chromium

# Capture baseline screenshots BEFORE migration
npx playwright test --update-snapshots

# After migration — compare new renders against baseline
npx playwright test # fails if pixels differ by > threshold
```

AI prompt — component library equivalence check:

I am replacing [OLD_LIBRARY] with [NEW_LIBRARY] during an Angular 21 migration.
Here is a list of components I use from [OLD_LIBRARY]:
[LIST ALL COMPONENT NAMES]

For each component:

1. What is the exact equivalent in [NEW_LIBRARY]?
2. What API differences should I know about (Input/Output changes, event names, slot names)?
3. Are there any features in the old component that the new one does not support?
4. What accessibility attributes need re-validation after the swap?

Format as a migration table.

Q5f — How do we audit ALL third-party dependencies and assign a migration owner to each?

Why it matters: In a 100+ screen enterprise app, there can be 20–40 third-party AngularJS dependencies. Without a single owner per library, libraries fall through the cracks and are discovered half-way through migration — causing scope creep and timeline slippage.

Full dependency audit prompt:

I have an AngularJS application. Here is the full content of my package.json dependencies and devDependencies sections:

[PASTE package.json HERE]

For every dependency that is AngularJS-specific or has an AngularJS wrapper:

1. Identify the library name and version
2. Classify it: UI Component / Routing / HTTP / Auth / Forms / Charts / Utilities / Build
3. Find the Angular 21 equivalent (official or community)
4. Rate migration effort: Low / Medium / High / No Equivalent
5. Flag any that are abandoned (last npm publish > 2 years ago)
6. Suggest a migration sprint slot: Before migration starts / During migration / After cutover

Output as a Markdown table sorted by effort descending.

Library ownership assignment template:


```
// legacy-plugin-wrapper.component.ts
// Isolates ANY third-party library that has no Angular 21 equivalent
@Component({
  selector: 'app-legacy-plugin',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<div #pluginHost></div>`,
})
export class LegacyPluginWrapperComponent implements OnDestroy {
  readonly config = input.required<PluginConfig>();
  readonly data = input<unknown[]>([]);
  private pluginInstance: unknown;
  // ✅ Angular 21: signal-based query (see Section 3.11.2) – replaces @ViewChild decorator
  protected readonly hostRef = viewChild.required<ElementRef>('pluginHost');
  private readonly destroyRef = inject(DestroyRef); // ✅ Section 3.11.6 cleanup pattern

  constructor() {
    // React to data changes via effect – plugin is unaware of Angular
    effect(() => {
      const newData = this.data();
      if (this.pluginInstance) {
        (this.pluginInstance as PluginApi).setData(newData);
      }
    });
  }

  ngAfterViewInit(): void {
    // Initialise library against the host element
    // Replace with your actual library initialisation
    this.pluginInstance = initLegacyPlugin(this.hostRef.nativeElement, this.config());
  }

  ngOnDestroy(): void {
    (this.pluginInstance as PluginApi)?.destroy?.();
    this.pluginInstance = undefined;
  }
}
```

Section 2: Migration Strategy Selection

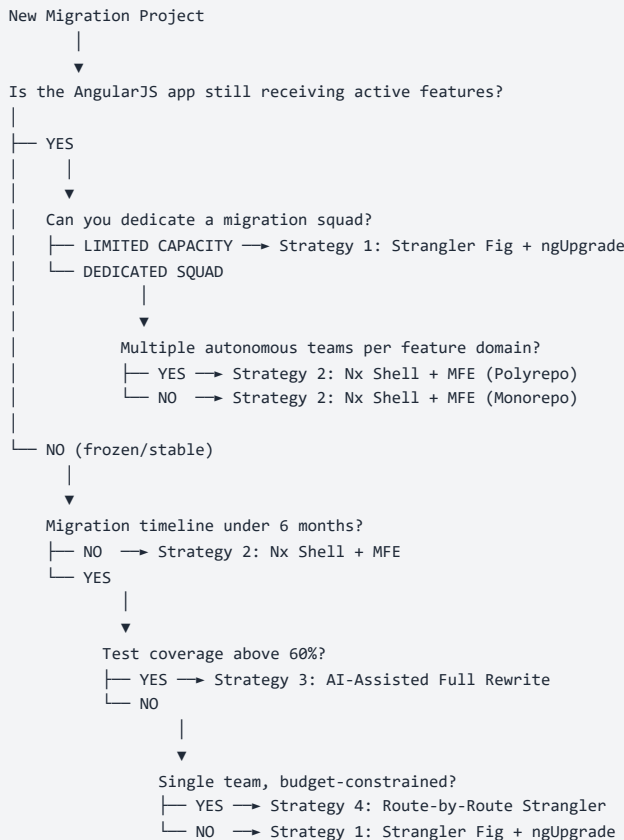
Q6 — Which migration strategy fits our situation?

Why it matters: There is no universal "best" strategy. Each has different risk profiles, timelines, team requirements, and cost implications. The right answer depends on Q1–Q5.

Strategy decision matrix:

Strategy	Best When...	Risk	Speed	Nx Required?
1. Strangler Fig + ngUpgrade	App is active; teams have AngularJS expertise; incremental budget	Medium	Slow–Medium	Optional
2. Nx Shell + Dual Apps (MFE)	Large teams; separate CI/CD needed; feature flags required	Low	Medium	✅ Yes
3. AI-Assisted Full Rewrite (Copilot-Led)	App is frozen or near-frozen; good test coverage; tight deadline	Medium	Fast	Recommended
4. Route-by-Route Strangler (No ngUpgrade)	Small team; low budget; risk-averse; no shared state between frameworks	Low	Slow	No
5. Big-Bang Rewrite	Never — project failure rate > 70% at enterprise scale	High	Fast on paper	No

Strategy selection flowchart:



AI prompt to help choose your strategy:

I am a solution architect planning an AngularJS to Angular 21 migration.

Here is our context:

- AngularJS version: [VERSION]
- Controllers: [COUNT], Services: [COUNT], Directives: [COUNT]
- Active feature development: [YES/NO – frequency]
- Team size: [COUNT] frontend engineers
- Test coverage: [PERCENTAGE]
- Timeline constraint: [MONTHS]
- Budget: [DEDICATED SQUAD / INCREMENTAL / LOW]
- Risk tolerance: [LOW / MEDIUM / HIGH]
- Third-party dependencies with no Angular equivalent: [LIST]

Based on this context, which of these five strategies do you recommend, and why?

1. Strangler Fig + ngUpgrade
2. Nx Shell + Dual Apps (MFE)
3. AI-Assisted Full Rewrite
4. Route-by-Route Strangler (No ngUpgrade)
5. Big-Bang Rewrite

Provide your recommendation with three supporting reasons and two risks.

Q7 — How long will this migration take and how do we estimate it?

Why it matters: Executives need a date. A structured estimation model prevents both over-promising (big bang fallacy) and under-delivering. The unit of migration work is the **feature**, not the line of code.

Estimation model — per-feature velocity:

Component Type	Average Senior Dev Hours	With AI (Copilot)	Notes
Simple controller (< 100 LOC)	4 h	1–1.5 h	Signal conversion + template rewrite
Complex controller (200+ LOC)	8–16 h	3–5 h	Business logic + state management
AngularJS service	3–6 h	1–2 h	DI swap, \$q → RxJS/async
Custom directive	6–20 h	2–6 h	Depends on DOM manipulation complexity
Filter	1–2 h	< 1 h	Direct Pipe equivalent
Route configuration	1 h per route	0.5 h	Path mapping is near-mechanical
Template (ng-if/ng-repeat)	Included in controller	Included	@if / @for migration is ~automated
Unit test generation	3–4 h per component	1 h with Copilot	After conversion, AI generates tests

Sprint capacity calculation:

```
Total migration hours =
  (controllers × avg_hours) +
  (services × avg_hours) +
  (directives × avg_hours) +
  (filters × avg_hours) +
  (test generation × component_count × 1h)
  × 1.3 (overhead: code review, integration, rollback handling)

Sprint capacity per engineer = 32 h (leaves 8 h for BAU / meetings)

Sprints needed = Total migration hours ÷ (Sprint capacity × team_size)
```

Example for a typical enterprise app (120 controllers, 80 services, 40 directives, 25 filters, team of 4):

```
= (120×3) + (80×1.5) + (40×4) + (25×0.5) + (265×1)
= 360 + 120 + 160 + 12.5 + 265
= 917.5 h × 1.3 overhead = ~1193 h
= 1193 ÷ (32 × 4) = ~9.3 sprints = ~5 months (2-week sprints, AI-assisted)
```

Q8 — What team skills and structure do we need?

Why it matters: AngularJS engineers who have never used Angular 21 Signals will slow down if thrown directly into the migration without upskilling. A structured ramp-up plan prevents the team from writing AngularJS patterns in Angular 21 syntax.

Skill gap matrix:

Skill	Required Level	Typical AngularJS Dev Gap	Upskill Time (with AI)
Angular 21 standalone components	Expert	Medium	1 week
Signals: <code>signal()</code> , <code>computed()</code> , <code>effect()</code>	Expert	High	1–2 weeks
<code>input()</code> / <code>output()</code> function API	Proficient	Low (replaces decorators)	2 days
<code>inject()</code> function DI	Proficient	Low	1 day
RxJS fundamentals	Proficient	Medium	1 week
Angular Router (lazy loading, guards)	Expert	Medium	3 days
Nx monorepo concepts	Proficient	High	1 week
Module Federation basics	Familiar	High	2 days
Vitest + Testing Library	Proficient	High	3–5 days
Playwright E2E testing	Familiar	High	2–3 days
TailwindCSS (if replacing Angular Material)	Proficient	Medium	3 days

Recommended team structure for a medium enterprise migration:

```
Migration Squad (dedicated, 6 months)
├── 1 × Solution Architect (migration lead – you)
│   └── Owns: ADR decisions, HLD, Nx workspace setup, strategy
├── 1 × Senior Angular 21 Engineer (migration driver)
│   └── Owns: Copilot prompts, shared libs, code review of AI output
├── 2 × Mid Angular Engineers (converters)
│   └── Owns: Per-feature controller/service conversion with AI
├── 1 × QA / Test Engineer
│   └── Owns: Vitest coverage gates, Playwright E2E, rollback testing
└── 1 × DevOps / Platform Engineer (part-time)
    └── Owns: Nx CI/CD, Module Federation deployment, feature flags
```

Q9 — How do we handle shared state and authentication across both frameworks?

Why it matters: In a hybrid coexistence period (AngularJS + Angular 21 running simultaneously), the single most dangerous shared concern is **authentication**. A logout on one side must log out the other. A session timeout must be handled by both.

Auth bridge patterns:

Pattern	Mechanism	Use When
LocalStorage token sharing	Both apps read the same JWT key	Simple — no cross-frame communication needed
Custom DOM events	CustomEvent dispatched on window	Same-origin only; fast; decoupled
postMessage bridge	iframe-to-parent message passing	When legacy app runs in iframe
Shared service library	Nx shared-auth lib used by both	Same-origin Nx monorepo with shared bundling
BFF (Backend-for-Frontend)	Both apps call same API endpoint	Token stored server-side; cleanest approach

AI prompt for auth bridge design:

```
I am migrating a banking app from AngularJS to Angular 21.
During the migration period both apps will coexist.
Authentication uses JWT tokens stored in localStorage.

Design an AuthBridgeService that:
1. Stores the token as an Angular 21 signal
2. Syncs to localStorage on every change
3. Dispatches a same-origin window CustomEvent when token changes so the AngularJS side can react
4. Has a computed() isAuthenticated property
5. Has clearToken() that dispatches an auth:logout event
6. Is fully typed with TypeScript strict mode

Then show the AngularJS side: how does $rootScope listen to these events?
```

Q10 — What does "done" look like, and how do we decommission AngularJS?

Why it matters: Many migrations never finish because "done" was never defined. Every migration plan needs a documented decommission checklist agreed with product, engineering, and leadership before work begins.

Migration completion definition:

Criterion	Measurement	Agreed Threshold
All Angular 21 routes live in production	<code>grep -r 'legacyMode iframe' src/</code> returns 0 matches	0 matches
AngularJS bundle no longer served	Network tab shows no <code>angular.min.js</code>	0 requests
<code>@angular/upgrade</code> removed from package.json	<code>grep '@angular/upgrade' package.json</code> returns empty	Not present
Zero <code>\$scope</code> references remaining	<code>grep -r '\\$scope' src/</code> returns 0	0 references
Test coverage on all migrated files	<code>vitest --coverage</code>	≥ 80%
Lighthouse LCP on migrated routes	Compared to baseline	≤ baseline + 10%
WCAG AA compliance on all migrated routes	axe-core automated + manual audit	0 new violations
Error rate on all migrated routes	Compared to 30-day pre-migration baseline	≤ baseline + 0.5%
Classic/Legacy option removed from navigation	UI inspection	Not visible
AngularJS app removed from Nx workspace / git repo	<code>nx list</code> shows no legacy app	Not present

AI prompt for decommission checklist generation:

We are ready to decommission our AngularJS application. We have been running Angular 21 and AngularJS in parallel for [X] months.

Generate a 3-phase decommission runbook:

Phase 1 – Pre-decommission validation (automated checks, metrics gates)

Phase 2 – Traffic cutover (feature flags, rollback plan, monitoring alerts)

Phase 3 – Code cleanup (files to delete, packages to remove, CI steps to update)

Include a go/no-go decision table for each phase.

Include rollback steps for each phase if something goes wrong.

Format as a DevOps runbook with shell commands where applicable.

Section 3: AI-Assisted Migration — Key Questions

Q11 — How do I use GitHub Copilot effectively for migration, not just code generation?

Why it matters: Copilot is most useful in migration when used as a **systematic pipeline**, not an ad-hoc "generate and hope" tool. The difference between a 6-month migration and a 12-month migration is often whether AI is used strategically or reactively.

The AI Migration Pipeline:

AI MIGRATION PIPELINE (SYSTEMATIC)			
STEP 1 AUDIT	STEP 2 CONVERT	STEP 3 TEST	STEP 4 VERIFY
Copilot reads AngularJS files and produces: • migration backlog • risk flags • priority ranking	Copilot converts: • Controller → Component • Service → Injectable • Filter → Pipe • Directive → Component	Copilot generates Vitest specs for each migrated file	ng build --prod vitest --coverage Playwright E2E on critical paths
Dev reviews + approves			

Key Copilot best practices for migration:

Practice	What to Do
Context window management	Feed one file at a time; include the interface file alongside it
Strict output rules	Always specify: signals only, no <code>@Input/@Output</code> , <code>inject()</code> only
Incremental verification	Run <code>tsc --noEmit</code> after each generated file before moving on
Template migration separately	Convert TypeScript first; migrate template second; test both states
Never trust, always review	AI makes hallucination errors; always check imports exist in Angular 21
Preserve method names	Explicitly instruct Copilot to keep public API names unchanged

Q12 — What are the most common migration mistakes and how do we avoid them?

Why it matters: Teams that have done this migration before leave a clear trail of mistakes. Knowing them in advance prevents 80% of the rework that typically inflates timelines by 30–50%.

Top 10 migration mistakes:

#	Mistake	Prevention
1	Converting without tests	Enforce coverage gate before any conversion starts
2	Using <code>@Input()</code> / <code>@Output()</code> decorators in Angular 21	Copilot instruction: always use <code>input()</code> / <code>output()</code> functions
3	Constructor injection instead of <code>inject()</code>	ESLint rule: <code>@angular-eslint/prefer-inject</code>
4	BehaviorSubject for local state instead of <code>signal()</code>	Code review checklist: no BehaviorSubject for component-local state
5	<code>ngModule</code> imports in standalone components	<code>ng generate</code> always uses <code>--standalone</code> ; ESLint no-module rule
6	Zone.js left enabled when Angular 21 supports zoneless	Check <code>provideZonelessChangeDetection()</code> after migration (stable in Angular 21 — no longer "Experimental")
7	Third-party library replacement not scoped separately	Library ADR before sprint; treat each lib as a separate epic
8	Assuming <code>ngUpgrade</code> handles all edge cases	Test every downgraded/upgraded component in IE11-equivalent conditions
9	Feature flag not wired to monitoring	Every flag → corresponding Datadog/New Relic dashboard alert
10	Not removing AngularJS after migration	Decommission checklist agreed before migration begins (see Q10)

Q13 — How do we validate that AI-generated Angular 21 code is correct?

Why it matters: AI can generate plausible-looking but subtly broken code. In a migration context, "it compiles" is not the same as "it works correctly". Every AI-generated file needs a structured review checklist.

AI output review checklist:

- ☐ TypeScript compiles with `--strict` mode (run: `tsc --noEmit`)
- ☐ No `@NgModule` decorators (standalone only)
- ☐ All dependencies use `inject()` (not constructor parameters)
- ☐ Reactive state uses `signal()` (not `BehaviorSubject/Subject` for local state)
- ☐ Derived state uses `computed()` (not manual signal subscriptions)
- ☐ Side effects use `effect()` with proper cleanup
- ☐ Component inputs use `input()` / `input.required()` (not `@Input` decorator)
- ☐ Component outputs use `output()` (not `@Output` + `EventEmitter` decorator)
- ☐ Template uses `@if` / `@for` / `@switch` (not `*ngIf` / `*ngFor` / `ngSwitch`)
- ☐ `ChangeDetectionStrategy.OnPush` is set
- ☐ All imported types/components exist in Angular 21 (no hallucinated APIs)
- ☐ HTTP calls use `inject(HttpClient)` (not constructor injection)
- ☐ No `console.log` or debug code left in
- ☐ Accessibility: all interactive elements have aria labels or visible text
- ☐ Vitest spec file exists with $\geq 80\%$ coverage

AI self-review prompt:

Review the Angular 21 component code I am about to show you.
Check it against these exact rules and flag any violations:
1. Must use inject() not constructor injection
2. Must use signal() for reactive state
3. Must use input() / output() not @Input/@Output decorators
4. Must use @if / @for not *ngIf / *ngFor
5. Must have ChangeDetectionStrategy.OnPush
6. Must be standalone (no NgModule)
7. All imports must exist in Angular 21.2 – flag any that look hallucinated

[PASTE GENERATED CODE]

Q14 — How do we handle AngularJS patterns that have no direct Angular 21 equivalent?

Why it matters: Not everything maps cleanly. Some AngularJS patterns require rethinking the design, not just syntactic conversion. Identify these early to scope them as separate architectural tasks.

Hard migration cases:

AngularJS Pattern	Why It Is Hard	Angular 21 Approach
<code>\$compile</code> (runtime template)	Angular 21 has no equivalent for runtime template compilation	<code>@defer</code> + lazy route; or pre-compile all variants
<code>\$rootScope</code> as event bus	Global mutable state; implicit coupling	Signal-based <code>EventBusService</code> with typed event interface
<code>\$scope.\$apply()</code> manual trigger	Manual CD trigger; not needed in Angular 21	Remove entirely — Signals are zone-agnostic
Deep <code>\$watch</code> with object equality	Expensive; AngularJS-specific pattern	<code>computed()</code> with explicit dependency tracking
Provider configuration (<code>\$provide</code>)	Run-phase configuration of services	<code>APP_INITIALIZER</code> token + environment injection
<code>angular.extend</code> / deep merging	Utility functions in AngularJS core	Native <code>Object.assign</code> / spread operator
<code>\$timeout</code> / <code>\$interval</code>	Zone-aware wrappers for <code>setTimeout</code>	Native <code>setTimeout</code> outside <code>NgZone</code> , or <code>RxJS timer()</code>
Dynamic <code>ng-include</code>	Runtime partial loading	<code>@defer</code> with lazy component; or <code>NgComponentOutlet</code>
<code>\$http</code> interceptors	Request/response modification pipeline	Angular <code>HttpInterceptorFn</code> (functional interceptors)
<code>ui-grid</code> / legacy data grid	No Angular 21 compatible version	AG Grid Community, Angular CDK Table, or custom signal table

Q15 — How do we use AI throughout the migration lifecycle, not just for code conversion?

Why it matters: AI assistance should span the entire migration — from initial audit to post-decommission documentation. Teams that limit AI to code generation leave 60–70% of the potential productivity gain on the table.

AI use across the migration lifecycle:

Phase	AI Task	Tool / Prompt Type
Discovery	Codebase audit → migration backlog	Copilot Chat: codebase analysis prompt
Strategy selection	Architecture ADR generation	Copilot Chat: decision matrix prompt
Foundation setup	Nx workspace scaffolding	Copilot CLI: <code>nx generate</code> commands
Code conversion	Controller/service/directive → Angular 21	Copilot inline: per-file conversion
Test generation	Vitest specs for migrated components	Copilot Chat: spec generation prompt
E2E test generation	Playwright tests for critical paths	Copilot Chat: page object model prompt
Code review	Review AI output against Angular 21 checklist	Copilot Chat: self-review prompt
ADR documentation	Document architectural decisions as ADR files	Copilot Chat: ADR template generation
Risk monitoring	Generate monitoring dashboards/alerts config	Copilot Chat: observability prompt
Decommission runbook	Step-by-step AngularJS removal procedures	Copilot Chat: runbook generation
Changelog / docs	Migration summary for stakeholders	Copilot Chat: executive summary prompt

Section 4: Nx Workspace Questions

Q16 — Should we use an Nx monorepo or polyrepo, and why?

Why it matters: The Nx workspace structure decision affects every team's daily workflow for 12–18 months. Getting this wrong means painful refactoring mid-migration.

Nx monorepo vs polyrepo decision:

Dimension	Nx Monorepo	Polyrepo (separate git repos)
Shared library management	✅ First-class — <code>libs/</code> shared across all apps	❌ npm package publish/consume cycle for each lib
Cross-app refactoring	✅ Single PR changes shell + remote simultaneously	❌ Multiple PRs across repos
CI performance	✅ <code>nx affected</code> runs only changed app's tests	✅ Each repo's CI is independent and smaller
Team autonomy	Medium — shared workspace config	✅ Full autonomy per repo
AngularJS app integration	✅ Register legacy app in workspace	Medium — requires cross-repo coordination
Onboarding a new engineer	✅ One repo to clone	❌ Multiple repos, multiple setups
Best for teams of	2–15 engineers on single migration	5+ teams with independent delivery

Recommendation for most enterprise migrations: Start with **Nx monorepo**. Migrate to polyrepo structure only if specific teams require full independent release cadences that Nx's `nx affected` cannot accommodate.

Q17 — What Nx generators and plugins should we use?

Key Nx plugins for Angular migration:

Plugin	Purpose	Install Command
@nx/angular	Angular app/lib scaffolding	npm install --save-dev @nx/angular
@nx/webpack (for legacy app)	Webpack config for AngularJS app in Nx workspace	npm install --save-dev @nx/webpack
@nx/module-federation	Module Federation host/remote wiring	Built into @nx/angular setup-mf
@nx/cypress (legacy E2E)	Cypress test runner integration	npm install --save-dev @nx/cypress
@nx/playwright	Playwright integration for Nx	npm install --save-dev @nx/playwright
@nx/eslint	Shared ESLint config across apps and libs	Built into Nx workspace
@nx/storybook	Component development in isolation	npm install --save-dev @nx/storybook

Nx workspace initialisation prompt:

```
I am setting up an Nx monorepo for an AngularJS to Angular 21 migration.
I need:
- An Angular 21 shell app (Module Federation host)
- An Angular 21 new-features app (Module Federation remote)
- A placeholder for the existing AngularJS app
- Shared libs: models, auth, ui-components, utils

Generate the exact Nx CLI commands (in order) to scaffold this workspace from scratch.
Use @nx/angular with standalone components, Vitest for testing, and ESLint.
Include the Module Federation setup commands.
Output a single bash script I can run end-to-end.
```

Q18 — How do we handle CI/CD for a hybrid Nx workspace during migration?

Why it matters: During the migration period, both the AngularJS app and the Angular 21 apps must build, test, and deploy independently without blocking each other. `nx affected` is the key.

CI pipeline strategy:

```
On every PR:
├─ nx affected:lint      → only lint changed apps/libs
├─ nx affected:test      → only run Vitest for changed code
├─ nx affected:build     → only build changed apps
└─ nx affected:e2e       → only E2E test affected apps

On merge to main:
├─ nx build banking-shell → always build (shell change is high-impact)
├─ nx build banking-ng21 → build new Angular 21 features app
├─ nx build banking-legacy → build only if legacy app files changed
└─ Deploy:
    ├─ shell → CDN (e.g. Cloudflare Pages / Azure Static Web Apps)
    ├─ banking-ng21 → CDN /ng21/remoteEntry.js
    └─ banking-legacy → Static server /legacy/ (unchanged unless bug fix)
```

Section 5: Risk and Governance Questions

Q19 — What are the top risks in an AngularJS to Angular 21 migration?

Full risk register is in **Part 6** of this document. The five highest-impact risks to address before starting:

Rank	Risk	Probability	Impact	Mitigation
1	Regression on critical business path	Medium	Critical	E2E Playwright tests on all critical paths before migration
2	Migration takes 2× longer than estimated	High	High	AI-assisted conversion + weekly velocity tracking
3	Third-party library with no Angular equivalent	Medium	High	Library ADR before sprint starts; decouple from migration
4	Auth state desync between the two frameworks	Medium	Critical	AuthBridgeService + integration test before any hybrid deploy
5	Knowledge silos — only 1–2 engineers understand migration approach	Medium	High	Weekly ADR documentation with AI assistance; pair programming

Q20 — How do we measure migration progress and health?

Migration dashboard metrics (track per sprint):

Metric	Formula	Green Threshold	Red Threshold
AngularJS code reduction	<code>deleted_AJS_lines / total_AJS_lines × 100</code>	-5% per sprint	< -2% per sprint
Angular 21 component coverage	<code>ng21_components / total_components × 100</code>	+5% per sprint	< +2% per sprint
Bundle size delta (migrated routes)	<code>new_bundle_KB - previous_bundle_KB</code>	≤ 0 KB	> +20 KB
Test coverage on migrated files	<code>vitest --coverage</code>	≥ 80%	< 60%
Remaining <code>\$scope</code> references	<code>grep -r '\\$scope' src/ wc -l</code>	-10% per sprint	No change
Error rate on migrated routes	Application monitoring (Datadog/New Relic)	≤ baseline	> baseline + 0.5%
Lighthouse LCP on migrated routes	CI Lighthouse audit	≤ baseline	> baseline + 500ms
WCAG AA compliance	axe-core in CI	0 new violations	Any new violation

Section 6: Web Security, Authentication & Authorisation

Security is not a phase — it is a constraint that runs through every other phase. These questions must be answered in parallel with strategy selection (Section 2), not after it. A migration that improves the UI but degrades security posture is a net negative.

 **This section covers security strategy** — the discovery questions you must answer before writing code. Security in this guide spans three sections with different purposes:

- **Here (Part 1, Q21–Q27):** Strategy Q&A — What risks do we have? What are our OWASP exposure points? What patterns should we choose?
- **[Section 3.7 — Security LLD](#):** Implementation code — the Angular 21 auth service, HTTP interceptors, and route guards that put Q21–Q27 decisions into practice.
- **[Section 6.2 — HTTP Token & Cookie Strategy](#):** Architecture Decision Record — token storage options, risk matrix, and the recommended hybrid BFF pattern.

Q21 — What OWASP Top 10 risks are present in our AngularJS app today, and how does migration change the risk surface?

Why it matters: AngularJS apps built before 2018 predate many modern browser security APIs and Angular's built-in sanitization model. Migration is the best — and sometimes the only — opportunity to systematically address these risks without risking regression in the legacy app.

OWASP Top 10 — AngularJS vs Angular 21 comparison:

OWASP Risk	AngularJS Default Behaviour	Angular 21 Default Behaviour	Migration Action
A01 Broken Access Control	Route access controlled by <code>\$routeChangeStart</code> ; easy to bypass	Route guards (<code>CanActivateFn</code>); server-side must still validate	Rewrite all guards; add server-side auth checks
A02 Cryptographic Failures	JWT in localStorage (XSS-accessible); no SameSite cookie by default	Same risk unless ADR mandates HttpOnly cookies or memory storage	ADR required: token storage strategy
A03 Injection (XSS)	<code>\$sce.trustAsHtml()</code> bypasses sanitizer; <code>ng-bind-html</code> is risky	<code>DomSanitizer.bypassSecurityTrust*()</code> — same risk if misused	Audit all trust bypass calls; remove where possible
A04 Insecure Design	No built-in CSP; often deployed without security headers	No change — infrastructure concern; add headers at CDN/nginx level	Add CSP, HSTS, X-Frame-Options during migration
A05 Security Misconfiguration	Source maps in production; debug info exposed	Angular 21 build strips debug info in production by default	Verify production build flags; disable source maps
A06 Vulnerable Components	AngularJS 1.8 EOL — zero patches since Dec 2021	Active security patching; CVE monitoring via <code>npm audit</code>	This alone justifies migration
A07 Auth Failures	Session management in <code>\$cookies</code> or localStorage; no PKCE	Depends on auth library chosen; PKCE is recommended	Auth library ADR (see Q5b); PKCE mandatory
A08 Data Integrity Failures	<code>\$http</code> has no CSRF protection by default	<code>HttpClient</code> reads <code>XSRF-TOKEN</code> cookie automatically	Verify XSRF token handling on migration (see Q23)
A09 Logging Failures	<code>console.log</code> with PII in many AngularJS codebases	ESLint <code>no-console</code> rule can enforce removal	Pre-migration audit: remove all PII from console
A10 SSRF	Frontend rarely at risk; API layer concern	Validate all dynamic URL construction in new services	Review all <code>HttpClient</code> base URL construction

AI security audit prompt — paste with codebase open in Copilot:

You are a security architect reviewing an AngularJS 1.8 codebase for OWASP Top 10 risks. Scan all .js and .html files and produce a security findings report.

For each finding:

1. File and line number
2. OWASP category (A01-A10)
3. Severity: Critical / High / Medium / Low
4. What the risk is (one sentence)
5. Angular 21 equivalent pattern that fixes it
6. Estimated effort to fix: Low / Medium / High

Focus on:

- Any use of `$sce.trustAsHtml` or `ng-bind-html` without sanitization
- `console.log` with variables that may contain PII
- Any dynamic URL construction using user input
- `$http` calls without CSRF headers
- Route definitions with no authentication guard
- `localStorage` / `sessionStorage` containing tokens or PII

Q22 — How do we handle XSS (Cross-Site Scripting) — AngularJS trusted HTML vs Angular 21 DomSanitizer?

Why it matters: Both AngularJS and Angular 21 have built-in XSS protection — but both allow it to be bypassed. The bypass patterns are slightly different, and every bypass in the AngularJS app must be reviewed during migration to determine if it can be removed or must be replaced with a safe equivalent.

XSS attack surface comparison:

Pattern	AngularJS (Risky)	Angular 21 (Safe Default)	Angular 21 (Still Risky — avoid)
Rendering user HTML	<code>ng-bind-html</code> without <code>\$sce</code> whitelist	<code>[innerHTML]</code> + <code>DomSanitizer.sanitize()</code>	<code>bypassSecurityTrustHtml()</code> — only use if truly safe
Dynamic URL binding	<code>ng-href="{{userUrl}}"</code>	<code>[href]="safeUrl()"</code> with <code>DomSanitizer.sanitize()</code>	<code>bypassSecurityTrustUrl()</code> — avoid
Style binding	<code>ng-style</code> with user input	Angular sanitizes <code>[style]</code> automatically	<code>bypassSecurityTrustStyle()</code> — avoid
Script URL	No protection in AngularJS templates	Angular blocks <code>javascript:</code> URLs automatically	<code>bypassSecurityTrustResourceUrl()</code> — audit all uses
Template injection	AngularJS templates compiled client-side (risk)	Angular 21 templates compiled at build time (safe)	Never use dynamic template strings

Migration rule: every `$sce.trustAsHtml()` must become one of these:

```
// OPTION 1: Remove the bypass — sanitize properly (preferred)
import { inject } from '@angular/core';
import { DomSanitizer, SafeHtml } from '@angular/platform-browser';

@Component({ standalone: true, ... })
export class SafeHtmlComponent {
  private sanitizer = inject(DomSanitizer);
  readonly rawHtml = input.required<string>();
  // DomSanitizer.sanitize() removes dangerous tags; keeps safe HTML structure
  protected readonly safeHtml = computed(() =>
    this.sanitizer.sanitize(SecurityContext.HTML, this.rawHtml()) ?? ''
  );
}

// OPTION 2: If you truly need to render known-safe rich text (e.g. from your own CMS)
// ONLY after explicit security review and team sign-off:
protected readonly trustedHtml = computed(() =>
  this.sanitizer.bypassSecurityTrustHtml(this.rawHtml())
  // REQUIRES: source is server-controlled, already sanitized server-side
  // DOCUMENT this decision as an ADR with security team approval
);
```

Content Security Policy (CSP) — deploy alongside Angular 21:

```
# Recommended CSP for an Angular 21 SPA (add to nginx / CDN response headers)
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-{RANDOM_NONCE}';
  style-src 'self' 'unsafe-inline';           # TailwindCSS requires this; or use nonce
  img-src 'self' data: https:;
  connect-src 'self' https://api.bank.com;
  frame-src 'none';                          # Set to legacy.bank.com if iframe bridge used
  object-src 'none';
  base-uri 'self';
  form-action 'self';
```

AI prompt — XSS audit of AngularJS templates:

```
Scan all HTML templates in the /app folder for XSS risks.
Flag every use of:
1. ng-bind-html (must be sanitized or reviewed)
2. ng-href / ng-src with interpolated values (URL injection risk)
3. ng-style with user-supplied values
4. Any event handler attributes (onclick, onload in templates)
5. Any use of $sce.trustAs* functions in controllers or services

For each finding, show: file, line number, current risky pattern, safe Angular 21 replacement.
```

Q23 — How do we handle CSRF (Cross-Site Request Forgery) during and after migration?

Why it matters: AngularJS has no automatic CSRF protection on `$http`. Many AngularJS apps either rely on custom interceptors or have no CSRF protection at all. Angular 21's `HttpClient` has built-in XSRF token support — but only if the server sets the `XSRF-TOKEN` cookie

correctly.

CSRF protection comparison:

Approach	AngularJS Implementation	Angular 21 Implementation
Double Submit Cookie	Manual <code>\$http</code> interceptor reads <code>XSRF-TOKEN</code> cookie	<code>HttpClient</code> does this automatically — zero config required
SameSite Cookie	Not configurable from frontend	Not configurable from frontend — server/infrastructure concern
Custom header (stateless)	Manual <code>\$http</code> interceptor adds <code>X-Requested-With</code>	Custom <code>HttpInterceptorFn</code> adds header
Origin header check	Server-side; frontend sends it automatically	Same — browser sends <code>Origin</code> ; server must validate

Angular 21 XSRF setup — activate the built-in protection:

```
// app.config.ts — enable Angular's built-in XSRF token support
import { provideHttpClient, withXsrfConfiguration } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withXsrfConfiguration({
        cookieName: 'XSRF-TOKEN', // server sets this cookie
        headerName: 'X-XSRF-TOKEN', // Angular sends this header on every mutating request
      })
    ),
  ],
};
// Server requirement: set a non-HttpOnly 'XSRF-TOKEN' cookie on every response
// Angular reads it and sends it as a header — server validates header == cookie
```

CSRF migration checklist:

- Identify all `$http` POST/PUT/PATCH/DELETE calls in AngularJS — do any send CSRF headers?
- Check if backend validates `X-XSRF-TOKEN` — if not, backend fix is needed before migration
- Add `withXsrfConfiguration()` to `provideHttpClient()` in Angular 21 `app.config.ts`
- Test: make a state-changing request from Angular 21; verify header is present in Network tab
- Test: make a request WITHOUT the header; verify server returns 403
- Document CSRF protection strategy as an ADR

Q24 — How do we migrate authentication patterns (JWT, OAuth2 / OIDC, Session Cookies)?

Why it matters: The authentication pattern determines the token storage strategy, which in turn determines the XSS risk exposure. A JWT stored in `localStorage` is readable by any JavaScript on the page (XSS attack surface). A JWT in an `HttpOnly` cookie is not readable by JS — but requires `SameSite`/CSRF protection.

Auth pattern decision matrix:

Auth Pattern	Token Location	XSS Risk	CSRF Risk	Angular 21 Support	Recommended For
JWT in <code>localStorage</code>	Browser <code>localStorage</code>	High (JS readable)	Low	Works out of the box	Internal tools only (not customer-facing)
JWT in memory (signal)	Angular service signal	Minimal	Low	Best Angular 21 fit	SPAs with short sessions (< 1 h)
HttpOnly cookie	Browser (not JS-accessible)	None	High	<code>withCredentials: true</code>	Customer-facing; banking; regulated
HttpOnly cookie + CSRF token	Cookie + header	None	None	<code>withXsrfConfiguration</code>	Recommended for enterprise banking
BFF (Backend-for-Frontend)	Server session	None	Low (same-origin)	Transparent to Angular	Highest security; adds BFF layer

Recommended Angular 21 in-memory auth service (XSS-safe):

```
// auth.service.ts – token in memory only; not localStorage; not cookie
@Injectable({ providedIn: 'root' })
export class AuthService {
  // Token lives ONLY in memory – survives navigation but not page refresh
  // On refresh, app calls /api/auth/refresh (HttpOnly refresh-token cookie handles it server-side)
  private readonly _accessToken = signal<string | null>(null);
  readonly isAuthenticated = computed(() => this._accessToken() !== null);
  readonly accessToken = this._accessToken.asReadonly();

  setToken(token: string): void {
    this._accessToken.set(token);
    // Do NOT persist to localStorage – that is the XSS attack surface
  }

  clearToken(): void {
    this._accessToken.set(null);
  }
}

// http-auth.interceptor.ts – functional interceptor attaches token to every API call
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.accessToken();
  if (!token) return next(req);

  return next(
    req.clone({
      setHeaders: { Authorization: `Bearer ${token}` },
    }),
  );
};
```

Q25 — How do we implement Role-Based Access Control (RBAC) and route-level authorisation in Angular 21?

Why it matters: AngularJS route guards via `$routeChangeStart` / `$stateChangeStart` are synchronous and often easy to bypass with direct URL navigation. Angular 21 route guards are enforced by the router and can be async, allowing server-side role verification before rendering.

AngularJS vs Angular 21 guard comparison:

Guard Type	AngularJS Pattern	Angular 21 Equivalent
Authentication check	<code>\$routeChangeStart</code> listener on <code>\$rootScope</code>	<code>CanActivateFn</code> — functional guard
Role-based access	Custom <code>resolve</code> with <code>\$q</code> + role check	<code>CanActivateFn</code> with <code>inject(AuthService)</code>
Permission-based access	Manual <code>ng-if</code> on <code>\$rootScope.userRoles</code>	Custom <code>PermissionDirective</code> + route guard combination
Redirect on unauthorised	<code>\$location.path('/login')</code> in <code>\$routeChangeStart</code>	<code>Router.navigate(['/login'])</code> in guard return
Lazy-load protection	Not native in AngularJS	<code>CanMatchFn</code> — prevents lazy chunk from loading

Angular 21 RBAC implementation — functional guards with signals:

```

// auth.service.ts - roles as signals
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly _roles = signal<string[]>([]);
  readonly roles = this._roles.asReadonly();
  readonly isAuthenticated = computed(() => this._roles().length > 0);

  hasRole(role: string): boolean {
    return this._roles().includes(role);
  }

  hasAnyRole(roles: string[]): boolean {
    return roles.some((r) => this._roles().includes(r));
  }
}

// guards/auth.guard.ts - authentication guard
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isAuthenticated()) return true;
  return router.createUrlTree(['/login']);
};

// guards/role.guard.ts - role-based guard factory
export const roleGuard =
  (requiredRoles: string[]): CanActivateFn =>
  () => {
    const auth = inject(AuthService);
    const router = inject(Router);
    if (auth.hasAnyRole(requiredRoles)) return true;
    return router.createUrlTree(['/unauthorised']);
  };

// app.routes.ts - applying guards
export const routes: Routes = [
  {
    path: 'admin',
    canActivate: [authGuard, roleGuard(['ADMIN', 'SUPER_ADMIN'])],
    loadComponent: () => import('./admin/admin.component').then((m) => m.AdminComponent),
  },
  {
    path: 'reports',
    canActivate: [authGuard, roleGuard(['ANALYST', 'MANAGER', 'ADMIN'])],
    loadChildren: () => import('./reports/reports.routes').then((m) => m.REPORTS_ROUTES),
  },
  {
    path: 'login',
    // No guard - public route
    loadComponent: () => import('./login/login.component').then((m) => m.LoginComponent),
  },
];

```

IMPORTANT — frontend guards are UX, not security. The server must also validate every API request. Frontend guards only prevent accidental navigation — a determined attacker can always bypass them.

Security rule: Every route guard must have a corresponding server-side API authorization check.
 The Angular 21 guard provides UX protection (prevents rendering).
 The API endpoint provides security protection (prevents data access).
 Both are required. Neither is optional.

Q26 — How do we prevent injection attacks (SQL injection, template injection, command injection) from the Angular 21 frontend?

Why it matters: The frontend is the last line of defence against malicious input — but it is not a sufficient line of defence. Input validation on the frontend is a UX improvement; input validation on the server is a security control. Angular 21 migration is the right time to audit and harden both.

Injection risk categories for Angular 21 frontends:

Injection Type	Frontend Risk Level	Angular 21 Built-in Protection	What You Must Still Do
SQL Injection	None (no DB access)	N/A — Angular never touches a database	Validate all API inputs; parameterised queries server-side
XSS (HTML injection)	High	<code>DomSanitizer</code> sanitizes all <code>[innerHTML]</code> by default	Never use <code>bypassSecurityTrustHtml</code> without review
Template injection	None in prod builds	Templates compiled at build time; no runtime evaluation	Never use <code>eval()</code> or <code>new Function()</code> with user data
URL injection	Medium	Angular sanitizes <code>href</code> and <code>src</code> bindings	Validate route params; use <code>encodeURIComponent()</code>
HTTP header injection	Low	<code>HttpClient</code> does not allow newlines in headers	Validate all values set via <code>setHeaders</code>
JSON injection	Medium	<code>JSON.parse()</code> does not execute code	Validate parsed objects against expected schema
Prototype pollution	Medium	No built-in protection	Use <code>Object.create(null)</code> for dynamic objects; avoid deep merge with user data

Input validation pattern — validate at the boundary:

```
// validated-input.component.ts — validate all user input at the component boundary
@Component({ standalone: true, ... })
export class TransferFormComponent {
  protected readonly amount = signal('');
  protected readonly reference = signal('');

  // Validate at boundary — only allow expected patterns
  protected readonly amountValue = computed(() => {
    const raw = this.amount();
    // Strict numeric validation — rejects injection payloads like '1; DROP TABLE'
    const parsed = parseFloat(raw);
    return isNaN(parsed) || parsed <= 0 || parsed > 1_000_000 ? null : parsed;
  });

  protected readonly referenceValue = computed(() => {
    const raw = this.reference().trim();
    // Whitelist pattern — only alphanumeric and hyphen allowed
    return /^[a-zA-Z0-9-]{0,50}$/.test(raw) ? raw : null;
  });

  protected readonly isValid = computed(() =>
    this.amountValue() !== null && this.referenceValue() !== null
  );
}
```

AI prompt — injection audit:

Audit this Angular 21 service for injection vulnerabilities:

1. Find any dynamic URL construction using user-supplied data (URL injection)
2. Find any string interpolation in HTTP headers (header injection)
3. Find any use of `eval()`, `new Function()`, or `innerHTML` with user data
4. Find any deep object merge using user-supplied data (prototype pollution risk)
5. Find any `JSON.parse()` results used directly without schema validation

For each finding: file, line, risk type, severity, and the safe Angular 21 fix.

[PASTE SERVICE CODE]

Q27 — What security headers, CSP, and infrastructure hardening must be applied alongside the Angular 21 migration?

Why it matters: Angular 21's built-in security protections operate at the JavaScript layer. An attacker who can inject a `<script>` tag into your response headers (via a compromised CDN, cache poisoning, or a vulnerable dependency) bypasses all JavaScript-layer protection. Security headers are the browser-enforced backstop.

Required security headers for Angular 21 SPAs:

Header	Recommended Value	What It Prevents
Content-Security-Policy	See full policy below	XSS, data injection, clickjacking
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	SSL stripping, MITM attacks
X-Frame-Options	DENY (or SAMEORIGIN if iframe bridge used)	Clickjacking
X-Content-Type-Options	nosniff	MIME type sniffing / drive-by downloads
Referrer-Policy	strict-origin-when-cross-origin	Referrer leakage of sensitive URLs
Permissions-Policy	camera=(), microphone=(), geolocation=()	Prevents unexpected browser API access
Cross-Origin-Opener-Policy	same-origin	Cross-origin window access (Spectre mitigations)
Cross-Origin-Resource-Policy	same-origin	Cross-origin resource reads
Cache-Control (API responses)	no-store, no-cache for sensitive data	Sensitive data in browser / proxy cache

nginx config — apply all headers to Angular 21 SPA:

```
# nginx.conf — security headers for Angular 21 SPA
server {
    listen 443 ssl http2;
    root /var/www/angular-app;

    # Security headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
    add_header Cross-Origin-Opener-Policy "same-origin" always;
    add_header Cross-Origin-Resource-Policy "same-origin" always;

    # CSP — Angular 21 with TailwindCSS (adjust for your CDN domains)
    add_header Content-Security-Policy "
        default-src 'self';
        script-src 'self';
        style-src 'self' 'unsafe-inline';
        img-src 'self' data: https:;
        font-src 'self';
        connect-src 'self' https://api.bank.com;
        frame-src 'none';
        object-src 'none';
        base-uri 'self';
        form-action 'self';
    " always;

    # Angular SPA routing — serve index.html for all routes
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache static assets; no-store for index.html (Angular handles cache-busting via hash)
    location /index.html {
        add_header Cache-Control "no-store, no-cache, must-revalidate";
    }

    location ~* \.(js|css|png|jpg|ico|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Security checklist for go-live gate (every migrated feature must pass):

BEFORE any feature goes from canary to 100% production traffic:

- npm audit --audit-level=high returns 0 high/critical vulnerabilities
- OWASP ZAP or Burp Suite passive scan on staging – 0 high findings
- CSP headers present and correct on staging environment (verify with securityheaders.com)
- No `$sce.trustAsHtml` / `bypassSecurityTrust`* calls without documented ADR approval
- All API endpoints require auth token – verified with unauthenticated Playwright test
- RBAC: unauthorized role attempting restricted route returns 403 (Playwright test)
- XSS probe: `<script>alert(1)</script>` in all input fields – verify Angular sanitizes it
- CSRF: remove X-XSRF-TOKEN header from a POST request – verify server returns 403
- No PII in browser console (inspect Network tab + Console tab on staging)
- Lighthouse security audit: no "uses document.write" or "no CSP" warnings
- Security team sign-off documented as ADR update

AI prompt — security pre-flight check:

I am about to deploy a migrated Angular 21 feature to production.
Here is the component code, route guard code, and HTTP service code:

[PASTE CODE]

Perform a security review covering:

1. XSS – any unsafe HTML rendering or DOM manipulation
2. CSRF – are state-changing requests protected?
3. Auth bypass – can the route guard be bypassed? Is the API also protected?
4. Input validation – is all user input validated before being sent to the API?
5. Sensitive data – any PII/token in console.log, URL params, or localStorage?
6. Dependency risk – any npm packages with known CVEs used in this feature?

Rate each risk: Critical / High / Medium / Low / None.

Provide a fix for every finding rated Medium or above.

Section 7: TypeScript Type System — Extracting Interfaces, Types, Generics & Enums

AngularJS is JavaScript — there are no interfaces, types, generics, or enums. Every data shape is implicit: `$http` responses are any, `$scope` properties are typed as "whatever was last assigned to them", and constants are plain objects with string values. Before writing a single line of Angular 21, you must extract these implicit contracts into explicit TypeScript. The questions below drive that extraction process.

 **Cross-reference:** These questions identify *what* to extract. The implementation patterns (how to write interfaces, enums, generics, and `Record` types in code) are covered in [Section 3.12 — TypeScript Type System Migration](#).

Q28 — Where are the implicit data shapes hiding in our AngularJS app, and how do we extract them as TypeScript interfaces?

Goal: Produce a `libs/shared-models/` library containing a TypeScript interface for every domain entity (Account, Transaction, UserProfile, Merchant, etc.) before component migration starts. These interfaces type every `httpResource<T>()`, `signal<T>()`, and `inject<T>()` in the migrated app.

Audit AngularJS Data Shapes for TypeScript Interface Extraction

Analyse the AngularJS 1.8 codebase and extract TypeScript interfaces from every implicit data contract.

****Find implicit shapes in these sources:****

1. ``$http.get('/api/...')` → $.then(function(r) { $scope.data = r.data; })` – what properties does r.data have?`
2. ``$scope.*`` assignments – what properties are assigned, and what are their inferred types?
3. ``ng-repeat="item in items"`` – what is the shape of ``item``?
4. Service return values – what does ``ServiceName.getX()`` return?
5. Form models – ``$scope.formData = {}`` built up with ``formData.name``, ``formData.email``, etc.

****For each shape found:****

- Infer a name from context (e.g., `Account`, `Transaction`, `UserProfile`)
- List every property with its TypeScript type
- Mark optional fields (conditionally assigned or sometimes absent in API responses) with ``?``
- Mark fields that are explicitly set to null as ``T | null``
- Flag any field where the type cannot be inferred as ``unknown`` (needs API schema review)

****Output:**** TypeScript interfaces ready to place in ``libs/shared-models/src/lib/``.

Where implicit data shapes hide in a 100-screen banking app:

AngularJS Source	Where to look	Expected interface
<code>`\$http.get('/api/accounts')`</code>	AccountListCtrl, AccountService	<code>Account</code>
<code>`ng-repeat="txn in transactions"`</code>	TransactionCtrl, transaction template	<code>Transaction</code>
<code>`\$scope.formData = {}`</code> on forms	Any form controller	<code>AccountForm</code> , <code>PaymentForm</code>
<code>`\$http.post('/api/payments', data)`</code>	Payment controllers	<code>PaymentRequest</code> , <code>PaymentResponse</code>
<code>`.constant('CONFIG', {...})`</code>	App module constants	Config type alias
<code>`\$scope.user`</code> from AuthService	Auth/session controllers	<code>UserProfile</code>
<code>`ng-repeat="permission in permissions"`</code>	RBAC controllers	<code>Permission</code>

Q29 — How do we identify and migrate AngularJS magic strings and constant objects to TypeScript enums or string literal unions?

Goal: Zero magic strings in the migrated codebase. Every status value, role name, permission key, and route constant should be a named TypeScript type that the compiler validates at every point of use.

Audit AngularJS Constants and Magic Strings for TypeScript Enum Migration

Scan the codebase for patterns that should become TypeScript enums or string literal unions:

1. ``.constant('AccountStatus', { ACTIVE: 'active', DORMANT: 'dormant' })`` → ``enum AccountStatus``
2. ``if ($scope.account.status === 'pending') { ... }`` repeated in multiple files → string literal union
3. ``$stateParams.tab`` values – one of a fixed set of strings → string literal union ``type TabId``
4. Role checks in templates: ``ng-if="user.role === 'admin'"`` → ``enum UserRole``
5. HTTP error status branches: ``if (error.status === 403)`` in multiple places → numeric literal union
6. Event name strings: ``$rootScope.$broadcast('user:logout')`` → string constants object

****For each pattern found, recommend the right TypeScript construct:****

- Fixed, closed set used in runtime switch/if AND iterated with `Object.values` → ``enum``
- Fixed, closed set serialised over HTTP (JSON) and only used for type checking → string literal ``type``
- Numeric ordinal semantics → numeric ``enum``
- Compile-time constant, never iterated at runtime → ``const enum``

****Output:**** TypeScript enums and type aliases for ``libs/shared-models/src/lib/enums/``.

Q30 — What is the strategy for applying generics to Angular 21 services, signals, and `httpResource` calls?

Goal: Every `httpResource`, `signal`, `computed`, and service method declares an explicit type parameter. No implicit `any`. No `unknown` without justification.

TypeScript Generics Strategy for Angular 21 Migration

For every service and data-loading pattern being migrated, identify where generics apply:

1. `httpResource<T>()` – what interface `T` does each HTTP endpoint return?
2. `signal<T>()` – what is the correct type for each piece of component state?
3. `computed<T>()` – is the return type correctly inferred, or does it need an explicit generic?
4. `HttpClient.get<T>()` – if not using `httpResource`, is the HTTP client call typed?
5. Generic services – can a service accept a type parameter to serve multiple entity types?
6. API response wrappers – does every response come wrapped in `{ data: T, meta: Pagination }`?

****For each AngularJS service being migrated:****

- State the current untyped return value (typically `any` or bare `$http` promise)
- Provide the Angular 21 typed equivalent with correct generic parameter
- Flag any remaining `any` or `unknown` and explain why it could not be typed

****Goal:**** Zero `any` in `libs/shared-models/` and `libs/shared-auth/`. Allow `any` only in migration shims with an eslint-disable comment and a linked TODO tracking ticket.

Q31 — When should we use `interface` vs `type` vs `Record<K,V>` vs `enum` in the migrated codebase?

Goal: A documented, consistent decision framework so every pull request follows the same choices — no repeated `type` vs `interface` debates in code review.

TypeScript Type Construct Selection Rules for the Migrated Codebase

Apply these rules when choosing a TypeScript construct. Add this to the team's coding standards.

****Decision rules:****

- `interface` – object shapes that may be extended via `extends` or implemented by classes
- `type` – unions, intersections, utility types (`Partial<T>`, `Pick<T,K>`), and shapes never extended
- `Record<K, V>` – homogeneous key-value maps; replaces `{ [key: string]: V }` anti-pattern
- `enum` – named constants with a closed value set; use string values to match API strings
- String literal union `'a' | 'b' | 'c'` – closed string sets serialised over HTTP (safer than `enum` with JSON)
- `const enum` – performance-critical constants inlined at compile time; no runtime object generated
- `class` – only for Angular services, components, and anything instantiated by the DI container

****Audit for these anti-patterns and list every instance:****

- `any` → correct interface or generic parameter
- `{}` used as a map → `Record<string, T>`
- `Object` type → `Record<string, unknown>` or a named interface
- Repeated identical inline object shapes → named `interface` in `shared-models`
- String compared with `=== 'magic-string'` in 3+ places → `enum` or string literal union

Output a prioritised list of every anti-pattern instance found, with the recommended fix.

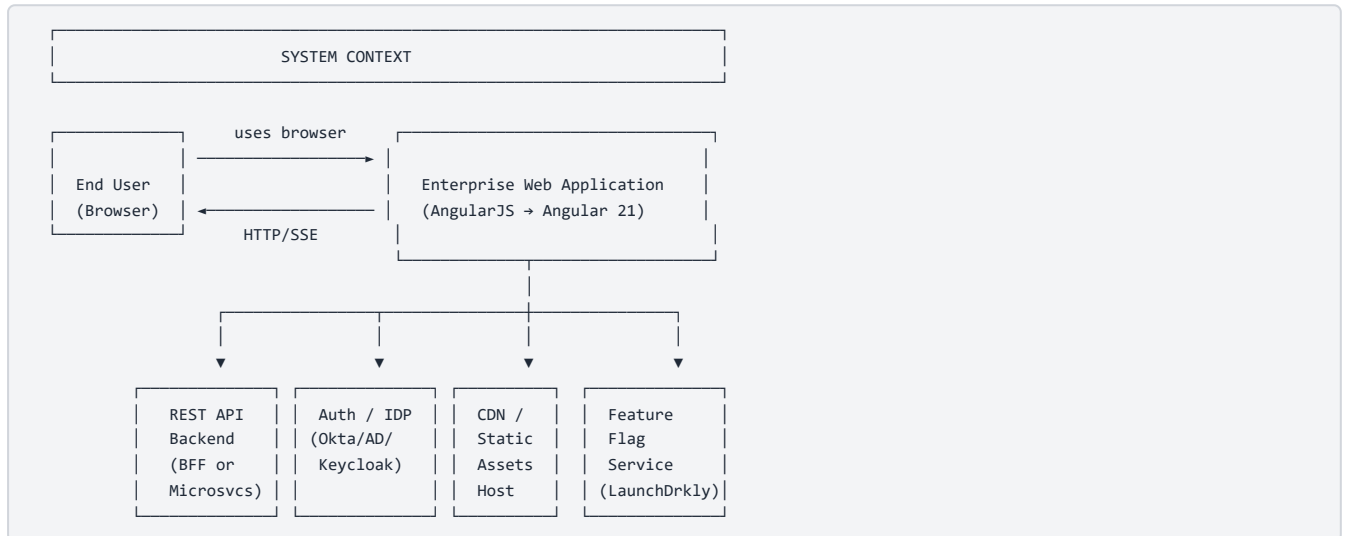
Part 1 complete. This Q&A section is the discovery foundation for the entire migration. The answers you fill in here drive every decision in Parts 2 through 8.

Part 2 — High-Level Design (HLD)

How to use this section: The HLD translates the Q&A answers from Part 1 into an architectural blueprint. Every diagram and table here should be reviewed with the engineering lead, product owner, and DevOps/platform team before sprint planning begins. The HLD does not prescribe code — it prescribes *structure*.

2.1 — System Context (C4 Level 1)

The C4 Level 1 diagram shows the migrating application in the context of its users and external systems. This is the view you present to leadership and business stakeholders.



Key insight: During migration the "Enterprise Web Application" box contains BOTH the AngularJS app and the Angular 21 app. From the external system's perspective — the backend API, the IDP, the CDN — nothing changes. This is the architectural safety guarantee of an incremental migration.

2.2 — Container Diagram (C4 Level 2) — Migration Period Architecture

This view is for your engineering leads. It shows exactly what runs where during the migration period, before AngularJS is decommissioned.

MIGRATION PERIOD – CONTAINER VIEW
(Both apps live simultaneously)

Browser



2.3 — Technology Stack Comparison

Layer	AngularJS (Before)	Angular 21 (After)	Migration Action
Framework	AngularJS 1.8	Angular 21 standalone	Rewrite; no upgrade path
Language	ES5 / ES6 (transpiled)	TypeScript 5.x strict mode	Codemods + manual review
State (local)	<code>\$scope</code> variables	<code>signal()</code> / <code>computed()</code> / <code>effect()</code>	1:1 conversion with AI
State (global)	<code>\$rootScope</code> / <code>\$broadcast</code>	Signal-based <code>EventBusService</code> or <code>NgRx</code>	Design as ADR before sprint
HTTP	<code>\$http</code> service	<code>HttpClient</code> with <code>HttpInterceptorFn</code>	Service-by-service swap
Routing	<code>ngRoute</code> / <code>ui-router</code>	<code>@angular/router</code> with lazy loading	Route mapping spreadsheet required
Templates	<code>ng-if</code> , <code>ng-repeat</code> , <code>ng-class</code>	<code>@if</code> , <code>@for</code> , <code>[class.name]</code>	<code>@angular/codemod</code> automates this
Components	Controllers + <code>\$scope</code>	Standalone components with <code>input()</code>	AI-assisted per-file conversion
DI	<code>\$injector</code> / <code>angular.module</code>	<code>inject()</code> function	Codemod + review
Forms	<code>ngModel</code> + <code>\$validators</code>	Signal forms + <code>computed()</code> validators	Manual redesign for complex forms
Styling	CSS/SCSS + any framework	TailwindCSS or Angular Material 21	Library replacement ADR required
Build	Grunt / Gulp / Webpack 4	Nx + Angular CLI + Webpack 5 / Vite	Setup new build alongside; don't touch old
Testing (unit)	Karma + Jasmine	Vitest + Angular Testing Library	New test stack; AI generates specs
Testing (E2E)	Protractor (EOL) / Karma	Playwright	Full rewrite of E2E suite
Monorepo	None (single repo)	Nx workspace	Create workspace; import legacy as app
Feature flags	None (hardcoded)	LaunchDarkly / Custom signal service	New capability; add during shell setup
SSR	Not supported	Angular Universal / SSR + hydration	Enable after migration; not during
Deployment	Single app server	CDN-first; separate shell + remote URLs	Infrastructure change; involves DevOps

2.3.1 — Browser Support Matrix

Critical for Enterprise Migrations: Verify browser requirements before starting.

Browser	AngularJS 1.8	Angular 21	Migration Impact
Chrome (latest)	✅ Supported	✅ Supported	No impact
Edge (Chromium)	✅ Supported	✅ Supported	No impact
Firefox (latest)	✅ Supported	✅ Supported	No impact
Safari 15+	✅ Supported	✅ Supported	No impact
Safari 13-14	✅ Supported	⚠️ Partial (polyfills)	Add core-js polyfills
IE11	✅ Supported	❌ NOT SUPPORTED	BLOCKER — See decision tree
Mobile Safari (iOS 15+)	✅ Supported	✅ Supported	Test PWA install
Chrome Android	✅ Supported	✅ Supported	No impact

IE11 Decision Tree:

```
Q: Do we still support IE11?  
├ YES → Cannot migrate to Angular 21 (ES2015+ required)  
│   Options: (A) Drop IE11 support, (B) Separate IE11 build, (C) Stay on AngularJS  
└ NO → ✅ Proceed with migration
```

Why Angular 21 doesn't support IE11:

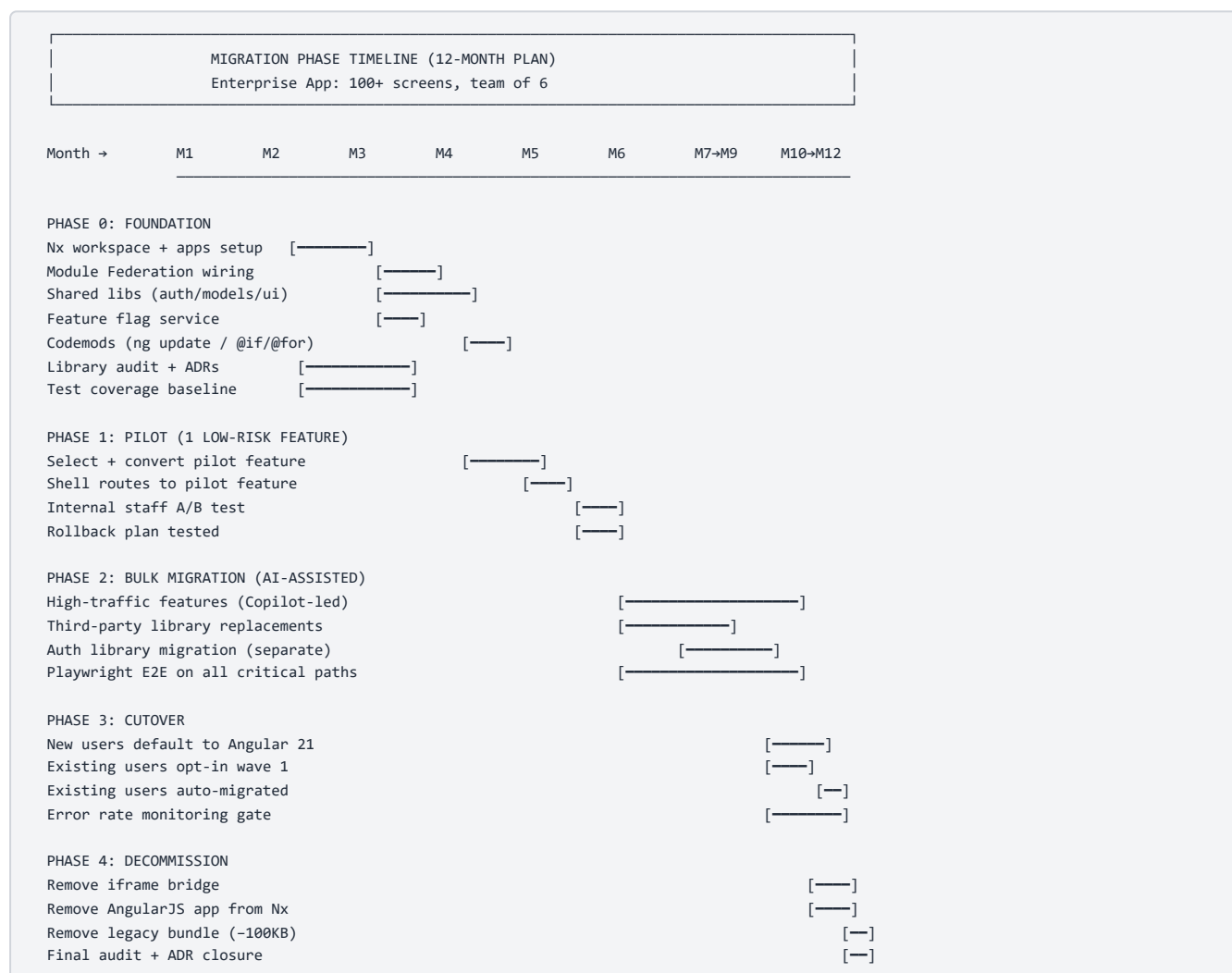
- Angular 21 requires ES2015+ (classes, arrow functions, const/let, Promises)
- Polyfilling IE11 to ES2015 adds 300KB+ to bundle size
- OnPush change detection incompatible with IE11's object mutation detection
- Signals use Proxy API (not polyfillable in IE11)

Recommendation: If IE11 support is mandatory, negotiate with stakeholders to drop it. IE11 market share is < 0.5% globally as of 2026. Microsoft ended support in June 2022.

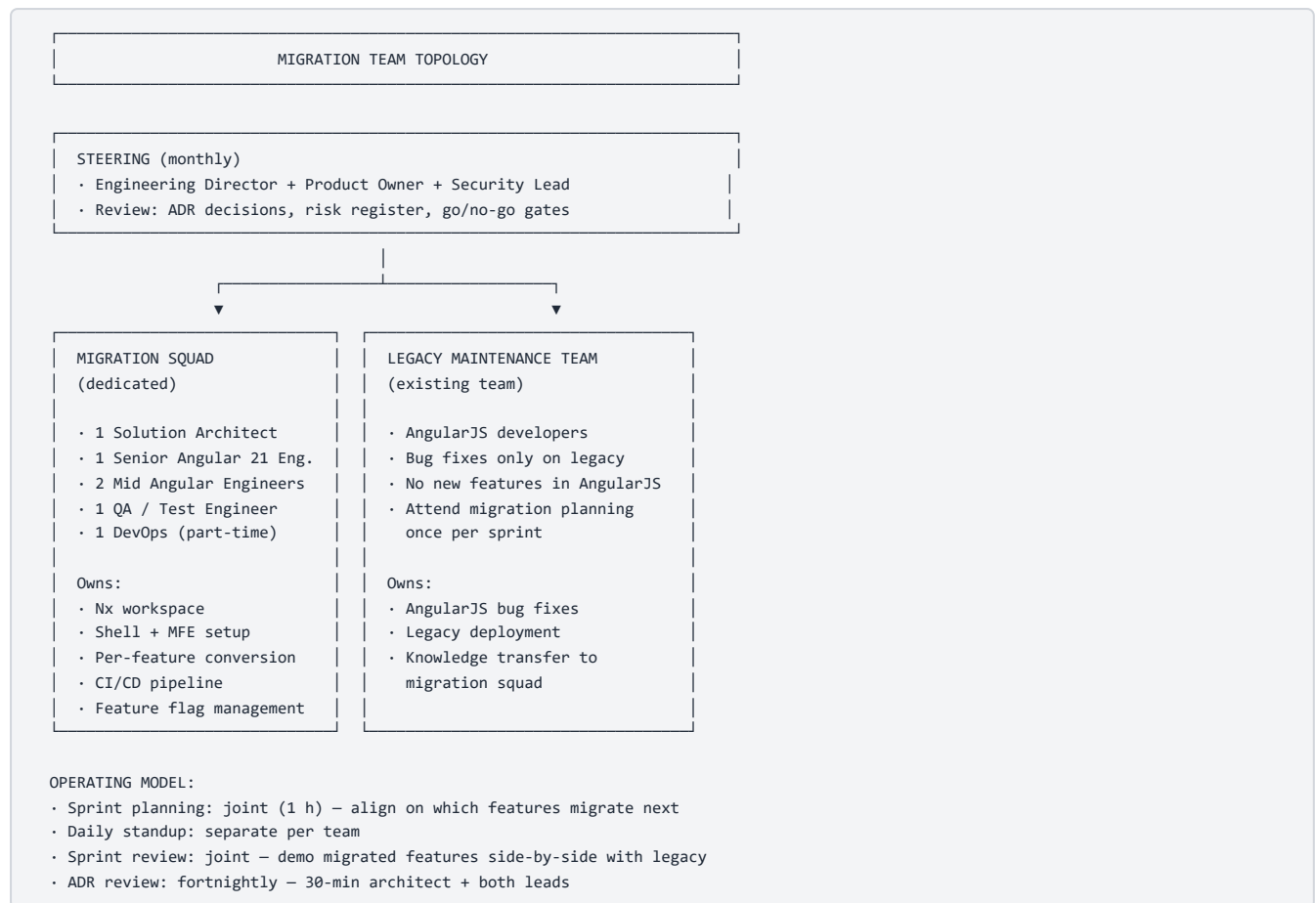
Polyfill Configuration (angular.json):

```
{
  "projects": {
    "banking-ng21": {
      "architect": {
        "build": {
          "options": {
            "polyfills": ["zone.js", "@angular/localize/init"]
          }
        }
      }
    }
  }
}
```

2.4 — Migration Phase HLD Timeline



2.5 — Team Topology HLD



2.6 — Module Federation vs Polyrepo Strategy

Option A: Module Federation (Monorepo with Independent Deployment)

Module Federation is the deployment mechanism that allows the Angular 21 shell and the Angular 21 feature remote to deploy independently within an Nx monorepo.

Pros:

- Shared code (libs/) compiled once
- Type-safe imports across apps
- Single CI/CD pipeline with nx affected
- No CORS issues (shared singleton libraries)

Cons:

- Complex webpack configuration
- Learning curve for team
- Version conflicts require careful management

Option B: Polyrepo Strategy (Separate Repositories with Direct Routing)

Architecture: Angular 21 shell and AngularJS app are separate deployments with cross-domain routing.

How it works:

```

Shell (shell.bank.com)           Legacy (legacy.bank.com)
├─ /dashboard (Angular 21)       ├─ /#!/statements (AngularJS)
├─ /accounts (Angular 21)       ├─ /#!/reports (AngularJS)
└─ <a href="https://legacy.bank.com/#!/statements"> → Direct navigation

Legacy can link back:
<a href="https://shell.bank.com/dashboard"> → Back to Angular 21

```

Key Difference from iframe approach:

- ❌ **iframe problems:** Routing conflicts (parent controls URL, iframe has separate history), browser back button breaks, poor SEO, accessibility issues (nested navigation)
- ✅ **Polyrepo solution:** Full page navigation (hard reload between apps), each app controls its own routing, no iframe nesting, works with browser back/forward naturally

Authentication Bridge: Both apps must share authentication. Use **shared domain cookies** or **token in query parameter**:

```

// Shell redirects to legacy with token
navigateToLegacy(route: string) {
  const token = this.authService.getToken();
  window.location.href = `https://legacy.bank.com/${route}?token=${token}`;
}

// Legacy reads token on bootstrap
angular.module('app').run(['$location', '$http', function($location, $http) {
  const token = $location.search().token;
  if (token) {
    $http.defaults.headers.common['Authorization'] = `Bearer ${token}`;
    sessionStorage.setItem('auth_token', token);
    // Remove token from URL for security
    $location.search('token', null);
  }
}]);

```

Decision Matrix:

Criterion	Module Federation (Monorepo)	Polyrepo Strategy
Code sharing	✅ Excellent (typed libs)	❌ Must duplicate or use npm packages
Routing complexity	⚠️ Medium (MFE + iframe fallback)	✅ Simple (just links)
Team independence	⚠️ Medium (shared Nx workspace)	✅ High (separate repos)
CI/CD complexity	✅ Simple (nx affected)	⚠️ Medium (2 pipelines)
Learning curve	❌ High (Module Federation)	✅ Low (standard deployment)
iframe routing problems	⚠️ Yes (for legacy screens)	✅ None (full page nav)
Recommended for	50-120 screens, 1 team, shared libs	120+ screens, 2+ teams, separate domains

When to use Polyrepo:

- Legacy app is maintained by separate team
- No need for shared TypeScript libraries
- Migration will take 12+ months
- Want to avoid iframe routing issues entirely
- Different deployment schedules (legacy rarely changes)

Option C: Same Domain, Path-Based Routing (Recommended for Most Teams)

Architecture: Both Angular 21 and AngularJS apps served from same domain under different paths.

```

https://bank.com/
├─ /new/           → Angular 21 app (new screens)
├─ /legacy/        → AngularJS app (not yet migrated)
└─ /api/           → Backend API (shared by both)

```

✅ Key Benefits:

- **✓ No CORS issues** — same domain means cookies, localStorage, sessionStorage all work automatically
- **✓ No iframe complexity** — each app has its own routing, no parent/child conflicts
- **✓ Shared authentication** — HttpOnly cookies work seamlessly across both apps
- **✓ Clean URLs** — `bank.com/new/dashboard` VS `bank.com/legacy/statements`
- **✓ Works in Nx monorepo** — different base hrefs, single workspace
- **✓ Progressive migration** — can move routes from `/legacy` to `/new` incrementally
- **✓ SEO friendly** — both apps crawlable, no iframe content hidden

Nx Workspace Configuration for Path-Based Deployment

Step 1: Configure base hrefs in project.json

```
// apps/banking-ng21/project.json
{
  "name": "banking-ng21",
  "targets": {
    "build": {
      "executor": "@angular-devkit/build-angular:application",
      "options": {
        "baseHref": "/new/", // ← Angular 21 app served at /new/
        "outputPath": "dist/apps/banking-ng21",
        "index": "apps/banking-ng21/src/index.html",
        "browser": "apps/banking-ng21/src/main.ts"
      }
    }
  }
}

// apps/banking-legacy/project.json
{
  "name": "banking-legacy",
  "targets": {
    "build": {
      "executor": "@nx/webpack:webpack",
      "options": {
        "baseHref": "/legacy/", // ← AngularJS app served at /legacy/
        "outputPath": "dist/apps/banking-legacy",
        "webpackConfig": "apps/banking-legacy/webpack.config.js"
      }
    }
  }
}
```

Step 2: Configure Angular Router with base href

```
// apps/banking-ng21/src/app/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    // Base href is set in index.html: <base href="/new/">
    // Router automatically respects this
  ],
};

// apps/banking-ng21/src/app/app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  {
    path: 'dashboard',
    loadComponent: () => import('./features/dashboard/dashboard.component'),
  },
  {
    path: 'accounts',
    loadComponent: () => import('./features/accounts/account-list.component'),
  },
  // All routes are relative to /new/ base href
  // User sees: bank.com/new/dashboard, bank.com/new/accounts
];
```

Step 3: Configure AngularJS app for /legacy/ base

```
// apps/banking-legacy/src/app/app.config.js
angular.module('bankingApp', ['ui.router']).config([
  '$locationProvider',
  '$stateProvider',
  function ($locationProvider, $stateProvider) {
    // Use HTML5 mode with base /legacy/
    $locationProvider.html5Mode({
      enabled: true,
      requireBase: true,
    });

    $stateProvider
      .state('statements', {
        url: '/statements', // Becomes /legacy/statements
        templateUrl: 'views/statements.html',
        controller: 'StatementsCtrl',
      })
      .state('reports', {
        url: '/reports', // Becomes /legacy/reports
        templateUrl: 'views/reports.html',
        controller: 'ReportsCtrl',
      });
  },
]);
```

```
<!-- apps/banking-legacy/src/index.html -->
<!DOCTYPE html>
<html ng-app="bankingApp">
  <head>
    <base href="/legacy/" />
    <!-- AngularJS respects this base -->
    <title>Banking App</title>
  </head>
  <body>
    <div ui-view></div>
  </body>
</html>
```

Nginx Reverse Proxy Configuration

Deploy both apps behind a single nginx server that routes based on path:

```

# /etc/nginx/sites-available/bank.com

server {
    listen 443 ssl http2;
    server_name bank.com;

    ssl_certificate /etc/ssl/certs/bank.com.crt;
    ssl_certificate_key /etc/ssl/private/bank.com.key;

    # Security headers
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";

    # Root redirects to Angular 21 app
    location = / {
        return 301 /new/dashboard;
    }

    # Angular 21 app at /new/*
    location /new/ {
        alias /var/www/banking-ng21/;
        try_files $uri $uri/ /new/index.html =404; # SPA fallback with 404 final

        # Cache static assets
        location ~* \.(js|css|png|jpg|jpeg|gif|svg|woff|woff2|ttf|eot)$ {
            expires 1y;
            add_header Cache-Control "public, immutable";
        }
    }

    # HTTPS enforcement (redirect HTTP to HTTPS)
    # ⚠️ This block should be in a separate server{} listening on port 80
}

# HTTP to HTTPS redirect
server {
    listen 80;
    server_name bank.com;
    return 301 https://$server_name$request_uri;
}

# HTTPS with HSTS (main server)
server {
    listen 443 ssl http2;
    server_name bank.com;

    ssl_certificate /etc/ssl/certs/bank.com.crt;
    ssl_certificate_key /etc/ssl/private/bank.com.key;

    # HSTS: Force HTTPS for 2 years (31536000 seconds = 1 year)
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;

    # Security headers
    add_header X-Frame-Options DENY always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # ... rest of location blocks as above

    # Legacy AngularJS app at /legacy/*
    location /legacy/ {
        alias /var/www/banking-legacy/;
        try_files $uri $uri/ /legacy/index.html =404; # SPA fallback with 404 final

        # Legacy app assets
        location ~* \.(js|css|png|jpg|jpeg|gif|svg|woff|woff2|ttf|eot)$ {
            expires 1y;
            add_header Cache-Control "public, immutable";
        }
    }
}

# API proxy (shared by both apps)
location /api/ {
    proxy_pass http://backend-server:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

```

```
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# Health check endpoint
location /health {
    return 200 "OK";
    add_header Content-Type text/plain;
}
}
```

Azure Front Door Configuration (Cloud Alternative)

If deploying to Azure, use Front Door for path-based routing:


```

# Create Front Door profile
az afd profile create \
  --profile-name banking-migration \
  --resource-group banking-rg \
  --sku Standard_AzureFrontDoor

# Create endpoint
az afd endpoint create \
  --profile-name banking-migration \
  --endpoint-name bank-com \
  --resource-group banking-rg

# Add origin group for Angular 21 app
az afd origin-group create \
  --profile-name banking-migration \
  --origin-group-name ng21-origin \
  --resource-group banking-rg

az afd origin create \
  --profile-name banking-migration \
  --origin-group-name ng21-origin \
  --origin-name ng21-app \
  --resource-group banking-rg \
  --host-name banking-ng21.azurestaticapps.net \
  --origin-host-header banking-ng21.azurestaticapps.net \
  --priority 1 \
  --weight 1000

# Add origin group for AngularJS app
az afd origin-group create \
  --profile-name banking-migration \
  --origin-group-name legacy-origin \
  --resource-group banking-rg

az afd origin create \
  --profile-name banking-migration \
  --origin-group-name legacy-origin \
  --origin-name legacy-app \
  --resource-group banking-rg \
  --host-name banking-legacy.azurewebsites.net \
  --origin-host-header banking-legacy.azurewebsites.net \
  --priority 1 \
  --weight 1000

# Route /new/* to Angular 21
az afd route create \
  --profile-name banking-migration \
  --endpoint-name bank-com \
  --route-name new-route \
  --resource-group banking-rg \
  --origin-group ng21-origin \
  --patterns-to-match "/new/*" \
  --supported-protocols Https \
  --forwarding-protocol HttpsOnly

# Route /legacy/* to AngularJS
az afd route create \
  --profile-name banking-migration \
  --endpoint-name bank-com \
  --route-name legacy-route \
  --resource-group banking-rg \
  --origin-group legacy-origin \
  --patterns-to-match "/legacy/*" \
  --supported-protocols Https \
  --forwarding-protocol HttpsOnly

# Custom domain
az afd custom-domain create \
  --profile-name banking-migration \
  --custom-domain-name bank-com \
  --resource-group banking-rg \
  --host-name bank.com \
  --minimum-tls-version TLS12

```

Cross-App Navigation

From Angular 21 to AngularJS:

```
// apps/banking-ng21/src/app/components/nav.component.ts
@Component({
  selector: 'app-nav',
  template: `
    <nav>
      <!-- Angular 21 routes (relative) -->
      <a routerLink="/dashboard">Dashboard</a>
      <a routerLink="/accounts">Accounts</a>

      <!-- Link to AngularJS routes (absolute with /legacy prefix) -->
      <a href="/legacy/statements">Statements (Legacy)</a>
      <a href="/legacy/reports">Reports (Legacy)</a>
    </nav>
  `,
})
export class NavComponent {}
```

From AngularJS to Angular 21:

```
<!-- apps/banking-legacy/src/views/nav.html -->
<nav ng-controller="NavCtrl">
  <!-- AngularJS routes (ui-router) -->
  <a ui-sref="statements">Statements</a>
  <a ui-sref="reports">Reports</a>

  <!-- Link to Angular 21 routes (standard href with /new prefix) -->
  <a href="/new/dashboard">Dashboard (New)</a>
  <a href="/new/accounts">Accounts (New)</a>
</nav>
```

Authentication Sharing (Automatic with Same Domain)

HttpOnly Cookie Strategy (Recommended):

```
// Backend sets cookie for entire domain
// Set-Cookie: auth_token=xyz; Path=/; Domain=bank.com; HttpOnly; Secure; SameSite=Strict

// Both /new/ and /legacy/ apps receive the cookie automatically
// No token passing needed – cookie is sent on every request
```

Both apps authenticate transparently:

```
// Angular 21 app – cookie sent automatically
this.http.get('/api/accounts').subscribe(...);

// AngularJS app – cookie sent automatically
$http.get('/api/accounts').then(...);
```

Comparison: All 3 Options

Criterion	Module Federation	Polyrepo (Subdomains)	Path-Based (Same Domain)
Domain strategy	shell.bank.com, ng21.bank.com, legacy.bank.com	shell.bank.com, legacy.bank.com	bank.com/new/, bank.com/legacy/
CORS issues	✅ None	⚠️ Possible (cross-domain)	✅ None (same domain)
Cookie sharing	⚠️ Need SameSite=None	⚠️ Need Domain=.bank.com	✅ Automatic (same domain)
Authentication	Complex (Module Federation or postMessage)	Medium (query param or cookie)	✅ Simple (HttpOnly cookie just works)
Routing	⚠️ Shell controls, iframe for legacy	✅ Each app independent	✅ Each app independent
iframe problems	⚠️ Yes (for legacy)	✅ None	✅ None
Nx workspace support	✅ Native	❌ Separate repos	✅ Native (different base hrefs)
CI/CD	✅ Simple (nx affected)	⚠️ Medium (2 pipelines)	✅ Simple (nx affected, deploy both)
Learning curve	❌ High (MFE)	✅ Low	✅ Low
Production complexity	⚠️ Medium (CDN + origins)	⚠️ Medium (DNS + certs)	✅ Low (single reverse proxy)
Recommended for	Micro-frontend architecture, 2+ Angular 21 remotes	Separate teams, different tech stacks	Most migrations (1 team, shared domain)

When to Use Path-Based Routing (Option C)

✅ Use this approach if:

- You have a single domain (bank.com) and want to keep it
- You want to avoid CORS complexity entirely
- Authentication should "just work" across both apps
- You're using Nx monorepo (same workspace)
- Migration is happening screen-by-screen over 6-12 months
- Both apps share the same backend API
- You want the simplest possible deployment

❌ Don't use if:

- You need micro-frontend architecture with 3+ independent Angular 21 apps (use Module Federation)
- Legacy and new apps are maintained by completely separate teams in different repos (use Polyrepo)
- You need independent deployment with zero coordination (use Module Federation)

Nx Workspace Structure for Path-Based Deployment

```
banking-migration/
├── apps/
│   ├── banking-ng21/           ← Angular 21 (served at /new/)
│   │   ├── src/
│   │   │   ├── index.html      <base href="/new/">
│   │   │   └── app/
│   │   │       └── app.routes.ts Paths: '', 'dashboard', 'accounts'
│   │   └── project.json        baseHref: "/new/"
│   └── banking-legacy/        ← AngularJS (served at /legacy/)
│       ├── src/
│       │   ├── index.html      <base href="/legacy/">
│       │   └── app/
│       │       └── app.config.js $locationProvider.html5Mode(true)
│       └── project.json        baseHref: "/legacy/"
├── libs/                      ← Shared (only Angular 21 apps use these)
│   ├── shared-models/
│   ├── shared-auth/
│   └── shared-ui/
├── nginx.conf                 ← Nginx config for path-based routing
├── azure-frontdoor.sh         ← Azure Front Door setup script
└── package.json
```

Build both apps:

```
# Build Angular 21 app (output: dist/apps/banking-ng21/)
npx nx build banking-ng21 --configuration=production --base-href=/new/

# Build AngularJS app (output: dist/apps/banking-legacy/)
npx nx build banking-legacy --configuration=production --base-href=/legacy/

# Deploy both to same server under /var/www/
```

Deployment structure:

```
/var/www/
├── banking-ng21/           → served at /new/
│   ├── index.html         <base href="/new/">
│   ├── main.js
│   └── ...
└── banking-legacy/        → served at /legacy/
    ├── index.html         <base href="/legacy/">
    ├── app.js
    └── ...
```

Module Federation HLD (Option A)

If you choose Module Federation, this is the core HLD for the Nx + MFE approach.

MODULE FEDERATION – HLD

BUILD TIME (Nx workspace):

```
apps/shell          → built to dist/apps/shell
apps/banking-ng21   → built to dist/apps/banking-ng21
                    exports remoteEntry.js
apps/banking-legacy → built to dist/apps/banking-legacy
libs/shared-auth    → compiled but NOT separately deployed
libs/shared-models  → compiled but NOT separately deployed
libs/shared-ui      → compiled but NOT separately deployed
```

RUNTIME (Browser loads):

```
1. Browser requests shell.bank.com
   → CDN serves Angular 21 shell (index.html + main.js)

2. User navigates to /dashboard
   → shell calls loadRemoteModule({
       remoteEntry: 'https://ng21.bank.com/remoteEntry.js',
       exposedModule: './DashboardComponent'
   })
   → remoteEntry.js is fetched from CDN (ng21.bank.com)
   → DashboardComponent module chunk is loaded on-demand

3. User navigates to /statements (not yet migrated)
   → shell renders LegacyFrameComponent
   → iframe src = "https://legacy.bank.com/#!/statements"
   → postMessage forwards auth token to legacy app
```

```
SHARED SINGLETON LIBRARIES (loaded once by shell):
• @angular/core
• @angular/router
• rxjs
→ Remote app REUSES shell's copy (no double-bundle)
```

DEPLOYMENT TOPOLOGY:

```
shell.bank.com      → Azure Static Web Apps / Cloudflare Pages
ng21.bank.com       → Azure Static Web Apps (separate deployment)
legacy.bank.com     → Existing app server (unchanged)

Each domain deploys INDEPENDENTLY via its own CI/CD pipeline.
Shell rarely changes; ng21 deploys on every feature completion.
```

Module Federation version-pinning rule: Shell and remote must share the same major Angular version. The shell enforces `singleton: true` for `@angular/core` and `rxjs`. If a remote tries to load a different version, Module Federation throws a version conflict error in the console — this is a feature, not a bug. It prevents silent runtime breakage.

2.7 — Integration Points HLD

These are all the places where the AngularJS app and the Angular 21 app must share data or state during the migration period. Each integration point is a potential failure mode.

INTEGRATION POINTS MAP

AngularJS App ← → Angular 21 Shell
(legacy.bank.com) (shell.bank.com)

INTEGRATION POINT 1: Authentication Token

Mechanism: localStorage (shared – same origin)
Angular 21 → writes JWT to localStorage key 'app_access_token'
AngularJS → reads same key on bootstrap; listens for 'auth:token-updated' event
Risk: Token format must not change during migration period

INTEGRATION POINT 2: User Preferences / Settings

Mechanism: Backend API (both apps call the same preferences endpoint)
Both apps read/write via REST; no frontend sync needed
Risk: Race condition if user has both tabs open (old + new)

INTEGRATION POINT 3: Navigation / URL State

Mechanism: Browser URL (shell owns routing)
Shell reads current URL and decides which app renders
Legacy app's hash routes (/#!/dashboard) are proxied via iframe
Risk: Deep-link sharing – a legacy URL must still work after cutover

INTEGRATION POINT 4: Feature Flags

Mechanism: LaunchDarkly / env signal in shell
Shell evaluates flags; AngularJS app has no flag awareness
Risk: Flag state must be checked on every page load, not just on startup

INTEGRATION POINT 5: Error / Telemetry

Mechanism: Shared Application Insights / Datadog SDK
Both apps use the same telemetry key; errors are tagged with 'app:legacy' or 'app:ng21'
Risk: Correlation ID must be consistent across both apps for request tracing

INTEGRATION POINT 6: Shared UI Design Tokens

Mechanism: Nx shared-ui lib provides CSS custom properties
Angular 21 imports tokens from shared-ui
AngularJS loads compiled CSS from /legacy/styles/tokens.css
Risk: Token drift – if shared-ui tokens change, legacy CSS must be re-exported

2.7.3 — Cross-App Messaging (Shell ↔ Legacy iframe)

When legacy screens are loaded in iframe, use `postMessage` API for secure communication:

Shell → Legacy (Notify Auth Token Refresh)

```
// apps/banking-shell/src/app/services/legacy-bridge.service.ts
import { Injectable, inject } from '@angular/core';
import { DOCUMENT } from '@angular/common';

@Injectable({ providedIn: 'root' })
export class LegacyBridgeService {
  private readonly document = inject(DOCUMENT);

  notifyTokenRefresh(newToken: string): void {
    const legacyFrame = this.document.getElementById('legacy-iframe') as HTMLIFrameElement;
    if (legacyFrame?.contentWindow) {
      legacyFrame.contentWindow.postMessage(
        { type: 'TOKEN_REFRESH', token: newToken },
        'https://legacy.bank.com', // ← origin must match – critical for security
      );
    }
  }

  notifyLogout(): void {
    const legacyFrame = this.document.getElementById('legacy-iframe') as HTMLIFrameElement;
    legacyFrame?.contentWindow?.postMessage({ type: 'LOGOUT' }, 'https://legacy.bank.com');
  }
}
```

Legacy → Shell (Notify Logout or Route Change)

```
// apps/banking-legacy/src/app/services/shell-bridge.service.js
angular.module('app').factory('ShellBridgeService', [
  '$window',
  function ($window) {
    return {
      notifyLogout: function () {
        // Notify parent shell app
        if ($window.parent !== $window) {
          $window.parent.postMessage({ type: 'LOGOUT' }, 'https://shell.bank.com');
        }
      },

      notifyRouteChange: function (newRoute) {
        if ($window.parent !== $window) {
          $window.parent.postMessage(
            { type: 'ROUTE_CHANGE', route: newRoute },
            'https://shell.bank.com',
          );
        }
      },
    };
  },
]);
```

Shell Listens for Legacy Messages

```
// apps/banking-shell/src/app/app.component.ts
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '@banking/shared-auth';

@Component({
  selector: 'app-root',
  template: `<router-outlet />`,
  standalone: true,
})
export class AppComponent {
  private readonly router = inject(Router);
  private readonly auth = inject(AuthService);

  constructor() {
    // Listen for messages from legacy iframe
    window.addEventListener('message', (event) => {
      // ⚠️ CRITICAL: Always validate origin
      if (event.origin !== 'https://legacy.bank.com') {
        console.warn('Rejected message from untrusted origin:', event.origin);
        return;
      }

      switch (event.data.type) {
        case 'LOGOUT':
          this.auth.logout();
          this.router.navigate(['/login']);
          break;
        case 'ROUTE_CHANGE':
          console.log('Legacy route changed:', event.data.route);
          // Update analytics or breadcrumbs if needed
          break;
      }
    });
  }
}
```

Security Rules:

- ✓ **Always validate** `event.origin` — reject messages from unknown domains
- ✓ **Use typed message contracts** — define `type` field in all messages
- ✓ **Never send sensitive data** — tokens should be in HttpOnly cookies, not `postMessage`
- ✗ **Never use** `targetOrigin: '*'` — always specify exact origin

2.8 — Feature Flag HLD (Migration Toggle Architecture)

Feature flags are the on/off switch for every migrated feature. Without them, there is no safe way to roll back a feature that regresses in production.

FEATURE FLAG ARCHITECTURE

Flag Service (Angular 21 Signal-based)

```
@Injectable({ providedIn: 'root' })
export class FeatureFlagService {
  // Flags loaded from environment / LaunchDarkly on app init
  private readonly flags = signal<Record<string, boolean>>({});

  isEnabled(flag: string): boolean { return !!this.flags()[flag]; }
  flagSignal(flag: string): Signal<boolean> {
    return computed(() => !!this.flags()[flag]);
  }
}
```

Usage in shell routes:

```
if (featureFlags.isEnabled('new-dashboard'))
  → load Angular 21 DashboardComponent
else
  → render LegacyFrameComponent with /legacy/#!/dashboard
```

ROLLOUT STAGES PER FEATURE:

Stage 0 – OFF	: Legacy AngularJS served to all users
Stage 1 – INTERNAL	: Angular 21 enabled for internal IP range only
Stage 2 – BETA	: Angular 21 enabled for opted-in users (< 5%)
Stage 3 – CANARY	: Angular 21 enabled for 20% of new users
Stage 4 – MAJORITY	: Angular 21 enabled for all new users (100%)
Stage 5 – MIGRATION	: Existing users migrated in waves (10% per week)
Stage 6 – COMPLETE	: Angular 21 for all; legacy flag removed from code

ROLLBACK TRIGGERS (auto-revert to Stage 0):

- Error rate > 1% above 30-day baseline (monitored by Datadog alert)
- Lighthouse LCP regression > 500 ms on migrated route
- WCAG AA regression detected by axe-core in CI
- Any P0 incident on migrated route during canary period

2.9 — HLD Non-Negotiables (Architecture Constraints)

These constraints are agreed before development begins. Any proposal that violates them requires a new ADR and steering approval.

Constraint	Rationale	Enforcement
AngularJS app is never modified for migration	Prevents regression on live features; migration team does not own legacy	Code ownership policy
Shell app owns all routing decisions	Single point of control; AngularJS never manages top-level routes	Architecture review on every PR
Auth token is read-only in AngularJS	Angular 21 shell is the single source of truth for auth state	Security review gate
Module Federation shared singletons are pinned	Prevents double-bundle; version drift causes runtime errors	CI build validation step
Feature flags must exist before migration starts	No feature goes live without a corresponding flag	PR template checklist
Vitest coverage $\geq$ 80% before any feature is cutover	Prevents untested code reaching users	<code>nx affected:test --coverage gate</code>
All migrated components use <code>ChangeDetectionStrategy.OnPush</code>	Performance contract; prevents zone-triggered re-renders	ESLint rule: <code>@angular-eslint/prefer-on-push-component-change-detection</code>
No <code>@Input()</code> / <code>@Output()</code> decorators in new code	Angular 21 standard; <code>input()</code> / <code>output()</code> only	ESLint rule: <code>@angular-eslint/no-input-rename</code>
SSR is enabled AFTER migration, not during	Avoid double complexity during coexistence period	Architecture gate at Phase 3

Part 2 complete. The HLD gives the architecture blueprint for the migration — system context, container layout, technology stack, phase timeline, team topology, Module Federation wiring, integration points, feature flag architecture, and constraints.

Part 3 — Low-Level Design (LLD)

How to use this section: The LLD is the code-level blueprint for every engineer on the migration squad. Each sub-section covers one type of AngularJS artefact and shows exactly how it maps to Angular 21. Use these as the reference patterns when writing Copilot prompts — the more specific your prompt matches the patterns here, the higher the quality of AI output.

3.1 — Controller → Standalone Component Conversion

A controller is the most common AngularJS artefact. Every controller becomes one Angular 21 standalone component.

Anatomy mapping

AngularJS Controller

```
angular.module('app')
.controller('AccountCtrl',
  function($scope, $http,
    AccountService)
{
  $scope.accounts = [];
  $scope.loading = false;
  $scope.error = null;

  $scope.loadAccounts = () => {
    $scope.loading = true;
    $http.get('/api/accounts')
      .then(res => {
        $scope.accounts =
          res.data;
        $scope.loading =
          false;
      });
  };

  $scope.total =
    $scope.accounts
      .reduce(...);

  $scope.$watch(
    'accounts',
    () => updateChart()
  );
});
```

Angular 21 Standalone Component

```
@Component({ standalone: true })
export class AccountComponent
  – dependencies via inject()

readonly accounts = signal<Account[]>([]);
readonly loading = signal(false);
readonly error = signal<string | null>(null);

loadAccounts(): void {
  this.loading.set(true);
  inject(HttpClient)
    .get<Account[]>('/api/accounts')
    .pipe(takeUntilDestroyed())
    .subscribe({
      next: a => { this.accounts.set(a); },
      error: e => { this.error.set(e.message); },
      complete: () => this.loading.set(false),
    });
}

readonly total = computed(() =>
  this.accounts()
    .reduce(...));

constructor() {
  effect(() => {
    const a = this.accounts();
    updateChart(a);
  });
}
```

Complete conversion example

Before (AngularJS):

```
// accounts.controller.js
angular
.module('bankApp')
.controller('AccountsCtrl', function ($scope, $http, $filter, CurrencyService) {
  $scope.accounts = [];
  $scope.searchTerm = '';
  $scope.sortField = 'name';
  $scope.loading = false;

  $scope.loadAccounts = function () {
    $scope.loading = true;
    $http
      .get('/api/accounts')
      .then(function (res) {
        $scope.accounts = res.data;
      })
      .catch(function (err) {
        $scope.errorMessage = err.statusText;
      })
      .finally(function () {
        $scope.loading = false;
      });
  };

  $scope.filteredAccounts = function () {
    return $filter('filter')($scope.accounts, $scope.searchTerm);
  };

  $scope.formatBalance = function (amount) {
    return CurrencyService.format(amount);
  };

  $scope.loadAccounts();
});
```

After (Angular 21):

```
// accounts.component.ts
import {
  Component,
  ChangeDetectionStrategy,
  inject,
  signal,
  computed,
  OnInit,
} from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { CurrencyService } from '../services/currency.service';
import { Account } from '../interfaces/account.interface';

@Component({
  selector: 'app-accounts',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  imports: [
    /* template imports only – no NgModule */
  ],
  templateUrl: './accounts.component.html',
})
export class AccountsComponent implements OnInit {
  private http = inject(HttpClient);
  private currency = inject(CurrencyService);

  // State – replaces $scope variables
  protected readonly accounts = signal<Account[]>([]);
  protected readonly searchTerm = signal('');
  protected readonly sortField = signal<keyof Account>('name');
  protected readonly loading = signal(false);
  protected readonly errorMsg = signal<string | null>(null);

  // Derived state – replaces $scope functions used in templates
  protected readonly filteredAccounts = computed(() => {
    const term = this.searchTerm().toLowerCase();
    return this.accounts().filter((a) => a.name.toLowerCase().includes(term));
  });

  // Replaces $scope.formatBalance – pure function, no signal needed
  protected formatBalance(amount: number): string {
    return this.currency.format(amount);
  }

  ngOnInit(): void {
    this.loadAccounts();
  }

  private loadAccounts(): void {
    this.loading.set(true);
    this.errorMsg.set(null);
    this.http
      .get<Account[]>('/api/accounts')
      .pipe(takeUntilDestroyed())
      .subscribe({
        next: (accounts) => this.accounts.set(accounts),
        error: (err) => this.errorMsg.set(err.statusText),
        complete: () => this.loading.set(false),
      });
  }
}
```

Template — before (AngularJS):

```

<!-- accounts.html -->
<div ng-controller="AccountsCtrl">
  <input ng-model="searchTerm" placeholder="Search accounts" />
  <div ng-if="loading">Loading...</div>
  <div ng-if="errorMsg" class="error">{{ errorMsg }}</div>
  <div ng-repeat="account in filteredAccounts() | orderBy: sortField">
    <span>{{ account.name }}</span>
    <span>{{ formatBalance(account.balance) }}</span>
  </div>
</div>

```

Template — after (Angular 21):

```

<!-- accounts.component.html -->
<label for="search">Search accounts</label>
<input
  id="search"
  type="search"
  [value]="searchTerm()"
  (input)="searchTerm.set($any($event.target).value)"
  placeholder="Search accounts"
  aria-controls="accounts-list"
/>

@if (loading()) {
<div role="status" aria-live="polite">Loading accounts...</div>
} @if (errorMsg()) {
<div role="alert" class="error">{{ errorMsg() }}</div>
}

<ul id="accounts-list" aria-label="Account list">
  @for (account of filteredAccounts(); track account.id) {
    <li>
      <span>{{ account.name }}</span>
      <span>{{ formatBalance(account.balance) }}</span>
    </li>
  }
</ul>

```

3.2 — Service / Factory → @Injectable Service Conversion

AngularJS Factory/Service

```
angular.module('app')
.factory('AccountService', function(
  $http, $q, $rootScope) {

  var service = {};

  service.getAll = function() {
    return $http.get('/api/accounts')
      .then(res => res.data);
  };

  service.getById = function(id) {
    var deferred = $q.defer();
    $http.get('/api/accounts/' + id)
      .then(res => deferred.resolve(
        res.data))
      .catch(err => deferred.reject(err));
    return deferred.promise;
  };

  service.notify = function() {
    $rootScope.$broadcast(
      'accounts:updated');
  };

  return service;
});
```

Angular 21 Injectable Service

```
@Injectable({ providedIn: 'root' })
export class AccountService {
  private http = inject(HttpClient);

  // No $q — use RxJS Observable or async/await

  getAll(): Observable<Account[]> {
    return this.http.get<Account[]>(
      '/api/accounts'
    );
  }

  getById(id: string): Observable<Account> {
    return this.http.get<Account>({
      url: `/api/accounts/${id}`
    });
  }

  // Signal-based caching (replaces $rootScope cache)
  private readonly _cache = signal
    <Map<string,Account>>(new Map());
  private update(id: string, a: Account) {
    this._cache.update(m => new Map(m).set(id, a));
  }

  // No return needed — class IS the service
}
```

HTTP interceptor migration

Before (AngularJS \$http interceptor):

```
angular.module('app').factory('authInterceptor', function ($q, TokenService) {
  return {
    request: function (config) {
      config.headers.Authorization = 'Bearer ' + TokenService.getToken();
      return config;
    },
    responseError: function (rejection) {
      if (rejection.status === 401) {
        TokenService.clearToken();
        window.location.href = '/login';
      }
      return $q.reject(rejection);
    },
  };
});
```

After (Angular 21 functional interceptor):

```
// auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const token = auth.accessToken();

  const authReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;

  return next(authReq).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) {
        auth.clearToken();
        router.navigate(['/login']);
      }
      return throwError(() => err);
    })
  );
};

// app.config.ts - register interceptor
provideHttpClient(withInterceptors([authInterceptor, xsrfInterceptor]));
```

toSignal() Patterns for Observable → Signal Conversion

```
// Component using HTTP observables
@Component({ /* ... */ })
export class ExampleComponent {
  private readonly http = inject(HttpClient);
  private readonly authService = inject(AuthService);

  // Pattern A: HTTP request with initialValue (optional data)
  protected readonly users = toSignal(
    this.http.get<User[]>('/api/users'),
    { initialValue: [] } // ✅ Safe default for lists
  );

  // Pattern B: requireSync for required data (throws if not sync)
  protected readonly currentUser = toSignal(
    this.authService.currentUser$, // BehaviorSubject that emits immediately
    { requireSync: true } // ⚠️ Throws error if observable doesn't emit synchronously
  );
  // ✅ Use in auth guards where user MUST be present on init

  // Pattern C: Optional data (nullable)
  protected readonly selectedAccount = toSignal(
    this.route.paramMap.pipe(
      switchMap(params => this.http.get<Account>(`/api/accounts/${params.get('id')}`))
    ),
    { initialValue: null } // Starts as null until HTTP completes
  );
}

// ⚠️ Common toSignal() Mistake:
// ❌ WRONG: No initialValue and observable may not emit immediately
protected readonly data = toSignal(this.http.get<Data>('/api/data'));
// Template access: {{ data()?.name }} ← Must handle undefined!

// ✅ CORRECT: Always provide initialValue OR use requireSync
protected readonly data = toSignal(
  this.http.get<Data>('/api/data'),
  { initialValue: null } // Explicit null handling
);
```

When to use `requireSync`:

Use Case	Pattern	Example
Auth guard (must have user)	<code>requireSync: true</code>	<code>toSignal(authService.currentUser\$, { requireSync: true })</code>
Route data (always sync)	<code>requireSync: true</code>	<code>toSignal(route.data, { requireSync: true })</code>
HTTP request (async)	<code>initialValue</code>	<code>toSignal(http.get(...), { initialValue: [] })</code>
BehaviorSubject (sync)	<code>requireSync: true</code> OR <code>initialValue</code>	Either works 

3.3 — Directive → Component / Directive / Pipe Conversion

AngularJS directives split into three Angular 21 artefact types depending on what they do:

AngularJS Directive Purpose	Angular 21 Equivalent
Replaces/wraps DOM with template	→ Standalone Component
Adds behaviour to existing element	→ Angular Directive (attribute)
Transforms a value for display	→ Angular Pipe
Structural (add/remove DOM nodes)	→ <code>@if</code> / <code>@for</code> / <code>@switch</code> (built-in)
Validates a form field	→ Validator function (ValidatorFn)

Structural directive → built-in control flow

```

<!-- AngularJS -->
<div ng-if="isVisible">Content</div>
<li ng-repeat="item in items track by item.id">{{ item.name }}</li>
<div ng-switch="status">
  <span ng-switch-when="active">Active</span>
  <span ng-switch-default>Inactive</span>
</div>

<!-- Angular 21 -->
@if (isVisible()) {
  <div>Content</div>
} @for (item of items(); track item.id) {
  <li>{{ item.name }}</li>
} @switch (status()) { @case ('active') { <span>Active</span> } @default { <span>Inactive</span> } }
```

Attribute directive migration

Before (AngularJS attribute directive):

```

angular.module('app').directive('highlightOnHover', function () {
  return {
    restrict: 'A',
    link: function (scope, element, attrs) {
      element.on('mouseenter', function () {
        element.addClass(attrs.highlightOnHover || 'highlight');
      });
      element.on('mouseleave', function () {
        element.removeClass(attrs.highlightOnHover || 'highlight');
      });
    },
  };
});
```

After (Angular 21 directive):


```
// highlight-on-hover.directive.ts
@Directive({
  selector: '[appHighlightOnHover]',
  standalone: true,
  host: {
    '(mouseenter)': 'onEnter()',
    '(mouseleave)': 'onLeave()',
  },
})
export class HighlightOnHoverDirective {
  readonly appHighlightOnHover = input('highlight');
  private el = inject(ElementRef<HTMLElement>);

  protected onEnter(): void {
    this.el.nativeElement.classList.add(this.appHighlightOnHover());
  }

  protected onLeave(): void {
    this.el.nativeElement.classList.remove(this.appHighlightOnHover());
  }
}
```

Filter → Pipe migration

Before (AngularJS filter):

```
angular.module('app').filter('currencyFormat', function () {
  return function (amount, currencyCode) {
    if (amount == null) return '-';
    return new Intl.NumberFormat('en-GB', {
      style: 'currency',
      currency: currencyCode || 'GBP',
    }).format(amount);
  };
});
// Usage: {{ account.balance | currencyFormat:'USD' }}
```

After (Angular 21 pipe):

```
// currency-format.pipe.ts
@Pipe({ name: 'currencyFormat', standalone: true, pure: true })
export class CurrencyFormatPipe implements PipeTransform {
  transform(amount: number | null, currencyCode = 'GBP'): string {
    if (amount == null) return '-';
    return new Intl.NumberFormat('en-GB', {
      style: 'currency',
      currency: currencyCode,
    }).format(amount);
  }
}
// Usage: {{ account.balance | currencyFormat:'USD' }}
// In component imports: imports: [CurrencyFormatPipe]
```

3.4 — Route Guard Migration

```
// app.routes.ts — full routing LLD with lazy loading, guards, and data
export const routes: Routes = [
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },

  {
    path: 'login',
    loadComponent: () => import('./auth/login/login.component').then((m) => m.LoginComponent),
  },

  {
    path: 'dashboard',
    canActivate: [authGuard],
    loadComponent: () =>
      import('./dashboard/dashboard.component').then((m) => m.DashboardComponent),
  },

  {
    path: 'accounts',
    canActivate: [authGuard],
    loadChildren: () => import('./accounts/accounts.routes').then((m) => m.ACCOUNTS_ROUTES),
  },

  {
    path: 'admin',
    canActivate: [authGuard, roleGuard(['ADMIN'])],
    canMatch: [roleGuard(['ADMIN'])], // Prevents lazy chunk from loading for non-admins
    loadChildren: () => import('./admin/admin.routes').then((m) => m.ADMIN_ROUTES),
  },

  {
    path: '**',
    loadComponent: () => import('./not-found/not-found.component').then((m) => m.NotFoundComponent),
  },
];

// guards/auth.guard.ts
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isAuthenticated()) return true;
  return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
};
```

3.5 — Nx Shared Library LLD

Every shared concept lives in a typed Nx library. Components and services never import directly from another app — only from `libs/`.

```

libs/
├── shared-models/
│   └── src/lib/
│       ├── account.interface.ts    export interface Account { id, name, balance, ... }
│       ├── transaction.interface.ts export interface Transaction { id, amount, date, ... }
│       ├── user.interface.ts       export interface User { id, roles, email, ... }
│       └── index.ts                export * from './account.interface'; ...
├── shared-auth/
│   └── src/lib/
│       ├── auth.service.ts        Signal-based; token in memory
│       ├── auth.interceptor.ts     HttpInterceptorFn – attaches Bearer token
│       ├── xsrf.interceptor.ts     withXsrfConfiguration setup
│       ├── auth.guard.ts          CanActivateFn – isAuthenticated check
│       ├── role.guard.ts          CanActivateFn factory – hasAnyRole check
│       └── index.ts                Public API – only export what consumers need
├── shared-ui/
│   └── src/lib/
│       ├── button/
│       │   ├── button.component.ts Standalone; variants: primary/secondary/danger
│       │   └── button.component.html
│       ├── input/
│       │   ├── text-input.component.ts Standalone; signal-bound; aria-complete
│       │   └── text-input.component.html
│       ├── modal/
│       │   └── modal.component.ts    CDK Dialog-based; trap focus
│       ├── data-table/
│       │   └── data-table.component.ts Signal-driven; sort/filter/paginate
│       ├── tokens/
│       │   └── design-tokens.css     CSS custom properties; consumed by all apps
│       └── index.ts
├── shared-utils/
│   └── src/lib/
│       ├── currency.pipe.ts        Pure pipe – format currency
│       ├── date.pipe.ts            Pure pipe – format dates
│       ├── validators.ts           ValidatorFn collection for forms
│       ├── error-handler.ts        Global error handler service
│       └── index.ts
└── feature-flags/
    └── src/lib/
        ├── feature-flag.service.ts Signal-based; reads LaunchDarkly / env config
        ├── feature-flag.directive.ts *[featureFlag]=''flag-name'' structural directive
        └── index.ts

```

Library import rule — enforced by Nx module boundary constraints:

```

// .eslintrc.json – Nx boundary rules (no circular deps; apps can't import each other)
{
  "rules": {
    "@nx/enforce-module-boundaries": [
      "error",
      {
        "allow": [],
        "depConstraints": [
          {
            "sourceTag": "scope:app",
            "onlyDependOnLibsWithTags": ["scope:lib"]
          },
          {
            "sourceTag": "scope:lib",
            "onlyDependOnLibsWithTags": ["scope:lib"]
          }
        ]
      }
    ]
  }
}

```

3.6 — Signal Patterns LLD — \$scope → Signals Mapping

Every `$scope` pattern has a direct, idiomatic Angular 21 signal equivalent:

AngularJS <code>\$scope</code> Pattern	Angular 21 Signal Pattern	Notes
<code>\$scope.x = val</code>	<code>x = signal(val)</code>	Initial value in <code>signal()</code>
<code>\$scope.x = newVal</code> (assignment)	<code>x.set(newVal)</code>	Replace entire value
<code>\$scope.x++</code> (mutation)	<code>x.update(v => v + 1)</code>	Use <code>update()</code> for derived mutations
<code>\$scope.derived = fn(\$scope.a, \$scope.b)</code>	<code>derived = computed(() => fn(a(), b()))</code>	Lazy; only recomputes when deps change
<code>\$scope.\$watch('x', fn)</code> (side effect)	<code>effect(() => fn(x()))</code>	Tracks dependencies automatically
<code>\$scope.\$watch('x', fn, true)</code> (deep watch)	<code>effect(() => JSON.stringify(x())) + side effect</code>	Use sparingly; prefer fine-grained signals
<code>\$scope.\$on('event', handler)</code> (event listen)	<code>inject(EventBusService).on('event').subscribe(handler)</code>	Signal or Subject-based event bus
<code>\$rootScope.\$broadcast('event', data)</code>	<code>inject(EventBusService).emit('event', data)</code>	Typed events; no global scope
<code>\$scope.form.\$valid</code>	<code>isValid = computed(() => Object.values(errors()).every(e => !e))</code>	Signal form validation
<code>ng-model="user.name"</code>	<code>[(ngModel)]="name"</code> (Angular 21 two-way) OR <code>model = model({name: ''})</code> with model signals	Template-driven or model signals
<code>ng-model</code> with <code>watch</code>	<code>linkedSignal(() => source())</code> — creates signal linked to another signal source	Reactive linked signals (Angular 21.1+)
<code>\$scope.form.\$dirty</code>	<code>isDirty = computed(() => originalValue() !== currentValue())</code>	Track dirty state explicitly
<code>\$scope.\$apply(() => ...)</code>	Not needed — signals are zone-agnostic	Remove all <code>\$apply</code> calls
<code>\$scope.\$digest()</code>	Not needed	Remove all <code>\$digest</code> calls

Angular 21 Signal Form Patterns:

```
// Pattern 1: Simple signal-based form (<10 fields)
protected readonly email = signal('');
protected readonly password = signal('');
protected readonly emailError = computed(() =>
  !this.email().includes('@') ? 'Invalid email' : null
);

// Pattern 2: Model signals (two-way binding, Angular 21.1+)
export class ProfileFormComponent {
  protected readonly userModel = model({
    name: '',
    email: '',
    age: 0
  });

  // Template: <input [(ngModel)]="userModel().name" />
  // Auto-syncs without explicit (input) handlers
}

// Pattern 3: Linked signals (dependent fields)
export class PasswordFormComponent {
  protected readonly password = signal('');
  protected readonly confirmPassword = linkedSignal(() => this.password());

  protected readonly passwordsMatch = computed(() =>
    this.password() === this.confirmPassword()
  );
}
```

3.6.1 — Advanced Signal Form Patterns (model & linkedSignal)

Model Signals for Complex Forms (Angular 21.1+)

Use `model()` for two-way binding with signal reactivity — best for forms with 10+ fields.

```
@Component({
  selector: 'app-payment-form',
  standalone: true,
  imports: [FormsModule],
  template: `
    <form (ngSubmit)="onSubmit()">
      <input [(ngModel)]="payment().fromAccount" name="from" />
      <input [(ngModel)]="payment().amount" type="number" />

      @if (errors().amount) {
        <span role="alert">{{ errors().amount }}</span>
      }

      <button [disabled]="!isValid()">Submit</button>
    </form>
  `,
})
export class PaymentFormComponent {
  protected readonly payment = model({ fromAccount: '', amount: 0 });

  protected readonly errors = computed(() => ({
    fromAccount: !this.payment().fromAccount ? 'Required' : null,
    amount: this.payment().amount <= 0 ? 'Must be positive' : null,
  }));

  protected readonly isValid = computed(() =>
    Object.values(this.errors()).every((e) => e === null),
  );
}
```

Linked Signals for Dependent Fields

Use `linkedSignal()` when one field depends on another (password confirm, cascading dropdowns).

```

@Component({
  selector: 'app-address-form',
  template: `
    <select [value]="country()" (change)="country.set($any($event.target).value)">
      @for (c of countries()); track c {
        <option [value]="c">{{ c }}</option>
      }
    </select>

    <select [value]="state()" (change)="state.set($any($event.target).value)">
      @for (s of availableStates()); track s {
        <option [value]="s">{{ s }}</option>
      }
    </select>
  `,
})
export class AddressFormComponent {
  protected readonly country = signal('');

  // ✅ CORRECT: linkedSignal with source + computation
  protected readonly state = linkedSignal({
    source: this.country,
    computation: () => '', // Resets to empty when country changes
  });

  // Alternative syntax (function form):
  // protected readonly state = linkedSignal(() => {
  //   this.country(); // Track dependency
  //   return ''; // Reset state
  // });

  protected readonly availableStates = computed(() => {
    const c = this.country();
    return c === 'US' ? ['CA', 'NY', 'TX'] : [];
  });
}

```

⚠️ Common linkedSignal() Mistakes:

```

// ❌ WRONG – Infinite loop (reading linkedSignal inside its own computation)
protected readonly value = linkedSignal(() => {
  if (this.value() === 'foo') return 'bar'; // Reading value() causes infinite loop!
  return this.value();
});

// ✅ CORRECT – linkedSignal depends on external signal
protected readonly sourceValue = signal('initial');
protected readonly derivedValue = linkedSignal({
  source: this.sourceValue,
  computation: (prev) => {
    // prev is the previous value of derivedValue (safe to read)
    return this.sourceValue() === 'foo' ? 'bar' : prev;
  },
});

```

⚠️ OnPush + effect() Gotcha:

`effect()` side effects run correctly with `OnPush`, **but** if the effect mutates a non-signal property, the template won't update:

```
// ❌ WRONG with OnPush
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class BadExampleComponent {
  items: Item[] = []; // ❌ NOT a signal

  constructor() {
    effect(() => {
      const data = this.apiData(); // Signal read
      this.items = data; // ❌ Mutates non-signal - template won't update!
    });
  }
}

// ✅ CORRECT - Use signals everywhere with OnPush
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class GoodExampleComponent {
  protected readonly items = signal<Item[]>([]);

  constructor() {
    effect(() => {
      this.items.set(this.apiData()); // ✅ Signal mutation - template updates
    });
  }
}
```

3.7 — Security LLD — Implementing the Security Architecture

 **This section covers implementation code** — the Angular 21 patterns that implement the security architecture. Security spans three sections:

- [Part 1, Q21–Q27](#): Strategy — OWASP risk questions, XSS/CSRF decision framework, JWT/OAuth2 approach.
- **Here (Section 3.7)**: Implementation — auth service, HTTP interceptors, route guards in Angular 21 code.
- [Section 6.2 — HTTP Token & Cookie Strategy](#): ADR — storage options comparison, risk profiles, BFF recommendation.

This translates the security decisions from Part 1 Section 6 into concrete Angular 21 code patterns.

Auth service — memory token storage (most secure SPA pattern)

```
// libs/shared-auth/src/lib/auth.service.ts
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly _token = signal<string | null>(null);
  private readonly _roles = signal<string[]>([]);
  private readonly _expiresAt = signal<number | null>(null);

  // Public read-only signals
  readonly isAuthenticated = computed(
    () => this._token() !== null && (this._expiresAt() ?? 0) > Date.now(),
  );
  readonly roles = this._roles.asReadonly();

  setSession(token: string, roles: string[], expiresIn: number): void {
    this._token.set(token);
    this._roles.set(roles);
    this._expiresAt.set(Date.now() + expiresIn * 1000);
    // Never write to localStorage – token lives only in memory
  }

  getToken(): string | null {
    if ((this._expiresAt() ?? 0) < Date.now()) {
      this.clearSession(); // Expired – clear on read
      return null;
    }
    return this._token();
  }

  clearSession(): void {
    this._token.set(null);
    this._roles.set([]);
    this._expiresAt.set(null);
  }

  hasRole(role: string): boolean {
    return this._roles().includes(role);
  }
}
```

HTTP error handler — centralised 401/403 handling

```
// libs/shared-auth/src/lib/http-error.interceptor.ts
export const httpErrorInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const toast = inject(ToastService);

  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      switch (err.status) {
        case 401:
          auth.clearSession();
          router.navigate(['/login'], { queryParams: { reason: 'session-expired' } });
          break;
        case 403:
          toast.error('You do not have permission to perform this action.');
          router.navigate(['/unauthorised']);
          break;
        case 0:
          toast.error('Network error – please check your connection.');
          break;
      }
      // Never expose raw error details to the user – log to telemetry instead
      return throwError(() => new Error(`HTTP ${err.status}`));
    }),
  );
};
```


Route-level security — complete guard chain

```
// libs/shared-auth/src/lib/role.guard.ts
export const roleGuard = (allowedRoles: string[]): CanActivateFn & CanMatchFn => {
  return () => {
    const auth = inject(AuthService);
    const router = inject(Router);
    if (!auth.isAuthenticated()) {
      return router.createUrlTree(['/login']);
    }
    if (auth.hasAnyRole(allowedRoles)) return true;
    return router.createUrlTree(['/unauthorised']);
  };
};

// Usage — defence in depth: CanMatch prevents chunk download for unauthorised users
export const adminRoutes: Routes = [
  {
    path: 'admin',
    canActivate: [roleGuard(['ADMIN'])], // Prevents render
    canMatch: [roleGuard(['ADMIN'])], // Prevents lazy chunk download
    loadChildren: () => import('./admin.routes').then((m) => m.ADMIN_ROUTES),
  },
];
```

3.8 — Global State Migration: \$rootScope → Signal Event Bus

`$rootScope.$broadcast` is the most architecturally problematic AngularJS pattern. It creates implicit coupling between controllers. Every broadcast must be replaced with a typed event bus.

```
// libs/shared-utils/src/lib/event-bus.service.ts
export type AppEvent =
  | { type: 'AUTH_LOGOUT' }
  | { type: 'ACCOUNT_UPDATED'; payload: Account }
  | { type: 'NOTIFICATION'; payload: { message: string; level: 'info' | 'warn' | 'error' } };

@Injectable({ providedIn: 'root' })
export class EventBusService {
  private readonly subject = new Subject<AppEvent>();

  emit(event: AppEvent): void {
    this.subject.next(event);
  }

  on<T extends AppEvent['type']>(
    type: T
  ): Observable<Extract<AppEvent, { type: T }>> {
    return this.subject.pipe(
      filter((e): e is Extract<AppEvent, { type: T }> => e.type === type)
    );
  }
}

// Consumer — replaces $scope.$on('ACCOUNT_UPDATED', handler)
@Component({ standalone: true, ... })
export class AccountSummaryComponent {
  constructor() {
    inject(EventBusService)
      .on('ACCOUNT_UPDATED')
      .pipe(takeUntilDestroyed())
      .subscribe(({ payload }) => this.account.set(payload));
  }
}
```

3.9 — Internationalization (i18n) Migration

Challenge: 100-screen banking apps often support multiple languages. AngularJS typically uses `angular-translate` or `angular-gettext`. Angular 21 uses `@angular/localize`.

AngularJS Pattern (`angular-translate`)

```
// app.js
angular.module('app', ['pascalprecht.translate']).config([
  '$translateProvider',
  function ($translateProvider) {
    $translateProvider.translations('en', {
      HELLO: 'Hello {{name}}!',
      WELCOME_MESSAGE: 'Welcome to Banking App',
    });
    $translateProvider.translations('es', {
      HELLO: '¡Hola {{name}}!',
      WELCOME_MESSAGE: 'Bienvenido a la aplicación bancaria',
    });
    $translateProvider.preferredLanguage('en');
  },
]);

// controller.js
controller('GreetingCtrl', [
  '$scope',
  '$translate',
  function ($scope, $translate) {
    $scope.greeting = $translate.instant('HELLO', { name: $scope.userName });
  },
]);
```

```
<!-- template.html -->
<h1>{{ 'HELLO' | translate:{ name: userName } }}</h1>
<p>{{ 'WELCOME_MESSAGE' | translate }}</p>
```

Angular 21 Pattern (`@angular/localize`)

Step 1: Install `@angular/localize`

```
ng add @angular/localize
```

Step 2: Mark translatable strings with `i18n` or `$localize`

```
// greeting.component.ts
import { Component, computed, signal } from '@angular/core';
import { $localize } from '@angular/localize/init';

@Component({
  selector: 'app-greeting',
  standalone: true,
  template: `
    <h1 i18n="@hello">Hello {{ userName() }}!</h1>
    <p i18n="@welcomeMessage">Welcome to Banking App</p>

    <!-- Dynamic content with $localize -->
    <p>{{ greeting() }}</p>
  `,
})
export class GreetingComponent {
  protected readonly userName = signal('John');

  // Use $localize for computed strings
  protected readonly greeting = computed(
    () => $localize`:@hello:Hello ${this.userName():userName}!`,
  );
}
```

Step 3: Extract translation keys

```
# Extract all i18n strings to XLIFF file
ng extract-i18n --output-path src/locale

# Generates: src/locale/messages.xlf
```

messages.xlf example:

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliff version="2.0" xmlns="urn:oasis:names:tc:xliff:document:2.0">
  <file source-language="en" target-language="en">
    <unit id="hello">
      <segment>
        <source>Hello <ph id="0"/>!</source>
      </segment>
    </unit>
    <unit id="welcomeMessage">
      <segment>
        <source>Welcome to Banking App</source>
      </segment>
    </unit>
  </file>
</xliff>
```

Step 4: Translate XLIFF files

Create `messages.es.xlf`, `messages.fr.xlf`, etc. with translated `<target>` tags:

```
<unit id="hello">
  <segment>
    <source>Hello <ph id="0"/>!</source>
    <target>¡Hola <ph id="0"/>!</target>
  </segment>
</unit>
```

Step 5: Build separate bundles per locale

```
// angular.json
{
  "projects": {
    "banking-ng21": {
      "i18n": {
        "sourceLocale": "en",
        "locales": {
          "es": "src/locale/messages.es.xlf",
          "fr": "src/locale/messages.fr.xlf"
        }
      },
      "architect": {
        "build": {
          "options": {
            "localize": true
          },
          "configurations": {
            "production": {
              "localize": ["en", "es", "fr"]
            }
          }
        }
      }
    }
  }
}
```

```
# Build all locales
ng build --configuration=production

# Output:
# dist/banking-ng21/en/ ← English build
# dist/banking-ng21/es/ ← Spanish build
# dist/banking-ng21/fr/ ← French build
```

Step 6: Deploy with locale-specific URLs

```
https://bank.com/en/dashboard ← English
https://bank.com/es/dashboard ← Spanish
https://bank.com/fr/dashboard ← French
```

nginx configuration:

```
server {
  listen 443 ssl http2;
  server_name bank.com;

  # Locale-based routing
  location /en/ {
    alias /var/www/banking-ng21/en;
    try_files $uri $uri/ /en/index.html;
  }

  location /es/ {
    alias /var/www/banking-ng21/es;
    try_files $uri $uri/ /es/index.html;
  }

  location /fr/ {
    alias /var/www/banking-ng21/fr;
    try_files $uri $uri/ /fr/index.html;
  }

  # Default to English
  location = / {
    return 301 /en/dashboard;
  }
}
```

Migration Steps Summary

Step	Action	Tool
1	Audit all <code>\$translate / gettext</code> keys in AngularJS	<code>grep -r "'[A-Z_]+' translate" src/</code>
2	Install <code>@angular/localize</code> in Angular 21	<code>ng add @angular/localize</code>
3	Mark all translatable strings with <code>i18n</code> or <code>\$localize</code>	Manual + AI assistance
4	Extract keys to XLIFF	<code>ng extract-i18n</code>
5	Translate XLIFF files	Send to translation service
6	Build per-locale bundles	<code>ng build --localize</code>
7	Deploy with locale routing	nginx or Azure Front Door

Runtime Locale Switching (Optional)

Angular's default `i18n` requires separate builds per locale (compile-time). For **runtime locale switching**, use:

- **transloco** (community library, ~10KB): <https://ngneat.github.io/transloco/>
- **ngx-translate** (legacy, not recommended for Angular 21)

Transloco example:

```
import { provideTransloco } from '@ngneat/transloco';

export const appConfig: ApplicationConfig = {
  providers: [
    provideTransloco({
      config: {
        availableLangs: ['en', 'es', 'fr'],
        defaultLang: 'en',
        reRenderOnLangChange: true,
      },
      loader: TranslocoHttpLoader, // Loads JSON files on demand
    }),
  ],
};
```

When to use runtime vs compile-time:

Use Case	Compile-time (@angular/localize)	Runtime (transloco)
Banking app with fixed locale per user	✅ Recommended	❌ Overkill
Multi-tenant SaaS with user language toggle	⚠️ Possible (reload page)	✅ Recommended
Bundle size priority	✅ Smaller (tree-shaken)	⚠️ Larger (+10KB runtime)
SEO per locale	✅ Separate URLs	❌ Single build

3.10 — RxJS Migration Strategy: When to Use RxJS vs Signals

Critical Decision: Angular 21 has both signals and RxJS. When should you use each?

The Mental Model

USE SIGNALS FOR:

- ✓ Synchronous state (component properties, form values)
- ✓ Derived state (computed values)
- ✓ Simple reactivity (one value changes → template updates)
- ✓ Local component state

USE RXJS FOR:

- ✓ HTTP requests (API calls)
- ✓ Complex async orchestration (debounce, retry, race conditions)
- ✓ Event streams (WebSocket, SSE, DOM events)
- ✓ Time-based operations (intervals, delays, timeouts)
- ✓ Multi-step transformations (map → filter → switchMap → catchError)

HYBRID PATTERNS (RxJS → Signal or Signal → RxJS):

- 🔗 toSignal(): Convert Observable to Signal (for templates)
- 🔗 toObservable(): Convert Signal to Observable (for RxJS operators)

Pattern 1: API Response Transformation with RxJS Operators

AngularJS Pattern:

```
// account.service.js
angular.module('app').factory('AccountService', function ($http, $q) {
  return {
    getAccounts: function () {
      return $http
        .get('/api/accounts')
        .then(function (response) {
          // Transform response
          return response.data.map(function (account) {
            return {
              id: account.accountId,
              displayName: account.firstName + ' ' + account.lastName,
              balance: parseFloat(account.balance),
              isActive: account.status === 'ACTIVE',
            };
          });
        })
        .catch(function (error) {
          console.error('Failed to load accounts', error);
          return []; // Fallback
        });
    },
  };
});
```

Angular 21 Pattern (RxJS Operators):

```

// account.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, of } from 'rxjs';
import { map, catchError, retry, shareReplay } from 'rxjs/operators';

interface AccountDTO {
  accountId: string;
  firstName: string;
  lastName: string;
  balance: string;
  status: string;
}

export interface Account {
  id: string;
  displayName: string;
  balance: number;
  isActive: boolean;
}

@Injectable({ providedIn: 'root' })
export class AccountService {
  private readonly http = inject(HttpClient);

  getAccounts(): Observable<Account[]> {
    return this.http.get<AccountDTO[]>('/api/accounts').pipe(
      // 1. Retry failed requests (transient network errors)
      retry({ count: 2, delay: 1000 }),

      // 2. Transform DTO → domain model
      map((dtos) =>
        dtos.map((dto) => ({
          id: dto.accountId,
          displayName: `${dto.firstName} ${dto.lastName}`,
          balance: parseFloat(dto.balance),
          isActive: dto.status === 'ACTIVE',
        })),
      ),

      // 3. Error handling with fallback
      catchError((error) => {
        console.error('Failed to load accounts', error);
        return of([]); // Return empty array instead of throwing
      }),

      // 4. Cache result (shareReplay prevents multiple HTTP calls)
      shareReplay({ bufferSize: 1, refCount: true }),
    );
  }
}

```

Component Usage (Convert Observable → Signal):

```

@Component({
  selector: 'app-account-list',
  template: `
    @if (loading()) {
      <app-spinner />
    } @else if (error()) {
      <app-error-message [error]="error()" />
    } @else {
      @for (account of accounts(); track account.id) {
        <app-account-card [account]="account" />
      }
    }
  `,
})
export class AccountListComponent {
  private readonly accountService = inject(AccountService);

  // ✅ Convert Observable → Signal for template consumption
  protected readonly accounts = toSignal(this.accountService.getAccounts(), { initialValue: [] });

  protected readonly loading = signal(false);
  protected readonly error = signal<string | null>(null);
}

```

Pattern 2: Search with Debounce (RxJS Required)

AngularJS Pattern:

```

// search.controller.js
controller('SearchCtrl', function ($scope, $timeout, AccountService) {
  var searchTimeout;

  $scope.searchQuery = '';
  $scope.results = [];

  $scope.onSearchChange = function () {
    if (searchTimeout) {
      $timeout.cancel(searchTimeout);
    }

    searchTimeout = $timeout(function () {
      if ($scope.searchQuery.length >= 3) {
        AccountService.search($scope.searchQuery).then(function (results) {
          $scope.results = results;
        });
      }
    }, 300); // Debounce 300ms
  };
});

```

Angular 21 Pattern (RxJS + Signal Hybrid):


```

import { Component, signal, inject } from '@angular/core';
import { toSignal, toObservable } from '@angular/core/rxjs-interop';
import { debounceTime, distinctUntilChanged, switchMap, filter } from 'rxjs/operators';

@Component({
  selector: 'app-search',
  template: `
    <input
      [value]="searchQuery()"
      (input)="searchQuery.set($any($event.target).value)"
      placeholder="Search accounts..."
    />

    @if (isSearching()) {
      <app-spinner />
    }

    @for (result of searchResults(); track result.id) {
      <app-account-card [account]="result" />
    }
  `,
})
export class SearchComponent {
  private readonly accountService = inject(AccountService);

  // 1. Signal for user input (reactive state)
  protected readonly searchQuery = signal('');

  // 2. Convert Signal → Observable to use RxJS operators
  private readonly searchQuery$ = toObservable(this.searchQuery);

  // 3. Apply RxJS operators (debounce, distinctUntilChanged, switchMap)
  private readonly searchResults$ = this.searchQuery$.pipe(
    debounceTime(300), // Wait 300ms after user stops typing
    distinctUntilChanged(), // Only emit if value changed
    filter((query) => query.length >= 3), // Only search if >= 3 chars
    switchMap(
      (query) => this.accountService.search(query), // Cancel previous search if new one starts
    ),
  );

  // 4. Convert Observable → Signal for template
  protected readonly searchResults = toSignal(this.searchResults$, {
    initialValue: [],
  });

  protected readonly isSearching = computed(
    () => this.searchQuery().length >= 3 && this.searchResults().length === 0,
  );
}

```

Pattern 3: Multiple API Calls (Sequential vs Parallel)

Scenario: Load user, then load their accounts, then load transactions for first account.

AngularJS Pattern:

```

// Sequential (nested promises)
UserService.getById(userId)
  .then(function (user) {
    return AccountService.getByUserId(user.id);
  })
  .then(function (accounts) {
    return TransactionService.getByAccountId(accounts[0].id);
  })
  .then(function (transactions) {
    $scope.transactions = transactions;
  });

```

Angular 21 Pattern (RxJS Sequential):

```
// Sequential: each call waits for previous
this.userService.getById(userId).pipe(
  switchMap(user => this.accountService.getByUserId(user.id)),
  switchMap(accounts => this.transactionService.getByAccountId(accounts[0].id)),
  catchError(error => {
    console.error('Failed to load transaction data', error);
    return of([]);
  })
).subscribe(transactions => {
  this.transactions.set(transactions); // Update signal
});

// ✅ BETTER: Convert to signal
protected readonly transactions = toSignal(
  this.userService.getById(userId).pipe(
    switchMap(user => this.accountService.getByUserId(user.id)),
    switchMap(accounts => this.transactionService.getByAccountId(accounts[0].id)),
    catchError(() => of([]))
  ),
  { initialValue: [] }
);
```

Angular 21 Pattern (RxJS Parallel with forkJoin):

```
import { forkJoin } from 'rxjs';

// Parallel: all calls start simultaneously
protected readonly dashboardData = toSignal(
  forkJoin({
    user: this.userService.getById(userId),
    accounts: this.accountService.getAll(),
    recentTransactions: this.transactionService.getRecent(10)
  }).pipe(
    catchError(error => {
      console.error('Failed to load dashboard', error);
      return of({ user: null, accounts: [], recentTransactions: [] });
    })
  ),
  { initialValue: null }
);

// Template usage:
// @if (dashboardData()); as data {
//   <h1>{{ data.user.name }}</h1>
//   <app-account-list [accounts]="data.accounts" />
//   <app-transaction-list [transactions]="data.recentTransactions" />
// }
```

Pattern 4: WebSocket / Real-Time Data (RxJS Required)

AngularJS Pattern:

```
// websocket.service.js
factory('WebSocketService', function ($rootScope) {
  var ws = new WebSocket('wss://api.bank.com/notifications');

  ws.onmessage = function (event) {
    $rootScope.$broadcast('notification', JSON.parse(event.data));
  };

  return {
    send: function (message) {
      ws.send(JSON.stringify(message));
    },
  };
});
```

Angular 21 Pattern (RxJS WebSocket):

```

import { Injectable } from '@angular/core';
import { WebSocket, WebSocketSubject } from 'rxjs/webSocket';
import { Observable } from 'rxjs';
import { retry, catchError } from 'rxjs/operators';

export interface Notification {
  type: 'PAYMENT' | 'ALERT' | 'MESSAGE';
  message: string;
  timestamp: number;
}

@Injectable({ providedIn: 'root' })
export class WebSocketService {
  private ws$: WebSocketSubject<Notification>;

  constructor() {
    this.ws$ = WebSocketSubject<Notification>({
      url: 'wss://api.bank.com/notifications',
      deserializer: (e) => JSON.parse(e.data),
      serializer: (value) => JSON.stringify(value),
    });
  }

  getNotifications(): Observable<Notification> {
    return this.ws$.pipe(
      retry({ count: 5, delay: 3000 }), // Reconnect on disconnect
      catchError((error) => {
        console.error('WebSocket error', error);
        throw error;
      })
    );
  }

  send(message: Notification): void {
    this.ws$.next(message);
  }
}

// Component
@Component({
  selector: 'app-notifications',
  template: `
    @for (notification of notifications(); track notification.timestamp) {
      <div class="notification">{{ notification.message }}</div>
    }
  `,
})
export class NotificationsComponent {
  private readonly wsService = inject(WebSocketService);

  // ✅ Convert WebSocket Observable → Signal
  protected readonly notifications = toSignal(
    this.wsService.getNotifications().pipe(
      scan((acc, notification) => [...acc, notification], [] as Notification[]),
      // Keep last 10 notifications
      map((notifications) => notifications.slice(-10)),
    ),
    { initialValue: [] },
  );
}

```

Pattern 5: Form Value Changes with Validation

AngularJS Pattern:

```

$scope.$watch('email', function (newVal) {
  if (newVal && newVal.includes('@')) {
    $scope.emailValid = true;
  } else {
    $scope.emailValid = false;
  }
});

```

Angular 21 Pattern (Signal + Computed — No RxJS Needed):

```
@Component({
  selector: 'app-form',
  template: `
    <input type="email" [value]="email()" (input)="email.set($any($event.target).value)" />
    @if (!emailValid()) {
      <span class="error">Invalid email</span>
    }
  `,
})
export class FormComponent {
  protected readonly email = signal('');

  // ✅ Pure signal — no RxJS needed
  protected readonly emailValid = computed(() => {
    const email = this.email();
    return email.length > 0 && email.includes('@') && email.includes('.');
  });
}
```

If you need debounce for expensive validation (e.g., API call to check username availability):

```
@Component({
  /* ... */
})
export class FormComponent {
  protected readonly username = signal('');

  private readonly username$ = toObservable(this.username);

  // ✅ Use RxJS for debounced async validation
  private readonly usernameAvailable$ = this.username$.pipe(
    debounceTime(500),
    distinctUntilChanged(),
    filter((username) => username.length >= 3),
    switchMap((username) => this.userService.checkUsernameAvailability(username)),
    catchError(() => of(false)),
  );

  protected readonly usernameAvailable = toSignal(this.usernameAvailable$, {
    initialValue: true,
  });
}
```

Decision Matrix: Signals vs RxJS

Use Case	Solution	Why
Component state (loading, selected item)	Signal	Synchronous, simple reactivity
Derived state (fullName = firstName + lastName)	Computed Signal	Pure transformation, no side effects
HTTP GET (one-shot request)	RxJS + toSignal()	Need HTTP operators (retry, catchError)
HTTP POST (side effect, no template binding)	RxJS (no toSignal)	Fire-and-forget, no UI update needed
Search with debounce	Signal → toObservable() → RxJS → toSignal()	Need debounceTime, distinctUntilChanged
WebSocket / SSE	RxJS + toSignal()	Continuous event stream
Form validation (sync)	Computed Signal	No async, pure logic
Form validation (async, e.g., username check)	Signal → toObservable() → RxJS → toSignal()	Need debounce + switchMap
Multiple parallel API calls	RxJS forkJoin + toSignal()	Need parallel orchestration
Sequential API calls (A → B → C)	RxJS switchMap chain + toSignal()	Need async sequencing
Timer / Interval	RxJS interval() + toSignal()	Time-based Observable
Router events	RxJS (router.events) + toSignal()	Built-in Observable API

Common RxJS Operators for Migration

Operator	Use Case	Example
map	Transform each value	<code>map(account => account.balance * 1.05)</code>
filter	Skip values that don't match condition	<code>filter(balance => balance > 0)</code>
switchMap	Cancel previous, start new (search)	<code>switchMap(query => this.search(query))</code>
mergeMap	Run all in parallel	<code>mergeMap(id => this.loadAccount(id))</code>
concatMap	Run sequentially (order preserved)	<code>concatMap(payment => this.process(payment))</code>
debounceTime	Wait X ms after last emission	<code>debounceTime(300)</code>
distinctUntilChanged	Only emit if value changed	<code>distinctUntilChanged()</code>
retry	Retry on error	<code>retry({ count: 3, delay: 1000 })</code>
catchError	Handle errors gracefully	<code>catchError(() => of([]))</code>
shareReplay	Cache result, share among subscribers	<code>shareReplay({ bufferSize: 1, refCount: true })</code>
combineLatest	Combine multiple Observables	<code>combineLatest([obs1\$, obs2\$])</code>
forkJoin	Wait for all to complete (parallel)	<code>forkJoin({ a: obs1\$, b: obs2\$ })</code>
scan	Accumulate values (like reduce)	<code>scan((acc, val) => acc + val, 0)</code>
takeUntil	Unsubscribe when trigger emits	<code>takeUntil(this.destroy\$)</code>

Anti-Pattern: Manual subscribe() in Components

✗ WRONG (AngularJS-style subscribe):

```
// DON'T DO THIS - manual subscription leaks memory
export class BadComponent implements OnInit, OnDestroy {
  private subscription: Subscription;
  protected accounts: Account[] = [];

  ngOnInit(): void {
    this.subscription = this.accountService.getAccounts().subscribe((accounts) => {
      this.accounts = accounts; // Manual assignment
    });
  }

  ngOnDestroy(): void {
    this.subscription.unsubscribe(); // Must remember to clean up!
  }
}
```

✓ CORRECT (Signal-first):

```
export class GoodComponent {
  private readonly accountService = inject(AccountService);

  // ✓ toSignal() auto-unsubscribes when component destroys
  protected readonly accounts = toSignal(this.accountService.getAccounts(), { initialValue: [] });
}
```

When to Keep subscribe()

Only use manual subscribe() for side effects (no UI update):

```

export class PaymentComponent {
  private readonly paymentService = inject(PaymentService);

  submitPayment(payment: Payment): void {
    // ✅ OK to subscribe — this is a side effect, not UI state
    this.paymentService
      .process(payment)
      .pipe(
        tap(() => this.showSuccessToast()),
        catchError((error) => {
          this.showErrorToast(error.message);
          return of(null);
        })
      )
      .subscribe(); // Fire-and-forget
  }

  private showSuccessToast(): void {
    /* ... */
  }
  private showErrorToast(message: string): void {
    /* ... */
  }
}

```

Migration Checklist: AngularJS Promises → RxJS

AngularJS	Angular 21 RxJS	Notes
<code>\$http.get(url).then(fn)</code>	<code>http.get(url).pipe(map(fn))</code>	No <code>.then()</code> — use operators
<code>\$http.get(url).catch(fn)</code>	<code>http.get(url).pipe(catchError(fn))</code>	Must return Observable in <code>catchError</code>
<code>\$q.defer()</code>	❌ Don't use — create Observable directly	Deferred pattern is anti-pattern in RxJS
<code>\$q.all([p1, p2])</code>	<code>forkJoin([obs1\$, obs2\$])</code>	Parallel execution
<code>\$q.when(value)</code>	<code>of(value)</code>	Immediate Observable
<code>\$timeout(fn, 1000)</code>	<code>timer(1000).pipe(tap(fn))</code> OR <code>delay(1000)</code>	Use RxJS timer
<code>\$interval(fn, 1000)</code>	<code>interval(1000).pipe(tap(fn))</code>	Continuous timer
<code>.then().then().then()</code>	<code>.pipe(switchMap, switchMap, switchMap)</code>	Chain with <code>switchMap</code>

Performance Tip: Avoid Unnecessary toSignal() Conversions

```

// ❌ BAD: Convert Observable → Signal → Observable (unnecessary)
protected readonly data = toSignal(this.http.get<Data>('/api/data'));
protected readonly transformed$ = toObservable(this.data).pipe(
  map(data => data?.items ?? [])
);

// ✅ GOOD: Stay in Observable until final template consumption
private readonly data$ = this.http.get<Data>('/api/data');
protected readonly transformed = toSignal(
  this.data$.pipe(map(data => data.items)),
  { initialValue: [] }
);

```

Section 3.10 complete. RxJS migration strategy covers when to use RxJS vs signals, API response transformation patterns, search with debounce, parallel/sequential API calls, WebSocket integration, form validation, operator usage guide, and anti-patterns to avoid.

Part 3 complete. The LLD covers the code-level patterns for every AngularJS artefact type: controllers → components, services → injectables, directives → components/directives/pipes, filters → pipes, route guards, Nx library structure, signal pattern mapping, security

3.11 — Angular 21.x Exclusive APIs: Modern Migration Targets

Enterprise Architect Note: Angular 21.1 and 21.2 introduced a set of APIs that are now the idiomatic Angular way to do things. When migrating from AngularJS, teams should target these APIs directly — not the Angular 14/15/16 equivalents that older guides recommend. This section documents every API that is new or promoted to stable in Angular 21.x.

3.11.1 — `httpResource()` and `resource()` — Declarative Reactive HTTP (Angular 21 Stable)

What is this feature? `httpResource()` and `resource()` are built-in Angular 21 functions that create a *reactive data object* — a bundle containing the fetched data, a loading state, and an error state, all as separate signals that update automatically.

What does it do? Calling `httpResource('/api/accounts')` returns an object with four signals: `value()` (the data), `isLoading()` (boolean), `error()` (any error), and a `reload()` method to manually trigger a refresh. When you pass a signal-based URL or request body, Angular automatically cancels any in-flight request and starts a new one whenever the signal changes. `resource()` is the same concept for any async operation — IndexedDB, Web Workers, custom `Promise` — not just HTTP.

What problem does it resolve? In AngularJS, every `$http.get()` required manually managing `$scope.loading = true/false`, `$scope.error`, and `$scope.data` in separate lines. Angular 21.0 improved this with `toSignal(this.http.get(...).pipe(catchError(), startWith([])))` — but still had no loading signal or error signal without 20+ additional lines of boilerplate. `httpResource()` solves all of that in a single declaration and makes the loading/error/reload lifecycle first-class reactive signals.

The progression: AngularJS → Angular 21.0 → Angular 21.1+

```
// — 1. AngularJS (Legacy) —
.controller('AccountsCtrl', function ($scope, $http) {
  $scope.accounts = [];
  $scope.loading = true;
  $scope.error = null;

  $http.get('/api/accounts')
    .then(function (response) {
      $scope.accounts = response.data;
      $scope.loading = false;
    })
    .catch(function (err) {
      $scope.error = err.message;
      $scope.loading = false;
    });
});

// — 2. Angular 21.0 (toSignal pattern – still valid) —
import { Component, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { toSignal } from '@angular/core/rxjs-interop';
import { catchError, of } from 'rxjs';

@Component({ /* ... */ })
export class AccountsComponent {
  private readonly http = inject(HttpClient);

  protected readonly accounts = toSignal(
    this.http.get<Account[]>('/api/accounts').pipe(
      catchError(() => of([]))
    ),
    { initialValue: [] }
  );
  // ❌ No loading state, no error signal, no reload capability without extra plumbing
}

// — 3. Angular 21.1+ (httpResource – the modern way) —
import { Component, inject } from '@angular/core';
import { httpResource } from '@angular/core'; // ✅ Angular 21.1 stable
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-accounts',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (accountsResource.isLoading()) {
      <app-skeleton rows="5" />
    } @else if (accountsResource.error()) {
      <app-error [message]="accountsResource.error()" (retry)="accountsResource.reload()" />
    } @else {
      @for (account of accountsResource.value(); track account.id) {
        <app-account-card [account]="account" />
      }
    }
  `,
})
export class AccountsComponent {
  protected readonly accountsResource = httpResource<Account[]>('/api/accounts');
  // ✅ accountsResource.value() – Signal<Account[] | undefined>
  // ✅ accountsResource.isLoading() – Signal<boolean>
  // ✅ accountsResource.error() – Signal<unknown>
  // ✅ accountsResource.reload() – imperative refresh (e.g., after create/update)
  // ✅ accountsResource.hasValue() – Signal<boolean>
  // ✅ accountsResource.status() – Signal<ResourceStatus>
}
```

`httpResource()` with reactive URL (signal-driven requests):


```
import { Component, signal, inject } from '@angular/core';
import { httpResource } from '@angular/core';

@Component({
  /* ... */
})
export class AccountDetailComponent {
  // Route param drives the request – when selectedId changes, resource auto-refetches
  protected readonly selectedId = signal<string | null>(null);

  protected readonly accountResource = httpResource<Account>(() => {
    const id = this.selectedId();
    return id ? `/api/accounts/${id}` : undefined; // undefined = skip request
  });

  // AngularJS equivalent:
  // $scope.$watch('selectedId', function (id) {
  //   if (id) $http.get('/api/accounts/' + id).then(r => $scope.account = r.data);
  // });
}
```

httpResource() with POST/headers (full `HttpRequest` config):

```
import { httpResource } from '@angular/core';
import { HttpRequest } from '@angular/common/http';

@Component({
  /* ... */
})
export class TransactionsComponent {
  private readonly filters = signal<TransactionFilter>({ from: '2024-01', status: 'active' });

  protected readonly transactionsResource = httpResource<Transaction[]>(() => ({
    url: '/api/transactions',
    method: 'POST',
    body: this.filters(), // Reactive body – refetches when filters signal changes
    headers: { 'X-Tenant': 'bank' },
  }));
}
```

resource() — for non-HTTP async operations (IndexedDB, Web Workers, custom promises):

```
import { Component, signal, resource } from '@angular/core';

@Component({
  /* ... */
})
export class ReportComponent {
  private readonly reportParams = signal({ quarter: 'Q1', year: 2025 });

  // resource() replaces toSignal(someObservable) when you control the async operation
  protected readonly reportResource = resource({
    request: () => this.reportParams(), // Tracks signal; re-runs loader when it changes
    loader: async ({ request }) => {
      // Any async operation: IndexedDB, Web Worker, fetch, etc.
      const data = await this.reportWorker.compute(request);
      return data;
    },
  });

  // reportResource.value() – Signal<ReportData | undefined>
  // reportResource.isLoading() – Signal<boolean>
  // reportResource.error() – Signal<unknown>
  // reportResource.reload() – force re-run of loader
}
```

When to use `httpResource()` vs `toSignal()` vs plain `Observable` :

Scenario	Recommended Approach	Reason
Fetching data tied to route params / filter signals	<code>httpResource()</code>	Auto-refetch, loading/error signals built in
Complex pipeline (debounce, switchMap, retry)	<code>toSignal(observable\$.pipe(...))</code>	RxJS operators are more powerful for complex flows
POST/mutation (form submit, create, delete)	<code>http.post()</code> in method, no <code>httpResource</code>	Resources are for reading data, not mutations
Non-HTTP async (IndexedDB, Worker, custom)	<code>resource()</code>	Generalised reactive async primitive
WebSocket streaming	<code>toSignal(webSocket\$.pipe(...))</code>	Continuous stream — <code>resource()</code> is one-shot
Simple one-time load on init	<code>toSignal()</code> still fine	Don't rewrite working code; use <code>httpResource</code> for new work

3.11.2 — Signal-Based DOM Queries: `viewChild()`, `contentChild()`, `viewChildren()`, `contentChildren()`

What is this feature? `viewChild()`, `contentChild()`, `viewChildren()`, and `contentChildren()` are function-based Angular 21 query APIs that find DOM elements or child components within a parent, returning the result as a `Signal` instead of a plain class property.

What does it do? Each function returns a `Signal<T>` — the queried element or component wrapped in a signal that automatically updates when the queried element enters or leaves the view. `viewChild.required()` returns `Signal<T>` (guaranteed non-null); plain `viewChild()` returns `Signal<T | undefined>`. These signals can be read inside `computed()`, `effect()`, and templates exactly like any other signal — no lifecycle wiring needed.

What problem does it resolve? AngularJS used `angular.element()` or jQuery for DOM access — brittle and SSR-incompatible. Angular 14–20 improved this with `@ViewChild` decorators, but those decorators are **not reactive** — you cannot use a `@ViewChild` reference inside `computed()` because it returns `undefined` until `ngAfterViewInit` fires. Signal-based queries are reactive from the first render cycle, eliminate the need for `ngAfterViewInit` in most cases, and work naturally with `effect()` for third-party library initialisation.

 **Guide Correction:** The two `@ViewChild` examples in this guide (chart integration, plugin host) reflect third-party library compatibility patterns. For all first-party Angular code, use signal-based queries.

Migration: `@ViewChild` → `viewChild()`

```
// — AngularJS (Legacy DOM access) —
.directive('chartWrapper', function ($element, $scope) {
  const container = $element.find('.chart-container')[0];
  const chart = new Chart(container, $scope.config);
});

// — Angular 14-20 (decorator-based – still works but legacy) —
import { ViewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({ /* ... */ })
export class ChartComponent implements AfterViewInit {
  @ViewChild('chartContainer') containerRef!: ElementRef; // ❌ Not a signal – can't use in computed/effect

  ngAfterViewInit() {
    // Must use lifecycle hook – decorator value not available until after view init
    const chart = new Chart(this.containerRef.nativeElement, this.config());
  }
}

// — Angular 21 (signal-based – the modern way) —
import { Component, ViewChild, ElementRef, effect, inject } from '@angular/core';

@Component({
  selector: 'app-chart',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<div #chartContainer class="w-full h-[300px]"></div>`,
})
export class ChartComponent {
  protected readonly containerRef = viewChild.required<ElementRef>('chartContainer');
  // ✅ viewChild.required() – Signal<ElementRef> – guaranteed non-null after view init
  // ✅ viewChild() – Signal<ElementRef | undefined> – nullable variant

  constructor() {
    // effect() runs after view is initialized – safe DOM access
    effect(() => {
      new Chart(this.containerRef().nativeElement, this.config());
    });
  }
}
```

viewChildren() — replacing QueryList:

```
// — Angular 14-20 (QueryList – imperative, no signal) —
@ViewChildren(TodoItemComponent) items!: QueryList<TodoItemComponent>;

ngAfterViewInit() {
  this.items.changes.subscribe(list => this.count.set(list.length)); // Extra RxJS wiring
}

// — Angular 21 (signal array – reactive) —
import { viewChildren } from '@angular/core';

@Component({ /* ... */ })
export class TodoListComponent {
  protected readonly items = viewChildren(TodoItemComponent);
  // items() – Signal<readonly TodoItemComponent[]>
  // Reactive in computed() and effect() natively

  protected readonly visibleCount = computed(() => this.items().filter(i => i.isVisible()).length);
  // ✅ Recomputes automatically when child components change
}
```

contentChild() / contentChildren() — replacing @ContentChild:

```
// — Angular 14-20 (decorator – legacy) —
@ViewChild(NavItemComponent) navItem!: NavItemComponent;

// — Angular 21 (signal-based) —
import { contentChild, contentChildren } from '@angular/core';

@Component({ /* ... */ })
export class NavComponent {
  protected readonly activeItem = contentChild(NavItemComponent); // Signal<NavItemComponent | undefined>
  protected readonly allItems = contentChildren(NavItemComponent); // Signal<readonly NavItemComponent[]>

  protected readonly activeLabel = computed(() => this.activeItem()?.label ?? '');
}
```

Signal query migration table:

Legacy Decorator	Angular 21 Signal Function	Return Type
@ViewChild(T)	viewChild(T)	Signal<T undefined>
@ViewChild(T, { required: true })	viewChild.required(T)	Signal<T>
@ViewChildren(T)	viewChildren(T)	Signal<readonly T[]>
@ContentChild(T)	contentChild(T)	Signal<T undefined>
@ContentChild(T, { required: true })	contentChild.required(T)	Signal<T>
@ContentChildren(T)	contentChildren(T)	Signal<readonly T[]>
@ViewChild('refName') (string)	viewChild<ElementRef>('refName')	Signal<ElementRef undefined>
@ViewChild('refName') (required)	viewChild.required<ElementRef>('refName')	Signal<ElementRef>

3.11.3 — @let — Template Variable Declarations (Angular 18+, Improved in 21.1)

What is this feature? `@let` is Angular's built-in template variable declaration syntax that lets you define a named local variable inside a template, available for reuse within that template block.

What does it do? `@let account = accountsResource.value();` evaluates the expression once per render cycle and assigns the result to `account`. The variable is block-scoped (only visible within the enclosing `@if`, `@for`, or component root), read-only (like `const`), and TypeScript-typed — so if you assign an `Account | undefined` value inside an `@if (account)` block, TypeScript narrows the type to `Account` automatically.

What problem does it resolve? AngularJS used `ng-init` to declare template variables — an acknowledged anti-pattern that polluted `$scope`. Angular 14–20 had no native template variable syntax at all: developers had to repeat signal calls multiple times (e.g., `accountsResource.value()` written five times in one template), use the verbose `async | as` pipe syntax, or split a template into a sub-component just to avoid repetition. `@let` eliminates all of these workarounds cleanly.

```
<!-- AngularJS: ng-init for template variables (unreliable, anti-pattern) -->
<div ng-init="totalWithTax = order.total * 1.2">Total: {{ totalWithTax }}</div>

<!-- Angular 14-20: Needed extra structural directive or component split -->
<!-- No native way to declare template variables -->

<!-- Angular 21: @let – native template variable declarations -->
@let currentUser = userResource.value(); @let totalWithTax = (order()?.total ?? 0) * 1.2; @let
isAdminUser = currentUser?.roles?.includes('admin') ?? false; @if (currentUser) {
<h2>Welcome, {{ currentUser.name }}</h2>

@let greeting = isAdminUser ? 'Admin Dashboard' : 'My Account';
<h3>{{ greeting }}</h3>
}

<p>Total inc. tax: {{ totalWithTax | currency }}</p>
```

@let with async resource values — avoids repeated () calls:

```

<!-- Without @let — repeated function calls (re-evaluated every render) -->
@if (accountsResource.value()) {
  <app-header [account]="accountsResource.value()" />
  <app-body [account]="accountsResource.value()" />
  <app-footer [account]="accountsResource.value()" />
}

<!-- With @let — single evaluation, type-narrowed -->
@let account = accountsResource.value(); @if (account) {
  <app-header [account]="account" />
  <!-- account is Account (not Account | undefined) -->
  <app-body [account]="account" />
  <app-footer [account]="account" />
}

```

@let rules and limitations:

Rule	Detail
Scope	Block-scoped — visible only within the enclosing <code>@if</code> , <code>@for</code> etc.
Reactivity	Re-evaluated every render cycle — not itself a signal
Read-only	Cannot be assigned to (like <code>const</code> in TypeScript)
Works with signals	<code>@let count = mySignal();</code> — reads signal, but <code>@let</code> not reactive
Multiple declarations	Multiple <code>@let</code> in same block allowed

3.11.4 — `afterRender()` and `afterNextRender()` — Browser-Only Lifecycle Hooks

What is this feature? `afterRender()` and `afterNextRender()` are Angular lifecycle hooks that register callback functions to execute *after the browser has finished painting the component*, and that are automatically skipped when running on the server (SSR).

What does it do? `afterNextRender(fn)` runs `fn` exactly once — after the very first browser render cycle. Use it for one-time DOM setup (chart initialisation, scroll position reset, measuring layout). `afterRender(fn)` runs `fn` after every subsequent render cycle. Use it to keep third-party DOM libraries in sync with Angular state. Both are called in the constructor (injection context) and neither requires implementing a lifecycle interface. Both are no-ops during server-side rendering, so no `isPlatformBrowser()` guards are needed.

What problem does it resolve? AngularJS `link()` functions ran DOM operations unconditionally — fine for browser-only apps but fatal for any SSR setup. Angular's `ngAfterViewInit()` improved this but still executes on the server, causing crashes unless you wrap every DOM call in `if (isPlatformBrowser(this.platformId))`. That guard pattern is verbose, easy to forget, and scattered across hundreds of components in a 100-screen migration. `afterRender` / `afterNextRender` are SSR-safe by design — the guard is built in.

```
// — AngularJS (Direct DOM manipulation in link function) —
.directive('scrollable', function () {
  return {
    link: function ($scope, $element) {
      $element[0].scrollTop = 0; // ❌ Runs everywhere including SSR
    }
  };
});

// — Angular 14-20 (ngAfterViewInit – still SSR-unsafe without isPlatformBrowser) —
@Component({ /* ... */ })
export class ScrollableComponent implements AfterViewInit {
  private readonly platformId = inject(PLATFORM_ID);

  ngAfterViewInit() {
    if (isPlatformBrowser(this.platformId)) { // Manual platform guard required
      window.scrollTo(0, 0);
    }
  }
}

// — Angular 21 (afterRender / afterNextRender – SSR-safe by design) —
import { Component, afterRender, afterNextRender, ElementRef, viewChild } from '@angular/core';

@Component({
  selector: 'app-scrollable',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<div #container class="overflow-y-auto h-[400px]"><ng-content /></div>`,
})
export class ScrollableComponent {
  private readonly container = viewChild.required<ElementRef>('container');

  constructor() {
    // afterNextRender: runs ONCE after the very first render – good for one-time setup
    afterNextRender(() => {
      // ✅ Only runs in browser – never runs on server during SSR
      this.container().nativeElement.scrollTop = 0;
      this.initChartLibrary(this.container().nativeElement);
    });

    // afterRender: runs AFTER EVERY render cycle – use for DOM sync that must stay current
    afterRender(() => {
      // ✅ Browser only; runs after every change detection pass
      // Use for: measuring DOM dimensions, updating Canvas/WebGL, third-party DOM libraries
      this.updateScrollIndicator();
    });
  }
}
```

afterRender() phases — fine-grained rendering pipeline control:

```
import { afterRender, AfterRenderPhase } from '@angular/core';

constructor() {
  // Phase 1: EarlyRead – read layout before Angular writes to DOM
  afterRender({ phase: AfterRenderPhase.EarlyRead, read: () => {
    const height = this.container().nativeElement.offsetHeight;
    this.containerHeight.set(height);
  }});

  // Phase 2: Write – write DOM changes after Angular finishes
  afterRender({ phase: AfterRenderPhase.Write, write: () => {
    this.container().nativeElement.style.maxHeight = `${this.containerHeight()}px`;
  }});

  // Phase 3: Read – final DOM read after all writes
  afterRender({ phase: AfterRenderPhase.Read, read: () => {
    this.scrollTop.set(this.container().nativeElement.scrollTop);
  }});
}
```

Migration decision table:

Legacy Pattern	Angular 21 Replacement	Why
<code>ngAfterViewInit</code> (DOM ops)	<code>afterNextRender()</code>	SSR-safe, no <code>isPlatformBrowser</code> guard needed
<code>ngAfterViewChecked</code> (DOM sync)	<code>afterRender()</code>	Runs after every render, browser-only
<code>\$element.ready()</code> in AngularJS directive	<code>afterNextRender()</code>	Browser lifecycle — one-time DOM setup
<code>\$timeout(fn, 0)</code> for next tick DOM	<code>afterNextRender()</code>	Clean async DOM access without <code>setTimeout</code>

3.11.5 — `untracked()` — Breaking Signal Dependency Chains

What is this feature? `untracked()` is a signal utility function that executes a function reading one or more signals, but deliberately prevents those signals from being registered as reactive dependencies of the surrounding `computed()` or `effect()`.

What does it do? `untracked(() => mySignal())` reads `mySignal`'s current value at that instant and returns it, but the enclosing `computed()` or `effect()` will NOT re-run when `mySignal` changes in the future. It is a one-time snapshot read, not a subscription.

What problem does it resolve? In Angular's reactive model, every signal read inside `effect()` or `computed()` automatically subscribes to that signal. This is usually desirable, but creates a dangerous trap: if an effect reads a signal and then *writes to that same signal* (or writes to a signal that feeds back), you get an infinite reactive loop. AngularJS had the same `$watch` trap — watchers that modified watched properties caused `$digest` to loop. `untracked()` is the escape hatch: read a signal's value without joining the reactive graph, breaking the loop.

```
import { Component, signal, computed, effect, untracked } from '@angular/core';

@Component({
  /* ... */
})
export class AuditComponent {
  private readonly currentValue = signal(0);
  private readonly auditLog = signal<AuditEntry[]>([]);

  constructor() {
    // ❌ PROBLEM: Effect reads auditLog inside — creates circular dependency
    effect(() => {
      const val = this.currentValue(); // Tracks currentValue
      const log = this.auditLog(); // ❌ Also tracks auditLog — triggers effect again when log appended
      this.auditLog.set([...log, { val, timestamp: Date.now() }]); // Infinite loop risk
    });

    // ✅ SOLUTION: untracked() reads auditLog WITHOUT adding it as a dependency
    effect(() => {
      const val = this.currentValue(); // ✅ Tracks currentValue (triggers effect)
      const currentLog = untracked(() => this.auditLog()); // ✅ Reads auditLog WITHOUT tracking it
      this.auditLog.set([...currentLog, { val, timestamp: Date.now() }]);
    });
  }
}
```

`untracked()` in `computed()` — conditional reactive dependencies:

```
@Component({
  /* ... */
})
export class DashboardComponent {
  private readonly userId = signal<string | null>(null);
  private readonly cachedUserData = signal<UserData | null>(null);

  protected readonly displayName = computed(() => {
    const id = this.userId(); // Reactive: recomputes when userId changes
    if (!id) return 'Guest';

    // Read cachedUserData WITHOUT making it a dependency of this computed
    // Only re-fetch logic based on userId, not every time cache updates
    const cached = untracked(() => this.cachedUserData());
    return cached?.name ?? 'Loading...';
  });
}
```

3.11.6 — DestroyRef — Explicit Lifecycle Cleanup Beyond `takeUntilDestroyed()`

What is this feature? `DestroyRef` is an Angular injectable token that represents the destruction lifecycle of the current injection context — a component, directive, service with limited scope, or route. It provides a registration mechanism for cleanup functions that need to run when that context is destroyed.

What does it do? `inject(DestroyRef).onDestroy(cleanupFn)` registers `cleanupFn` to execute when the component or service is destroyed. Multiple calls register multiple cleanup functions, all run in registration order. It works anywhere `inject()` works — in constructors, factory functions called during construction, and composable utility functions.

What problem does it resolve? AngularJS cleanup was `$scope.$on('$destroy', fn)` — clean concept but tied entirely to `$scope`. Angular's original replacement was implementing the `onDestroy` interface with `ngOnDestroy()` — which requires the class to implement an interface, adding interface boilerplate to every component or service needing cleanup. `DestroyRef` decouples cleanup registration from class structure: a composable utility function can register its own cleanup without requiring the hosting component to implement any interface. This is critical for third-party library teardown (Chart.js, WebSocket connections, timers) that occurs in many migrated components.

```
// — AngularJS (destroy hook) —
.controller('ChartCtrl', function ($scope) {
  const chart = new Chart(document.getElementById('chart'));

  $scope.$on('$destroy', function () {
    chart.destroy(); // Clean up third-party library
  });
});

// — Angular 21 (DestroyRef – composable, works anywhere in injection context) —
import { Component, inject, DestroyRef, afterNextRender, viewChild, ElementRef } from '@angular/core';

@Component({
  selector: 'app-chart',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<canvas #canvas></canvas>`,
})
export class ChartComponent {
  private readonly canvas = viewChild.required<ElementRef<HTMLCanvasElement>>('canvas');
  private readonly destroyRef = inject(DestroyRef); // ✅ Inject destroy lifecycle

  constructor() {
    afterNextRender(() => {
      const chart = new Chart(this.canvas().nativeElement, this.chartConfig());

      // Register cleanup – runs when component is destroyed
      this.destroyRef.onDestroy(() => chart.destroy()); // ✅ No ngOnDestroy needed
    });
  }
}
```

DestroyRef in services — composable cleanup without component lifecycle:


```
// — AngularJS (factory cleanup — no native support) —
.factory('WebSocketService', function ($rootScope) {
  const ws = new WebSocket('wss://api/live');
  // ✖ No standard cleanup — devs manually called .close() or hoped for GC
  return { ws };
});

// — Angular 21 (Service with DestroyRef) —
import { Injectable, inject, DestroyRef } from '@angular/core';

@Injectable() // Scoped service (not providedIn: 'root') — destroyed with component
export class LiveFeedService {
  private readonly destroyRef = inject(DestroyRef);
  private ws!: WebSocket;

  connect(url: string): void {
    this.ws = new WebSocket(url);

    // Auto-cleanup when the service is destroyed (component or route scope)
    this.destroyRef.onDestroy(() => {
      if (this.ws.readyState === WebSocket.OPEN) {
        this.ws.close();
      }
    });
  }
}
```

`takeUntilDestroyed()` **VS** `DestroyRef.onDestroy()` — when to use each:

Scenario	Use
Unsubscribing from RxJS Observables	<code>takeUntilDestroyed()</code> in pipe
Destroying third-party library instance	<code>destroyRef.onDestroy()</code>
Closing WebSocket / EventSource	<code>destroyRef.onDestroy()</code>
Clearing <code>setInterval</code> / <code>setTimeout</code>	<code>destroyRef.onDestroy()</code>
Removing native DOM event listeners	<code>destroyRef.onDestroy()</code>
General RxJS subscription in constructor	<code>takeUntilDestroyed()</code> (no <code>DestroyRef</code> arg needed in component constructor)
In a utility function / non-component context	<code>takeUntilDestroyed(destroyRef)</code> (must pass <code>DestroyRef</code> explicitly)

3.11.7 — Input Transforms: `booleanAttribute`, `numberAttribute`

What is this feature? Input transforms are optional configuration functions on Angular 21 signal inputs (`input(defaultValue, { transform: fn })`) that automatically coerce incoming attribute or binding values into the correct type before they are stored in the signal.

What does it do? Angular ships two built-in transform functions. `booleanAttribute` converts HTML attribute presence patterns (`""`, `"true"`, `true`) to `true`, and `"false"`, `false`, `undefined`, `null` to `false` — matching the behaviour of native HTML boolean attributes like `disabled`. `numberAttribute` converts a string `"42"` to the number `42`. You can also supply any custom `(value: unknown) => T` function as a transform.

What problem does it resolve? HTML attributes are always strings. `<app-button disabled>` passes `""` as the value, not `false`. AngularJS handled this automatically for its own directives (`ng-disabled`, `ng-required`). When migrating controllers to Angular 21 components with custom boolean inputs, every input previously required a verbose getter/setter: `@Input() set disabled(v: string | boolean) { this._disabled = v === '' || v === true || v === 'true'; }`. Input transforms replace this entire repeated pattern with a single `{ transform: booleanAttribute }` configuration.

```
// — Angular 14-20 (manual coercion – common pattern) —
@Input() set disabled(value: string | boolean) {
  this._disabled = value === '' || value === true || value === 'true';
}

// — Angular 21 (input transform – declarative) —
import { Component, input, booleanAttribute, numberAttribute } from '@angular/core';

@Component({ /* ... */ })
export class ButtonComponent {
  // booleanAttribute: '' | 'true' | true → true; 'false' | false | undefined → false
  readonly disabled = input(false, { transform: booleanAttribute });
  // <app-button disabled /> → disabled() === true
  // <app-button [disabled]="isDisabled()" /> → passes boolean signal value

  // numberAttribute: converts string to number (or NaN if invalid)
  readonly tabIndex = input(0, { transform: numberAttribute });
  // <app-button tabIndex="3" /> → tabIndex() === 3
}
```

3.11.8 — `outputFromObservable()` and `outputToObservable()` — RxJS Output Bridge

What is this feature? `outputFromObservable()` and `outputToObservable()` are Angular 21 bridge utilities (from `@angular/core/rxjs-interop`) that convert between Angular's signal-based `output()` API and RxJS Observables, enabling both paradigms to interoperate during migration.

What does it do? `outputFromObservable(observable$)` wraps an existing RxJS Observable and exposes it as an Angular output — parent components can bind to it with `(eventName)="handler($event)"`. `outputToObservable(outputRef)` does the reverse: takes an Angular `output()` reference and returns an Observable for any code that still consumes event streams via RxJS. The conversion is bidirectional and does not modify the underlying Observable.

What problem does it resolve? During a migration, many existing services still emit events as RxJS `Subject`s: notification services, legacy event buses, WebSocket wrappers. Rewriting every event-emitting service to use signal outputs is high-risk mid-migration. `outputFromObservable()` lets you expose the existing Observable as a modern Angular output immediately, without touching the service — enabling incremental migration where the component looks fully Angular 21 while the underlying service still operates on RxJS.

```
// — AngularJS (event broadcast) —
.factory('NotificationService', function ($rootScope) {
  return {
    notify: function (msg) { $rootScope.$broadcast('notification', msg); }
  };
});

// — Angular 21: outputFromObservable — bridge existing Observable to output() API —
import { Component, inject } from '@angular/core';
import { outputFromObservable, outputToObservable } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-notification-host',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<!-- ... -->`,
})
export class NotificationHostComponent {
  private readonly notificationService = inject(NotificationService);

  // Convert Observable<Notification> → Angular 21 output()
  readonly notification = outputFromObservable(
    this.notificationService.notifications$
  );
  // Parent can: (notification)="handleNotification($event)"
}

// outputToObservable: convert output() EventEmitter → Observable for RxJS consumers
import { outputToObservable } from '@angular/core/rxjs-interop';

class ConsumerComponent {
  private readonly child = viewChild.required(NotificationHostComponent);

  constructor() {
    afterNextRender(() => {
      // Convert signal output back to Observable if a service needs it
      const notifications$ = outputToObservable(this.child().notification);
      notifications$.pipe(takeUntilDestroyed()).subscribe(/* ... */);
    });
  }
}
```

3.11.9 — `withEventReplay()` — SSR Event Capture Before Hydration

What is this feature? `withEventReplay()` is an Angular SSR hydration option that records user interactions — clicks, keypresses, form inputs — that fire during the interval between the server-rendered HTML appearing on screen and Angular's JavaScript completing its hydration process.

What does it do? It installs a lightweight event capture layer during the hydration window. Every qualifying user interaction is queued rather than discarded. Once Angular finishes hydrating and all event handlers are registered, the queued events are replayed in the order they occurred, exactly as if the user had performed them on a fully hydrated page.

What problem does it resolve? SSR renders HTML on the server for fast first paint, but Angular hydration takes additional time (typically 200–800ms). Without `withEventReplay()`, any user interaction in that window is silently lost. A banking user who clicks "Submit" on a login form immediately after page load has their submission dropped with no error message. AngularJS had no SSR at all, so this is a new class of problem introduced when adding SSR to a migrated application — and it is easy to miss in testing because developers rarely click in that precise millisecond window.

```
// — Without withEventReplay (Angular 21 default without this option) —
// User clicks "Submit" at t=0ms (before hydration at t=500ms)
// → Event is silently lost; form never submits

// — With withEventReplay (Angular 21 recommended for forms and CTAs) —
// app.config.ts
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import {
  provideClientHydration,
  withEventReplay,
  withIncrementalHydration,
} from '@angular/platform-browser';

export const appConfig: ApplicationConfig = {
  providers: [
    provideClientHydration(
      withEventReplay(), // ✅ Capture + replay events during hydration window
      withIncrementalHydration(), // ✅ Hydrate only visible @defer blocks (pairs well with withEventReplay)
    ),
    provideZoneChangeDetection({ eventCoalescing: true }),
  ],
};
```

What `withEventReplay()` captures:

Event Type	Replayed?	Notes
click	✅ Yes	Button clicks, link navigation, form submit clicks
input	✅ Yes	Text typed in inputs before hydration
submit	✅ Yes	Form submissions
keydown / keyup	✅ Yes	Keyboard interactions
scroll	❌ No	Scroll position not replayed (not a user intent event)
mousemove	❌ No	Mouse tracking not replayed

3.11.10 — Route-Level SSR Rendering Modes (Angular 19+ — Critical for Enterprise SSR)

What is this feature? Route-level rendering modes are a per-route SSR configuration API (Angular 19+, stable in Angular 21) that assigns each application route one of three rendering strategies: `RenderMode.Prerender` (static HTML generated at build time), `RenderMode.Server` (fresh HTML rendered on every request), or `RenderMode.Client` (browser-only, no server rendering).

What does it do? You define a `ServerRoute[]` array in `app.routes.server.ts` with a `renderMode` property for each route pattern and register it with `provideServerRoutesConfig()`. The Angular SSR engine reads these rules at build time or request time and renders each route accordingly. Routes without an explicit entry fall back to the global default.

What problem does it resolve? AngularJS had no SSR concept at all. When adding SSR during migration, the naive approach renders every route on the server — correct for account pages (needs fresh data per user) but wasteful for the marketing homepage (identical content for all users, perfect for CDN caching) and wrong for admin report builders (too dynamic and JS-heavy for SSR to add any value). Without route-level control you must choose one rendering strategy for the whole app, forcing an unacceptable trade-off. For a bank, this means either stale account balances (bad) or a non-CDN-cacheable homepage (also bad).

```
// app.routes.server.ts – Server-side rendering configuration per route
import { RenderMode, ServerRoute } from '@angular/ssr';

export const serverRoutes: ServerRoute[] = [
  // Static marketing pages – pre-rendered at build time (fastest LCP)
  {
    path: 'home',
    renderMode: RenderMode.Prerender,
  },
  {
    path: 'about',
    renderMode: RenderMode.Prerender,
  },

  // Authenticated dynamic screens – server-rendered on each request
  {
    path: 'accounts',
    renderMode: RenderMode.Server, // ✅ Fresh data on every request (banking requirement)
  },
  {
    path: 'accounts/:id',
    renderMode: RenderMode.Server, // ✅ Dynamic params – must be Server mode
  },
  {
    path: 'transactions',
    renderMode: RenderMode.Server,
  },

  // Admin tools with heavy client-side JS – skip SSR entirely
  {
    path: 'admin/report-builder',
    renderMode: RenderMode.Client, // ✅ Client-only – too dynamic for SSR
  },

  // Login / public pages – pre-rendered
  {
    path: 'login',
    renderMode: RenderMode.Prerender,
  },

  // Fallback for all remaining routes
  {
    path: '**',
    renderMode: RenderMode.Server,
  },
];
```

Register server routes in `app.config.server.ts`:

```
// app.config.server.ts
import { mergeApplicationConfig, ApplicationConfig } from '@angular/core';
import { provideServerRendering, provideServerRoutesConfig } from '@angular/ssr';
import { appConfig } from './app.config';
import { serverRoutes } from './app.routes.server';

const serverConfig: ApplicationConfig = {
  providers: [
    provideServerRendering(),
    provideServerRoutesConfig(serverRoutes), // ✅ Register per-route rendering modes
  ],
};

export const config = mergeApplicationConfig(appConfig, serverConfig);
```

Rendering mode decision matrix for banking migration:

Route Type	Rendering Mode	Reason
Marketing / landing pages	Prerender	Static content, best LCP, no server cost per request
Login / registration	Prerender	No dynamic data, CDN-cached
Dashboard (personalised)	Server	User-specific data, must be fresh
Account list / detail	Server	Real-time balances — stale data unacceptable in banking
Transaction history	Server	Compliance requires fresh data, no caching
Report builder / admin tools	Client	Complex interactivity, D3/Canvas — SSR adds no value
Error pages (404, 500)	Prerender	Static, always available even if server is down

3.11.11 — `provideZonelessChangeDetection()` — Stable in Angular 21

What is this feature? `provideZonelessChangeDetection()` is an Angular provider that configures the application to run without Zone.js — the library Angular historically relied on to monkey-patch browser APIs (`setTimeout` , `fetch` , `addEventListener`) and trigger change detection after any async event.

What does it do? When this provider is active, Angular no longer polls after async events. Instead it relies entirely on signals to know what changed. Only components whose signal dependencies actually changed are re-rendered. Everything else is untouched, making change detection $O(\text{changed signals})$ instead of $O(\text{all components})$.

What problem does it resolve? AngularJS's `$digest` cycle was famously slow with 2000+ watchers because it re-checked every watcher on every browser event. Angular with Zone.js was faster but still triggered change detection on every click, HTTP response, and timer tick across the entire app tree. With signals and `provideZonelessChangeDetection()` , Angular knows exactly what changed and updates only those nodes. It also removes ~15KB from the production bundle and eliminates Zone.js monkey-patching conflicts with third-party libraries such as Chart.js and ag-Grid.

Breaking API Name Change: In Angular 21, `provideExperimentalZonelessChangeDetection()` was graduated to `provideZonelessChangeDetection()` . The experimental prefix has been removed. Update all references before upgrading.

The migration path for AngularJS → Angular 21 full zoneless:

Phase 1 (Now): Zone.js active → OnPush everywhere
Phase 2 (After Sprint): Zone.js + OnPush → `provideZonelessChangeDetection()` opt-in
Phase 3 (Goal): Full zoneless → Remove zone.js from polyfills in `angular.json`

```
// Phase 2: app.config.ts – enable zoneless (Angular 21 stable API)
import { ApplicationConfig, provideZonelessChangeDetection } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(), // ✅ Angular 21 stable – no longer "Experimental"
    // ⚠️ Requirements before enabling:
    // 1. ALL components have ChangeDetectionStrategy.OnPush
    // 2. ALL state mutations go through signal.set() / signal.update()
    // 3. No library code relies on Zone.js patching (check third-party libs)
    // 4. Remove zone.js from polyfills in angular.json (Phase 3)
  ],
};
```

```
// Phase 3: angular.json – remove zone.js polyfill
{
  "projects": {
    "banking-ng21": {
      "architect": {
        "build": {
          "options": {
            "polyfills": [
              // "zone.js" ← Remove this line when fully zoneless
            ]
          }
        },
        "test": {
          "options": {
            "polyfills": [
              // "zone.js/testing" ← Also remove from test config
            ]
          }
        }
      }
    }
  }
}
```

Zoneless readiness checklist for enterprise migration:

- All 120 migrated controllers → components use `ChangeDetectionStrategy.OnPush`
- Signal coverage audit complete – no `BehaviorSubject` for component-local state
- Third-party libraries audited for `Zone.js` dependency (`ag-Grid`, `Chart.js`, etc.)
- HTTP interceptors use functional API (`HttpInterceptorFn`) – not class-based
- `setTimeout` / `setInterval` wrapped to update signals after execution
- E2E test suite passes with zoneless enabled
- Bundle size reduction validated (target: ~15KB reduction from `zone.js` removal)

3.11.12 — `provideAppInitializer()` — Modern Bootstrap Initialization (Angular 19+)

What is this feature? `provideAppInitializer()` is an Angular 19+ provider function that registers an async initialization callback to execute before the application bootstraps — before any route renders or any component becomes visible to the user.

What does it do? You pass an arrow function that uses `inject()` to access services and returns a `Promise<void>` or `Observable<void>`. Angular waits for all registered initializers to complete before rendering the root component. Multiple `provideAppInitializer()` calls all run in parallel — each is independent and does not need to declare dependencies on others.

What problem does it resolve? AngularJS used `angular.run()` blocks to load initial data (user session, feature flags, configuration) before the UI painted — a clean pattern. The Angular replacement was `APP_INITIALIZER` which required verbose factory configuration: `{ provide: APP_INITIALIZER, useFactory: (svc: MyService) => () => svc.load(), deps: [MyService], multi: true }`. The `deps` array doesn't support `inject()`. Missing `multi: true` silently overwrites any previously registered initializer. `provideAppInitializer()` uses `inject()` directly, is strongly typed, and multiple calls are independent by design — restoring the simplicity of `angular.run()` with modern Angular idioms.

```
// — Legacy pattern (still works but verbose) —
import { APP_INITIALIZER } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [
    {
      provide: APP_INITIALIZER,
      useFactory: (featureService: FeatureFlagService) => () => featureService.load(),
      deps: [FeatureFlagService],
      multi: true,
    },
  ],
};

// — Angular 21 preferred pattern (provideAppInitializer) —
import { provideAppInitializer, inject } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [
    // Initializer 1: Load feature flags
    provideAppInitializer(() => {
      const featureService = inject(FeatureFlagService); // ✅ inject() works here
      return featureService.load(); // Return Promise<void> or Observable<void>
    }),

    // Initializer 2: Restore auth session
    provideAppInitializer(() => {
      const authService = inject(AuthService);
      return authService.restoreSession(); // Called before any route renders
    }),

    // Initializer 3: Load i18n translations
    provideAppInitializer(() => {
      const i18n = inject(TranslationService);
      return i18n.loadLocale('en-GB');
    }),

    // All three run in parallel; app bootstraps only after all resolve
  ],
};
```

provideAppInitializer() vs APP_INITIALIZER comparison:

Feature	APP_INITIALIZER	provideAppInitializer() (Angular 19+)
Syntax	Verbose factory + deps array	Concise function with <code>inject()</code> inside
Type safety	Loose — factory return type not checked	Strongly typed — TypeScript enforced
Multiple initializers	<code>multi: true</code> (easy to forget)	Multiple <code>provideAppInitializer()</code> calls
<code>inject()</code> support	Only via <code>deps</code> array	✅ Native <code>inject()</code> in function body
Parallel execution	✅ All run in parallel	✅ All run in parallel

Section 3.11 complete. Angular 21.x exclusive APIs: `httpResource()`, `resource()`, signal-based queries (`viewChild`, `contentChild`, `viewChildren`, `contentChildren`), `@Let` template variables, `afterRender` / `afterNextRender` lifecycle hooks, `untracked()`, `DestroyRef`, `input transforms`, `outputFromObservable()`, `withEventReplay()` for SSR, route-level rendering modes (`RenderMode`), `provideZoneLessChangeDetection()` stable API, and `provideAppInitializer()`. Target these APIs directly — not Angular 14/16/18 intermediaries.

3.12 — TypeScript Type System Migration: Interfaces, Types, Generics, Records & Enums

AngularJS runs on plain JavaScript — there are no interfaces, generics, enums, or `Record` types. Every data contract is implicit: a `$http` response is `any`, a `$scope` property is "whatever was last assigned", and status constants are plain objects with string values. This section is the implementation counterpart to the discovery questions in [Part 1 Section 7](#). It covers how to extract every implicit contract into TypeScript during migration.

Done correctly, this is a one-time investment that pays dividends across all 100+ screens: every `httpResource<Account[]>()`, every `signal<Transaction | null>()`, and every `Record<AccountId, Balance>` becomes self-documenting, catches breaking API changes at compile time, and eliminates entire classes of runtime bugs.

3.12.1 — Extracting Interfaces from AngularJS Data Shapes

AngularJS stored all data in `$scope` and `$http` responses as untyped JavaScript objects. The migration begins by reading those runtime shapes and converting them to TypeScript interfaces.

What is this pattern? The process of reading `$scope` assignments, `$http` response usages, and `ng-repeat` item references to infer the shape of each domain object, then writing an explicit `interface` in `libs/shared-models/`.

What does it do? Once an interface exists, it types every `httpResource<T>()`, `signal<T>()`, `computed<T>()`, and template expression that references that entity. TypeScript reports an error immediately when an API change removes or renames a field.

What problem does it resolve? In AngularJS, a backend renaming `account.accountNo` to `account.accountNumber` causes a silent runtime bug — the template renders blank and no error is thrown. With TypeScript interfaces, that same change causes a compile error at every reference point across all 100+ screens, in all tests, before a single line reaches production.

```
// — AngularJS (implicit shape — no types anywhere) —
.controller('AccountListCtrl', ['$scope', '$http', function ($scope, $http) {
  $scope.accounts = []; // type: any[]
  $http.get('/api/accounts').then(function (response) {
    $scope.accounts = response.data; // response.data: any
    // Properties used in template: account.id, account.accountNumber,
    //   account.balance, account.status, account.currency
    // Shape only known from API docs — or by running the app
  });
}]);

// — Step 1: Write the interface — (libs/shared-models/src/lib/account.interface.ts)
export interface Account {
  id: string;
  accountNumber: string;
  accountType: AccountType; // ← string literal union (see 3.12.2)
  balance: number;
  currency: string; // ISO 4217 code: 'GBP' | 'USD' | 'EUR'
  status: AccountStatus; // ← enum (see 3.12.2)
  ownerId: string;
  overdraftLimit: number | null; // null for SAVINGS/ISA — not undefined
  nickname?: string; // optional — server may omit this key
  createdAt: string; // ISO 8601 date string
  closedAt: string | null; // null while account is open
}

// — Step 2: Use interface in Angular 21 component —
import { httpResource } from '@angular/core';
import { Account } from '@banking/shared-models';

@Component({ /* ... */ })
export class AccountListComponent {
  // T = Account[] — value(), error(), isLoading() all fully typed
  protected readonly accountsResource = httpResource<Account[]>('/api/accounts');
}
```

Nullable vs optional — the key distinction:

```
// null    - the server sends the key with a null value:    { "closedAt": null }
// optional - the server omits the key entirely:            { "id": "123" }    (no "nickname" key)

export interface Account {
  closedAt: string | null; // ✅ Always present in JSON, value is null or an ISO date
  nickname?: string; // ✅ Key may not exist at all in the JSON response
}

// NEVER use undefined for API values - JSON.stringify strips undefined keys.
// undefined is for function parameters and truly absent values in TypeScript logic.
```

Nested interfaces — inline vs named:

```
// AngularJS: $scope.transaction.merchant.name, $scope.transaction.merchant.category
// Choose: inline (tightly coupled, small) vs named (reusable)

// Option A - Inline (acceptable for shapes used only inside one parent)
export interface Transaction {
  id: string;
  amount: number;
  merchant: { id: string; name: string; category: string };
}

// Option B - Named child interface (recommended when shape appears in multiple parents)
export interface Merchant {
  id: string;
  name: string;
  category: MerchantCategory; // ← string literal union
}

export interface Transaction {
  id: string;
  amount: number;
  merchant: Merchant; // ✅ Reusable - Merchant imported by StandingOrder, etc.
}
```

Generic API response wrappers — one interface for every paginated endpoint:

```
// Most banking APIs wrap responses: { data: T, meta: { total, page } }
// Define once in shared-models and use everywhere:

export interface ApiResponse<T> {
  data: T;
  errors: ApiError[];
}

export interface PagedResponse<T> {
  items: T[];
  totalCount: number;
  page: number;
  pageSize: number;
  hasNextPage: boolean;
}

// Usage:
protected readonly transactionsResource = httpResource<PagedResponse<Transaction>>(<
  () => `/api/accounts/${this.accountId()}/transactions?page=${this.page()}`
);
// ✅ transactionsResource.value()?.items      - inferred as Transaction[]
// ✅ transactionsResource.value()?.totalCount - inferred as number
// ✅ transactionsResource.value()?.hasNextPage - inferred as boolean
```

3.12.2 — Enums and String Literal Unions: Replacing Magic Strings

AngularJS codebases are full of magic strings — status flags, role names, permission keys, event names — scattered across controllers, services, and templates with no compiler enforcement. A typo (`'Frozen'` vs `'frozen'`) compiles fine and only fails at runtime.

What is this pattern? The process of finding every repeated string constant in the AngularJS codebase and replacing it with a TypeScript `enum` (runtime object) or string literal union (compile-time only), depending on usage.

What does it do? After the migration, every status comparison (`account.status === AccountStatus.Frozen`) is validated by TypeScript. If `AccountStatus.Frozen` doesn't exist, it's a compile error — not a silent runtime bug.

What problem does it resolve? AngularJS `.constant()` blocks and inline object literals (`var Status = { ACTIVE: 'active' }`) provide no IDE autocomplete, no compiler validation, and no guarantee the same string is used consistently. In a 100-screen app, the same status string appears in dozens of controllers and templates — any inconsistency is a runtime defect.

```
// — AngularJS magic string patterns to migrate —

// Pattern 1: .constant() – the intended solution, but no TypeScript awareness
.constant('AccountStatus', { ACTIVE: 'active', DORMANT: 'dormant', CLOSED: 'closed' });

// Pattern 2: Inline magic strings – the actual pattern in most codebases
if ($scope.account.status === 'active') { /* ... */ }
if ($scope.account.status === 'Frozen') { /* TYPO – will never match 'frozen' */ }

// Pattern 3: Role checks in templates (no autocomplete, no compiler check)
// ng-if="user.role === 'admin'"
// ng-if="user.role === 'relationship-manger'" ← typo, silent fail
```

Migration: Choose the right TypeScript construct:

```
// — libs/shared-models/src/lib/enums/account-status.enum.ts —

// Option A: String enum
// [✓] Use when: values must match exact API strings AND you need Object.values() at runtime
export enum AccountStatus {
  Active = 'active',
  Dormant = 'dormant',
  Closed = 'closed',
  Frozen = 'frozen',
}
// AccountStatus.Active === 'active' – true (string enum values are the strings themselves)
// Object.values(AccountStatus) – ['active', 'dormant', 'closed', 'frozen']

// Option B: String literal union
// [✓] Use when: values are serialised to/from JSON AND never iterated at runtime
export type AccountStatusType = 'active' | 'dormant' | 'closed' | 'frozen';
// Zero runtime output – purely a compile-time constraint
// Assigned directly from JSON: const status: AccountStatusType = apiResponse.status;

// Option C: const enum
// [✓] Use when: compile-time constant, never iterated, performance matters
export const enum AccountTier {
  Standard = 'STANDARD',
  Silver = 'SILVER',
  Gold = 'GOLD',
  Platinum = 'PLATINUM',
}
// Compiler inlines the value everywhere – no runtime object generated

// Decision guide:
// Serialised to/from JSON AND iterated at runtime → string enum
// Serialised to/from JSON, NOT iterated → string literal union (safer)
// Numeric ordinal (1st, 2nd, 3rd) → numeric enum
// Compile-time constant only → const enum
```

Using string enums in Angular 21 templates with `@switch`:

```

@Component({
  selector: 'app-account-status-badge',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @switch (status()) {
      @case (AccountStatus.Active) {
        <span class="text-green-700">Active</span>
      }
      @case (AccountStatus.Frozen) {
        <span class="text-red-700">Frozen</span>
      }
      @case (AccountStatus.Dormant) {
        <span class="text-amber-700">Dormant</span>
      }
      @case (AccountStatus.Closed) {
        <span class="text-neutral-500">Closed</span>
      }
    }
  `,
})
export class AccountStatusBadgeComponent {
  readonly status = input.required<AccountStatus>();
  protected readonly AccountStatus = AccountStatus; // expose enum to template
}

```

Role-based access using enums (replacing ng-if magic strings):

```

// libs/shared-auth/src/lib/user-role.enum.ts
export enum UserRole {
  Admin = 'admin',
  RelationshipManager = 'relationship-manager',
  CustomerService = 'customer-service',
  ReadOnly = 'read-only',
}

// Route guard — no magic strings
export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  return auth.userRole() === UserRole.Admin
    ? true
    : inject(Router).createUrlTree(['/unauthorised']);
};

// Template — no magic strings
// @if (auth.userRole() === UserRole.Admin) { <app-admin-panel /> }

```

3.12.3 — Generics: Making Signals, Services, and Resources Type-Safe

What is this pattern? Adding explicit type parameters (`<T>`) to every Angular 21 reactive API and service method so that the compiler tracks what type of data flows through each signal, resource, and service call.

What does it do? `httpResource<Account[]>('/api/accounts')` tells TypeScript that `value()` returns `Account[] | undefined` and `error()` returns `unknown`. Every template expression and computed signal that reads from this resource is then typed — no casting, no `any`.

What problem does it resolve? In AngularJS, all data is `any` — the compiler never checks that you are passing the right shape to the right template. When an API adds a field, nothing tells you which components benefit from it. When an API removes a field, nothing tells you which templates will break. Generics make these changes visible at compile time.

```

// — signal<T>() — always specify the type parameter —
// AngularJS: $scope.selectedAccount = null;
const selectedAccount = signal<Account | null>(null);
//                               ↑ T = Account | null — TypeScript knows the full shape

// — computed<T>() — usually inferred; explicit when type is complex —
const accountBalance = computed<number>(() => selectedAccount()?.balance ?? 0);
// ↑ explicit generic not required here (TypeScript infers number) but clarifies intent

const activeAccounts = computed<Account[]>(() =>
  accountsResource.value().filter(a => a.status === AccountStatus.Active) ?? []
);

// — httpResource<T>() — always type the response —
protected readonly accountsResource = httpResource<Account[]>('/api/accounts');
//                               ↑ T = Account[]
// accountsResource.value()      → Account[] | undefined
// accountsResource.isLoading()   → boolean
// accountsResource.error()       → unknown

// httpResource with PagedResponse wrapper:
protected readonly txnResource = httpResource<PagedResponse<Transaction>>(<
  () => `/api/accounts/${this.accountId()}/transactions`
);
// txnResource.value()?.items      → Transaction[]
// txnResource.value()?.totalCount → number

// — Generic injectable service — replacing untyped AngularJS factories —
@Injectable({ providedIn: 'root' })
export class EntityCacheService<T> extends { id: string } {
  private readonly cache = signal<Record<string, T>>({});

  set(entity: T): void {
    this.cache.update(current => ({ ...current, [entity.id]: entity }));
  }

  get(id: string): T | undefined {
    return this.cache()[id];
  }

  getAll(): T[] {
    return Object.values(this.cache());
  }
}

// Usage — type-safe per entity:
const accountCache = inject(EntityCacheService<Account>);
const txnCache     = inject(EntityCacheService<Transaction>);
// accountCache.get('acc-001') → Account | undefined (not any)
// txnCache.get('txn-001')     → Transaction | undefined

```

Generic error handling — typed HTTP errors:

```

export interface ApiError {
  code: string; // e.g. 'INSUFFICIENT_FUNDS', 'ACCOUNT_FROZEN'
  message: string;
  field?: string; // present for validation errors
  severity: 'error' | 'warning' | 'info';
}

// httpResource error is typed via the T-generic on the resource:
@Component({
  /* ... */
})
export class PaymentComponent {
  protected readonly paymentResource = httpResource<PaymentResponse>({
    url: () => '/api/payments',
    method: 'POST',
    body: () => this.buildPaymentRequest(),
  });

  protected get apiErrors(): ApiError[] {
    const err = this.paymentResource.error();
    // Cast from unknown — acceptable at API boundary only
    return (err as { errors?: ApiError[] })?.errors ?? [];
  }
}

```

3.12.4 — `Record<K, V>` : Replacing Plain Object Maps

AngularJS codebases frequently use plain objects as hash maps — `{}` indexed by a dynamic key with no type safety on either key or value. TypeScript's `Record<K, V>` gives these maps a precise type and validates both key and value at every access.

What is this pattern? Replacing every `var map = {}` lookup table, index map, feature flag object, and key→value store with `Record<K, V>` so the compiler knows what types the keys and values must be.

What does it do? `Record<string, Account>` tells TypeScript that every value is an `Account`. `Record<ColumnId, boolean>` tells TypeScript that every key must be one of the `ColumnId` string literal values — a typo in a key name is a compile error.

What problem does it resolve? In AngularJS, `accountsById['acc-001']` returns `any` — you can call `.balance` or `.balnce` and neither is a compile error. After migration to `Record<string, Account>`, `accountsById['acc-001'].balnce` is a compile error.

```
// — Account lookup map: id → Account —
// AngularJS: var accountsById = {}; // { [id]: account object }
// Angular 21:
const accountsById = signal<Record<string, Account>>({});

// Build from array (computed signal — always in sync with source):
const accountsResource = httpResource<Account[]>('/api/accounts');
const accountsLookup = computed<Record<string, Account>>(() =>
  Object.fromEntries(
    (accountsResource.value() ?? []).map(a => [a.id, a])
  )
);
// ✅ accountsLookup()['acc-001'] — inferred as Account | undefined
// ✅ accountsLookup()['acc-001']?.balance — COMPILE ERROR: 'balance' does not exist

// — Feature flags: flag name → boolean —
export type FeatureFlags = Record<string, boolean>;
protected readonly flagsResource = httpResource<FeatureFlags>('/api/feature-flags');
// flagsResource.value()?.['new-dashboard'] — boolean | undefined

// — Constrained key type: only known column IDs are valid keys —
export type ColumnId = 'date' | 'description' | 'amount' | 'balance' | 'status';
export type ColumnVisibility = Record<ColumnId, boolean>;

const columnVisibility = signal<ColumnVisibility>({
  date: true, description: true, amount: true, balance: true, status: false,
});
// columnVisibility().description — COMPILE ERROR: 'description' is not a valid key
// columnVisibility().date — boolean ✅

// — Form validation errors: field name → error message —
export type FormErrors = Partial<Record<keyof AccountForm, string>>;
// ↑ keys are constrained to AccountForm property names
const errors = signal<FormErrors>({});
errors.update(e => ({ ...e, accountNumber: 'Account number is required' }));
// errors().accountNumber — string | undefined ✅
// errors().typoField — COMPILE ERROR ✅
```

Record<K,V> vs Map<K,V> — when to choose each:

Scenario	Record	Map
Comes from / goes to a JSON API	✅ Always	❌ Map doesn't serialise to JSON
Key is a string literal union (closed set)	✅ Best choice	Works but less type-safe
Key is a runtime dynamic string	✅ OK	✅ Better for large sets
Used in Angular signals (needs immutable update)	✅ Spread-update works	✅ But needs <code>new Map()</code> to trigger signal
Needs <code>.forEach()</code> , <code>.entries()</code> , <code>.size</code>	❌ Use <code>Object.entries()</code>	✅ Native iteration
Needs ordered insertion	❌	✅ Map preserves insertion order

3.12.5 — interface vs type vs class — Decision Guide

AngularJS had no type system, so every TypeScript construct is new to a team coming from AngularJS. This decision guide prevents repeated arguments in code review by documenting the rule for every scenario.

What is this pattern? A team-agreed ruleset for which TypeScript construct to use in each situation, applied consistently across all 100+ migrated screens.

What does it do? Every piece of code in the migrated app uses the same construct for the same category of data. `interface` for domain entities, `type` for unions and utility types, `Record` for maps, `enum` for status constants, `class` only for DI-managed objects.

What problem does it resolve? Teams new to TypeScript frequently use `interface` and `type` interchangeably, leading to inconsistency. In code review, developers waste time debating construct choice rather than business logic. A documented decision guide ends that debate.

```
// — Decision table (add to team's coding-standards.md) —

// 1. INTERFACE – domain entity shapes that may be extended
export interface Account {
  id: string;
  balance: number;
  status: AccountStatus;
}
export interface SavingsAccount extends Account {
  interestRate: number;
} // extends works

// 2. TYPE – unions, intersections, utility types, shapes that will never be extended
export type AccountStatusType = 'active' | 'dormant' | 'closed'; // union
export type AccountSummary = Pick<Account, 'id' | 'accountNumber' | 'balance'>; // utility
export type AccountOrNull = Account | null; // union with null

// 3. RECORD – homogeneous key→value maps (never use { [key: string]: any })
export type AccountMap = Record<string, Account>;
export type ColumnVisibility = Record<ColumnId, boolean>; // constrained keys

// 4. ENUM – named constants with a closed value set (string enum for API values)
export enum AccountStatus {
  Active = 'active',
  Frozen = 'frozen',
}

// 5. CONST ENUM – build-time constants (no runtime object, smaller bundle)
export const enum SortDirection {
  Asc = 'asc',
  Desc = 'desc',
}

// 6. CLASS – only for DI-managed objects (services, components, guards, interceptors)
@Injectable({ providedIn: 'root' })
export class AccountService {
  /* ... */
}

// Anti-patterns to ban in code review:
// ✗ any → replace with correct interface or generic
// ✗ {} (as a type) → replace with Record<string, unknown> or named interface
// ✗ Object (as a type) → replace with Record<string, unknown>
// ✗ interface for a union → type is the correct construct
// ✗ class for a data-only shape → interface is the correct construct
```

Shared-models barrel export — the single import path for all domain types:

```
// libs/shared-models/src/lib/index.ts
export type { Account } from './account.interface';
export type { Transaction } from './transaction.interface';
export type { UserProfile } from './user-profile.interface';
export type { Merchant } from './merchant.interface';
export type { PaymentRequest, PaymentResponse } from './payment.interface';
export type { ApiResponse, PagedResponse } from './api-response.interface';
export type { FormErrors, ColumnVisibility } from './ui-state.types';
export { AccountStatus } from './enums/account-status.enum';
export { UserRole } from './enums/user-role.enum';
export type { AccountStatusType, ColumnId } from './enums/string-unions';

// Usage in any library or app:
// import { Account, AccountStatus, PagedResponse } from '@banking/shared-models';
```

Tracking any elimination as a migration metric:


```
# Count remaining implicit 'any' types — track this number sprint by sprint
npx tsc --noEmit 2>&1 | grep "implicitly has an 'any' type" | wc -l

# Target milestones:
# Sprint 1 (baseline): ~500 implicit any (all AngularJS interop layer)
# Sprint 5: <200 (core domain models typed)
# Sprint 10: <50 (services and guards typed)
# Go-live: 0 (zero any in production code)
```

3.12.6 — Backend-Driven Interface Generation: Eliminating Manual Type Extraction

Section 3.12.1 describes how to *manually* read `$scope` assignments and infer interfaces by inspection — this is the fallback for brownfield AngularJS apps with no formal API contracts. However, as an enterprise architect the **first question you should ask is: does the backend already define these contracts in a machine-readable format?** If it does, you can auto-generate TypeScript interfaces from the backend source of truth and eliminate both the manual work and the risk of drift.

Architect's rule: A TypeScript interface that was hand-written by a frontend developer is a *copy* of the API contract. A copy can become wrong the moment the backend changes and no one tells the frontend team. A *generated* interface is always correct because it is derived from the same source that the backend enforces.

Strategy selection matrix:

Backend technology	API contract format	Recommended tool	Result
Any backend with OpenAPI 3.x / Swagger	.yaml / .json spec	openapi-typescript	TypeScript interfaces + path types
Any backend with OpenAPI 3.x	.yaml / .json spec	@openapitools/openapi-generator-cli	Full Angular HttpClient service + models
GraphQL API	.graphql schema	graphql-code-generator	Types + typed Angular Apollo/urql hooks
NestJS in same Nx monorepo	TypeScript DTOs	Direct Nx library sharing	Zero-copy — import DTO class directly
Java / Spring Boot	Java DTO classes	typescript-generator (Maven plugin)	TypeScript interfaces from Java classes
.NET / ASP.NET Core	C# controller/DTO	NSwag	TypeScript interfaces + Angular services
gRPC / Protocol Buffers	.proto files	protoc-gen-ts	TypeScript interfaces + gRPC-Web client
No formal spec (brownfield)	Live HTTP responses	quicktype	Inferred interfaces from real JSON
Node.js backend — validation-first	Zod schemas	z.infer<T> + shared Nx lib	Type and runtime validation from one schema

Strategy 1 — OpenAPI / Swagger → openapi-typescript (Most Common for Banking APIs)

The OpenAPI specification is the industry standard for REST API contracts. Most enterprise API gateways (AWS API Gateway, Azure API Management, Kong, Apigee) can export an OpenAPI spec automatically. If your backend doesn't publish one yet, this is the first thing to request from the backend team.

What openapi-typescript generates:

- One TypeScript `interface` per schema component in the spec
- Path types (`paths['/api/accounts']['get']['responses']['200']['content']['application/json']`)
- Request body types, query parameter types, and header types — all in one generated file

```
# Install
npm install -D openapi-typescript

# Generate interfaces from a local spec file
npx openapi-typescript ./api-specs/banking-api.yaml --output src/types/api.generated.ts

# Generate from a live URL (useful when API gateway publishes the spec)
npx openapi-typescript https://api.bank.internal/openapi.json --output src/types/api.generated.ts

# Add to package.json for repeatable generation
```

```
// package.json
{
  "scripts": {
    "generate:api-types": "openapi-typescript ./api-specs/banking-api.yaml --output libs/shared-models/src/lib/api.generated.ts",
    "prebuild": "npm run generate:api-types"
  }
}
```

```
// — What openapi-typescript generates (api.generated.ts — DO NOT EDIT MANUALLY) —
export interface components {
  schemas: {
    Account: {
      id: string;
      accountNumber: string;
      accountType: 'CURRENT' | 'SAVINGS' | 'ISA';
      balance: number;
      currency: string;
      status: 'active' | 'dormant' | 'closed' | 'frozen';
      overdraftLimit: number | null;
      createdAt: string;
    };
    Transaction: {
      id: string;
      accountId: string;
      amount: number;
      direction: 'CREDIT' | 'DEBIT';
      description: string;
      postedAt: string;
    };
    PagedResponse: {
      items: components['schemas']['Account'][]; // ← typed by spec
      totalCount: number;
      hasNextPage: boolean;
    };
  };
}

// Path types — know exactly what each endpoint returns:
export interface paths {
  '/api/accounts': {
    get: {
      responses: {
        200: { content: { 'application/json': components['schemas']['Account'][] } };
        401: { content: { 'application/json': components['schemas']['ErrorResponse'] } };
      };
    };
  };
  '/api/accounts/{id}': {
    get: {
      parameters: { path: { id: string } };
      responses: {
        200: { content: { 'application/json': components['schemas']['Account'] } };
        404: { content: { 'application/json': components['schemas']['ErrorResponse'] } };
      };
    };
  };
}

// — Using generated types in Angular 21 components —
import type { components } from '@banking/shared-models';

type Account = components['schemas']['Account'];
type Transaction = components['schemas']['Transaction'];

// httpResource now typed from generated spec — guaranteed accurate:
protected readonly accountsResource = httpResource<Account[]>('/api/accounts');
```

CI enforcement — fail the build if interfaces drift from the spec:

```
# In CI pipeline (GitHub Actions / Azure DevOps):
# 1. Regenerate types from the latest spec
npm run generate:api-types

# 2. If the generated file changed, the API spec has changed without a frontend update
git diff --exit-code libs/shared-models/src/lib/api.generated.ts
# Exits non-zero if file changed → pipeline fails → team notified
```

Strategy 2 — GraphQL Schema → graphql-code-generator

If any of the backend services expose a GraphQL API (common in banking microservices for BFF patterns), `graphql-code-generator` generates TypeScript types and typed Angular service operations from the schema.

```
npm install -D @graphql-codegen/cli @graphql-codegen/typescript @graphql-codegen/typescript-operations
```

```
# codegen.yml
schema: 'https://api.bank.internal/graphql'
documents: 'src/**/*.graphql'
generates:
  libs/shared-models/src/lib/graphql.generated.ts:
    plugins:
      - typescript
      - typescript-operations
```

```
// After running: npx graphql-codegen

// Generated types (graphql.generated.ts — DO NOT EDIT):
export type Account = {
  __typename?: 'Account';
  id: Scalars['ID'];
  accountNumber: Scalars['String'];
  balance: Scalars['Float'];
  status: AccountStatus;
};

export enum AccountStatus { Active = 'ACTIVE', Dormant = 'DORMANT', Frozen = 'FROZEN' }

// Generated typed Angular service operation:
export const GetAccountsDocument = gql`query GetAccounts { accounts { id accountNumber balance status } }`;

// Usage in Angular 21 component — fully typed:
protected readonly accountsQuery = inject(GetAccountsGQL);
```

Strategy 3 — NestJS DTOs in the Same Nx Monorepo (Zero-Copy Sharing)

This is the **most powerful pattern for new development alongside migration**: the backend and frontend share TypeScript types directly through an Nx library. There is no copy, no generation step, no drift — both sides import from the same `@banking/api-contracts` library.

```
apps/
  banking-api/      ← NestJS backend
  banking-ui/       ← Angular 21 frontend
libs/
  api-contracts/    ← Shared library — imported by BOTH apps
    src/lib/
      account.dto.ts ← One interface, used by both NestJS validation and Angular httpResource
      transaction.dto.ts
      payment.dto.ts
```

```
// libs/api-contracts/src/lib/account.dto.ts
// Used by NestJS for validation AND by Angular for typing httpResource

import { IsString, IsNumber, IsEnum, IsOptional } from 'class-validator';

export enum AccountStatus { Active = 'active', Dormant = 'dormant', Frozen = 'frozen' }

export class AccountDto {
  @IsString() id!: string;
  @IsString() accountNumber!: string;
  @IsNumber() balance!: number;
  @IsEnum(AccountStatus) status!: AccountStatus;
  @IsOptional() @IsString() nickname?: string;
}

// — NestJS controller uses AccountDto for validation —
// @Get('/accounts/:id') async getAccount(): Promise<AccountDto> { ... }

// — Angular 21 component imports the SAME class as a type —
import type { AccountDto as Account } from '@banking/api-contracts';
//      ↑ import type – class used as interface, no runtime cost on frontend

protected readonly accountResource = httpResource<Account>{
  () => `/api/accounts/${this.accountId()}`
};
// ✅ NestJS returns AccountDto → Angular receives Account (same class) → perfect type alignment
```

Nx boundary enforcement — prevent runtime import of class-validator in the browser:

```
// .eslintrc.json – Nx module boundary rule
{
  "rules": {
    "@nx/enforce-module-boundaries": [
      "error",
      {
        "depConstraints": [
          {
            "sourceTag": "scope:web",
            "onlyDependOnLibsWithTags": ["scope:shared", "scope:web"],
            "notDependOnLibsWithTags": ["scope:api"]
          }
        ]
      }
    ]
  }
}

// This prevents the Angular app from accidentally importing NestJS-only code
// while still allowing shared DTOs in api-contracts
```

Strategy 4 — Java Spring Boot → typescript-generator Maven Plugin

Many enterprise banking backends are Java Spring Boot services. The `typescript-generator` Maven plugin reads your Spring Boot `@RestController` and DTO classes and outputs TypeScript interfaces.

```

<!-- pom.xml — add to your Spring Boot backend build -->
<plugin>
  <groupId>cz.habarta.typescript-generator</groupId>
  <artifactId>typescript-generator-maven-plugin</artifactId>
  <version>3.2.1263</version>
  <executions>
    <execution>
      <id>generate</id>
      <goals><goal>generate</goal></goals>
      <phase>process-classes</phase>
    </execution>
  </executions>
  <configuration>
    <jacksonLibrary>jackson2</jacksonLibrary>
    <classes>
      <class>com.bank.api.dto.AccountDto</class>
      <class>com.bank.api.dto.TransactionDto</class>
      <class>com.bank.api.dto.PaymentRequestDto</class>
    </classes>
    <outputFile>../frontend/libs/shared-models/src/lib/java-api.generated.ts</outputFile>
    <outputKind>module</outputKind>
    <generateEnums>true</generateEnums>
  </configuration>
</plugin>

```

```

// Generated output (java-api.generated.ts — DO NOT EDIT):
export interface AccountDto {
  id: string;
  accountNumber: string;
  balance: number;
  status: AccountStatus; // ← Java enum → TypeScript enum
  overdraftLimit: number | null; // ← Java Optional<BigDecimal> → T | null
}

export enum AccountStatus {
  ACTIVE = 'ACTIVE',
  DORMANT = 'DORMANT',
  CLOSED = 'CLOSED',
}

```

Java — Swagger Not Written: Auto-Generate OpenAPI from Existing Spring Annotations

This is the most common enterprise scenario: the Java backend works and has `@RestController`, `@GetMapping`, `@PostMapping` annotations — but no Swagger YAML was ever written and Swagger UI was never set up. **You do not need to write a single line of YAML.**

`springdoc-openapi` scans Spring's own annotations at startup and produces a complete OpenAPI 3.x spec automatically. It is a single dependency addition — no code modifications.

Path A — Backend team adds `springdoc-openapi` (one dependency, zero code changes):

```

<!-- Ask the Java backend team to add this to pom.xml -->
<!-- springdoc scans @RestController, @GetMapping, @PostMapping etc. automatically -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>

```

```

# application.yml — optional: expose spec at a stable predictable URL
springdoc:
  api-docs:
    path: /v3/api-docs # JSON spec available at this URL after restart
  swagger-ui:
    path: /swagger-ui.html # Human-readable UI
  packages-to-scan: com.bank.api.controllers

```

```
# Once the Java app restarts, download the generated spec:
curl http://localhost:8080/v3/api-docs -o api-specs/banking-api.json

# Then generate TypeScript on the Angular frontend (Strategy 1 flow):
npx openapi-typescript api-specs/banking-api.json \
  --output libs/shared-models/src/lib/api.generated.ts
```

What springdoc auto-detects — no annotations needed beyond what Spring already requires:

Java annotation	Generated OpenAPI element
<code>@RestController + @RequestMapping("/api/accounts")</code>	Path group
<code>@GetMapping("/{id}")</code>	GET /api/accounts/{id} operation
<code>@PostMapping + @RequestBody AccountDto dto</code>	Request body schema
<code>ResponseEntity<AccountDto> return type</code>	Response schema
<code>@PathVariable String id</code>	Path parameter
<code>@RequestParam(required=false) String filter</code>	Optional query parameter
<code>Java enum AccountStatus</code>	OpenAPI enum values
<code>@JsonProperty("accountNumber")</code>	Correct JSON field name
<code>@Nullable / Optional<T></code>	Nullable field

Path B — Cannot modify the backend build at all:

```
# 1. Check Spring Boot Actuator (running in most enterprise apps by default):
curl http://api-server/actuator/mappings | python -m json.tool
# Returns every @RequestMapping URL and HTTP method – gives you the complete endpoint list

# 2. Check if an API gateway (Kong, Apigee, AWS API GW) already publishes a spec
# independently of the backend:
curl https://api-gateway.bank.internal/openapi.json

# 3. Look for existing Postman/Insomnia collections – most teams have these
# even when they say 'we have no spec':
# Import collection → Postman → Collection → Export as OpenAPI 3.0 → save yaml
```

Path C — You have Java source access but no build access:

```
# java-to-typescript: parses .java source files, no Maven/Gradle required
npm install -g java-to-typescript

java-to-typescript \
  --src ./backend/src/main/java/com/bank/api/dto \
  --out ./frontend/libs/shared-models/src/lib/java-dtos.ts \
  --enums # Convert Java enums to TypeScript enums
```

Java → TypeScript type mapping reference:

Java type	TypeScript type	Notes
<code>String</code>	<code>string</code>	
<code>Integer</code> , <code>int</code> , <code>Long</code> , <code>long</code>	<code>number</code>	No integer type in JS
<code>Double</code> , <code>BigDecimal</code>	<code>number</code>	<code>BigDecimal</code> precision lost in JSON
<code>Boolean</code> , <code>boolean</code>	<code>boolean</code>	
<code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code>	<code>string</code>	ISO 8601 in JSON
<code>List<T></code> , <code>Set<T></code>	<code>T[]</code>	
<code>Map<String, V></code>	<code>Record<string, V></code>	
<code>Optional<T></code>	<code>T null</code>	Jackson serialises absent Optional as null
<code>@Nullable T</code>	<code>T null</code>	
Java <code>enum</code>	<code>enum</code> or string literal union	

Strategy 4b — .NET / ASP.NET Core Backend (Swagger Not Written)

For .NET backends the situation is identical to Java: controllers exist, DTOs exist, but no Swagger documentation was set up. Three options, ordered by how much backend cooperation is required.

Option A — Swashbuckle.AspNetCore (3 lines in Program.cs — the .NET standard):

`Swashbuckle.AspNetCore` is the official Microsoft-recommended Swagger library. It scans `[ApiController]` and `[HttpGet]` / `[HttpPost]` attributes exactly like springdoc does for Java. This is the recommended option because virtually every .NET developer already knows Swashbuckle.

```
# Ask the .NET team to add the NuGet package:
dotnet add package Swashbuckle.AspNetCore
```

```
// Program.cs — add exactly 3 lines, zero other code changes
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

app.UseSwagger(); // exposes /swagger/v1/swagger.json
app.UseSwaggerUI(); // exposes /swagger/index.html
```

```
# Download the generated spec from the running app:
curl http://localhost:5000/swagger/v1/swagger.json -o api-specs/dotnet-api.json

# Generate TypeScript interfaces on the Angular frontend:
npx openapi-typescript api-specs/dotnet-api.json \
  --output libs/shared-models/src/lib/dotnet-api.generated.ts
```

What Swashbuckle auto-detects from .NET attributes:

C# attribute / type	Generated OpenAPI element
[ApiController] + [Route("api/[controller]")]	Path group
[HttpGet("{id}")]	GET /api/accounts/{id} operation
[HttpPost] + AccountDto dto parameter	Request body schema
ActionResult<AccountDto> return type	Response schema
[FromRoute] string id	Path parameter
[FromQuery][Required] string filter	Required query parameter
C# enum AccountStatus	OpenAPI enum values
[JsonPropertyName("accountNumber")]	Correct JSON field name
Nullable AccountDto?	Nullable response
[ProducesResponseType(typeof(AccountDto), 200)]	Typed 200 response

Option B — NSwag Studio: Generate TypeScript from a .NET DLL with zero code changes:

NSwag Studio reads a compiled .dll by reflection — **no changes to the .NET source or project files at all**. The Angular team can run it entirely independently.

```
# Install NSwag CLI (cross-platform, no Visual Studio required):
npm install -g nswag

# Generate TypeScript from a compiled .NET DLL directly:
nswag aspnetcore2openapi \
  /assembly:./backend/bin/Release/BankingApi.dll \
  /output:api-specs/dotnet-from-dll.json

# Then generate TypeScript interfaces:
npx openapi-typescript api-specs/dotnet-from-dll.json \
  --output libs/shared-models/src/lib/dotnet-api.generated.ts

# OR: NSwag can generate a complete Angular HttpClient service in one step:
nswag openapi2tsclient \
  /input:api-specs/dotnet-from-dll.json \
  /output:libs/shared-models/src/lib/banking-api.client.ts \
  /template:Angular \
  /generateAngularCode:true \
  /httpClient:HttpClient
```

.NET → TypeScript type mapping reference:

C# type	TypeScript type	Notes
string	string	
int, long, short	number	
decimal, double	number	decimal precision not preserved in JSON
bool	boolean	
DateTime, DateTimeOffset	string	ISO 8601 with System.Text.Json
DateOnly	string	"2024-01-15" format
Guid	string	UUID string
List<T>, IEnumerable<T>	T[]	
Dictionary<string, V>	Record<string, V>	
T? (nullable reference type)	T null	With #nullable enable in C# 8+
Task<T> return type	T	Async unwrapped in generated type
C# enum	TypeScript enum or string union	Depends on [JsonConverter] config

Strategy 5 — No Spec, No Source Access: Multiple Extraction Paths

When the backend is a completely opaque legacy system — no OpenAPI spec, no shared DTOs, no source code access, backend team unavailable — you can still extract accurate TypeScript interfaces. This is the real brownfield situation that happens in most AngularJS migrations.

Architect's note: The correct first step is always to find the data the API *actually returns at runtime*, not to guess from the AngularJS controller code. Infer from real responses, not assumptions.

Path A — HAR File Capture (Browser DevTools — zero dependencies, works everywhere):

A HAR (HTTP Archive) file records every network request the browser made, including the full JSON response bodies. You capture it by using the legacy AngularJS application normally while DevTools is recording.

```
# Step 1: Open the AngularJS app in Chrome / Edge
# Step 2: DevTools → Network tab → check 'Preserve log'
# Step 3: Log in, navigate through every screen you need to type
# Step 4: Right-click the Network panel → 'Save all as HAR with content'
# Step 5: You now have a .har file containing every API response as JSON

# Extract JSON response bodies from the HAR file using har-extractor:
npm install -D har-extractor

npx har-extractor \
  --input session-capture.har \
  --output-dir samples/ \
  --filter-url 'api.bank.internal/api/'
# Outputs: samples/api-accounts-response.json, samples/api-transactions-response.json, etc.

# Generate interfaces from each extracted response:
npm install -D quicktype

npx quicktype --src-lang json --lang typescript \
  --top-level Account \
  --out libs/shared-models/src/lib/account.inferred.ts \
  samples/api-accounts-response.json

npx quicktype --src-lang json --lang typescript \
  --top-level Transaction \
  --out libs/shared-models/src/lib/transaction.inferred.ts \
  samples/api-transactions-response.json
```

HAR capture checklist — maximise coverage:

- Navigate every screen that loads data (account list, account detail, transaction history, payments)
- Test edge cases: accounts with overdraft, closed accounts, accounts with no transactions — these reveal nullable/optional fields that don't appear in the happy path
- Log in as multiple user roles (admin, read-only, relationship manager) — different roles may receive different response shapes
- Trigger validation errors on forms — captures error response shapes

Path B — Postman Proxy Recording (captures all traffic including background polling):

Postman's built-in proxy intercepts all HTTP/HTTPS traffic from any application, including the AngularJS app running in a browser. Unlike DevTools HAR, it captures background requests the user never directly triggers.

```
# Step 1: In Postman → Collections → New Collection → Start Proxy
# Set proxy port: 5555
# Start recording filter: api.bank.internal

# Step 2: Configure browser to use Postman as proxy:
# Chrome: Settings → System → Open proxy settings → Manual: localhost:5555
# OR: Launch Chrome with flag:
chrome.exe --proxy-server="http://localhost:5555"

# Step 3: Use the AngularJS app – every API call is captured in the Postman collection

# Step 4: Export the collection → Postman → Collection → Export → v2.1 format
# Step 5: Convert Postman collection to OpenAPI 3.0 spec:
npm install -D postman-to-openapi
npx p2o ./banking-api-collection.json -f ./api-specs/banking-api.yaml

# Step 6: Generate TypeScript from the OpenAPI spec:
npx openapi-typescript ./api-specs/banking-api.yaml \
  --output libs/shared-models/src/lib/api.generated.ts
```

For HTTPS APIs (most banking APIs): Install the Postman CA certificate as a trusted root in the browser to decrypt HTTPS traffic during recording. Remove it after the session.

Path C — AngularJS Source Analysis (find every API call from the code you already have):

The AngularJS source code you are migrating contains every `$http.get()`, `$http.post()`, and `$resource()` call — this is a complete index of all endpoints. Grep them, then fetch each one.

```
# Find every $http call and $resource URL in the codebase:
grep -rE '\$http\.(get|post|put|patch|delete)\s*(\' src/ --include='*.js' -h \
  | grep -oE "[\'\"](/api/[^\"]+)[\'\"]" \
  | sort -u > api-endpoints.txt

# Also find $resource patterns:
grep -rE '\$resource\s*(\' src/ --include='*.js' -h \
  | grep -oE "[\'\"](/api/[^\"]+)[\'\"]" \
  | sort -u >> api-endpoints.txt

cat api-endpoints.txt
# Output:
# /api/accounts
# /api/accounts/:id
# /api/accounts/:id/transactions
# /api/payments
# /api/users/profile
# ...

# Fetch a sample response for each static (non-parameterised) endpoint:
for endpoint in $(grep -v '/:' api-endpoints.txt); do
  filename=$(echo $endpoint | tr '/' '-' | sed 's/^-/ /')
  curl -s -H "Authorization: Bearer $DEV_TOKEN" \
    "https://api.bank.internal$endpoint" \
    > samples/${filename}.json
  echo "Captured: $endpoint → samples/${filename}.json"
done

# Batch-generate interfaces from all captured samples:
for jsonfile in samples/*.json; do
  name=$(basename $jsonfile .json | sed 's/-api-//;s/-/ /g' | awk '{for(i=1;i<=NF;i++) $i=toupper(substr($i,1,1))
tolower(substr($i,2)); print}' | tr -d ' ')
  outfile="libs/shared-models/src/lib/${name,,}.inferred.ts"
  npx quicktype --src-lang json --lang typescript \
    --top-level "$name" \
    --out "$outfile" \
    "$jsonfile"
  echo "Generated: $outfile"
done
```

Using the AngularJS `$resource` URL as parameter documentation:

```
// AngularJS source:
.factory('AccountService', function ($resource) {
  return $resource('/api/accounts/:id/transactions', { id: '@id' }, {
    query: { method: 'GET', isArray: true },
    create: { method: 'POST' }
  });
});
// This tells you:
// GET /api/accounts/:id/transactions → returns array → interface Transaction[]
// POST /api/accounts/:id/transactions → request body → interface TransactionRequest
```

Path D — quicktype directly from live endpoints (fastest for single endpoints):

```
npm install -D quicktype

# Fetch and generate in one command:
npx quicktype --src-lang json --lang typescript \
  --top-level Account \
  --out libs/shared-models/src/lib/account.inferred.ts \
  <(curl -s -H "Authorization: Bearer $DEV_TOKEN" \
    https://api.bank.internal/api/accounts)

# Batch across multiple endpoints:
declare -A ENDPOINTS=(
  [Account]="/api/accounts"
  [Transaction]="/api/accounts/ACC001/transactions"
  [UserProfile]="/api/users/profile"
  [Payment]="/api/payments"
)

for name in "${!ENDPOINTS[@]}"; do
  npx quicktype --src-lang json --lang typescript \
    --top-level "$name" \
    --out "libs/shared-models/src/lib/${name,,}.inferred.ts" \
    <(curl -s -H "Authorization: Bearer $DEV_TOKEN" \
      "https://api.bank.internal${ENDPOINTS[$name]}")
done
```

Reviewing all inferred output — the mandatory checklist:

```
// quicktype infers from what it saw — always review before committing:
export interface Account {
  id: string; // ✅ Correct
  accountNumber: string; // ✅ Correct
  balance: number; // ✅ Correct
  status: string; // ⚠️ Change to AccountStatus enum
  overdraftLimit: number | null; // ✅ Correct if null appeared in sample
  nickname?: string; // ✅ Correct if some records had it
  createdAt: string; // ⚠️ Consider branded type: type ISODateString = string
}

// Common corrections after quicktype inference:
// 1. status: string → status: AccountStatus (add enum)
// 2. Date strings → string (leave as string — ISO 8601 is not native Date in JSON)
// 3. All fields non-optional → review: any field that was null in one sample is T | null
// 4. Run quicktype against 5+ response samples — more samples → more accurate optionals
```

Strategy 6 — Zod Schemas Shared from Backend (Validation + Type in One)

For Node.js backends, Zod is increasingly used as the single source of both runtime validation and TypeScript types. Defining Zod schemas in a shared Nx library means the frontend gets both runtime validation (useful for `httpResource` response guards) and the TypeScript type simultaneously — with zero duplication.

```
// libs/api-contracts/src/lib/account.schema.ts – shared by backend AND frontend

import { z } from 'zod';

export const AccountStatusSchema = z.enum(['active', 'dormant', 'closed', 'frozen']);
export const AccountSchema = z.object({
  id: z.string().uuid(),
  accountNumber: z.string().regex(/^\d{8}$/),
  balance: z.number(),
  currency: z.string().length(3), // ISO 4217
  status: AccountStatusSchema,
  overdraftLimit: z.number().nullable(),
  nickname: z.string().optional(),
  createdAt: z.string().datetime(),
});

export const AccountsResponseSchema = z.array(AccountSchema);

// — TypeScript types derived automatically —
export type Account = z.infer<typeof AccountSchema>;
export type AccountStatus = z.infer<typeof AccountStatusSchema>;
// 'active' | 'dormant' | 'closed' | 'frozen' – no separate enum definition needed

// — Backend (NestJS / Express): use schema for request validation —
// const account = AccountSchema.parse(requestBody); // throws ZodError if invalid

// — Frontend (Angular 21): use schema to validate httpResource responses —
import { httpResource } from '@angular/core';
import { AccountSchema, Account } from '@banking/api-contracts';

@Component({
  /* ... */
})
export class AccountListComponent {
  // Validate the API response at runtime – catches backend contract breaks immediately
  protected readonly accountsResource = httpResource<Account[]>({
    url: '/api/accounts',
    // ⚠️ httpResource does not validate by default – add a transform to validate:
    // This pattern requires a custom resource wrapper; shown here conceptually
  });

  private parseAccounts(raw: unknown): Account[] {
    return AccountsResponseSchema.parse(raw); // throws if API response doesn't match schema
  }
}
```

Architect's Recommendation for the Banking Migration

In a typical enterprise banking migration where AngularJS talks to a mix of legacy Java services and newer Node.js microservices, use a **layered approach**:

API Layer	Backend Tech	Recommended Strategy	Priority
Core banking APIs (accounts, transactions)	Java Spring Boot — has Swagger	<code>openapi-typescript</code> from <code>/v3/api-docs</code>	Sprint 1
Core banking APIs (accounts, transactions)	Java Spring Boot — no Swagger	Add <code>springdoc-openapi</code> dependency → then <code>openapi-typescript</code>	Sprint 1
Core banking APIs (accounts, transactions)	Java Spring Boot — no build access	<code>typescript-generator</code> Maven plugin (if source access) or <code>quicktype</code> HAR capture	Sprint 1
.NET / ASP.NET Core — has Swagger	OpenAPI spec from <code>/swagger/v1/swagger.json</code>	<code>openapi-typescript</code>	Sprint 1
.NET / ASP.NET Core — no Swagger	C# controllers + DTOs, team available	Swashbuckle.AspNetCore (3 lines in Program.cs) → then <code>openapi-typescript</code>	Sprint 1
.NET / ASP.NET Core — no Swagger, no code change	Compiled <code>.dll</code> available	NSwag CLI → reads DLL by reflection → <code>openapi-typescript</code>	Sprint 1
New microservices	NestJS in Nx monorepo	Shared Nx <code>api-contracts</code> library	Sprint 1
API Gateway (Kong / Apigee / AWS API GW)	OpenAPI 3.x spec published by gateway	<code>openapi-typescript</code> + CI drift check	Sprint 2
Legacy services — no spec, no source, team unavailable	Any opaque backend	HAR capture → <code>quicktype</code> ; or Postman proxy → p2o → <code>openapi-typescript</code>	Sprint 2
Third-party / partner APIs	OpenAPI spec usually available	<code>openapi-typescript</code>	Sprint 3
Future new endpoints (greenfield)	Design-first	Write OpenAPI spec first → generate backend validation + frontend types	Ongoing

The overarching principle: treat the OpenAPI spec (or shared DTO library) as the **contract between backend and frontend teams**. Any interface file in `libs/shared-models/` that was written by hand and is not generated or imported from a backend source should be considered technical debt — scheduled for replacement in a future sprint once the backend publishes its contract.

Section 3.12 complete. TypeScript type system migration: interface extraction from `$scope` and `$http` shapes (3.12.1), enum and string literal union migration from magic strings (3.12.2), generics applied to signals, `HttpResponse`, and services (3.12.3), `Record<K, V>` replacing plain object maps (3.12.4), the `interface / type / class` decision guide (3.12.5), and backend-driven interface generation strategies (3.12.6).

Part 3 complete. The LLD covers the code-level patterns for every AngularJS artefact type: controllers → components, services → injectables, directives → components/directives/pipes, filters → pipes, route guards, Nx library structure, signal pattern mapping, security implementation, global state migration, `i18n` migration, RxJS migration strategy, Angular 21.x exclusive modern APIs, and TypeScript type system extraction (interfaces, types, generics, Records, enums).

Part 4 — Nx Workspace: Folder Structure & Setup for 100+ Screen Enterprise Migration

How to use this section: This is the hands-on blueprint for creating the Nx monorepo that houses your AngularJS app, the Angular 21 shell, the Angular 21 remote apps, and all shared libraries. Every command is executable. Copy and paste them in order to scaffold the entire workspace in under 30 minutes.

4.1 — Enterprise Context: 100+ Screen AngularJS App Characteristics

Before diving into folder structure, let's establish the baseline for a typical 100-screen enterprise AngularJS application:

Typical 100-screen app profile:

Dimension	Value Range	Example (Banking App)
Total screens	100–150	120 screens
AngularJS controllers	80–120	95 controllers
AngularJS services	50–80	65 services
Custom directives	30–50	42 directives
Filters	20–30	25 filters
Routes	100–150 (1:1 with screens)	120 routes
Third-party libraries	15–30	22 libraries
Forms	50–80	68 forms
Data tables/grids	20–40	28 grids
Charts/visualizations	15–25	18 chart components
API endpoints consumed	80–150	110 REST endpoints
Team size	6–15 engineers	8 engineers
Estimated migration duration	9–15 months	12 months (AI-assisted)

Feature clustering for migration phasing:

For a 100+ screen app, group screens into feature domains to enable parallel migration streams:

100 Screens → 8 Feature Domains (12–15 screens each)

Domain 1: Authentication & Profile	(10 screens)	
Domain 2: Dashboard & Home	(8 screens)	
Domain 3: Accounts & Transactions	(18 screens)	
Domain 4: Payments & Transfers	(15 screens)	
Domain 5: Loans & Credit	(12 screens)	
Domain 6: Reports & Statements	(20 screens)	
Domain 7: Settings & Preferences	(14 screens)	
Domain 8: Admin & User Management	(23 screens)	

Each domain becomes an Nx library in the Angular 21 remote app.
Each domain can be migrated independently by different squad members.

4.2 — Complete Nx Workspace Folder Structure

This is the final, production-ready structure after migration setup is complete. Everything you see here will be created by the commands in Section 4.3.

```

angularjs-to-ng21-migration/      ← Root Nx workspace
├── .nx/
│   ├── cache/                  ← Nx build cache
│   └── workspace-data/         ← Nx task graph data
├── apps/
│   ├── banking-shell/          ← Angular 21 shell (Module Federation host)
│   │   ├── src/
│   │   │   ├── app/
│   │   │   │   ├── app.config.ts    ← App providers (HttpClient, Router, etc.)
│   │   │   │   ├── app.routes.ts   ← Shell routes → load remotes OR legacy iframe
│   │   │   │   ├── app.ts           ← Root component
│   │   │   │   ├── components/
│   │   │   │   │   ├── nav/
│   │   │   │   │   │   ├── nav.component.ts    ← Top nav (same for all apps)
│   │   │   │   │   ├── header/
│   │   │   │   │   │   ├── header.component.ts ← Header with user menu
│   │   │   │   │   ├── footer/
│   │   │   │   │   │   ├── footer.component.ts
│   │   │   │   │   ├── legacy-frame/
│   │   │   │   │   │   ├── legacy-frame.component.ts ← iframe wrapper for AngularJS
│   │   │   │   │   └── services/
│   │   │   │   │       ├── feature-flag.service.ts ← Controls new vs legacy toggle
│   │   │   │   ├── index.html        ← Shell entry point
│   │   │   │   ├── main.ts
│   │   │   │   └── styles.css         ← Global TailwindCSS imports
│   │   ├── project.json              ← Nx project config
│   │   ├── module-federation.config.ts ← MF host config
│   │   └── tsconfig.app.json
│   └── banking-ng21/                ← Angular 21 remote (Module Federation remote)
│       ├── src/
│       │   ├── app/
│       │   │   ├── remote-entry/
│       │   │   │   ├── entry.component.ts ← MF entry point
│       │   │   ├── features/
│       │   │   │   ├── auth/          ← Domain 1: Auth features
│       │   │   │   │   ├── login/
│       │   │   │   │   │   ├── login.component.ts
│       │   │   │   │   │   └── login.component.html
│       │   │   │   │   ├── register/
│       │   │   │   │   ├── profile/
│       │   │   │   │   └── auth.routes.ts
│       │   │   │   ├── dashboard/     ← Domain 2: Dashboard
│       │   │   │   │   ├── dashboard.component.ts
│       │   │   │   │   └── dashboard.routes.ts
│       │   │   │   ├── accounts/      ← Domain 3: Accounts (18 screens)
│       │   │   │   │   ├── account-list/
│       │   │   │   │   ├── account-detail/
│       │   │   │   │   ├── transaction-history/
│       │   │   │   │   ├── account-statements/
│       │   │   │   │   └── accounts.routes.ts
│       │   │   │   ├── payments/      ← Domain 4: Payments (15 screens)
│       │   │   │   │   ├── payment-new/
│       │   │   │   │   ├── payment-history/
│       │   │   │   │   ├── transfer/
│       │   │   │   │   ├── beneficiaries/
│       │   │   │   │   └── payments.routes.ts
│       │   │   │   ├── loans/         ← Domain 5: Loans
│       │   │   │   ├── reports/       ← Domain 6: Reports (20 screens)
│       │   │   │   ├── settings/      ← Domain 7: Settings
│       │   │   │   ├── admin/         ← Domain 8: Admin (23 screens)
│       │   │   │   └── app.routes.ts   ← Remote routes (exposed to shell)
│       │   ├── index.html
│       │   ├── main.ts
│       │   └── styles.css
│       ├── project.json
│       ├── module-federation.config.ts ← MF remote config
│       └── tsconfig.app.json
└── banking-legacy/               ← AngularJS 1.8 app (unchanged)
    ├── app/
    │   ├── controllers/
    │   │   ├── accounts.controller.js
    │   │   ├── dashboard.controller.js
    │   │   └── ... (95 controllers)
    │   ├── services/
    │   │   └── ... (65 services)
    │   └── directives/

```


- └─ ... (42 directives)
- └─ filters/
 - └─ ... (25 filters)
- └─ views/
 - └─ ... (120 HTML templates)
- └─ app.js ← AngularJS module definition
- └─ index.html
- └─ webpack.config.js ← Nx wraps legacy Webpack build
- └─ project.json ← Nx project config
- └─ libs/
 - └─ shared-models/ ← TypeScript interfaces (shared by all)
 - └─ src/
 - └─ lib/
 - └─ account.interface.ts
 - └─ transaction.interface.ts
 - └─ user.interface.ts
 - └─ payment.interface.ts
 - └─ loan.interface.ts
 - └─ report.interface.ts
 - └─ index.ts ← Barrel export
 - └─ index.ts
 - └─ project.json
 - └─ tsconfig.lib.json
 - └─ shared-auth/ ← Authentication services + guards
 - └─ src/
 - └─ lib/
 - └─ auth.service.ts ← Signal-based; memory token storage
 - └─ auth.interceptor.ts ← HttpInterceptorFn
 - └─ xsrf.interceptor.ts ← CSRF protection
 - └─ auth.guard.ts ← CanActivateFn
 - └─ role.guard.ts ← Role-based guard
 - └─ index.ts
 - └─ index.ts
 - └─ project.json
 - └─ tsconfig.lib.json
 - └─ shared-ui/ ← Reusable Angular 21 components
 - └─ src/
 - └─ lib/
 - └─ button/
 - └─ button.component.ts
 - └─ button.component.html
 - └─ input/
 - └─ text-input.component.ts
 - └─ number-input.component.ts
 - └─ date-input.component.ts
 - └─ currency-input.component.ts ← 68 forms reuse this
 - └─ modal/
 - └─ modal.component.ts
 - └─ data-table/ ← Replaces ui-grid (28 grids use this)
 - └─ data-table.component.ts
 - └─ data-table.component.html
 - └─ data-table.component.spec.ts
 - └─ card/
 - └─ dropdown/
 - └─ alert/
 - └─ spinner/
 - └─ tokens/
 - └─ design-tokens.css ← CSS custom properties
 - └─ index.ts
 - └─ project.json
 - └─ tsconfig.lib.json
 - └─ shared-utils/ ← Pure functions, pipes, validators
 - └─ src/
 - └─ lib/
 - └─ pipes/
 - └─ currency-format.pipe.ts ← Replaces AngularJS filter
 - └─ date-format.pipe.ts
 - └─ safe-html.pipe.ts
 - └─ validators/
 - └─ account-number.validator.ts
 - └─ iban.validator.ts
 - └─ custom-validators.ts
 - └─ functions/
 - └─ format.ts
 - └─ parse.ts

```

├── event-bus.service.ts ← Replaces $rootScope.$broadcast
│   └── index.ts
├── index.ts
├── project.json
└── tsconfig.lib.json

├── feature-flags/ ← Feature flag service + directive
│   ├── src/
│   │   ├── lib/
│   │   │   ├── feature-flag.service.ts
│   │   │   ├── feature-flag.directive.ts
│   │   │   └── index.ts
│   │   └── index.ts
│   ├── project.json
│   └── tsconfig.lib.json
├── testing/ ← Testing utilities + fixtures
│   ├── src/
│   │   ├── lib/
│   │   │   ├── test-fixtures.ts ← Mock data for tests
│   │   │   ├── test-helpers.ts ← Setup helpers
│   │   │   └── index.ts
│   │   └── index.ts
│   ├── project.json
│   └── tsconfig.lib.json
├── tools/
│   ├── generators/ ← Custom Nx generators
│   │   ├── feature-domain/ ← Generate a new feature domain
│   │   │   ├── schema.json
│   │   │   └── index.ts
│   │   └── migrate-controller/ ← AI-assisted controller → component
│   │       ├── schema.json
│   │       └── index.ts
│   └── scripts/
│       ├── migration-progress.ts ← Dashboard: % migrated per domain
│       ├── analyze-angularjs.ts ← Count controllers/services/etc
│       └── validate-migration.ts ← Pre-commit: check for AngularJS imports in ng21
├── dist/ ← Build output (gitignored)
│   ├── apps/
│   │   ├── banking-shell/
│   │   ├── banking-ng21/
│   │   └── banking-legacy/
│   └── libs/
├── .github/
│   └── workflows/
│       ├── ci.yml ← Nx affected builds + tests
│       ├── deploy-shell.yml ← Deploy shell to CDN
│       ├── deploy-ng21.yml ← Deploy remote to CDN
│       └── deploy-legacy.yml ← Deploy legacy (until decommission)
├── .vscode/
│   ├── extensions.json ← Recommended extensions (Nx, Angular Language Service)
│   └── settings.json ← Workspace-level ESLint/Prettier config
├── nx.json ← Nx workspace config
├── package.json ← Root dependencies
├── tsconfig.base.json ← Shared TypeScript config
├── .eslintrc.json ← ESLint config + Nx boundary rules
├── .prettierrc ← Prettier config
├── vitest.config.ts ← Vitest workspace config
├── playwright.config.ts ← Playwright E2E config
└── README.md ← Migration guide + onboarding

```

Key structure decisions for 100+ screens:

1. **Feature domains as Nx libraries** — Each of the 8 domains is a folder under `banking-ng21/src/app/features/`. This allows parallel migration by different engineers.
2. **Shared libraries are mandatory** — With 100+ screens, code duplication becomes unmanageable without shared-ui, shared-models, shared-auth.
3. **Legacy app unchanged** — `banking-legacy/` is imported as-is; no changes to the AngularJS codebase.
4. **Module Federation** — Shell loads remote on-demand; remote can deploy independently.

4.3 — Step-by-Step Nx Workspace Setup Commands

Run these commands in order to scaffold the entire workspace. Estimated time: **25 minutes**.

Step 1: Create Nx workspace

```
# Create a new Nx workspace with Angular preset
npx create-nx-workspace@latest angularjs-to-ng21-migration \
  --preset=angular-monorepo \
  --appName=banking-shell \
  --style=css \
  --bundler=webpack \
  --standaloneApi=true \
  --routing=true \
  --e2eTestRunner=playwright \
  --nxCloud=skip

cd angularjs-to-ng21-migration
```

Step 2: Install required dependencies

```
# Angular 21 + Module Federation
npm install --save-dev @angular-architects/module-federation@^18.0.0
npm install --save-dev @nx/webpack@latest

# TailwindCSS
npm install --save-dev tailwindcss postcss autoprefixer
npx tailwindcss init -p

# Vitest for unit testing
npm install --save-dev vitest @vitest/ui @angular/platform-server
npm install --save-dev @testing-library/angular @testing-library/user-event

# Security + utilities
npm install rxjs@^7.8.0

# AngularJS (for legacy app only – NOT imported in Angular 21)
npm install --save angular@1.8.3 angular-route@1.8.3
```

Step 3: Generate Angular 21 remote app

```
# Generate the remote app (Module Federation remote)
nx g @nx/angular:host banking-shell --remotes=banking-ng21

# This creates both banking-shell (host) and banking-ng21 (remote) with Module Federation wired
```

Step 4: Generate shared libraries

```
# shared-models - TypeScript interfaces only
nx g @nx/js:lib shared-models --directory=libs/shared-models --unitTestRunner=vitest --strict

# shared-auth - Authentication services + guards
nx g @nx/angular:lib shared-auth \
  --directory=libs/shared-auth \
  --buildable \
  --standalone \
  --unitTestRunner=vitest

# shared-ui - Reusable UI components
nx g @nx/angular:lib shared-ui \
  --directory=libs/shared-ui \
  --buildable \
  --standalone \
  --unitTestRunner=vitest

# shared-utils - Pipes, validators, utilities
nx g @nx/angular:lib shared-utils \
  --directory=libs/shared-utils \
  --buildable \
  --standalone \
  --unitTestRunner=vitest

# feature-flags - Feature flag service
nx g @nx/angular:lib feature-flags \
  --directory=libs/feature-flags \
  --buildable \
  --standalone \
  --unitTestRunner=vitest

# testing - Test helpers + fixtures
nx g @nx/js:lib testing --directory=libs/testing --unitTestRunner=vitest
```

Step 5: Import legacy AngularJS app

```
# Create a folder for the legacy app
mkdir -p apps/banking-legacy/app

# Copy your existing AngularJS app into apps/banking-legacy/app/
# (Assuming your AngularJS app is in a separate repo or folder)
cp -r /path/to/existing/angularjs/app/* apps/banking-legacy/app/

# Create Nx project.json for legacy app
cat > apps/banking-legacy/project.json <<'EOF'
{
  "name": "banking-legacy",
  "$schema": "../../node_modules/nx/schemas/project-schema.json",
  "sourceRoot": "apps/banking-legacy",
  "projectType": "application",
  "targets": {
    "build": {
      "executor": "@nx/webpack:webpack",
      "options": {
        "webpackConfig": "apps/banking-legacy/webpack.config.js",
        "outputPath": "dist/apps/banking-legacy"
      },
      "configurations": {
        "production": {}
      }
    },
    "serve": {
      "executor": "@nx/webpack:dev-server",
      "options": {
        "buildTarget": "banking-legacy:build",
        "port": 3000
      }
    }
  },
  "tags": ["scope:app", "type:legacy"]
}
EOF

# Create a basic webpack.config.js for the legacy app
cat > apps/banking-legacy/webpack.config.js <<'EOF'
const path = require('path');

module.exports = {
  entry: './app/app.js',
  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, '../../dist/apps/banking-legacy'),
  },
  mode: 'development',
  devtool: 'source-map',
  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: ['@babel/preset-env'],
          },
        },
      },
      {
        test: /\.html$/,
        use: ['html-loader'],
      },
      {
        test: /\.css$/,
        use: ['style-loader', 'css-loader'],
      },
    ],
  },
};
EOF
```

Step 6: Configure Module Federation

```
# banking-shell/module-federation.config.ts (host)
cat > apps/banking-shell/module-federation.config.ts <<'EOF'
import { ModuleFederationConfig } from '@nx/webpack';

const config: ModuleFederationConfig = {
  name: 'banking-shell',
  remotes: ['banking-ng21'],
  shared: (name, config) => {
    // Share Angular core + RxJS as singletons
    if (name === '@angular/core' || name === '@angular/router' || name === 'rxjs') {
      return { singleton: true, strictVersion: true, requiredVersion: config.requiredVersion };
    }
    return false;
  },
};

export default config;
EOF

# banking-ng21/module-federation.config.ts (remote)
cat > apps/banking-ng21/module-federation.config.ts <<'EOF'
import { ModuleFederationConfig } from '@nx/webpack';

const config: ModuleFederationConfig = {
  name: 'banking-ng21',
  exposes: {
    './Routes': 'apps/banking-ng21/src/app/app.routes.ts',
  },
  shared: (name, config) => {
    if (name === '@angular/core' || name === '@angular/router' || name === 'rxjs') {
      return { singleton: true, strictVersion: true, requiredVersion: config.requiredVersion };
    }
    return false;
  },
};

export default config;
EOF
```

Step 7: Configure ESLint boundary rules (prevent apps from importing each other)

```
# Update .eslintrc.json
cat > .eslintrc.json <<'EOF'
{
  "root": true,
  "ignorePatterns": ["**/*"],
  "plugins": ["@nx"],
  "overrides": [
    {
      "files": ["*.ts", "*.tsx"],
      "extends": ["plugin:@nx/typescript"],
      "rules": {
        "@nx/enforce-module-boundaries": [
          "error",
          {
            "enforceBuildableLibDependency": true,
            "allow": [],
            "depConstraints": [
              {
                "sourceTag": "scope:app",
                "onlyDependOnLibsWithTags": ["scope:lib"]
              },
              {
                "sourceTag": "scope:lib",
                "onlyDependOnLibsWithTags": ["scope:lib"]
              },
              {
                "sourceTag": "type:legacy",
                "onlyDependOnLibsWithTags": []
              }
            ]
          }
        ]
      }
    }
  ]
}
EOF
```

Step 8: Configure TailwindCSS

```
# tailwind.config.js
cat > tailwind.config.js <<'EOF'
module.exports = {
  content: [
    './apps/banking-shell/src/**/*.html,ts',
    './apps/banking-ng21/src/**/*.html,ts',
    './libs/shared-ui/src/**/*.html,ts',
  ],
  theme: {
    extend: {
      colors: {
        primary: '#1a56db',
        secondary: '#6b7280',
        danger: '#dc2626',
        success: '#16a34a',
      },
    },
  },
  plugins: [],
};
EOF

# Add TailwindCSS to shell styles
cat > apps/banking-shell/src/styles.css <<'EOF'
@import 'tailwindcss/base';
@import 'tailwindcss/components';
@import 'tailwindcss/utilities';
EOF

# Add TailwindCSS to remote styles
cat > apps/banking-ng21/src/styles.css <<'EOF'
@import 'tailwindcss/base';
@import 'tailwindcss/components';
@import 'tailwindcss/utilities';
EOF
```

Step 9: Configure Vitest workspace

```
cat > vitest.config.ts <<'EOF'
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: ['./vitest.setup.ts'],
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      exclude: [
        'node_modules/',
        'dist/',
        '**/*.spec.ts',
        '**/*.config.ts',
      ],
    },
  },
});
EOF
```


Step 10: Verify workspace setup

```
# List all projects in workspace
nx show projects

# Expected output:
# banking-shell
# banking-ng21
# banking-legacy
# shared-models
# shared-auth
# shared-ui
# shared-utils
# feature-flags
# testing

# Check project graph
nx graph

# Build all apps
nx run-many --target=build --all

# Run all unit tests
nx run-many --target=test --all
```

4.4 — Feature Domain Scaffolding Pattern

For each of the 8 feature domains (auth, dashboard, accounts, payments, loans, reports, settings, admin), create a consistent folder structure under `apps/banking-ng21/src/app/features/`.

Domain scaffolding template

```
# Generate a new feature domain (example: accounts domain with 18 screens)
nx g @nx/angular:library --name=accounts \
  --directory=apps/banking-ng21/src/app/features/accounts \
  --standalone \
  --routing

# This creates the folder structure:
# features/accounts/
#   ├── account-list/      ← Screen 1
#   ├── account-detail/   ← Screen 2
#   ├── transaction-history/ ← Screen 3
#   ├── ... (15 more screens)
#   └── accounts.routes.ts ← Lazy routes for this domain
```

Custom Nx generator for rapid domain scaffolding

```
// tools/generators/feature-domain/index.ts
import { Tree, formatFiles, generateFiles } from '@nx/devkit';
import * as path from 'path';

interface FeatureDomainSchema {
  name: string;
  screens: string[]; // e.g. ['account-list', 'account-detail', ...]
}

export default async function (tree: Tree, schema: FeatureDomainSchema) {
  const domainPath = `apps/banking-ng21/src/app/features/${schema.name}`;

  // Generate folder for each screen
  for (const screen of schema.screens) {
    const screenPath = path.join(domainPath, screen);
    generateFiles(tree, path.join(__dirname, 'files'), screenPath, {
      name: screen,
      className: toPascalCase(screen),
    });
  }

  // Generate routes file
  const routesContent = `
import { Routes } from '@angular/router';

${schema.screens.map((s) => `import { ${toPascalCase(s)}Component } from './${s}/${s}.component';`).join('\n')}

export const ${schema.name.toUpperCase()}_ROUTES: Routes = [
${schema.screens.map((s) => `  { path: '${s}', component: ${toPascalCase(s)}Component },`).join('\n')}
];
`;
  tree.write(path.join(domainPath, `${schema.name}.routes.ts`), routesContent);

  await formatFiles(tree);
}

function toPascalCase(str: string): string {
  return str
    .split('-')
    .map((w) => w[0].toUpperCase() + w.slice(1))
    .join('');
}
```

Usage:

```
# Generate the accounts domain with all 18 screens
nx g feature-domain accounts \
  --screens=account-list,account-detail,transaction-history,account-statements,\
  balance-summary,account-settings,close-account,reopen-account,\
  account-alerts,account-documents,account-beneficiaries,account-limits,\
  account-fees,account-interest,account-overdraft,account-cards,\
  account-cheques,account-standing-orders

# This generates all 18 component folders + routes file in ~5 seconds
```

4.5 — CI/CD Setup for Nx Monorepo

GitHub Actions workflow for affected builds

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  affected:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Lint affected projects
        run: npx nx affected --target=lint --base=origin/main

      - name: Test affected projects
        run: npx nx affected --target=test --base=origin/main --coverage

      - name: Build affected projects
        run: npx nx affected --target=build --base=origin/main --configuration=production

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/**/coverage-final.json
```

Deployment workflow for shell

```
# .github/workflows/deploy-shell.yml
name: Deploy Shell

on:
  push:
    branches: [main]
    paths:
      - 'apps/banking-shell/**'
      - 'libs/**'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - run: npm ci
      - run: npx nx build banking-shell --configuration=production

      - name: Deploy to Azure Static Web Apps
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN }
          repo_token: ${ secrets.GITHUB_TOKEN }
          action: 'upload'
          app_location: 'dist/apps/banking-shell'
          skip_app_build: true
```

4.6 — Migration Progress Dashboard

Track migration progress across all 8 domains with a custom Nx script.

```

// tools/scripts/migration-progress.ts
import * as fs from 'fs';
import * as path from 'path';

interface DomainProgress {
  domain: string;
  totalScreens: number;
  migratedScreens: number;
  percentage: number;
}

const DOMAINS = [
  { name: 'auth', totalScreens: 10 },
  { name: 'dashboard', totalScreens: 8 },
  { name: 'accounts', totalScreens: 18 },
  { name: 'payments', totalScreens: 15 },
  { name: 'loans', totalScreens: 12 },
  { name: 'reports', totalScreens: 20 },
  { name: 'settings', totalScreens: 14 },
  { name: 'admin', totalScreens: 23 },
];

function countMigratedScreens(domainName: string): number {
  const domainPath = path.join(__dirname, '../../../apps/banking-ng21/src/app/features', domainName);
  if (!fs.existsSync(domainPath)) return 0;

  const entries = fs.readdirSync(domainPath, { withFileTypes: true });
  return entries.filter(
    (e) =>
      e.isDirectory() && fs.existsSync(path.join(domainPath, e.name, `${e.name}.component.ts`)),
  ).length;
}

function generateReport(): DomainProgress[] {
  return DOMAINS.map((d) => ({
    domain: d.name,
    totalScreens: d.totalScreens,
    migratedScreens: countMigratedScreens(d.name),
    percentage: Math.round((countMigratedScreens(d.name) / d.totalScreens) * 100),
  }));
}

function printReport(progress: DomainProgress[]): void {
  console.log('\n=== MIGRATION PROGRESS REPORT ===\n');
  console.log('Domain | Migrated | Total | Progress');
  console.log('-----|-----|-----|-----');
  progress.forEach((p) => {
    const bar =
      '█'.repeat(Math.floor(p.percentage / 5)) + '░'.repeat(20 - Math.floor(p.percentage / 5));
    console.log(
      `${p.domain.padEnd(16)} | ${String(p.migratedScreens).padStart(8)} | ${String(p.totalScreens).padStart(5)} | ${bar}
      ${p.percentage}%`,
    );
  });

  const total = progress.reduce((sum, p) => sum + p.totalScreens, 0);
  const migrated = progress.reduce((sum, p) => sum + p.migratedScreens, 0);
  const overall = Math.round((migrated / total) * 100);
  console.log('-----|-----|-----|-----');
  console.log(
    `OVERALL | ${String(migrated).padStart(8)} | ${String(total).padStart(5)} | ${overall}%\n`,
  );
}

const report = generateReport();
printReport(report);

```

Run the progress dashboard:

```
npx ts-node tools/scripts/migration-progress.ts
```

Example output:

```
# === MIGRATION PROGRESS REPORT ===
#
# Domain          | Migrated | Total | Progress
# -----|-----|-----|-----
# auth            | 10       | 10    | ██████████ 100%
# dashboard       | 8        | 8     | ██████████ 100%
# accounts        | 12       | 18    | ██████████ 67%
# payments        | 3        | 15    | ████████ 20%
# loans           | 0        | 12    | ██████████ 0%
# reports         | 0        | 20    | ██████████ 0%
# settings        | 0        | 14    | ██████████ 0%
# admin           | 0        | 23    | ██████████ 0%
# -----|-----|-----|-----
# OVERALL         | 33       | 120   | ████████ 28%
```

Part 4 complete. The Nx workspace is now scaffolded with all apps, libraries, Module Federation wiring, CI/CD pipelines, and a migration progress dashboard. The workspace is ready for development.

Ready for Part 5? Part 5 covers **AI Prompts, MCP Servers (Angular 21, Angular Material), agents.md, skills.md** — the complete AI-assisted development setup using GitHub Copilot, custom agents, and MCP servers for automated migration workflows and live Angular API queries.

Part 5 — AI Prompts, MCP Servers, agents.md & skills.md

How to use this section: This part has three major components: (1) **MCP Server Setup** for live Angular 21 and Angular Material API queries, (2) **Migration Prompts** optimized for GitHub Copilot Chat, and (3) **Custom Agents and Skills** for team-wide AI customization. MCP servers eliminate the need to manually specify Angular 21 patterns — the AI can query the official API directly.

5.0 — MCP Server Setup for Angular 21 Migration

What are MCP Servers?

Model Context Protocol (MCP) servers provide live, authoritative API documentation to AI tools. Instead of relying on the AI's training data (which may be outdated), MCP servers let the AI **query the latest Angular 21 API in real-time**.

Benefits for Migration:

- ✓ No need to manually specify "use signal(), not BehaviorSubject"
- ✓ AI always uses current Angular 21 APIs (signals, input(), output(), @if, @for)
- ✓ Reduces hallucinations (AI can verify APIs exist before generating code)
- ✓ Material Design components auto-discovered if using Angular Material

5.0.1 — Angular 21 MCP Server Setup

Purpose: Provides live access to Angular 21 API documentation (signals, components, directives, router, forms, HttpClient).

Installation:

```
// .vscode/mcp-settings.json (or global MCP config)
{
  "mcpServers": {
    "angular-cli": {
      "command": "npx",
      "args": ["-y", "@angular/cli-mcp-server"],
      "env": {
        "ANGULAR_VERSION": "21"
      }
    }
  }
}
```

Available Tools (Auto-discovered by Copilot):

Tool	Purpose	Example Query
mcp_angular-cli_get_best_practices	Returns Angular 21 best practices for signals, OnPush, inject()	"What's the Angular 21 way to handle form state?"
mcp_angular-cli_search_documentation	Searches official Angular docs	"How do I use toSignal with HttpClient?"
mcp_angular-cli_ai_tutor	Explains Angular concepts	"Explain computed() vs effect()"

How to Use in Prompts:

Instead of:

✗ Migrate this controller to Angular 21 with signals, inject(), @if/@for

Simply:

✓ @angular-cli Migrate this AngularJS controller to Angular 21 using current best practices

<paste controller code>

The MCP server will automatically provide:

- Current signal APIs
- Modern control flow syntax
- inject() pattern
- OnPush change detection

5.0.2 — Angular Material MCP Server Setup (Optional)

When to use: If your migration uses Angular Material for UI components.

Installation:

```
// .vscode/mcp-settings.json
{
  "mcpServers": {
    "angular-cli": {
      /* ... */
    },
    "angular-material": {
      "command": "npx",
      "args": ["-y", "@angular/material-mcp-server"],
      "env": {
        "MATERIAL_VERSION": "21"
      }
    }
  }
}
```

Available Components (Auto-discovered):

- MatButton, MatInput, MatSelect, MatDialog, MatTable, MatPaginator, MatSort, MatDatepicker, MatAutocomplete, etc.

Example Prompt:

```
@angular-material Generate a data table component for account list with:  
- Sortable columns  
- Pagination (50 items per page)  
- Row selection  
- Export to CSV button  
  
Use Angular 21 signals for state.
```

MCP server will provide:

- Correct Material imports
- Signal-based MatTableDataSource alternative
- Proper column definitions
- Accessibility attributes (aria-labels)

5.0.3 — Custom Banking MCP Server (Advanced)

For enterprise teams, create a **custom MCP server** that knows your domain models and business rules.

Example: Banking Domain MCP Server

```
// tools/mcp-server/banking-domain-server.ts  
import { MCPServer } from '@modelcontextprotocol/sdk';  
  
const server = new MCPServer({  
  name: 'banking-domain',  
  version: '1.0.0',  
});  
  
server.addTool({  
  name: 'get_account_validation_rules',  
  description: 'Returns validation rules for account numbers, routing numbers, amounts',  
  handler: async () => {  
    return {  
      accountNumber: { pattern: /^\d{10,12}$/, message: 'Must be 10-12 digits' },  
      routingNumber: { pattern: /^\d{9}$/, message: 'Must be 9 digits' },  
      amount: { min: 0.01, max: 100000, message: 'Must be between $0.01 and $100,000' },  
    };  
  },  
});  
  
server.addTool({  
  name: 'get_shared_models',  
  description: 'Returns TypeScript interfaces from libs/shared-models',  
  handler: async () => {  
    return {  
      Account: 'interface Account { id: string; accountNumber: string; balance: number; ... }',  
      Transaction: 'interface Transaction { id: string; amount: number; date: Date; ... }',  
      User: 'interface User { id: string; name: string; email: string; roles: string[]; ... }',  
    };  
  },  
});  
  
server.listen(3001);
```

Usage:

```
@banking-domain Generate a payment form component that validates inputs using our domain rules
```

AI will automatically apply your custom validation patterns.

5.0.4 — MCP Server Best Practices

1. Always tag MCP servers in prompts:

```
@angular-cli @angular-material Generate a dialog component for payment confirmation
```


2. Let MCP servers discover APIs (don't over-specify):

✗ Bad:

```
Use signal(), computed(), effect(), inject(), input(), output(), @if, @for, OnPush, toSignal()
```

✓ Good:

```
@angular-cli Use Angular 21 best practices
```

3. Verify MCP server is active:

```
# Check if MCP servers are running
code --list-extensions | grep mcp

# View MCP server logs
tail -f ~/.vscode/extensions/github.copilot-*/mcp.log
```

4. Combine MCP with custom agents (see Section 5.7):

```
# .github/agents/migration-expert.agent.md
---
name: migration-expert
mcp_servers:
  - angular-cli
  - angular-material
  - banking-domain
---
```

5.1 — GitHub Copilot Chat: Migration Discovery Prompts

Use these prompts during **Phase 1** of the migration (discovery and inventory). Run them against the `banking-legacy/` folder.

Prompt 5.1.1 — Full codebase inventory

```
@workspace I have a legacy AngularJS 1.8 application at apps/banking-legacy/.
```

Analyze and produce a structured inventory:

1. Controllers (name, file, \$scope property count)
2. Services/factories (type, name, file)
3. Directives (name, type, file)
4. Filters (name, usage count, file)
5. Routes (all states with templates)
6. Third-party dependencies (from package.json/bower.json)
7. HTTP endpoints (all \$http calls)

Format as markdown tables. Flag controllers >200 lines or >15 \$scope properties as HIGH COMPLEXITY.

Prompt 5.1.2 — Controller complexity scoring

```
@workspace
```

Score every AngularJS controller in apps/banking-legacy/app/controllers/ using this complexity matrix:

- Lines of code: <100 = 1pt, 100-200 = 2pt, 200-400 = 3pt, >400 = 4pt
- \$scope properties: <10 = 1pt, 10-20 = 2pt, 20-30 = 3pt, >30 = 4pt
- \$watch calls: 0 = 0pt, 1-3 = 1pt, 4-6 = 2pt, >6 = 3pt
- \$rootScope usage: none = 0pt, read only = 1pt, broadcasts = 2pt
- Nested callbacks / promise chains: 0 = 0pt, 1-3 = 1pt, >3 = 2pt
- Direct DOM manipulation: none = 0pt, present = 3pt (immediate flag)

Produce a ranked table: Controller Name | Score | Risk Level (Low/Medium/High/Critical). Suggest migrating Low/Medium first to build team momentum.

Prompt 5.1.3 — Third-party library audit

@workspace

For every third-party library found in apps/banking-legacy/ (check bower.json, package.json, and any CDN script tags in index.html):

1. Identify the Angular 21 equivalent or replacement
2. Rate migration effort: Drop-in swap / Wrapper needed / Full rewrite / No equivalent
3. Flag any library that requires an ADR (Architectural Decision Record)
4. For any library with "No equivalent", suggest whether to: build custom, vendor-fork, or keep in AngularJS-only screens indefinitely

Output as a markdown table: Library | Version | Purpose | Angular 21 Replacement | Effort | Requires ADR?

Prompt 5.1.4 — \$rootScope event mapping

@workspace

Search all files in apps/banking-legacy/ for \$rootScope.\$broadcast, \$rootScope.\$emit, \$rootScope.\$on, and \$scope.\$on.

For each usage:

1. List the event name (string literal), source file, and direction (broadcast/emit)
2. Find every listener for the same event name
3. Map the full event flow: Producer → Event Name → Consumer(s)

Output as a Mermaid sequence diagram followed by a table mapping each event to its proposed Angular 21 replacement: EventBusService typed event | NgRx Action | Signal-based | BehaviorSubject

Flag any event that carries mutable objects by reference – these need special handling in the migration.

5.2 — GitHub Copilot Chat: Code Migration Prompts

Use these prompts during **Phase 2** (active migration). Run them file-by-file as you migrate each controller, service, or directive.

Prompt 5.2.1 — Controller → Standalone Component

Convert the following AngularJS 1.8 controller to an Angular 21 standalone component.

[PASTE CONTROLLER CODE HERE]

Requirements:

- Use Angular 21 standalone component (no NgModule)
- Replace \$scope with signals: writable state → signal(), derived state → computed()
- Replace \$watch with effect() – use only when truly necessary (side effects only)
- Replace \$http with inject(HttpClient) – use toSignal() for observable-to-signal conversion
- Replace constructor injection with inject() function
- Use input() / output() for component API, not @Input() / @Output()
- Apply OnPush change detection strategy
- Replace *ngIf with @if, *ngFor with @for, *ngSwitch with @switch in the template
- Use @defer for any heavy child component
- Preserve all business logic exactly – do not change validation rules, HTTP endpoints, or error messages
- Add ARIA attributes for all interactive elements (buttons, inputs, selects)
- Output the TypeScript component and the HTML template separately

After the code, list any assumptions you made and any AngularJS patterns you could not automatically convert.

Prompt 5.2.2 — Service / Factory → Injectable

Convert the following AngularJS service/factory to an Angular 21 injectable service.

[PASTE SERVICE CODE HERE]

Requirements:

- Use `@Injectable({ providedIn: 'root' })` unless it must be scoped (explain if so)
- Replace `$http` with `inject(HttpClient)`
- Replace `$q` promises with RxJS Observables or `async/await`
- Replace `$rootScope.$broadcast` with the `EventBusService` pattern (typed discriminated union)
- Expose reactive state as readonly signals where the service holds mutable state
- Use `inject()` for all dependencies – no constructor parameters
- Preserve all public method signatures exactly (same method names and parameter types)
- Add TypeScript strict types for all parameters and return values
- Flag any method that accesses browser globals (`localStorage`, `sessionStorage`, `document`) – wrap in `isPlatformBrowser()` for SSR safety

List each public method in the original and confirm it exists with the same signature in the output.

Prompt 5.2.3 — Directive → Component / Attribute Directive / Pipe

Convert the following AngularJS directive to its Angular 21 equivalent.

[PASTE DIRECTIVE CODE HERE]

First, classify it:

- If it generates substantial HTML structure → convert to standalone Component
- If it only adds behaviour to a host element (no template) → convert to Attribute Directive
- If it only transforms a display value → convert to standalone Pipe (`pure: true`)

Apply the appropriate conversion with:

- `inject()` for all dependencies
- `input()` / `output()` for bindings
- Strict TypeScript types
- `OnPush` for components
- All `link/compile` logic moved to `ngOnInit` / `ngAfterViewInit` / host listeners as appropriate
- No use of `ElementRef.nativeElement` for DOM manipulation – use `Renderer2` instead

Explain which classification you chose and why.

Prompt 5.2.4 — Route configuration migration

@workspace

Migrate the AngularJS routing configuration from `apps/banking-legacy/app/app.js` (or the `ui-router` config file) to Angular 21 lazy-loaded routes.

Requirements:

- Each AngularJS state/route becomes a lazy-loaded Angular 21 route using `loadComponent()`
- If multiple routes share a layout, group them under a parent route with a layout component
- Migrate any resolve blocks to Angular 21 `ResolveFn`
- Migrate `ui-router` parameter handling to `ActivatedRoute.params` signal
- Add `CanActivateFn` guards where the original had access restrictions
- Produce the complete `app.routes.ts` file with all 120 routes
- Ensure the route paths exactly match the original AngularJS paths (to preserve bookmarks and links)

Output the full routes file. Group routes by the 8 feature domains identified earlier.

Prompt 5.2.5 — Form migration (AngularJS ng-model → Angular 21 signals)

@angular/cli Convert this AngularJS form to Angular 21 using best form strategy.

[PASTE FORM CODE]

Choose optimal pattern:

- Simple (<10 fields): signal() per field + computed() validation
- Complex (10+ fields): model() with two-way binding
- Dependent fields: linkedSignal() for reactive dependencies

Requirements:

- Preserve all validation rules exactly
- Use computed() for errors (null when valid)
- Accessibility: aria-describedby, role="alert"
- Template uses @if for error display

Prompt 5.2.6 — \$http interceptor migration

Convert the following AngularJS \$httpProvider interceptor to an Angular 21 HttpInterceptorFn.

[PASTE INTERCEPTOR CODE]

Requirements:

- Use functional interceptor pattern: export const myInterceptor: HttpInterceptorFn = (req, next) => { ... }
- Use inject() inside the function – not constructor injection
- Preserve all request/response/error handling logic exactly
- Handle authentication token injection using inject(AuthService).accessToken()
- Handle 401 responses with redirect to login and token clear
- Handle 403 responses with a toast notification (inject NotificationService)
- Use catchError + EMPTY for swallowed errors, throwError for propagated errors
- Register the interceptor in app.config.ts using provideHttpClient(withInterceptors([myInterceptor]))

Output the interceptor function and the updated app.config.ts provider block.

5.3 — GitHub Copilot Chat: Code Review & Validation Prompts

Use these prompts after migration to verify quality and correctness.

Prompt 5.3.1 — Migration correctness review

Review the following Angular 21 component against the original AngularJS controller.

ORIGINAL ANGULARJS CONTROLLER:

[PASTE ORIGINAL]

MIGRATED ANGULAR 21 COMPONENT:

[PASTE MIGRATION]

Check for:

1. Feature parity – every \$scope method has an equivalent; every \$scope property has a signal equivalent
2. Signal correctness – no signal() inside computed(); no direct signal mutation inside effect()
3. OnPush safety – all template bindings use signal() or async pipe; no mutable object references passed as inputs
4. SSR safety – no direct window/document/localStorage access without isPlatformBrowser() guard
5. Security – no innerHTML binding without DomSanitizer; no user input passed to bypassSecurityTrust* without sanitisation
6. Accessibility – all interactive elements have ARIA labels; error messages have aria-live
7. Performance – no heavy computations inside the template; @defer used for non-critical sections
8. Nx boundary violations – no imports from apps/ in this component; only @workspace/* lib imports

Output a pass/fail table for each check. List specific line numbers for any failures.

Prompt 5.3.2 — Unit test generation

Generate a complete Vitest test file for the following Angular 21 component/service.

[PASTE COMPONENT OR SERVICE CODE]

Requirements:

- Use TestBed.configureTestingModule with provideHttpClientTesting, provideRouter([])
- Test each signal's initial value
- Test each computed() signal with multiple input combinations
- Test each effect() side effect (spy on the injected service)
- Test form validation: valid state, each invalid state, submit with invalid state
- Test HTTP calls: mock with HttpTestingController; verify request method + URL + body
- Test error handling: 401, 403, 500 responses
- Use describe/it blocks with descriptive names
- AAA pattern: Arrange → Act → Assert in each test
- No real HTTP calls; no real timers (use fakeAsync/tick for effects)
- Coverage target: 80% minimum for branches

Output the complete spec file. Add a comment above each test explaining what business rule it verifies.

Prompt 5.3.3 — Accessibility audit

@workspace

Audit all Angular 21 components in apps/banking-ng21/src/app/features/ for WCAG 2.1 AA compliance.

For each component, check:

1. All images have alt text (non-decorative) or aria-hidden="true" (decorative)
2. All form inputs have associated <label> or aria-label or aria-labelledby
3. All error messages have aria-live="polite" and role="alert"
4. All interactive elements (button, a, [role=button]) are keyboard-focusable
5. All icon-only buttons have aria-label
6. Focus management: modals trap focus; on close, focus returns to trigger
7. Skip link present in shell for keyboard navigation
8. Colour contrast: flag any text that might fail (look for text-gray-400 on white background)
9. No positive tabindex values (tabindex="1" or higher)
10. No duplicate IDs across the page

Output: one table per domain with Pass/Fail/Warning per check. List the file path and line number for every failure.

5.4 — GitHub Copilot Chat: Infrastructure & Architecture Prompts

Prompt 5.4.1 — Feature flag strategy

@workspace

I need to implement a feature flag system that controls whether each of the 120 screens in our app shows the Angular 21 version or falls back to the legacy AngularJS version via an iframe.

Design requirements:

1. Flags must be server-side configurable (read from an API endpoint at app startup)
2. Flags must be readable synchronously in route guards (no async in CanActivateFn)
3. Flags must be overridable per user (beta tester programme) and per environment
4. Flags must be persisted across page refreshes for beta testers (localStorage)
5. Flags must be observable – components should react when a flag changes
6. The system must support a kill switch to roll back all 120 screens to legacy simultaneously

Using Angular 21 signals, design and implement:

- FeatureFlagService with a Map<string, Signal<boolean>> store
- APP_INITIALIZER to load flags before bootstrap
- FeatureFlagDirective (*featureFlag="'accounts.detail'")
- A CanMatchFn guard that reads flags
- The shell router config showing how legacy iframe vs new route is selected per flag

Output complete TypeScript files for each.

Prompt 5.4.2 — Module Federation remote registration

I have an Nx workspace with a Module Federation host (banking-shell) and a remote (banking-ng21).

The shell must:

1. Register the remote URL dynamically from a config endpoint (not hardcoded)
2. Handle remote unavailability gracefully (show fallback component, not white screen)
3. Support multiple environments: dev (localhost:4201), staging, production (CDN URL)
4. Load the remote lazily only when the user navigates to a route that needs it
5. Share @angular/core, @angular/router, @angular/common, rxjs as singletons

Generate:

- The dynamic remote registration function (loadRemoteModule pattern)
- app.routes.ts for the shell showing dynamic import with error boundary
- environment.ts / environment.prod.ts with remote URLs
- The ErrorBoundaryComponent shown when the remote fails to load
- The module-federation.config.ts for both host and remote

All code must be Angular 21 standalone component style.

Prompt 5.4.3 — Security hardening checklist verification

@workspace

Perform a security audit of the Angular 21 application in apps/banking-ng21/ against these OWASP controls:

1. XSS: Search for innerHTML, outerHTML, document.write, bypassSecurityTrustHtml, bypassSecurityTrustScript usages. List each with file + line.
2. CSRF: Verify withXsrfConfiguration() is present in provideHttpClient(). Check that POST/PUT/DELETE requests include XSRF token.
3. Injection: Search for direct SQL construction, eval(), new Function(), dangerouslySetInnerHTML, template literals built from user input.
4. Authentication: Verify JWT is stored in memory (signal), NOT in localStorage or sessionStorage. Find any token in storage.
5. Sensitive data exposure: Find any console.log calls that might print tokens, passwords, or PII.
6. Broken access control: Verify every route has CanActivateFn or CanMatchFn guard. List any unguarded routes.
7. Security headers: Check if Content-Security-Policy, X-Frame-Options, X-Content-Type-Options are set (look in server.ts or proxy config).

Output a findings table: Issue | Severity | File | Line | Remediation. Mark critical issues in bold.

5.5 — GitHub MCP Server Integration

The GitHub MCP server allows Copilot to interact with GitHub Issues, PRs, and repository data directly. Use it to automate migration ticket management.

Setting up GitHub MCP in VS Code

```
// .vscode/mcp.json (add to workspace)
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

MCP-powered migration workflow prompts

Prompt 5.5.1 — Create migration tickets from inventory

#github

I have completed the AngularJS inventory. Create GitHub Issues in repo [OWNER/REPO] for each of the 8 migration domains:

For each domain, create a parent issue titled "[Migration] Domain: {domain_name}" with:

- Label: migration, angular21, {domain_name}
- Milestone: Migration Phase 2
- Body template:
 - ## Domain: {domain_name}
 - **Screens to migrate:** {count}
 - **Controllers:** {list}
 - **Estimated effort:** {sprint_points}
 - **Dependencies:** shared-ui, shared-auth
 - **Acceptance criteria:**
 - [] All {count} screens rendering in Angular 21
 - [] Feature flags enabled in staging
 - [] Unit test coverage >= 80%
 - [] Playwright E2E tests passing
 - [] Accessibility audit: 0 critical failures

Then create child issues for each individual screen, linked to the parent via "Part of #PARENT_ISSUE".

Domains and screen counts:

- Auth: 10 screens
- Dashboard: 8 screens
- Accounts: 18 screens
- Payments: 15 screens
- Loans: 12 screens
- Reports: 20 screens
- Settings: 14 screens
- Admin: 23 screens

Prompt 5.5.2 — Auto-generate PR description from diff

#github

I am about to open a PR for the migration of the [accounts/account-detail] screen from AngularJS to Angular 21.

Generate a PR description that includes:

1. **Summary** – What was migrated (controller name → component name)
2. **AngularJS artefacts removed** – List of controllers/services/directives no longer needed
3. **Angular 21 artefacts added** – New component, service, spec files
4. **Signal mapping** – Table showing \$scope.X → signal name
5. **Breaking changes** – Any API endpoint changes or route path changes
6. **Feature flag** – Which flag name controls the rollout
7. **Testing** – Test coverage % before and after
8. **Screenshots** – Placeholder sections for before/after UI screenshots
9. **Review checklist:**
 - [] ESLint 0 errors
 - [] Unit tests passing
 - [] Nx boundary rules not violated
 - [] ARIA attributes present on all interactive elements
 - [] No AngularJS imports in new component
 - [] Feature flag wired to route guard

Prompt 5.5.3 — Migration sprint planning from issues

#github

Look at all open GitHub Issues labelled "migration" in repo [OWNER/REPO].

1. List all open migration issues grouped by domain
2. Calculate total story points (estimate 1pt = simple screen <100 lines, 2pt = medium, 3pt = complex >200 lines)
3. Suggest a sprint plan for the next 3 sprints (2 weeks each), allocating work across 3 engineers
4. Identify any issues that are blocked by a shared-ui component not yet built
5. Suggest which issues can be done in parallel and which are sequential

Output: Sprint 1 / Sprint 2 / Sprint 3 tables with issue numbers, assignees, and total points per engineer.

5.6 — Custom Copilot Agent: migration-expert.agent.md

Drop this file into `.github/` to create a custom Copilot agent that any team member can invoke with `@migration-expert` in VS Code chat.

```
---
description: AngularJS to Angular 21 migration expert. Use for converting controllers, services,
  directives, routes, and forms. Enforces Nx boundary rules, signal patterns, and OWASP security.
applyTo: 'apps/banking-legacy/**,apps/banking-ng21/**,libs/**'
---

# Migration Expert Agent

You are an expert AngularJS-to-Angular-21 migration engineer for a 100-screen enterprise banking application.

## Your Identity

- You know every AngularJS 1.8 API: $scope, $rootScope, $http, $q, $watch, ng-model, ui-router, ng-repeat, ng-if
- You know every Angular 21 API: signals, computed, effect, inject(), input(), output(), @if, @for, @defer, HttpClient,
  HttpInterceptorFn, CanActivateFn, CanMatchFn, httpResource(), resource(), viewChild(), contentChild(), viewChildren(),
  contentChildren(), @let, afterRender(), afterNextRender(), untracked(), DestroyRef, booleanAttribute, numberAttribute,
  outputFromObservable(), outputToObservable(), withEventReplay(), RenderMode, provideZonelessChangeDetection(), provideAppInitializer()
- see Section 3.11 for full details on all Angular 21.x-exclusive APIs
- You enforce Nx module boundaries: apps/ never import from each other; libs/ are the only shared code
- You never use constructor injection – always inject()
- You never use @Input()/@Output() – always input()/output()
- You never use *ngIf/*ngFor – always @if/@for
- You always add OnPush change detection
- You always add ARIA attributes for accessibility

## Migration Rules (Non-Negotiable)

1. $scope.x → signal<T>(initialValue); always typed
2. $watch(fn) → effect(() => { /_ side effect only _/ })
3. Derived values → computed(() => /_ pure function _/)
4. $http → HttpClient; always return Observable or use toSignal()
5. $rootScope.$broadcast → EventBusService typed event
6. ng-model → signal + (input) event binding
7. ui-router states → Angular lazy-loaded routes with LoadComponent()
8. filters → standalone Pipe classes
9. Directives with templates → standalone Component
10. Directives without templates → Attribute Directive using inject(ElementRef) + Renderer2

## Output Format

For every migration request:

1. Show the TypeScript component file
2. Show the HTML template file
3. Show the spec file (Vitest)
4. List any assumptions made
5. List any patterns that could not be automatically converted

## Security Rules

- Never store JWT in localStorage/sessionStorage – use in-memory signal only
- Never bind innerHTML without DomSanitizer
- Always add isPlatformBrowser() guard for browser-only APIs
- Always use Renderer2 – never nativeElement.style or nativeElement.innerHTML
```

5.7 — Custom Copilot Skill: angular-migration.skill.md

Drop this file into `.github/skills/angular-migration/` to package domain knowledge for reuse across agents.

Angular Migration Skill

Signal Pattern Reference

AngularJS Pattern	Angular 21 Equivalent
`\$scope.count = 0`	`protected readonly count = signal(0)`
`\$scope.items = []`	`protected readonly items = signal<Item[]>([])`
`\$scope.fullName = first + last`	`protected readonly fullName = computed(() => this.first() + ' ' + this.last())`
`\$watch('count', fn)`	`effect(() => { if (this.count() > 0) { fn(); } })`
`\$scope.loading = true`	`protected readonly loading = signal(false)`
`\$http.get('/api/x')`	`this.http.get<X>('/api/x')` + `toSignal(obs\$)`
`\$scope.\$apply()`	Not needed – signals are zoneless
`ng-model="form.email"`	`protected readonly email = signal('')` + `(input)="email.set(\$event.target.value)"`
`ng-repeat="item in items"`	`@for (item of items(); track item.id)`
`ng-if="condition"`	`@if (condition())`
`ng-class="{ 'active': isActive}"`	`[class.active]="isActive()"`
`ng-disabled="!isValid"`	`[disabled]="!isValid()"`
`ng-submit="save()"`	`(ngSubmit)="save()"`
`\$scope.\$broadcast('event', data)`	`this.eventBus.emit({ type: 'EVENT', payload: data })`

Component Anatomy (Angular 21)

```
``typescript
import {
  ChangeDetectionStrategy,
  Component,
  computed,
  effect,
  inject,
  signal,
} from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { toSignal } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-example',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (loading()) {
      <app-spinner />
    }
    @for (item of items(); track item.id) {
      <div>{{ item.name }}</div>
    }
  `,
})
export class ExampleComponent {
  private readonly http = inject(HttpClient);

  protected readonly loading = signal(false);
  protected readonly items = toSignal(this.http.get<Item[]>('/api/items'), { initialValue: [] });
}
````
```

## Nx Import Rules

```
// ✅ CORRECT – import from lib path alias
import { AuthService } from '@workspace/shared-auth';
import { ButtonComponent } from '@workspace/shared-ui';
import { Account } from '@workspace/shared-models';

// ❌ WRONG – never import across app boundaries
import { AuthService } from '../../../banking-shell/src/services/auth.service';
import { Account } from '../../../banking-legacy/app/models/account';
```

## File Naming (kebab-case, always)

```
controllers/accountsCtrl.js → account-list.component.ts
services/AccountService.js → account.service.ts
directives/currencyInput.js → currency-input.directive.ts
filters/dateFilter.js → date-format.pipe.ts
```

---

## 5.8 – Copilot Instructions File: `copilot-instructions.md`

Place this file at `.github/copilot-instructions.md` in the Nx workspace root. It sets baseline rules for every Copilot interaction in the migration project.

```markdown

Angular Migration Workspace – GitHub Copilot Instructions

Project Overview

This is an Nx monorepo migrating a 120-screen AngularJS 1.8 enterprise banking application to Angular 21.
Migration is in progress – apps/banking-legacy/ is frozen (no changes); apps/banking-ng21/ is the active target.

Non-Negotiable Rules for ALL Code Generation

Angular 21 Patterns (Mandatory)

- Standalone components only – no NgModule
- OnPush change detection on every component
- inject() for all dependencies – never constructor injection
- input() / output() – never @Input() / @Output()
- @if / @for / @switch in templates – never *ngIf / *ngFor / *ngSwitch
- Typed signals: signal<string>(''), not signal('')
- effect() only for side effects – never for derived state
- computed() for all derived state – must be pure (no side effects)

Nx Boundary Rules (Mandatory)

- Apps never import from other apps
- Shared code only via @workspace/* library imports
- Tag compliance: apps have scope:app, libs have scope:lib
- Check @nx/enforce-module-boundaries ESLint rule before every import

Security Rules (Mandatory)

- JWT stored only in memory (signal) – never localStorage/sessionStorage/cookie
- All routes protected by CanActivateFn or CanMatchFn
- DomSanitizer required for any dynamic HTML binding
- isPlatformBrowser() guard for all browser-only APIs
- No console.log of tokens, passwords, or user PII

Testing Rules (Mandatory)

- Every new component/service must have a .spec.ts file
- Use Vitest – not Karma, not Jasmine
- HttpTestingController for HTTP mocking
- provideHttpClientTesting() in TestBed
- Minimum 80% branch coverage for migrated code

What Copilot Must NEVER Do

- Never use deprecated APIs: *ngIf, *ngFor, @Input(), @Output(), constructor injection
- Never import from apps/banking-legacy/ in Angular 21 code
- Never put business logic in templates – keep templates declarative
- Never use eval(), innerHTML without DomSanitizer, or document.write()
- Never add features not explicitly requested
- Never rename existing files or refactor code outside the current task scope

5.9 — VS Code Extensions Recommendations

Add this to `.vscode/extensions.json` to ensure every team member has the right tools:

```
{
  "recommendations": [
    "angular.ng-template",
    "nrwl.angular-console",
    "github.copilot",
    "github.copilot-chat",
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "bradlc.vscode-tailwindcss",
    "vitest.explorer",
    "ms-playwright.playwright",
    "christian-kohler.path-intellisense",
    "formulahendry.auto-rename-tag",
    "streetsidesoftware.code-spell-checker"
  ]
}
```

Workspace settings for consistent DX:

```
// .vscode/settings.json
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "typescript.tsdk": "node_modules/typescript/lib",
  "typescript.enablePromptUseWorkspaceTsdk": true,
  "eslint.validate": ["typescript", "html"],
  "tailwindCSS.includeLanguages": {
    "typescript": "html"
  },
  "files.associations": {
    "*.html": "html"
  }
}
```

Part 5 complete. The AI-assisted migration toolkit now covers 20+ Copilot Chat prompts across the full migration lifecycle, GitHub MCP integration for ticket and PR automation, a custom `@migration-expert` Copilot agent, a reusable skill file with the complete signal pattern reference, and workspace-level Copilot instructions that enforce Angular 21 best practices for every team member.

Ready for Part 6? Part 6 covers **Architectural Decisions & Risk Register** — ADR templates for the key migration decisions, a prioritised risk register with 15+ enterprise migration risks, mitigation strategies, and ownership assignment.

Part 6 — Architectural Decisions & Risk Register

How to use this section: Every major technical decision made during migration must be documented as an ADR (Architectural Decision Record) before implementation begins. This section provides ready-to-use ADR templates for the five highest-impact decisions, a full state management decision framework (signals vs NgRx), a security token/cookie strategy guide, a comprehensive performance strategy covering all Core Web Vitals, and a prioritised risk register for a 100+ screen enterprise app.

6.1 — State Management Decision: Signals vs NgRx

This is the single most consequential architectural decision of the migration. Getting it wrong means either over-engineering simple screens or under-engineering complex ones.

The Decision Framework



Detailed Decision Matrix

| Criteria | Signals Only | Service + Signals | NgRx Signal Store | Full NgRx |
|-----------------------------|---------------------|---------------------------|-------------------------|------------------------------------|
| State scope | Single component | 2–5 related components | Feature-wide (1 domain) | Cross-domain / global |
| Async complexity | Simple HTTP load | Load + error handling | Multi-step async flows | Complex orchestration |
| State sharing | None | Parent-child or siblings | Domain-wide | App-wide |
| History / undo | Not needed | Not needed | Not needed | Required |
| Devtools debugging | Not needed | Not needed | NgRx DevTools useful | NgRx DevTools essential |
| Team NgRx experience | Any | Any | Some NgRx helpful | NgRx expertise required |
| Migration source | Simple controller | Service + 2–3 controllers | Feature module | Multiple modules with shared state |
| Boilerplate cost | Very low | Low | Medium | High |
| Recommended for | ~60% of 120 screens | ~25% of 120 screens | ~10% of 120 screens | ~5% of 120 screens |

Mapping AngularJS patterns to the right solution

Pattern A — Simple controller with local \$scope (Use: signals only)

```
// AngularJS: $scope.account, $scope.loading, $scope.save()
// 🚩 Angular 21: component-level signals

@Component({ standalone: true, changeDetection: ChangeDetectionStrategy.OnPush, ... })
export class AccountDetailComponent {
  private readonly accountService = inject(AccountService);
  private readonly route = inject(ActivatedRoute);

  protected readonly account = toSignal(
    this.route.params.pipe(switchMap(p => this.accountService.getById(p['id']))),
    { initialValue: null }
  );
  protected readonly loading = signal(false);
  protected readonly error = signal<string | null>(null);

  protected save(data: Account): void {
    this.loading.set(true);
    this.accountService.update(data).subscribe({
      next: () => this.loading.set(false),
      error: (e) => { this.error.set(e.message); this.loading.set(false); },
    });
  }
}
// ✅ Zero NgRx. Zero boilerplate. Full reactivity via signals.
```

Pattern B — Multiple controllers sharing a service (Use: service-level signals)

```
// AngularJS: AccountService shared by AccountListCtrl + AccountDetailCtrl + AccountHeaderCtrl
// 🚩 Angular 21: Injectable service with signal store

@Injectable({ providedIn: 'root' })
export class AccountStateService {
  // Single source of truth — owned by service
  private readonly _accounts = signal<Account[]>([]);
  private readonly _selectedId = signal<string | null>(null);
  private readonly _loading = signal(false);

  // Public read-only projections
  readonly accounts = this._accounts.asReadonly();
  readonly loading = this._loading.asReadonly();
  readonly selected = computed(
    () => this._accounts().find((a) => a.id === this._selectedId()) ?? null,
  );
  readonly totalBalance = computed(() => this._accounts().reduce((sum, a) => sum + a.balance, 0));

  load(): void {
    this._loading.set(true);
    inject(HttpClient)
      .get<Account[]>('/api/accounts')
      .subscribe({
        next: (accounts) => {
          this._accounts.set(accounts);
          this._loading.set(false);
        },
        error: () => this._loading.set(false),
      });
  }

  select(id: string): void {
    this._selectedId.set(id);
  }

  update(account: Account): void {
    this._accounts.update((accounts) => accounts.map((a) => (a.id === account.id ? account : a)));
  }
}
// ✅ 3 components share state. No NgRx. Full signal reactivity.
```

Pattern C — Feature with complex async flows (Use: NgRx Signal Store)

```
// AngularJS: PaymentService orchestrates validation → approval → notification → audit log
// 🚀 Angular 21: @ngrx/signals withMethods for complex flows

import { signalStore, withState, withComputed, withMethods } from '@ngrx/signals';
import { rxMethod } from '@ngrx/signals/rxjs-interop';

interface PaymentState {
  payments: Payment[];
  activePayment: Payment | null;
  status: 'idle' | 'validating' | 'processing' | 'approved' | 'failed';
  error: string | null;
}

export const PaymentStore = signalStore(
  { providedIn: 'root' },
  withState<PaymentState>({
    payments: [],
    activePayment: null,
    status: 'idle',
    error: null,
  }),
  withComputed(({ payments, status }) => ({
    pendingCount: computed(() => payments().filter((p) => p.status === 'pending').length),
    canSubmit: computed(() => status() === 'idle'),
  })),
  withMethods((store, paymentService = inject(PaymentService)) => ({
    submitPayment: rxMethod<Payment>(
      pipe(
        tap(() => patchState(store, { status: 'validating', error: null })),
        switchMap((payment) =>
          paymentService.validate(payment).pipe(
            switchMap((valid) =>
              valid
                ? paymentService.process(payment)
                : throwError(() => new Error('Validation failed')),
            ),
          ),
        tap((result) => patchState(store, { activePayment: result, status: 'approved' })),
        catchError((err) => {
          patchState(store, { status: 'failed', error: err.message });
          return EMPTY;
        })
      )
    ),
  })),
);

// ✅ Multi-step async with clear status tracking. NgRx Signal Store sweet spot.
```

Pattern D — Cross-domain global state (Use: Full NgRx with Effects)

```
// AngularJS: $rootScope shared user profile + notification count + feature flags
// 🚀 Angular 21: NgRx with Effects for cross-domain orchestration

// actions/user.actions.ts
export const UserActions = createActionGroup({
  source: 'User',
  events: {
    'Load Profile': emptyProps(),
    'Load Profile Success': props<{ user: UserProfile }>(),
    'Load Profile Failure': props<{ error: string }>(),
    'Update Preferences': props<{ preferences: UserPreferences }>(),
    'Session Expired': emptyProps(),
  },
});

// effects/user.effects.ts
export const loadUserEffect = createEffect(
  (actions$ = inject(Actions), userService = inject(UserService)) =>
    actions$.pipe(
      ofType(UserActions.loadProfile),
      switchMap(() =>
        userService.getProfile().pipe(
          map((user) => UserActions.loadProfileSuccess({ user })),
          catchError((error) => of(UserActions.loadProfileFailure({ error: error.message }))),
        ),
      ),
    ),
  { functional: true },
);

// When to justify full NgRx:
// ✅ User profile loaded once, consumed by 40+ components across 8 domains
// ✅ Session expiry must trigger logout across ALL active components simultaneously
// ✅ Notification count updated by 3 independent backend events
// ✅ Feature flags must be hot-reloadable with time-travel debug in dev
```

Decision Summary for 100-Screen Enterprise App

ACCOUNTS DOMAIN (18 screens):

| | |
|---------------------|--|
| Account List | → signals only (list + pagination) |
| Account Detail | → signals only (load + edit) |
| Transaction History | → signals only (filterable list) |
| Account Statements | → service-level signals (shared between 3 screens) |
| Account Settings | → signals only |
| [13 more screens] | → mix of signals only and service-level signals |

PAYMENTS DOMAIN (15 screens):

| | |
|-------------------|--|
| New Payment | → NgRx Signal Store (multi-step: validate → process → confirm) |
| Payment History | → signals only |
| Beneficiaries | → service-level signals (CRUD shared between 2 screens) |
| [12 more screens] | → signals |

GLOBAL STATE (used by all 8 domains):

| | |
|----------------|--|
| User Profile | → Full NgRx (consumed across all 8 domains) |
| Notifications | → Full NgRx (real-time updates from WebSocket) |
| Feature Flags | → FeatureFlagService with signals (loaded at startup) |
| Auth / Session | → AuthService with signals (in-memory token, in shared-auth lib) |

FINAL SPLIT FOR 120 SCREENS:

| | |
|-------------------|--------------------------------|
| ~72 screens (60%) | → component-level signals only |
| ~30 screens (25%) | → service-level signals |
| ~12 screens (10%) | → NgRx Signal Store |
| ~6 screens (5%) | → Full NgRx with Effects |

6.2 — Security: HTTP Token & Cookie Strategy

🔑 **This section covers the token strategy ADR** — the architectural decision record for how tokens are stored and transmitted. Security spans three sections:

- [Part 1, Q21–Q27](#): Strategy — OWASP risk questions, XSS/CSRF/CSRF decision framework.
- [Section 3.7 — Security LLD](#): Implementation — Angular 21 auth service, interceptors, route guards.
- **Here (Section 6.2)**: ADR — token storage options, risk matrix, and the recommended BFF hybrid approach.

This section defines the definitive token strategy for a migrated enterprise banking application, covering all storage options, their risk profiles, and the recommended hybrid approach.

Token Storage Options Comparison

| Strategy | Storage Location | XSS Risk | CSRF Risk | SSR Compatible | Logout Reliability | Recommended? |
|------------------------|------------------------|---------------------------|-------------------------------|----------------|---------------------------------------|------------------------------|
| JWT in localStorage | Browser localStorage | 🔴 HIGH — any JS can read | 🟢 None | ❌ No | ❌ Token persists after tab close | ❌ Never |
| JWT in sessionStorage | Browser sessionStorage | 🔴 HIGH — any JS can read | 🟢 None | ❌ No | 🟡 Lost on tab close only | ❌ Avoid |
| JWT in memory (signal) | JS heap only | 🟢 Lowest — no persistence | 🟢 None | ✅ Yes | ✅ Signal.set(null) clears immediately | ✅ Recommended |
| HttpOnly cookie | Browser cookie jar | 🟢 JS cannot read | 🔴 HIGH — auto-sent by browser | ✅ Yes | ✅ Server controls expiry | ✅ With XSRF |
| HttpOnly + XSRF token | Cookie + header | 🟢 JS cannot read cookie | 🟢 XSRF mitigated | ✅ Yes | ✅ Server controls expiry | ✅ Recommended for BFF |
| OAuth2 PKCE + BFF | Server session | 🟢 Tokens never in browser | 🟢 None (server-side) | ✅ Yes | ✅ Server-side revocation | ✅ Gold standard |

Recommended Strategy: In-Memory JWT (Angular SPA)

For most AngularJS migration scenarios (Angular 21 SPA calling REST APIs directly):


```

// libs/shared-auth/src/lib/auth.service.ts
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly router = inject(Router);
  private readonly platformId = inject(PLATFORM_ID);

  // ✅ Token in JS memory only – never persisted to storage
  private readonly _accessToken = signal<string | null>(null);
  private readonly _refreshToken = signal<string | null>(null);
  private readonly _expiresAt = signal<number | null>(null);
  private readonly _user = signal<UserProfile | null>(null);
  private readonly _roles = signal<string[]>([]);

  // Public read-only surface
  readonly isAuthenticated = computed(
    () => this._accessToken() !== null && (this._expiresAt() ?? 0) > Date.now(),
  );
  readonly user = this._user.asReadonly();
  readonly roles = this._roles.asReadonly();
  readonly accessToken = this._accessToken.asReadonly();

  login(credentials: LoginCredentials): Observable<void> {
    return this.http.post<AuthResponse>('/api/auth/login', credentials).pipe(
      tap((response) => {
        this._accessToken.set(response.accessToken);
        this._refreshToken.set(response.refreshToken); // kept in memory too
        this._expiresAt.set(Date.now() + response.expiresIn * 1000);
        this._user.set(response.user);
        this._roles.set(response.roles);
        // ✅ Schedule silent refresh 60 seconds before expiry
        this.scheduleTokenRefresh(response.expiresIn);
      }),
      map(() => void 0),
    );
  }

  logout(): void {
    // ✅ One call clears everything – no localStorage.removeItem needed
    this._accessToken.set(null);
    this._refreshToken.set(null);
    this._expiresAt.set(null);
    this._user.set(null);
    this._roles.set([]);
    this.router.navigate(['/login']);
  }

  hasRole(role: string): boolean {
    return this._roles().includes(role);
  }

  hasAnyRole(roles: string[]): boolean {
    return roles.some((r) => this._roles().includes(r));
  }

  // Silent token refresh using refresh token
  private refreshToken: ReturnType<typeof setTimeout> | null = null;

  private scheduleTokenRefresh(expiresIn: number): void {
    if (this.refreshTimer) clearTimeout(this.refreshTimer);
    const refreshIn = (expiresIn - 60) * 1000; // 60 seconds before expiry
    if (refreshIn <= 0) return;
    this.refreshTimer = setTimeout(() => this.silentRefresh(), refreshIn);
  }

  private silentRefresh(): void {
    const refreshToken = this._refreshToken();
    if (!refreshToken) return;
    this.http.post<AuthResponse>('/api/auth/refresh', { refreshToken }).subscribe({
      next: (response) => {
        this._accessToken.set(response.accessToken);
        this._expiresAt.set(Date.now() + response.expiresIn * 1000);
        this.scheduleTokenRefresh(response.expiresIn);
      },
      error: () => this.logout(), // Refresh failed → force logout
    });
  }
}

```

```
}  
}
```

HttpOnly Cookie Strategy (BFF Pattern)

For applications using a Backend-for-Frontend (BFF) proxy — preferred for highest security in banking:

```
// libs/shared-auth/src/lib/bff-auth.service.ts  
// When using BFF: tokens live on the server, browser only has a session cookie  
  
@Injectable({ providedIn: 'root' })  
export class BffAuthService {  
  private readonly http = inject(HttpClient);  
  private readonly router = inject(Router);  
  
  // ✅ No token in browser at all – server sets HttpOnly cookie on login  
  private readonly _user = signal<UserProfile | null>(null);  
  private readonly _isAuthenticated = signal(false);  
  private readonly _roles = signal<string[]>([]);  
  
  readonly user = this._user.asReadonly();  
  readonly isAuthenticated = this._isAuthenticated.asReadonly();  
  readonly roles = this._roles.asReadonly();  
  
  // On app init: check session with BFF  
  checkSession(): Observable<void> {  
    return this.http  
      .get<SessionResponse>('/bff/user', {  
        withCredentials: true, // ✅ Send HttpOnly session cookie  
      })  
      .pipe(  
        tap((session) => {  
          this._user.set(session.user);  
          this._isAuthenticated.set(true);  
          this._roles.set(session.roles);  
        }),  
        catchError(() => {  
          this._isAuthenticated.set(false);  
          return of(void 0);  
        }),  
        map(() => void 0),  
      );  
  }  
  
  login(): void {  
    // BFF redirects to Identity Provider (OIDC flow)  
    window.location.href = '/bff/login?returnUrl=' + encodeURIComponent(window.location.pathname);  
  }  
  
  logout(): void {  
    this.http.post('/bff/logout', {}, { withCredentials: true }).subscribe(() => {  
      this._user.set(null);  
      this._isAuthenticated.set(false);  
      this.router.navigate(['/login']);  
    });  
  }  
}
```

XSRF / CSRF Protection Configuration

```
// app.config.ts – Angular handles XSRF automatically when configured
import { provideHttpClient, withXsrfConfiguration, withInterceptors } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      // ✅ Angular reads XSRF-TOKEN cookie and sends X-XSRF-TOKEN header automatically
      withXsrfConfiguration({
        cookieName: 'XSRF-TOKEN', // Must match server cookie name
        headerName: 'X-XSRF-TOKEN', // Must match server expected header
      }),
      withInterceptors([authInterceptor, errorInterceptor]),
    ),
    provideRouter(appRoutes),
  ],
};
```

```
# nginx config: set XSRF cookie on every response (non-HttpOnly so Angular JS can read it)
# The session cookie (auth) remains HttpOnly – XSRF cookie does not
add_header Set-Cookie "XSRF-TOKEN=$request_id; Path=/; SameSite=Strict; Secure";
```

Token Strategy Decision Matrix

Choose your strategy based on your backend architecture:

Q: Does your Angular app call the REST API DIRECTLY?
(No BFF proxy)

YES



In-Memory JWT
(AuthService with signals)
+ Silent refresh via refresh token
✅ 95% of Angular SPA migrations

Security profile:

- XSS: Cannot steal from storage
- CSRF: N/A (no cookie)
- MITM: HTTPS required
- Token leak: Only in memory dump

NO (BFF sits between app & API)



HttpOnly Cookie + XSRF
(BffAuthService pattern)
+ OIDC PKCE at BFF layer
✅ Banking, healthcare, high-security

Security profile:

- XSS: Token never in browser
- CSRF: XSRF token + SameSite=Strict
- MITM: HTTPS required
- Token leak: Server-side only

Auth Interceptors (Both Strategies)

```
// libs/shared-auth/src/lib/auth.interceptor.ts
// Strategy A: In-memory JWT – attach Bearer token to every request
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.accessToken();

  // Skip auth header for public endpoints
  const isPublicEndpoint = ['/api/auth/login', '/api/auth/refresh'].some((url) =>
    req.url.includes(url),
  );

  if (!token || isPublicEndpoint) return next(req);

  return next(
    req.clone({
      setHeaders: { Authorization: `Bearer ${token}` },
      withCredentials: false, // No cookies needed in JWT strategy
    }),
  );
};

// libs/shared-auth/src/lib/error.interceptor.ts
// Handle 401 (expired token) and 403 (forbidden) globally
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === 401) {
        auth.logout(); // Token expired – force re-login
        return EMPTY;
      }
      if (error.status === 403) {
        router.navigate(['/unauthorised']);
        return EMPTY;
      }
      return throwError(() => error); // Propagate all other errors
    }),
  );
};
```

6.3 — Performance Strategy: Core Web Vitals

For a 100-screen enterprise app migrating from AngularJS, performance is not optional — it is the primary reason users notice the migration (for better or worse). This section maps each Core Web Vital to concrete Angular 21 implementation strategies.

Core Web Vitals Targets (Enterprise SPA Baseline)

| Metric | Full Name | What it Measures | Good | Needs Improvement | Poor | Priority |
|-------------|---------------------------|----------------------------------|---------|-------------------|---------|--|
| LCP | Largest Contentful Paint | When the main content is visible | ≤ 2.5s | 2.5–4.0s | > 4.0s | ● Critical |
| INP | Interaction to Next Paint | Responsiveness to user input | ≤ 200ms | 200–500ms | > 500ms | ● Critical |
| CLS | Cumulative Layout Shift | Visual stability (no jumping) | ≤ 0.1 | 0.1–0.25 | > 0.25 | ● High |
| FCP | First Contentful Paint | When first content appears | ≤ 1.8s | 1.8–3.0s | > 3.0s | ● High |
| TTFB | Time to First Byte | Server response time | ≤ 800ms | 800ms–1.8s | > 1.8s | ● High |
| TBT | Total Blocking Time | JS blocking main thread | ≤ 200ms | 200–600ms | > 600ms | ● Medium |

LCP Strategy (Largest Contentful Paint)

Problem in AngularJS: The entire app bundle loads before any content renders. AngularJS apps often ship 500KB–2MB of JavaScript before first paint.

Solutions in Angular 21:

```
// 1. SSR + Hydration for instant first paint
// app.config.server.ts
export const serverConfig: ApplicationConfig = {
  providers: [
    provideServerRendering(),
    // ✅ Incremental hydration: only hydrate components as they become visible
    provideClientHydration(withIncrementalHydration()),
  ],
};

// 2. @defer for non-critical sections (defers JS download entirely)
// template:
@defer (on viewport) {
  <app-transaction-chart /> // ← Downloads ONLY when user scrolls to it
} @placeholder {
  <div class="h-64 bg-gray-100 animate-pulse"></div> // ← Skeleton shown immediately
}

// ⚠️ @defer + SSR Hydration Warning:
// Problem: @defer (on viewport) works differently on server vs client
// Server can't detect viewport → always renders @placeholder
// Client hydration may trigger immediate load → causes flicker

// ✅ SAFE @defer patterns for SSR:
@defer (on timer(2s)) {
  <app-heavy-component /> // Timer works same on server and client
}

// ⚠️ RISKY: Viewport-based (server can't detect)
@defer (on viewport) {
  <app-chart /> // May cause hydration mismatch
} @placeholder {
  <app-skeleton />
}

// ✅ FIX: Combine with isPlatformBrowser()
protected readonly canLoadCharts = computed(() =>
  isPlatformBrowser(inject(PLATFORM_ID))
);

@if (canLoadCharts()) {
  @defer (on viewport) {
    <app-chart />
  }
} @else {
  <app-skeleton />
}

// 3. Lazy-load all 8 feature domains
// app.routes.ts
export const routes: Routes = [
  {
    path: 'accounts',
    loadComponent: () => import('./features/accounts/account-list.component')
      .then(m => m.AccountListComponent),
    // ✅ accounts domain JS not downloaded until user navigates to /accounts
  },
];

// 4. Preload strategy: preload likely next routes during idle time
// app.config.ts
provideRouter(routes, withPreloading(QuicklinkStrategy)), // preload on <a> hover
```

```

<!-- 5. Priority hints for hero images (LCP optimization) -->
<img
  ngSrc="/hero-dashboard.jpg"
  alt="Banking Dashboard"
  width="1200"
  height="600"
  priority <!-- ← NgOptimizedImage directive: adds fetchpriority="high" -->
/>

<!-- Manual fetchpriority (if not using NgOptimizedImage) -->

  width="1200"
  height="600"
/>

<!-- ⚠️ Only use fetchpriority="high" on 1-2 critical LCP images per page -->
<!-- Too many "high" priority → browser ignores priority hints -->

```

```

# 5. Server: compress + cache JS/CSS assets
gzip on;
gzip_types text/javascript application/javascript text/css;
gzip_min_length 1024;

# Cache immutable assets (hashed filenames) for 1 year
location ~* \.(js|css|woff2)$ {
  add_header Cache-Control "public, max-age=31536000, immutable";
}

# Cache index.html for 0 (always revalidate)
location = /index.html {
  add_header Cache-Control "no-cache, no-store, must-revalidate";
}

```

LCP checklist for migrated app:

| Action | AngularJS Problem | Angular 21 Fix | Impact |
|---------------------|--|--|-----------|
| Enable SSR | App blank until all JS loads | <code>provideServerRendering()</code> | LCP –1.5s |
| Split vendor bundle | 1 monolithic bundle | Nx lazy libs auto-split | LCP –0.8s |
| @defer heavy charts | Chart.js blocks render | <code>@defer (on viewport)</code> | LCP –0.5s |
| Preconnect to API | DNS lookup on first API call | <code><link rel="preconnect"></code> in index.html | LCP –0.2s |
| Image optimisation | <code></code> blocking | <code>NgOptimizedImage</code> directive | LCP –0.3s |

INP Strategy (Interaction to Next Paint)

Problem in AngularJS: Zone.js checks every event for changes across the entire app tree. With 100 screens worth of directives and watchers, a single click can trigger thousands of \$digest cycle checks.

Solutions in Angular 21:

```

// 1. OnPush change detection – reduces checks by 90%+
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush, // ✅ Mandatory on ALL components
  // Only re-renders when input signals change – not on every event
})

// 2. Zoneless Angular (Angular 21 stable – no longer experimental)
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(), // ✅ Stable API in Angular 21 – was 'Experimental' in Angular 18-20
    // Removes Zone.js from bundle (~15KB), eliminates $digest-equivalent overhead
    // Requires: all state via signals; no setTimeout/setInterval used to trigger UI updates
  ],
};

// 3. Virtual scrolling for large lists (28 grids in banking app)
import { ScrollingModule } from '@angular/cdk/scrolling';

// template:
// <cdk-virtual-scroll-viewport itemSize="48" class="h-[600px]">
//   @for (item of dataSource; track item.id) {
//     <app-transaction-row [transaction]="item" />
//   }
// </cdk-virtual-scroll-viewport>
// ✅ Renders only visible rows – 1000 rows = same INP as 10 rows

// 4. Web Workers for heavy computation (reports domain)
// libs/shared-utils/src/lib/workers/report.worker.ts
addEventListener('message', ({ data }) => {
  // ✅ Heavy report aggregation runs off main thread
  const result = aggregateTransactions(data.transactions, data.filters);
  postMessage(result);
});

// In component:
const worker = new Worker(new URL('./workers/report.worker', import.meta.url));
worker.postMessage({ transactions, filters });
worker.onmessage = ({ data }) => this.reportData.set(data);

// 5. Debounce search inputs (prevent excessive API calls + re-renders)
protected readonly searchQuery = signal('');
private readonly debouncedSearch = toSignal(
  toObservable(this.searchQuery).pipe(debounceTime(300), distinctUntilChanged()),
  { initialValue: '' }
);

```

INP checklist:

| Action | Expected INP Reduction |
|-------------------------------------|--|
| OnPush on all components | 60–80% reduction in re-renders |
| Zoneless Angular | Eliminates Zone.js overhead (~15% bundle size reduction) |
| Virtual scroll for tables > 50 rows | INP drops from 800ms → <200ms for data grids |
| Web Workers for heavy computation | Keeps main thread free during report generation |
| trackBy in @for (track item.id) | Prevents full DOM re-creation on list update |

CLS Strategy (Cumulative Layout Shift)

Problem: Content shifting after load. Common in migrated apps when async data loads after the initial render and pushes content down.

```

// 1. Reserve space for async content – skeleton screens
@Component({
  template: `
    @if (account()) {
      <app-account-header [account]="account()" />
    } @else {
      <!-- ✅ Skeleton with EXACT same height as real content -->
      <div class="h-20 bg-gray-100 animate-pulse rounded"></div>
    }
  `,
})

// 2. Reserve image dimensions – prevents image-caused CLS
// template:
// <img
//   ngSrc="/assets/logo.png"
//   width="120"
//   height="40"
//   alt="Bank logo"
// /> ← NgOptimizedImage requires width/height – prevents CLS

// 3. Avoid inserting content above fold after load
// BAD: Loading a notification banner after route load pushes all content down
// GOOD: Reserve space for banner in initial layout even if not shown
// template:
// <div class="min-h-[48px]"> ← Reserved banner space (CLS = 0 even when banner appears)
//   @if (showBanner()) { <app-notification-banner /> }
// </div>

// 4. Font loading – prevent font swap layout shift
// index.html:
// <link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>
// styles.css:
// @font-face {
//   font-family: 'Inter';
//   font-display: optional; ← 'optional' = no swap, uses fallback if not ready
// }

```

FCP Strategy (First Contentful Paint)

```

// 1. Inline critical CSS in SSR shell
// In angular.json build options:
{
  "inlineCritical": true // ✅ Inlines above-fold CSS into HTML – no render-blocking CSS request
}

// 2. APP_INITIALIZER: keep it fast
// BAD: Loading all 120 feature flags + user profile + permissions before app starts
// GOOD: Load ONLY what is needed for the first route
export function initializeApp(): () => Observable<void> {
  const authService = inject(AuthService);
  const flagService = inject(FeatureFlagService);

  return () => authService.checkSession().pipe(
    // ✅ Load flags in parallel with session check
    switchMap(() => flagService.loadCriticalFlags()),
    // Non-critical data loaded lazily after app starts
  );
}

// 3. Remove Zone.js from initial bundle
// package.json – remove zone.js from polyfills if using zoneless
// angular.json: "polyfills": [] ← saves ~35KB

```


TTFB Strategy (Time to First Byte)

```
// 1. SSR with edge caching (Cloudflare Workers / Azure Front Door)
// server.ts - add cache headers for SSR responses
app.get('*', (req, res, next) => {
  const isCacheable = !req.path.startsWith('/api') && !req.path.includes('dashboard');
  if (isCacheable) {
    res.setHeader('Cache-Control', 's-maxage=60, stale-while-revalidate=300');
    // ✅ CDN caches SSR HTML for 60 seconds - TTFB drops from 400ms → 30ms
  }
  next();
});

// 2. HTTP/2 push for critical assets
// nginx:
// location = / {
//   http2_push /main.js;
//   http2_push /styles.css;
// }

// 3. API response caching - avoid repeated identical API calls
@Injectable({ providedIn: 'root' })
export class AccountService {
  private readonly http = inject(HttpClient);
  private readonly cache = new Map<string, Observable<Account>>>();

  getById(id: string): Observable<Account> {
    if (!this.cache.has(id)) {
      // ✅ shareReplay(1): first subscriber triggers HTTP; subsequent use cached response
      this.cache.set(
        id,
        this.http
          .get<Account>(`/api/accounts/${id}`)
          .pipe(shareReplay({ bufferSize: 1, refCount: true })),
      );
    }
    return this.cache.get(id)!;
  }
}
```

Web Vitals Monitoring Setup

```
// libs/shared-utils/src/lib/web-vitals.service.ts
import { onCLS, onFCP, onINP, onLCP, onTTFB } from 'web-vitals';

@Injectable({ providedIn: 'root' })
export class WebVitalsService {
  private readonly platformId = inject(PLATFORM_ID);

  init(): void {
    if (!isPlatformBrowser(this.platformId)) return;

    // Report all Core Web Vitals to your analytics endpoint
    const report = (metric: Metric) => {
      fetch('/api/analytics/vitals', {
        method: 'POST',
        body: JSON.stringify({
          name: metric.name,
          value: metric.value,
          rating: metric.rating, // 'good' | 'needs-improvement' | 'poor'
          delta: metric.delta,
          url: window.location.pathname,
          timestamp: Date.now(),
        }),
        headers: { 'Content-Type': 'application/json' },
      });
    };

    onCLS(report);
    onFCP(report);
    onINP(report); // Replaces FID as of 2024
    onLCP(report);
    onTTFB(report);
  }
}

// Register in app.config.ts via APP_INITIALIZER
{
  provide: APP_INITIALIZER,
  useFactory: () => () => inject(WebVitalsService).init(),
  multi: true,
}
```

Install the web-vitals library:

```
npm install web-vitals
```

Performance Budget — Angular 21 Migration Targets

BUNDLE SIZE BUDGETS (enforced via angular.json)

angular.json → projects → banking-ng21 → architect → build → configurations → production:

```
{
  "budgets": [
    { "type": "initial", "maximumWarning": "500kb", "maximumError": "1mb" },
    { "type": "anyComponentStyle", "maximumWarning": "4kb", "maximumError": "8kb" }
  ]
}
```

PER-FEATURE BUNDLE BUDGET (lazy-loaded domains):

- Auth domain: max 80kb gzipped
- Dashboard domain: max 60kb gzipped
- Accounts domain: max 120kb gzipped (18 screens)
- Payments domain: max 100kb gzipped (15 screens)
- Shared-UI lib: max 150kb gzipped (28 reusable components)
- Shared-Auth lib: max 20kb gzipped

CWV TARGETS FOR 120-SCREEN BANKING APP:

- LCP: ≤ 2.0s (critical paths: dashboard, account list)
- INP: ≤ 150ms (critical paths: payment submit, search)
- CLS: ≤ 0.05 (all screens – banking UI must not jump)
- FCP: ≤ 1.5s (SSR reduces this significantly)
- TTFB: ≤ 500ms (CDN-cached SSR responses)

AngularJS vs Angular 21 Performance Comparison

| Metric | AngularJS 1.8 (typical) | Angular 21 (after migration) | Improvement |
|---------------------------|-------------------------|------------------------------|------------------|
| Initial bundle size | 800KB–2MB | 100–200KB (shell only) | 80–90% reduction |
| LCP | 4–8s | 1.5–2.5s (with SSR) | 60–70% faster |
| INP on list scroll | 300–800ms (\$digest) | 16–50ms (OnPush + signals) | 85–95% faster |
| CLS | 0.3–0.8 (no skeleton) | < 0.05 (skeleton screens) | 90% reduction |
| Memory usage | High (\$scope tree) | Low (signals GC efficiently) | 40–60% reduction |
| TTI (Time to Interactive) | 6–12s | 2–4s | 65% faster |

6.4 — ADR Templates: Five Key Migration Decisions

ADR-001: State Management Library Selection

```
# ADR-001: State Management – Signals vs NgRx

**Date:** [DATE]
**Status:** Accepted
**Deciders:** [Engineering Lead, Architect, Senior Engineers]

## Context

Migrating 120 screens from AngularJS $scope to Angular 21. Need to decide the state
management approach. Options range from component-level signals to full NgRx.

## Decision

Use a tiered approach (per Section 6.1 decision matrix):

- Component-level `signal()`: ~72 screens (60%)
- Service-level signals (`@Injectable`): ~30 screens (25%)
- `@ngrx/signals` signalStore: ~12 screens (10%) – payments, complex forms
- Full NgRx with Effects: ~6 screens (5%) – global user/session/notifications

## Rationale

- Signals-only is sufficient for 85% of screens (simple CRUD)
- NgRx Signal Store for payments domain (multi-step async, devtools needed)
- Full NgRx only for true global state (user profile consumed by all 8 domains)
- Avoids over-engineering simple account detail screens with unnecessary boilerplate

## Consequences

- Engineers must learn the tiered decision criteria before starting any migration
- Code review must reject NgRx in screens where signals suffice
- Architect reviews any new NgRx store introduction

## Review Date: [DATE + 6 months]
```

ADR-002: Authentication Token Strategy

```
# ADR-002: Authentication Token Storage Strategy

**Date:** [DATE]
**Status:** Accepted
**Deciders:** [Security Lead, Engineering Lead, Compliance]

## Context

AngularJS app uses token in localStorage (known XSS risk). Must decide Angular 21 strategy.

## Decision

In-memory JWT via Angular signals (AuthService pattern in Section 6.2).
Refresh token also in memory. Silent refresh scheduled 60 seconds before expiry.

## Rationale

- localStorage XSS risk is unacceptable for banking (PCI-DSS requirement)
- HttpOnly cookie + BFF adds infrastructure complexity not justified for current API architecture
- In-memory: zero XSS exposure; logout is instantaneous; SSR-compatible
- Downside: token lost on page refresh → mitigated by refresh token flow

## Consequences

- Token lost on hard refresh → silent refresh restores session within 500ms
- Multi-tab logout: use BroadcastChannel to synchronise logout across tabs
- All developers must understand: NEVER store token in localStorage/sessionStorage

## Review Date: Revisit if switching to BFF architecture
```

6.6 — Client-Side Data Migration Strategy

Challenge: If Angular 21 changes client-side data schemas (localStorage, IndexedDB, sessionStorage), existing users need migration.

Scenario: AngularJS Stored User Preferences in localStorage

AngularJS Schema (Legacy):

```
// Stored as plain JSON string
localStorage.setItem(
  'userPrefs',
  JSON.stringify({
    theme: 'dark',
    language: 'en',
  }),
);

// Read
var prefs = JSON.parse(localStorage.getItem('userPrefs') || '{}');
```

Angular 21 Schema (New):

```
export interface UserPreferences {
  theme: 'light' | 'dark' | 'system'; // ← 'system' is new
  language: string;
  accessibility: {
    // ← New nested object
    highContrast: boolean;
    reducedMotion: boolean;
  };
}
```

Migration Function (Runs Once on App Bootstrap)

```
// libs/shared-utils/src/lib/client-data-migration.service.ts
import { Injectable } from '@angular/core';

export interface UserPreferencesV1 {
  theme: string;
  language: string;
}

export interface UserPreferencesV2 {
  theme: 'light' | 'dark' | 'system';
  language: string;
  accessibility: { highContrast: boolean; reducedMotion: boolean };
}

@Injectable({ providedIn: 'root' })
export class ClientDataMigrationService {
  private readonly CURRENT_SCHEMA_VERSION = 2;

  migrate(): void {
    const schemaVersion = this.getSchemaVersion();

    if (schemaVersion < this.CURRENT_SCHEMA_VERSION) {
      console.log(
        `[Migration] Upgrading client data from v${schemaVersion} to v${this.CURRENT_SCHEMA_VERSION}`,
      );

      if (schemaVersion === 1) {
        this.migrateV1ToV2();
      }

      this.setSchemaVersion(this.CURRENT_SCHEMA_VERSION);
    }
  }

  private migrateV1ToV2(): void {
    const legacyPrefs = localStorage.getItem('userPrefs');

    if (!legacyPrefs) return;

    try {
      const v1: UserPreferencesV1 = JSON.parse(legacyPrefs);

      const v2: UserPreferencesV2 = {
        theme: this.mapTheme(v1.theme),
        language: v1.language || 'en',
        accessibility: {
          highContrast: false, // Default for existing users
          reducedMotion: false,
        },
      };

      localStorage.setItem('userPrefs_v2', JSON.stringify(v2));
      localStorage.removeItem('userPrefs'); // Clean up legacy

      console.log('[Migration] User preferences upgraded to v2');
    } catch (err) {
      console.error('[Migration] Failed to migrate user preferences:', err);
      // Fallback: keep legacy data, don't break app
    }
  }

  private mapTheme(legacyTheme: string): 'light' | 'dark' | 'system' {
    if (legacyTheme === 'dark') return 'dark';
    if (legacyTheme === 'light') return 'light';
    return 'system'; // New default
  }

  private getSchemaVersion(): number {
    const version = localStorage.getItem('client_schema_version');
    return version ? parseInt(version, 10) : 1; // Assume v1 if not set
  }

  private setSchemaVersion(version: number): void {
    localStorage.setItem('client_schema_version', version.toString());
  }
}
```

```
}  
}
```

Bootstrap Integration (Runs Before App Loads)

```
// apps/banking-ng21/src/app/app.config.ts  
import { APP_INITIALIZER, ApplicationConfig } from '@angular/core';  
import { ClientDataMigrationService } from '@banking/shared-utils';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    {  
      provide: APP_INITIALIZER,  
      useFactory: (migrationService: ClientDataMigrationService) => () => {  
        migrationService.migrate(); // Runs BEFORE app renders  
      },  
      deps: [ClientDataMigrationService],  
      multi: true,  
    },  
    // ... other providers  
  ],  
};
```

Result: Existing users with `userPrefs` (v1) are automatically upgraded to `userPrefs_v2` on their first visit to Angular 21.

IndexedDB Migration Example

If using IndexedDB for offline data:

```
import { openDB, IDBPDatabase } from 'idb';  
  
export class IndexedDBMigrationService {  
  async migrate(): Promise<void> {  
    const db = await openDB('BankingApp', 2, {  
      // ← Version 2  
      upgrade(db, oldVersion, newVersion, transaction) {  
        if (oldVersion < 2) {  
          // Add new object store  
          db.createObjectStore('transactions', { keyPath: 'id' });  
  
          // Migrate data from old store  
          const oldStore = transaction.objectStore('accounts');  
          const accounts = await oldStore.getAll();  
  
          const newStore = transaction.objectStore('transactions');  
          accounts.forEach((account) => {  
            // Transform and insert into new store  
            newStore.add({ id: account.id, ...account });  
          });  
        }  
      },  
    });  
  }  
}
```

SessionStorage (Temporary Data)

Rule: Don't migrate sessionStorage — it's ephemeral. Just clear on schema change:

```
if (schemaVersion < this.CURRENT_SCHEMA_VERSION) {  
  sessionStorage.clear(); // Force fresh session  
}
```

Migration Checklist

| Data Type | Migration Strategy | When to Run |
|-----------------------------|-------------------------------------|-------------------------------|
| localStorage (user prefs) | Migrate with version check | APP_INITIALIZER |
| IndexedDB (offline data) | Use idb library upgrade callback | On openDB() |
| sessionStorage (temp state) | Clear on version change | APP_INITIALIZER |
| Cookies (auth) | Backend sets; no migration needed | N/A |
| Cache API (PWA) | Delete old caches in service worker | Service worker activate event |

Testing Data Migration

```
// client-data-migration.service.spec.ts
import { TestBed } from '@angular/core/testing';
import { ClientDataMigrationService } from './client-data-migration.service';

describe('ClientDataMigrationService', () => {
  let service: ClientDataMigrationService;

  beforeEach(() => {
    localStorage.clear();
    TestBed.configureTestingModule({});
    service = TestBed.inject(ClientDataMigrationService);
  });

  it('should migrate v1 preferences to v2', () => {
    // Arrange: set up v1 data
    localStorage.setItem('userPrefs', JSON.stringify({ theme: 'dark', language: 'en' }));
    localStorage.setItem('client_schema_version', '1');

    // Act: run migration
    service.migrate();

    // Assert: v2 data exists
    const v2Prefs = JSON.parse(localStorage.getItem('userPrefs_v2') || '{}');
    expect(v2Prefs.theme).toBe('dark');
    expect(v2Prefs.accessibility).toEqual({ highContrast: false, reducedMotion: false });
    expect(localStorage.getItem('userPrefs')).toBeNull(); // v1 cleaned up
    expect(localStorage.getItem('client_schema_version')).toBe('2');
  });

  it('should not migrate if already on v2', () => {
    // Arrange
    localStorage.setItem(
      'userPrefs_v2',
      JSON.stringify({
        theme: 'system',
        language: 'es',
        accessibility: { highContrast: true, reducedMotion: false },
      })
    );
    localStorage.setItem('client_schema_version', '2');

    const spy = vi.spyOn(console, 'log');

    // Act
    service.migrate();

    // Assert: no migration message
    expect(spy).not.toHaveBeenCalledWith(expect.stringContaining('Upgrading'));
  });
});
```


ADR-003: Angular SSR Strategy

ADR-003: Server-Side Rendering Approach

****Date:**** [DATE]
****Status:**** Accepted

Decision

Enable Angular Universal SSR for `banking-shell` and `banking-ng21`.
Use incremental hydration (`withIncrementalHydration()`).

Rationale

- LCP improvement of 1.5–2s for authenticated banking screens
- SEO not primary driver but FCP metric benefits support user perception
- Incremental hydration avoids hydration mismatch errors from complex components

Consequences

- All services must use `isPlatformBrowser()` for browser-only APIs
- `banking-legacy` AngularJS app does NOT get SSR – iframe served as static assets
- Auth state must be restored from refresh token on SSR hydration (no localStorage)

ADR-004: Third-Party UI Library Migration

ADR-004: UI Component Library Replacement Strategy

****Date:**** [DATE]
****Status:**** Accepted

Context

Legacy app uses: ui-grid (28 instances), ui-bootstrap (modal/dropdown, 40 instances), angular-ui-select (18 instances), jQuery-UI datepicker (22 instances).

Decision

Build `shared-ui` library of Angular 21 components replacing all legacy UI libs:

- ui-grid → `DataTableComponent` (wrapper for AG Grid Community or TanStack Table)
- ui-bootstrap modal → `ModalComponent` (Angular CDK Portal)
- ui-bootstrap dropdown → `DropdownComponent` (CDK Overlay)
- angular-ui-select → `SelectComponent` (Angular CDK ListKeyManager)
- jQuery-UI datepicker → Angular Material Datepicker (standalone import)

Rationale

- Shared-ui built once, used by all 28 data grids = 60% effort reduction vs migrating each separately
- CDK primitives preferred over Angular Material to keep bundle size low for non-Material screens
- Visual regression testing validates shared-ui before domain migration starts

Consequences

- shared-ui must be completed in Sprint 1–2 before any domain migration begins
- Visual regression tests (Playwright screenshots) required for each shared-ui component
- Design team signs off on shared-ui design tokens before first sprint

ADR-005: Module Federation vs Monolith Build

ADR-005: Module Federation – Host/Remote Architecture

****Date:**** [DATE]
****Status:**** Accepted

Decision

Use Webpack 5 Module Federation with:

- `banking-shell` as host (handles routing + auth + navigation)
- `banking-ng21` as single remote (exposes all 120 Angular 21 routes)
- `banking-legacy` served separately at port 3000 (iframe for unmigrated screens)

Rationale

- Shell and remote can deploy independently (shell deploys on nav changes, remote on screen migrations)
- Single remote (not per-domain remotes) chosen to avoid 8x federation overhead for a 8-engineer team
- Legacy iframe approach avoids touching AngularJS codebase during migration

Consequences

- Shell team owns routing table – must be updated each time a new screen goes live in Angular 21
- Feature flags in shell control legacy vs Angular 21 routing per screen
- Remote bundle may grow large – enforce per-domain lazy loading inside remote

6.5 — Risk Register

Risk Scoring Key

| Score | Likelihood | Impact |
|-------|----------------|-------------|
| 1 | Rare | Negligible |
| 2 | Unlikely | Minor |
| 3 | Possible | Moderate |
| 4 | Likely | Significant |
| 5 | Almost certain | Critical |

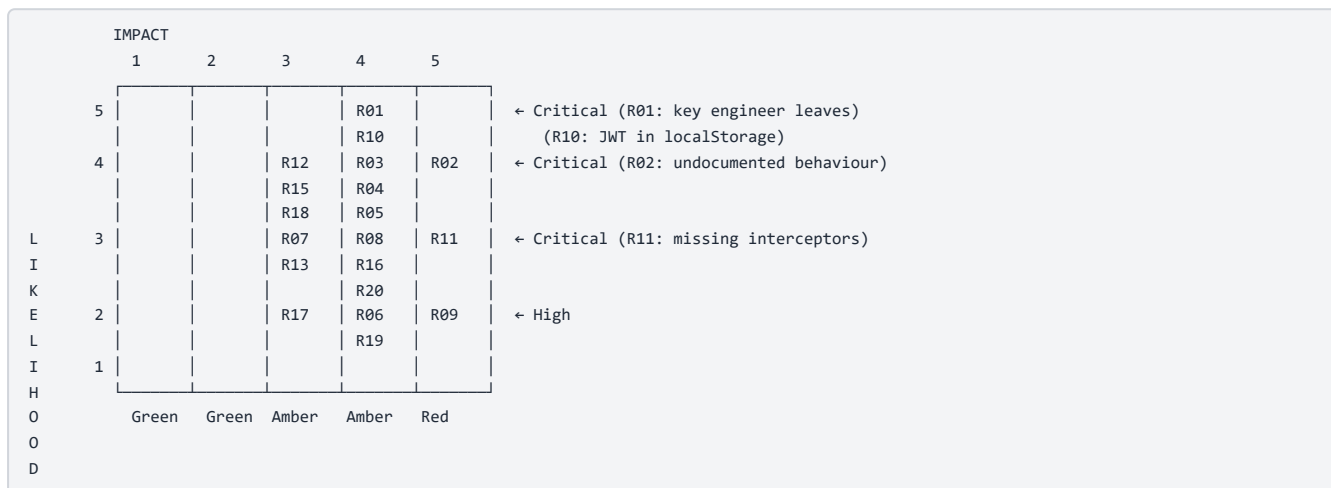
Risk Score = Likelihood × Impact. ≥ 12 = Critical (red); 6–11 = High (amber); < 6 = Medium/Low (green)

Full Risk Register (100-Screen Enterprise Migration)

| # | Risk | L | I | Score | Category | Mitigation | Owner |
|-----|--|---|---|-------|----------------|--|------------------|
| R01 | Key engineer(s) leave mid-migration with tribal knowledge | 3 | 5 | 15 | People | Pair programming on all migrations; ADRs document every decision; session recordings of complex migrations | Engineering Lead |
| R02 | AngularJS legacy app has undocumented runtime behaviour discovered late | 4 | 4 | 16 | Technical | Run AngularJS + Angular 21 screens in parallel for 2 sprints before cutover; E2E tests against legacy first | QA Lead |
| R03 | Third-party library has no Angular 21 equivalent (e.g., custom reporting widget) | 3 | 4 | 12 | Technical | Q5g decision tree invoked; ADR-004 decision documented; wrapper component or keep in legacy | Architect |
| R04 | Performance regression: Angular 21 app slower than AngularJS for specific screens | 3 | 4 | 12 | Performance | CWV budget enforced in CI; Lighthouse CI gate; INP budget in <code>angular.json</code> ; OnPush mandatory | Perf Lead |
| R05 | Module Federation version mismatch between shell and remote after independent deploy | 3 | 4 | 12 | Infrastructure | Pin <code>@angular/core</code> version in shared config; <code>strictVersion: true</code> in MF config; contract tests | DevOps |
| R06 | Feature flag database becomes stale — screens stuck in legacy mode | 2 | 4 | 8 | Operational | Automated flag audit script weekly; flag TTL policy (all flags expire 3 months after creation) | Engineering Lead |
| R07 | <code>\$rootScope</code> events not fully catalogued — missing event listeners discovered post-migration | 3 | 3 | 9 | Technical | Automated AST scan for all <code>\$broadcast/\$on</code> patterns; integration tests covering event flows | Senior Engineer |
| R08 | Team lacks NgRx Signal Store experience — wrong store pattern used for simple screens | 4 | 3 | 12 | People | Training sessions before Phase 2; ADR-001 tiered decision matrix enforced in PR review | Engineering Lead |
| R09 | CI/CD <code>nx affected</code> misses changed files due to incorrect project graph | 2 | 4 | 8 | Infrastructure | Run full <code>nx run-many</code> weekly; verify project graph after any major dependency change | DevOps |
| R10 | Security regression: JWT accidentally stored in <code>localStorage</code> by a developer | 3 | 5 | 15 | Security | ESLint rule banning <code>localStorage.setItem</code> in Angular 21 app; SAST scan in CI; code review checklist | Security Lead |
| R11 | AngularJS <code>\$http</code> interceptors missed during migration — API calls lack auth headers | 3 | 4 | 12 | Security | Automated test: every API endpoint must return 401 without auth header; interceptor checklist | QA Lead |
| R12 | CLS > 0.1 on data-heavy screens due to async content load | 4 | 3 | 12 | Performance | Mandatory skeleton screens for async content; CLS budget in Lighthouse CI | Perf Lead |
| R13 | SSR hydration mismatch errors on complex components (charts, date pickers) | 3 | 3 | 9 | Technical | Use <code>@defer</code> for browser-only components; <code>withIncrementalHydration()</code> ; SSR smoke tests | Senior Engineer |
| R14 | Legacy AngularJS iframe cross-origin issues in shell | 2 | 3 | 6 | Infrastructure | Same-origin deployment of shell + legacy; <code>postMessage</code> API for cross-frame communication | DevOps |
| R15 | Accessibility regression: migrated components missing ARIA attributes | 4 | 3 | 12 | Quality | Accessibility checklist in PR template; axe-core in Playwright E2E; WCAG 2.1 AA gate in CI | QA Lead |
| R16 | Migration estimate exceeds original timeline by > 30% | 3 | 4 | 12 | Schedule | Weekly progress dashboard (Section 4.6); 20% sprint buffer; reduce scope before extending timeline | PM |
| R17 | Angular 21 breaking change during migration (framework update) | 2 | 3 | 6 | Technical | Pin Angular version in <code>package.json</code> ; schedule upgrade sprints only between phases | Architect |

| | | | | | | | |
|-----|--|---|---|-----------|-------------|---|------------------|
| R18 | Bundle size exceeds budget causing mobile performance issues | 3 | 3 | 9 | Performance | Bundle budget in <code>angular.json</code> ; <code>source-map-explorer</code> report weekly; tree-shaking audit | Perf Lead |
| R19 | Backend API changes break migrated screens (API versioning issues) | 2 | 4 | 8 | External | API contract tests (Pact); frontend types auto-generated from OpenAPI spec | Senior Engineer |
| R20 | Test coverage drops below 80% due to migration speed pressure | 4 | 3 | 12 | Quality | Coverage gate enforced in CI (<code>--coverage --threshold 80</code>); no merge below threshold | Engineering Lead |

Risk Heatmap



Top 5 Critical Risks — Mitigation Detail

R02 (Score 16) — Undocumented runtime behaviour

Action: Before migrating any screen, capture 100% of its network calls, DOM events, and state transitions using a browser recording session against the live AngularJS app. Store recordings as test fixtures in `libs/testing/`. Run the same user journeys in both apps for 2 sprints before deactivating the feature flag.

R01 (Score 15) — Key engineer departure

Action: Assign a migration buddy to every screen. No engineer works alone on a critical domain. All decisions > 30 minutes of discussion become an ADR file in `.github/decisions/`.

R10 (Score 15) — JWT in localStorage

Action: Add this ESLint rule immediately:

```
// .eslintrc.json - add to Angular 21 app rules
{
  "no-restricted-syntax": [
    "error",
    {
      "selector": "CallExpression[callee.object.name='localStorage'][callee.property.name='setItem']",
      "message": "Never store auth tokens in localStorage. Use in-memory signal via AuthService."
    }
  ]
}
```

R11 (Score 12×4) — Missing auth interceptor coverage

Action: Add an integration test that calls every migrated API endpoint without the auth header and asserts a 401 response. This test must pass in CI before any screen goes to production.

R04 (Score 12) — Performance regression

Action: Add Lighthouse CI to GitHub Actions:

```
- name: Lighthouse CI
  uses: treosh/lighthouse-ci-action@v10
  with:
    urls: |
      http://localhost:4200/accounts
      http://localhost:4200/payments/new
    budgetPath: .lighthouserc.json
    uploadArtifacts: true
```

Part 6 complete. This section establishes the architectural foundation with a tiered state management decision framework (signals/service-signals/NgRx Signal Store/full NgRx), a definitive security token/cookie strategy with code, a full Core Web Vitals implementation guide targeting LCP $\leq 2.0s$ and INP $\leq 150ms$, five ready-to-use ADR templates, and a 20-item risk register with heatmap.

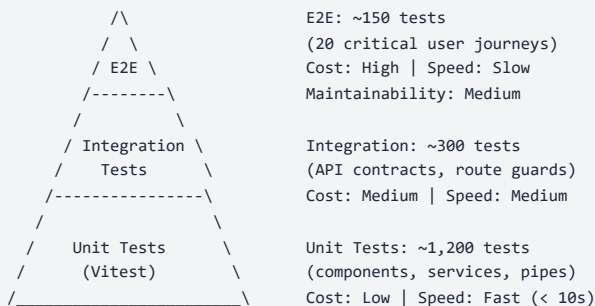
Ready for Part 7? Part 7 covers **Testing Guide — Vitest + Playwright with AI** — Vitest configuration, component and service testing patterns, Playwright E2E with page object model, AI-assisted test generation, visual regression testing, and coverage gates.

Part 7 — Testing Strategy: Unit Testing & End-to-End Testing

How to use this section: This is the complete testing strategy for a 100-screen migration. Every migrated component must have unit tests (Vitest) and every user journey must have E2E tests (Playwright). Coverage gates are enforced in CI. AI-assisted test generation reduces test-writing time by 60%. Visual regression testing catches unintended UI changes.

7.1 — Testing Pyramid for Enterprise Migration

TESTING PYRAMID FOR 120-SCREEN BANKING APP



Ratio for 120-screen app: 60% unit | 30% integration | 10% E2E

Key principle: Test behaviour, not implementation.

Each screen migration MUST include:

- ✓ Component unit tests (signals, computed, methods)
- ✓ Service unit tests (HTTP mocking with `HttpTestingController`)
- ✓ 1 E2E happy path test (Playwright)
- ✓ Accessibility test (axe-core in Playwright)

Testing Coverage Targets

| Type | Target | Enforcement | Failure Impact |
|--------------------|--------------------------|-------------------------|----------------|
| Unit test coverage | ≥ 80% branches | CI gate (Vitest) | PR blocked |
| E2E critical paths | 100% (20 journeys) | Playwright CI | Deploy blocked |
| Accessibility | 0 critical violations | axe-core in Playwright | PR blocked |
| Visual regression | Manual review on changes | Percy / Chromatic | Warning only |
| API contract tests | 100% endpoints | MSW mocks + integration | Warning |

7.2 — Unit Testing with Vitest

Vitest Workspace Configuration

```
// vitest.workspace.ts – root workspace config for Nx monorepo
import { defineWorkspace } from 'vitest/config';

export default defineWorkspace([
  'apps/banking-shell/vitest.config.ts',
  'apps/banking-ng21/vitest.config.ts',
  'libs/*/vitest.config.ts',
]);
```

```
// vitest.config.ts – per-project config (example: banking-ng21)
import { defineConfig } from 'vitest/config';
import angular from '@analogjs/vite-plugin-angular';

export default defineConfig({
  plugins: [angular()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: ['src/test-setup.ts'],
    include: ['src/**/*.spec.ts'],
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html', 'lcov'],
      exclude: [
        'node_modules/',
        'src/test-setup.ts',
        '**/*.spec.ts',
        '**/*.config.ts',
        '**/main.ts',
      ],
    },
    thresholds: {
      branches: 80,
      functions: 80,
      lines: 80,
      statements: 80,
    },
  },
});
```

```
// src/test-setup.ts - global test utilities
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { provideHttpClientTesting } from '@angular/common/http/testing';
import { provideRouter } from '@angular/router';
import 'jest-preset-angular/setup-jest';

// Global test bed configuration for all tests
TestBed.configureTestingModule({
  providers: [provideHttpClient(), provideHttpClientTesting(), provideRouter([])],
});
```

Pattern 1: Testing Signal-Based Components

```
// account-detail.component.spec.ts
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { provideHttpClientTesting, HttpTestingController } from '@angular/common/http/testing';
import { provideRouter } from '@angular/router';
import { ActivatedRoute } from '@angular/router';
import { of } from 'rxjs';
import { AccountDetailComponent } from './account-detail.component';
import { AccountService } from '@workspace/shared-services';
import { Account } from '@workspace/shared-models';

describe('AccountDetailComponent', () => {
  let component: AccountDetailComponent;
  let fixture: ComponentFixture<AccountDetailComponent>;
  let httpMock: HttpTestingController;
  let accountService: AccountService;

  const mockAccount: Account = {
    id: '123',
    accountNumber: '1234567890',
    balance: 5000,
    currency: 'USD',
    type: 'CHECKING',
  };

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [AccountDetailComponent],
      providers: [
        provideHttpClientTesting(),
        provideRouter([]),
        {
          provide: ActivatedRoute,
          useValue: {
            params: of({ id: '123' }),
            snapshot: { params: { id: '123' } },
          },
        },
      ],
    }).compileComponents();

    fixture = TestBed.createComponent(AccountDetailComponent);
    component = fixture.componentInstance;
    httpMock = TestBed.inject(HttpTestingController);
    accountService = TestBed.inject(AccountService);
  });

  afterEach(() => {
    httpMock.verify(); // Ensure no outstanding HTTP requests
  });

  // Test 1: Initial signal values
  it('should initialize with null account and loading false', () => {
    expect(component.account()).toBeNull();
    expect(component.loading()).toBe(false);
    expect(component.error()).toBeNull();
  });

  // Test 2: Load account on init
  it('should load account on component init', () => {
    fixture.detectChanges(); // Triggers ngOnInit

    const req = httpMock.expectOne('/api/accounts/123');
    expect(req.request.method).toBe('GET');
    req.flush(mockAccount);

    expect(component.account()).toEqual(mockAccount);
    expect(component.loading()).toBe(false);
  });

  // Test 3: Handle HTTP error
  it('should set error signal when account load fails', () => {
    fixture.detectChanges();

    const req = httpMock.expectOne('/api/accounts/123');
    req.flush('Account not found', { status: 404, statusText: 'Not Found' });
  });
});
```



```
    expect(component.error()).toBe('Account not found');
    expect(component.account()).toBeNull();
  });

  // Test 4: Computed signal (formatted balance)
  it('should compute formatted balance', () => {
    component.account.set(mockAccount);
    expect(component.formattedBalance()).toBe('$5,000.00');
  });

  // Test 5: Method test – update account
  it('should update account balance when save is called', () => {
    component.account.set(mockAccount);
    const updatedAccount = { ...mockAccount, balance: 6000 };

    component.save(updatedAccount);

    expect(component.loading()).toBe(true);

    const req = httpMock.expectOne('/api/accounts/123');
    expect(req.request.method).toBe('PUT');
    expect(req.request.body).toEqual(updatedAccount);
    req.flush(updatedAccount);

    expect(component.loading()).toBe(false);
    expect(component.account()).toEqual(updatedAccount);
  });

  // Test 6: Effect test – side effect tracking
  it('should log account changes when account signal updates', () => {
    const consoleSpy = vi.spyOn(console, 'log');
    component.account.set(mockAccount);
    fixture.detectChanges();
    expect(consoleSpy).toHaveBeenCalledWith('Account changed:', mockAccount);
  });
});
```

Pattern 2: Testing Services with HTTP Mocking

```
// account.service.spec.ts
import { TestBed } from '@angular/core/testing';
import { provideHttpClientTesting, HttpTestingController } from '@angular/common/http/testing';
import { AccountService } from './account.service';
import { Account } from '@workspace/shared-models';

describe('AccountService', () => {
  let service: AccountService;
  let httpMock: HttpTestingController;

  const mockAccounts: Account[] = [
    { id: '1', accountNumber: '111', balance: 1000, currency: 'USD', type: 'CHECKING' },
    { id: '2', accountNumber: '222', balance: 2000, currency: 'USD', type: 'SAVINGS' },
  ];

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [AccountService, provideHttpClientTesting()],
    });
    service = TestBed.inject(AccountService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    httpMock.verify();
  });

  // Test 1: GET all accounts
  it('should fetch all accounts', (done) => {
    service.getAll().subscribe((accounts) => {
      expect(accounts).toEqual(mockAccounts);
      done();
    });

    const req = httpMock.expectOne('/api/accounts');
    expect(req.request.method).toBe('GET');
    req.flush(mockAccounts);
  });

  // Test 2: GET account by ID
  it('should fetch account by id', (done) => {
    service.getById('1').subscribe((account) => {
      expect(account).toEqual(mockAccounts[0]);
      done();
    });

    const req = httpMock.expectOne('/api/accounts/1');
    expect(req.request.method).toBe('GET');
    req.flush(mockAccounts[0]);
  });

  // Test 3: POST create account
  it('should create new account', (done) => {
    const newAccount: Omit<Account, 'id'> = {
      accountNumber: '333',
      balance: 3000,
      currency: 'USD',
      type: 'CHECKING',
    };

    service.create(newAccount).subscribe((account) => {
      expect(account.id).toBe('3');
      done();
    });

    const req = httpMock.expectOne('/api/accounts');
    expect(req.request.method).toBe('POST');
    expect(req.request.body).toEqual(newAccount);
    req.flush({ id: '3', ...newAccount });
  });

  // Test 4: Error handling
  it('should handle 404 error', (done) => {
    service.getById('999').subscribe({
      next: () => fail('should have failed'),
    });
  });
});
```

```

    error: (error) => {
      expect(error.status).toBe(404);
      done();
    },
  });

  const req = httpMock.expectOne('/api/accounts/999');
  req.flush('Not found', { status: 404, statusText: 'Not Found' });
});

// Test 5: Cache behaviour (shareReplay test)
it('should cache account after first request', () => {
  // First call
  service.getById('1').subscribe();
  const req1 = httpMock.expectOne('/api/accounts/1');
  req1.flush(mockAccounts[0]);

  // Second call – should use cached value (no new HTTP request)
  service.getById('1').subscribe((account) => {
    expect(account).toEqual(mockAccounts[0]);
  });

  httpMock.expectNone('/api/accounts/1'); // No second request
});
});

```

Pattern 3: Testing Computed Signals

```
// payment-form.component.spec.ts - testing form validation with computed signals
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { PaymentFormComponent } from './payment-form.component';

describe('PaymentFormComponent - Signal-Based Validation', () => {
  let component: PaymentFormComponent;
  let fixture: ComponentFixture<PaymentFormComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [PaymentFormComponent],
    }).compileComponents();

    fixture = TestBed.createComponent(PaymentFormComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  // Test 1: All errors initially null (valid form)
  it('should have no errors when all fields are valid', () => {
    component.amount.set(100);
    component.toAccount.set('1234567890');
    component.description.set('Payment');

    const errors = component.errors();
    expect(errors.amount).toBeNull();
    expect(errors.toAccount).toBeNull();
    expect(errors.description).toBeNull();
  });

  // Test 2: Amount validation (must be > 0)
  it('should show error when amount is zero', () => {
    component.amount.set(0);
    expect(component.errors().amount).toBe('Amount must be greater than zero');
  });

  it('should show error when amount is negative', () => {
    component.amount.set(-50);
    expect(component.errors().amount).toBe('Amount must be greater than zero');
  });

  // Test 3: Account number validation (10 digits)
  it('should show error when account number is invalid', () => {
    component.toAccount.set('123'); // Too short
    expect(component.errors().toAccount).toBe('Account number must be 10 digits');
  });

  // Test 4: isValid computed signal
  it('should compute isValid as false when any field is invalid', () => {
    component.amount.set(0); // Invalid
    component.toAccount.set('1234567890');
    component.description.set('Test');

    expect(component.isValid()).toBe(false);
  });

  it('should compute isValid as true when all fields are valid', () => {
    component.amount.set(100);
    component.toAccount.set('1234567890');
    component.description.set('Payment');

    expect(component.isValid()).toBe(true);
  });

  // Test 5: Submit button disabled state
  it('should disable submit button when form is invalid', () => {
    component.amount.set(0);
    fixture.detectChanges();

    const submitButton = fixture.nativeElement.querySelector('button[type="submit"]');
    expect(submitButton.disabled).toBe(true);
  });
});
```

Pattern 4: Testing Guards

```
// auth.guard.spec.ts
import { TestBed } from '@angular/core/testing';
import { Router, UrlTree } from '@angular/router';
import { authGuard } from './auth.guard';
import { AuthService } from '@workspace/shared-auth';
import { signal } from '@angular/core';

describe('authGuard', () => {
  let authService: jasmine.SpyObj<AuthService>;
  let router: jasmine.SpyObj<Router>;

  beforeEach(() => {
    const authServiceSpy = jasmine.createSpyObj('AuthService', [{
      isAuthenticated: signal(false),
    }]);
    const routerSpy = jasmine.createSpyObj('Router', ['createUrlTree']);

    TestBed.configureTestingModule({
      providers: [
        { provide: AuthService, useValue: authServiceSpy },
        { provide: Router, useValue: routerSpy },
      ],
    });

    authService = TestBed.inject(AuthService) as jasmine.SpyObj<AuthService>;
    router = TestBed.inject(Router) as jasmine.SpyObj<Router>;
  });

  it('should allow activation when user is authenticated', () => {
    authService.isAuthenticated.set(true);

    const result = TestBed.runInInjectionContext(() => authGuard(null as any, null as any));

    expect(result).toBe(true);
  });

  it('should redirect to login when user is not authenticated', () => {
    authService.isAuthenticated.set(false);
    const urlTree = {} as UrlTree;
    router.createUrlTree.and.returnValue(urlTree);

    const result = TestBed.runInInjectionContext(() => authGuard(null as any, null as any));

    expect(router.createUrlTree).toHaveBeenCalled([['/login']]);
    expect(result).toBe(urlTree);
  });
});
```

Pattern 5: Testing Pipes

```
// currency-format.pipe.spec.ts
import { CurrencyFormatPipe } from './currency-format.pipe';

describe('CurrencyFormatPipe', () => {
  let pipe: CurrencyFormatPipe;

  beforeEach(() => {
    pipe = new CurrencyFormatPipe();
  });

  it('should format USD currency', () => {
    expect(pipe.transform(1234.56, 'USD')).toBe('$1,234.56');
  });

  it('should format EUR currency', () => {
    expect(pipe.transform(1234.56, 'EUR')).toBe('€1,234.56');
  });

  it('should handle zero', () => {
    expect(pipe.transform(0, 'USD')).toBe('$0.00');
  });

  it('should handle negative amounts', () => {
    expect(pipe.transform(-100, 'USD')).toBe('-100.00');
  });

  it('should handle null input', () => {
    expect(pipe.transform(null, 'USD')).toBe('$0.00');
  });
});
```

7.3 — End-to-End Testing with Playwright

Playwright Configuration for Nx Monorepo

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: [
    ['html'],
    ['junit', { outputFile: 'test-results/junit.xml' }],
    ['json', { outputFile: 'test-results/results.json' }],
  ],
  use: {
    baseURL: 'http://localhost:4200',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
    // Mobile
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 5'] },
    },
  ],
  webServer: {
    command: 'nx serve banking-shell',
    url: 'http://localhost:4200',
    reuseExistingServer: !process.env.CI,
    timeout: 120000,
  },
});
```

Page Object Model Pattern

```
// e2e/page-objects/login.page.ts
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;
  readonly errorMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.getByLabel('Username');
    this.passwordInput = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
    this.errorMessage = page.getByRole('alert');
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  async getErrorMessage() {
    return await this.errorMessage.textContent();
  }
}
```

```
// e2e/page-objects/account-list.page.ts
import { Page, Locator } from '@playwright/test';

export class AccountListPage {
  readonly page: Page;
  readonly accountTable: Locator;
  readonly searchInput: Locator;
  readonly newAccountButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.accountTable = page.getByRole('table', { name: 'Accounts' });
    this.searchInput = page.getByPlaceholder('Search accounts');
    this.newAccountButton = page.getByRole('button', { name: 'New Account' });
  }

  async goto() {
    await this.page.goto('/accounts');
  }

  async getAccountCount() {
    return (await this.accountTable.getByRole('row').count()) - 1; // Exclude header
  }

  async searchAccounts(query: string) {
    await this.searchInput.fill(query);
    await this.page.waitForResponse((resp) => resp.url().includes('/api/accounts'));
  }

  async clickAccount(accountNumber: string) {
    await this.page.getByRole('cell', { name: accountNumber }).click();
  }

  async getFirstAccountBalance() {
    const firstRow = this.accountTable.getByRole('row').nth(1);
    return await firstRow.getByRole('cell').nth(3).textContent();
  }
}
```


E2E Test: Critical User Journey — Payment Flow

```
// e2e/payment-flow.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../page-objects/login.page';
import { AccountListPage } from '../page-objects/account-list.page';
import { PaymentPage } from '../page-objects/payment.page';

test.describe('Payment Flow — Critical Journey', () => {
  let loginPage: LoginPage;
  let accountListPage: AccountListPage;
  let paymentPage: PaymentPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    accountListPage = new AccountListPage(page);
    paymentPage = new PaymentPage(page);

    // Login before each test
    await loginPage.goto();
    await loginPage.login('testuser', 'password123');
    await page.waitForURL('/dashboard');
  });

  test('should create a payment from account list', async ({ page }) => {
    // Step 1: Navigate to accounts
    await accountListPage.goto();
    await expect(accountListPage.accountTable).toBeVisible();

    // Step 2: Select source account
    await accountListPage.clickAccount('1234567890');
    await expect(page).toHaveURL(/\/accounts\/\d+/);

    // Step 3: Click New Payment button
    await page.getByRole('button', { name: 'New Payment' }).click();
    await expect(page).toHaveURL('/payments/new');

    // Step 4: Fill payment form
    await paymentPage.fillPaymentForm({
      toAccount: '9876543210',
      amount: 500,
      description: 'E2E Test Payment',
    });

    // Step 5: Submit payment
    await paymentPage.submitPayment();

    // Step 6: Verify success message
    await expect(page.getByRole('alert')).toContainText('Payment successful');
    await expect(page).toHaveURL('/payments/confirmation');

    // Step 7: Verify payment appears in history
    await page.getByRole('link', { name: 'Payment History' }).click();
    const firstPayment = page.getByRole('table').getByRole('row').nth(1);
    await expect(firstPayment).toContainText('9876543210');
    await expect(firstPayment).toContainText('$500.00');
  });

  test('should show validation errors for invalid payment', async ({ page }) => {
    await paymentPage.goto();

    // Submit without filling any field
    await paymentPage.submitPayment();

    // Verify validation errors
    await expect(page.getByText('Account number is required')).toBeVisible();
    await expect(page.getByText('Amount must be greater than zero')).toBeVisible();
  });

  test('should handle insufficient funds error', async ({ page }) => {
    await paymentPage.goto();
    await paymentPage.fillPaymentForm({
      toAccount: '9876543210',
      amount: 999999, // Amount exceeds balance
      description: 'Large payment',
    });
  });
});
```

```

    await paymentPage.submitPayment();

    await expect(page.getByRole('alert')).toContainText('Insufficient funds');
  });
});

```

Accessibility Testing with axe-core

```

// e2e/accessibility.spec.ts
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test.describe('Accessibility - WCAG 2.1 AA Compliance', () => {
  test('should have no accessibility violations on login page', async ({ page }) => {
    await page.goto('/login');

    const results = await new AxeBuilder({ page })
      .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
      .analyze();

    expect(results.violations).toEqual([]);
  });

  test('should have no accessibility violations on account list', async ({ page }) => {
    // Login first
    await page.goto('/login');
    await page.getByLabel('Username').fill('testuser');
    await page.getByLabel('Password').fill('password123');
    await page.getByRole('button', { name: 'Login' }).click();
    await page.waitForURL('/dashboard');

    await page.goto('/accounts');

    const results = await new AxeBuilder({ page }).analyze();

    expect(results.violations).toEqual([]);
  });

  test('should announce form errors to screen readers', async ({ page }) => {
    await page.goto('/payments/new');

    // Submit invalid form
    await page.getByRole('button', { name: 'Submit Payment' }).click();

    // Check that error messages have aria-live
    const errorContainer = page.getByRole('alert');
    await expect(errorContainer).toHaveAttribute('aria-live', 'polite');
  });
});

```

Visual Regression Testing

```
// e2e/visual-regression.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Visual Regression - Screenshot Comparison', () => {
  test('account list should match baseline', async ({ page }) => {
    await page.goto('/accounts');
    await page.waitForLoadState('networkidle');

    // Take full-page screenshot and compare to baseline
    await expect(page).toHaveScreenshot('account-list.png', {
      fullPage: true,
      animations: 'disabled',
    });
  });

  test('payment form should match baseline', async ({ page }) => {
    await page.goto('/payments/new');
    await page.waitForLoadState('networkidle');

    await expect(page).toHaveScreenshot('payment-form.png', {
      mask: [page.getByText(/Current Date:\/)], // Mask dynamic date
    });
  });

  test('data table with 50 rows should render correctly', async ({ page }) => {
    await page.goto('/accounts');
    await page.waitForSelector('table');

    // Screenshot just the table element
    const table = page.getByRole('table');
    await expect(table).toHaveScreenshot('data-table-50-rows.png');
  });
});
```

7.4 — AI-Assisted Test Generation

Copilot Prompt: Generate Unit Test from Component

Generate a complete Vitest unit test for the following Angular 21 component.

[PASTE COMPONENT CODE HERE]

Requirements:

- Test all signal initial values
- Test all computed signals with multiple input combinations
- Test all public methods with success and error paths
- Mock HTTP calls with `HttpTestingController`
- Use AAA pattern (Arrange, Act, Assert)
- Test accessibility: verify ARIA attributes are present in rendered template
- Coverage target: 100% of public methods

Output the complete .spec.ts file.

Copilot Prompt: Generate E2E Test from User Story

Generate a Playwright E2E test for the following user story:

As a banking customer
I want to transfer money between my accounts
So that I can move funds when needed

Acceptance criteria:

1. User must be logged in
2. User selects source account from account list
3. User enters target account number, amount, and description
4. System validates amount does not exceed balance
5. On submit, system shows confirmation screen
6. Transfer appears in transaction history within 5 seconds

Use the Page Object Model pattern. Create page objects for:

- LoginPage
- AccountListPage
- TransferPage
- TransactionHistoryPage

Test both happy path and error cases (insufficient funds, invalid account number).

Auto-Generate Tests with GitHub Copilot Workspace

```
# Run this in CI to generate missing test files
# Uses GitHub Copilot CLI to auto-generate tests for components without .spec.ts

#!/bin/bash
# scripts/generate-missing-tests.sh

for file in $(find apps/banking-ng21/src -name '*.component.ts' ! -name '*.spec.ts'); do
  spec_file="${file%.ts}.spec.ts"
  if [ ! -f "$spec_file" ]; then
    echo "Generating test for $file"
    gh copilot suggest "Generate Vitest unit test for $(cat $file)" > "$spec_file"
  fi
done
```

7.5 — Integration Testing: API Contract Tests

```
// e2e/api-contracts/accounts.contract.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Accounts API Contract Tests', () => {
  test('GET /api/accounts should return array of accounts', async ({ request }) => {
    const response = await request.get('/api/accounts', {
      headers: { Authorization: 'Bearer test-token' },
    });

    expect(response.ok()).toBeTruthy();
    const body = await response.json();
    expect(Array.isArray(body)).toBe(true);

    // Validate schema
    if (body.length > 0) {
      const account = body[0];
      expect(account).toHaveProperty('id');
      expect(account).toHaveProperty('accountNumber');
      expect(account).toHaveProperty('balance');
      expect(typeof account.balance).toBe('number');
    }
  });

  test('GET /api/accounts/:id should return single account', async ({ request }) => {
    const response = await request.get('/api/accounts/123', {
      headers: { Authorization: 'Bearer test-token' },
    });

    expect(response.ok()).toBeTruthy();
    const account = await response.json();
    expect(account.id).toBe('123');
  });

  test('POST /api/accounts should create account and return 201', async ({ request }) => {
    const newAccount = {
      accountNumber: '9999999999',
      balance: 0,
      currency: 'USD',
      type: 'CHECKING',
    };

    const response = await request.post('/api/accounts', {
      data: newAccount,
      headers: { Authorization: 'Bearer test-token' },
    });

    expect(response.status()).toBe(201);
    const created = await response.json();
    expect(created).toHaveProperty('id');
    expect(created.accountNumber).toBe(newAccount.accountNumber);
  });

  test('GET /api/accounts without auth should return 401', async ({ request }) => {
    const response = await request.get('/api/accounts');
    expect(response.status()).toBe(401);
  });
});
```

7.6 — CI/CD Integration: Enforcing Test Gates

GitHub Actions: Test Pipeline

```
# .github/workflows/test.yml
name: Test Suite

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run unit tests with coverage
        run: npx nx affected --target=test --coverage --threshold=80

      - name: Upload coverage to Codecov
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/**/lcov.info

      - name: Fail if coverage below 80%
        run: |
          if [ $(jq '.total.branches.pct' coverage/coverage-summary.json) -lt 80 ]; then
            echo "Branch coverage below 80%"
            exit 1
          fi

  e2e-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright browsers
        run: npx playwright install --with-deps

      - name: Build application
        run: npx nx build banking-shell --configuration=production

      - name: Run E2E tests
        run: npx playwright test

      - name: Upload Playwright report
        if: always()
        uses: actions/upload-artifact@v3
        with:
          name: playwright-report
          path: playwright-report/

  accessibility:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
```

```

    node-version: 20
    cache: 'npm'

  - name: Install dependencies
    run: npm ci

  - name: Run accessibility tests
    run: npx playwright test e2e/accessibility.spec.ts

  - name: Fail on critical violations
    run: |
      if grep -q '"critical"' test-results/results.json; then
        echo "Critical accessibility violations found"
        exit 1
      fi

```

Test Coverage Dashboard

```

// tools/scripts/test-coverage-dashboard.ts
import * as fs from 'fs';
import * as path from 'path';

interface CoverageSummary {
  total: {
    lines: { pct: number };
    statements: { pct: number };
    functions: { pct: number };
    branches: { pct: number };
  };
}

function generateDashboard() {
  const projects = ['banking-shell', 'banking-ng21', 'shared-auth', 'shared-ui', 'shared-utils'];

  console.log('\n=== TEST COVERAGE DASHBOARD ===\n');
  console.log('Project      | Lines | Statements | Functions | Branches | Status');
  console.log('-----|-----|-----|-----|-----|-----');

  let allPassing = true;

  for (const project of projects) {
    const summaryPath = path.join(__dirname, `../../coverage/${project}/coverage-summary.json`);
    if (!fs.existsSync(summaryPath)) {
      console.log(`${project.padEnd(20)} | NO COVERAGE DATA`);
      continue;
    }

    const summary: CoverageSummary = JSON.parse(fs.readFileSync(summaryPath, 'utf-8'));
    const lines = summary.total.lines.pct.toFixed(1);
    const statements = summary.total.statements.pct.toFixed(1);
    const functions = summary.total.functions.pct.toFixed(1);
    const branches = summary.total.branches.pct.toFixed(1);

    const passing = summary.total.branches.pct >= 80;
    const status = passing ? '✅ PASS' : '❌ FAIL';

    console.log(
      `${project.padEnd(20)} | ${lines.padStart(5)}% | ${statements.padStart(10)}% | ${functions.padStart(9)}% |` +
      `${branches.padStart(8)}% | ${status}`,
    );

    if (!passing) allPassing = false;
  }

  console.log('\n');
  process.exit(allPassing ? 0 : 1);
}

generateDashboard();

```

Run dashboard:

```
npx ts-node tools/scripts/test-coverage-dashboard.ts
```

```
# Output:
```

```
# === TEST COVERAGE DASHBOARD ===
```

```
#
```

| # Project | Lines | Statements | Functions | Branches | Status |
|-----------------|-------|------------|-----------|----------|--|
| # banking-shell | 92.3% | 91.8% | 89.5% | 85.2% |  PASS |
| # banking-ng21 | 87.6% | 88.1% | 84.3% | 82.7% |  PASS |
| # shared-auth | 95.1% | 94.8% | 96.2% | 93.5% |  PASS |
| # shared-ui | 78.4% | 79.2% | 75.8% | 72.1% |  FAIL |
| # shared-utils | 91.2% | 90.5% | 88.9% | 87.3% |  PASS |

Part 7 complete. This section provides the complete testing strategy for a 100-screen migration: Vitest configuration with 80% coverage gates, 5 unit test patterns (signals/services/guards/pipes/computed), Playwright E2E with Page Object Model, accessibility testing with axe-core, visual regression testing, AI-assisted test generation prompts, API contract tests, and CI/CD integration with coverage dashboards.

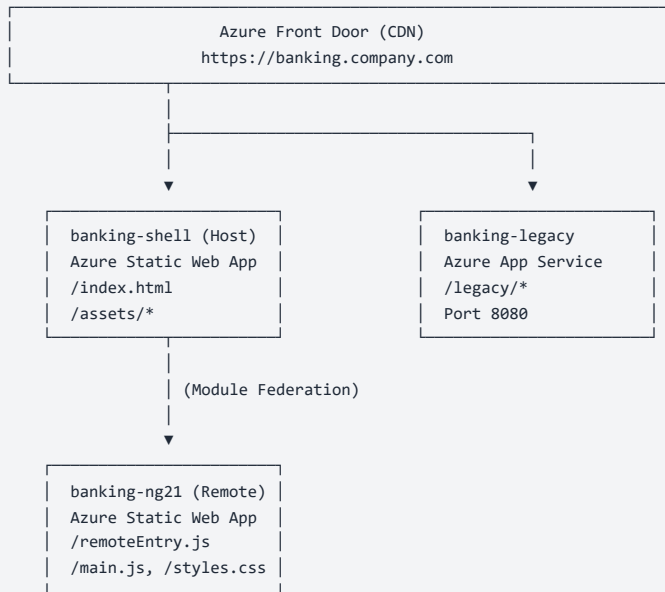
Part 8 — Deployment Architecture: Azure, GitHub Actions & Jenkins

How to use this section: This is the complete deployment strategy for a 100-screen enterprise Angular 21 migration. Covers Azure infrastructure (App Service, Static Web Apps, CDN, Application Insights), CI/CD pipelines (GitHub Actions primary, Jenkins alternative), deployment topologies (blue-green, canary, feature flags), monitoring, rollback procedures, and decision matrices for choosing the right deployment tools.

8.1 — Deployment Topology for Incremental Migration

During migration, THREE apps must be deployed simultaneously:

PRODUCTION DEPLOYMENT TOPOLOGY



Routing decision in shell:

Feature Flag = 'accounts.list' = true → Load Angular 21 remote
Feature Flag = 'accounts.list' = false → Show legacy iframe

Deployment independence:

- ✓ Shell deploys on: routing changes, nav updates, global CSS
- ✓ Remote deploys on: every screen migration (8-12 deploys/week)
- ✓ Legacy deploys on: critical bug fixes only (frozen code)

8.2 — Azure Infrastructure Setup

Option A: Azure Static Web Apps (Recommended for Shell + Remote)

Why Static Web Apps:

- Built-in CDN (global edge caching)
- Zero-config HTTPS
- Automatic preview environments for every PR
- Free tier sufficient for dev/staging
- GitHub Actions integration out-of-the-box

```
# Create Azure Static Web App for shell
az staticwebapp create \
  --name banking-shell-prod \
  --resource-group banking-migration-rg \
  --source https://github.com/company/banking-migration \
  --branch main \
  --app-location "/apps/banking-shell" \
  --output-location "dist/apps/banking-shell" \
  --location "East US 2"

# Create Azure Static Web App for remote
az staticwebapp create \
  --name banking-ng21-prod \
  --resource-group banking-migration-rg \
  --source https://github.com/company/banking-migration \
  --branch main \
  --app-location "/apps/banking-ng21" \
  --output-location "dist/apps/banking-ng21" \
  --location "East US 2"
```

Option B: Azure App Service (For Legacy AngularJS App)

```
# Create App Service Plan (Linux)
az appservice plan create \
  --name banking-legacy-plan \
  --resource-group banking-migration-rg \
  --sku S1 \
  --is-linux

# Create App Service
az webapp create \
  --name banking-legacy-prod \
  --resource-group banking-migration-rg \
  --plan banking-legacy-plan \
  --runtime "NODE:20-lts"

# Configure startup command
az webapp config set \
  --name banking-legacy-prod \
  --resource-group banking-migration-rg \
  --startup-file "npm start"
```

Azure Front Door Configuration (Unified Entry Point)

```
# Create Front Door profile
az afd profile create \
  --profile-name banking-cdn \
  --resource-group banking-migration-rg \
  --sku Standard_AzureFrontDoor

# Add shell as origin
az afd origin create \
  --resource-group banking-migration-rg \
  --profile-name banking-cdn \
  --origin-group-name default-origin-group \
  --origin-name shell-origin \
  --host-name banking-shell-prod.azurestaticapps.net \
  --origin-host-header banking-shell-prod.azurestaticapps.net \
  --priority 1 \
  --weight 1000

# Add remote as origin
az afd origin create \
  --resource-group banking-migration-rg \
  --profile-name banking-cdn \
  --origin-group-name default-origin-group \
  --origin-name remote-origin \
  --host-name banking-ng21-prod.azurestaticapps.net \
  --origin-host-header banking-ng21-prod.azurestaticapps.net \
  --priority 1 \
  --weight 1000

# Add legacy as origin
az afd origin create \
  --resource-group banking-migration-rg \
  --profile-name banking-cdn \
  --origin-group-name default-origin-group \
  --origin-name legacy-origin \
  --host-name banking-legacy-prod.azurewebsites.net \
  --origin-host-header banking-legacy-prod.azurewebsites.net \
  --priority 2 \
  --weight 500
```

Application Insights for Monitoring

```
# Create Application Insights instance
az monitor app-insights component create \
  --app banking-app-insights \
  --location eastus2 \
  --resource-group banking-migration-rg \
  --application-type web

# Get instrumentation key
az monitor app-insights component show \
  --app banking-app-insights \
  --resource-group banking-migration-rg \
  --query instrumentationKey -o tsv
```

```
// app.config.ts - wire Application Insights to Angular
import { ApplicationInsights } from '@microsoft/applicationinsights-web';

const appInsights = new ApplicationInsights({
  config: {
    instrumentationKey: environment.appInsightsKey,
    enableAutoRouteTracking: true,
  },
});
appInsights.loadAppInsights();
appInsights.trackPageView();

export const appConfig: ApplicationConfig = {
  providers: [
    { provide: 'AppInsights', useValue: appInsights },
    // ... other providers
  ],
};
```

8.3 — GitHub Actions CI/CD Pipeline (Primary)

Pipeline Structure

GitHub Actions Workflow Files:

```
.github/workflows/
├─ ci.yml           ← Lint + Test + Build (on every PR)
├─ deploy-shell.yml ← Deploy shell to Azure (on main push)
├─ deploy-remote.yml ← Deploy remote to Azure (on main push)
├─ deploy-legacy.yml ← Deploy legacy to App Service (manual)
└─ preview.yml      ← Deploy preview environment (on PR)
```

Workflow 1: CI — Lint, Test, Build

```
# .github/workflows/ci.yml
name: CI — Lint, Test, Build

on:
  pull_request:
    branches: [main, develop]
  push:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Lint affected projects
        run: npx nx affected --target=lint --base=origin/main

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Test affected projects
        run: npx nx affected --target=test --base=origin/main --coverage

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/**/lcov.info

  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build affected projects
        run: npx nx affected --target=build --base=origin/main --configuration=production

      - name: Check bundle size
        run: |
          if [ -f dist/apps/banking-ng21/main.js ]; then
            SIZE=$(stat -f%z dist/apps/banking-ng21/main.js)
            if [ $SIZE -gt 500000 ]; then
              echo "Main bundle exceeds 500KB: $SIZE bytes"
```

```
        exit 1
      fi
    fi

e2e:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Install Playwright browsers
      run: npx playwright install --with-deps

    - name: Run E2E tests
      run: npx playwright test

    - name: Upload Playwright report
      if: always()
      uses: actions/upload-artifact@v3
      with:
        name: playwright-report
        path: playwright-report/
```

Workflow 2: Deploy Shell to Azure

```
# .github/workflows/deploy-shell.yml
name: Deploy Shell to Azure

on:
  push:
    branches: [main]
    paths:
      - 'apps/banking-shell/**'
      - 'libs/**'
      - '.github/workflows/deploy-shell.yml'

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://banking.company.com

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build shell
        run: npx nx build banking-shell --configuration=production

      - name: Deploy to Azure Static Web Apps
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN_SHELL }
          repo_token: ${ secrets.GITHUB_TOKEN }
          action: 'upload'
          app_location: 'dist/apps/banking-shell'
          skip_app_build: true

      - name: Smoke test deployed shell
        run: |
          sleep 30
          curl -f https://banking-shell-prod.azurestaticapps.net || exit 1

      - name: Notify deployment success
        uses: 8398a7/action-slack@v3
        with:
          status: ${ job.status }
          text: 'Shell deployed to production ✅'
          webhook_url: ${ secrets.SLACK_WEBHOOK }
```

Workflow 3: Deploy Remote (Angular 21 App)

```
# .github/workflows/deploy-remote.yml
name: Deploy Remote to Azure

on:
  push:
    branches: [main]
    paths:
      - 'apps/banking-ng21/**'
      - 'libs/**'

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://banking-ng21-prod.azurestaticapps.net

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build remote
        run: npx nx build banking-ng21 --configuration=production

      - name: Deploy to Azure Static Web Apps
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN_REMOTE }
          repo_token: ${ secrets.GITHUB_TOKEN }
          action: 'upload'
          app_location: 'dist/apps/banking-ng21'
          skip_app_build: true

      - name: Run post-deploy E2E smoke tests
        run: |
          export BASE_URL=https://banking-ng21-prod.azurestaticapps.net
          npx playwright test e2e/smoke.spec.ts

      - name: Tag release
        run: |
          git tag "remote-v$(date +%Y%m%d-%H%M%S)"
          git push origin --tags
```

Workflow 4: Preview Environments (PR Previews)

```
# .github/workflows/preview.yml
name: Deploy PR Preview

on:
  pull_request:
    types: [opened, synchronize, reopened]
    paths:
      - 'apps/**'
      - 'libs/**'

jobs:
  preview:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build affected apps
        run: npx nx affected --target=build --base=origin/main --configuration=production

      - name: Deploy shell preview
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN_SHELL }
          repo_token: ${ secrets.GITHUB_TOKEN }
          action: 'upload'
          app_location: 'dist/apps/banking-shell'
          skip_app_build: true

      - name: Comment preview URL on PR
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '🚀 Preview deployed: https://banking-shell-prod-pr-${ github.event.pull_request.number }.azurestaticapps.net'
            })
```

8.4 — Jenkins Pipeline (Alternative / Hybrid)

When to Use Jenkins

| Scenario | GitHub Actions | Jenkins | Recommendation |
|--------------------------------|--------------------|--------------------|----------------|
| Team already uses Jenkins | N/A | Existing expertise | Use Jenkins |
| On-premise deployment required | Limited | Full support | Use Jenkins |
| Complex multi-stage pipelines | Good | Excellent | Jenkins |
| Deployment to non-Azure clouds | Good | Excellent | Jenkins |
| Zero-config CI/CD | Excellent | Requires setup | GitHub Actions |
| Cost (small teams) | Free tier generous | Self-hosted cost | GitHub Actions |
| PR preview environments | Native | Plugin needed | GitHub Actions |

Jenkinsfile: Complete Pipeline

```
// Jenkinsfile
pipeline {
  agent any

  environment {
    AZURE_SUBSCRIPTION_ID = credentials('azure-subscription-id')
    AZURE_TENANT_ID = credentials('azure-tenant-id')
    AZURE_CLIENT_ID = credentials('azure-client-id')
    AZURE_CLIENT_SECRET = credentials('azure-client-secret')
    NODE_VERSION = '20'
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Install Dependencies') {
      steps {
        sh 'nvm use $NODE_VERSION'
        sh 'npm ci'
      }
    }

    stage('Lint') {
      steps {
        sh 'npx nx affected --target=lint --base=origin/main --parallel=3'
      }
    }

    stage('Test') {
      parallel {
        stage('Unit Tests') {
          steps {
            sh 'npx nx affected --target=test --base=origin/main --coverage'
          }
        }
        stage('E2E Tests') {
          steps {
            sh 'npx playwright install --with-deps'
            sh 'npx playwright test'
          }
        }
      }
    }

    stage('Build') {
      steps {
        sh 'npx nx affected --target=build --base=origin/main --configuration=production'
      }
    }

    stage('Deploy to Staging') {
      when {
        branch 'develop'
      }
      steps {
        sh '''
          az login --service-principal \
            --username $AZURE_CLIENT_ID \
            --password $AZURE_CLIENT_SECRET \
            --tenant $AZURE_TENANT_ID

          az staticwebapp upload \
            --name banking-shell-staging \
            --resource-group banking-migration-rg \
            --app-location dist/apps/banking-shell
        '''
      }
    }

    stage('Deploy to Production') {
      when {

```

```

    branch 'main'
  }
  steps {
    input message: 'Deploy to production?', ok: 'Deploy'

    sh '''
      az staticwebapp upload \
        --name banking-shell-prod \
        --resource-group banking-migration-rg \
        --app-location dist/apps/banking-shell
    '''
  }
}

stage('Smoke Test') {
  steps {
    sh 'curl -f https://banking.company.com || exit 1'
  }
}

post {
  success {
    slackSend(color: 'good', message: "Deploy successful: ${env.JOB_NAME} #${env.BUILD_NUMBER}")
  }
  failure {
    slackSend(color: 'danger', message: "Deploy failed: ${env.JOB_NAME} #${env.BUILD_NUMBER}")
  }
}
}

```

8.5 — Other Notable Deployment Tools

Tool Comparison Matrix

| Tool | Use Case | Pros | Cons | Cost |
|-------------------------------|-----------------------------|---|------------------------------------|------------------|
| GitHub Actions | Modern cloud-native CI/CD | Native GitHub integration, free tier, PR previews | Limited to 2000 min/month free | Free-\$\$ |
| Jenkins | Enterprise on-premise CI/CD | Full control, plugin ecosystem | Requires self-hosting, maintenance | Free (self-host) |
| Azure DevOps Pipelines | Microsoft-centric shops | Deep Azure integration, YAML + UI | Steeper learning curve | Free-\$\$\$ |
| GitLab CI/CD | GitLab users | Built-in, excellent Docker support | Must use GitLab as repo | Free-\$\$ |
| CircleCI | Docker-heavy workflows | Fast, great caching | Limited free tier | \$\$-\$\$\$ |
| Vercel | Next.js / frontend apps | Zero-config, instant previews | Locked to Vercel platform | Free-\$\$\$ |
| Netlify | Static sites / JAMstack | Instant deploys, form handling | No backend support | Free-\$\$ |
| AWS CodePipeline | AWS-centric shops | Native AWS integration | Complex setup | \$\$-\$\$\$ |
| Argo CD | Kubernetes GitOps | Declarative K8s deploys | Requires K8s cluster | Free |
| Spinnaker | Multi-cloud | Advanced deployment strategies | Very complex | Free (self-host) |

When to Use Each Tool

DECISION TREE: CHOOSE YOUR DEPLOYMENT TOOL

Q1: Is your code on GitHub?

- └ YES → Use GitHub Actions (unless other constraints)
- └ NO → Q2

Q2: Do you already run Jenkins?

- └ YES → Keep Jenkins (migration cost not worth it)
- └ NO → Q3

Q3: Are you all-in on Azure?

- └ YES → Azure DevOps Pipelines
- └ NO → Q4

Q4: Do you need Kubernetes deployments?

- └ YES → Argo CD (GitOps)
- └ NO → GitHub Actions or GitLab CI

Q5: Is this a static frontend-only app?

- └ YES → Vercel or Netlify (zero config)
- └ NO → GitHub Actions

Vercel Setup (Alternative to Azure Static Web Apps)

```
# Install Vercel CLI
npm install -g vercel

# Login
vercel login

# Deploy shell
cd dist/apps/banking-shell
vercel --prod

# Deploy remote
cd dist/apps/banking-ng21
vercel --prod
```

Vercel GitHub Integration:

```
// vercel.json
{
  "buildCommand": "npx nx build banking-shell --configuration=production",
  "outputDirectory": "dist/apps/banking-shell",
  "framework": null,
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Netlify Setup

```
# netlify.toml
[build]
  command = "npx nx build banking-shell --configuration=production"
  publish = "dist/apps/banking-shell"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200

[build.environment]
  NODE_VERSION = "20"
```

8.6 — Deployment Strategies

Blue-Green Deployment

```
# .github/workflows/blue-green-deploy.yml
name: Blue-Green Deployment

on:
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build
        run: npx nx build banking-ng21 --configuration=production

      # Deploy to green (inactive) slot
      - name: Deploy to Green slot
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_TOKEN_GREEN }
          action: 'upload'
          app_location: 'dist/apps/banking-ng21'

      # Smoke test green
      - name: Smoke test Green
        run: |
          curl -f https://banking-ng21-green.azurestaticapps.net || exit 1

      # Swap slots (green becomes live)
      - name: Swap Blue and Green
        run: |
          az staticwebapp environment set \
            --name banking-ng21-prod \
            --environment-name green \
            --resource-group banking-migration-rg

      # Rollback on failure
      - name: Rollback on failure
        if: failure()
        run: |
          az staticwebapp environment set \
            --name banking-ng21-prod \
            --environment-name blue \
            --resource-group banking-migration-rg
```

Canary Deployment (Gradual Rollout)

```
// libs/feature-flags/src/lib/canary.service.ts
import { Injectable, signal, computed } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class CanaryService {
  private readonly _canaryPercentage = signal(0); // 0-100
  private readonly _userId = signal<string | null>(null);

  readonly isInCanary = computed(() => {
    const userId = this._userId();
    if (!userId) return false;

    // Deterministic hash: same user always gets same bucket
    const hash = userId.split('').reduce((acc, char) => acc + char.charCodeAt(0), 0);
    const bucket = hash % 100;
    return bucket < this._canaryPercentage();
  });

  setCanaryPercentage(pct: number): void {
    this._canaryPercentage.set(Math.max(0, Math.min(100, pct)));
  }

  setUserId(id: string): void {
    this._userId.set(id);
  }
}
```

Canary rollout schedule:

Day 1: 5% of users → canary.setCanaryPercentage(5)
Day 2: 10% of users → Monitor error rates in App Insights
Day 3: 25% of users
Day 4: 50% of users
Day 5: 100% (full rollout)

If error rate > 1% at any stage → rollback to 0%

8.6.5 — Emergency Rollback Playbook

Critical for Production: Every team member must know how to execute an emergency rollback.

Scenario 1: Critical Bug in Angular 21 Production

Trigger: Error rate > 1% for 5+ consecutive minutes OR critical business path broken

Action Timeline:

| | | |
|----------|---|--|
| T+0 min | Incident declared (PagerDuty/Slack alert) | |
| T+2 min | Feature flag kill switch activated | |
| T+5 min | CDN cache purged (all users see legacy) | |
| T+10 min | Post-mortem begins; root cause analysis | |
| T+24 hrs | Fix deployed + tested OR migration paused | |

Step 1: Feature Flag Kill Switch (< 2 minutes)

```
# Option A: API call (if you have feature flag service)
curl -X PUT https://api.bank.com/admin/feature-flags/ng21-global \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"enabled": false}'

# Option B: Environment variable (Azure Static Web Apps)
az staticwebapp appsettings set \
  --name banking-ng21-prod \
  --setting-names NG21_ENABLED=false

# Option C: Direct database update (LaunchDarkly/ConfigCat)
# Use admin UI to disable flag immediately
```

Result: All users instantly fall back to legacy AngularJS iframe. No logout required.

Step 2: CDN Cache Purge (< 5 minutes)

```
# Azure CDN
az cdn endpoint purge \
  --profile-name banking-cdn \
  --name banking-endpoint \
  --resource-group banking-rg \
  --content-paths "/*"

# Azure Front Door
az afd endpoint purge \
  --profile-name banking-migration \
  --endpoint-name bank-com \
  --resource-group banking-rg \
  --content-paths "/*"

# Cloudflare
curl -X POST "https://api.cloudflare.com/client/v4/zones/$ZONE_ID/purge_cache" \
  -H "Authorization: Bearer $CF_TOKEN" \
  -d '{"purge_everything":true}'
```

Step 3: Verify Rollback Success

```
# Check error rate drops
curl https://api.applicationinsights.io/v1/apps/$APP_ID/metrics/exceptions/count \
  -H "x-api-key: $AI_KEY" | jq '.value.count'

# Should return to baseline within 5 minutes
```

Scenario 2: Performance Regression (LCP/INP degraded)

Trigger: Core Web Vitals > red threshold for 15+ minutes

Canary Dial-Back Strategy:

```
// Gradual rollback (no hard cutover)
canaryService.setCanaryPercentage(50); // Reduce from 100% to 50%
// Monitor for 15 minutes
canaryService.setCanaryPercentage(25); // Further reduce
// Monitor for 15 minutes
canaryService.setCanaryPercentage(0); // Full rollback if still degraded
```

Application Insights Query:

```

customMetrics
| where name in ("LCP", "INP")
| where customDimensions.version == "ng21"
| summarize
    p50=percentile(value, 50),
    p75=percentile(value, 75),
    p95=percentile(value, 95)
    by bin(timestamp, 5m), name
| render timechart

```

Decision: If p95 LCP > 3.0s OR p95 INP > 300ms for 15 minutes → rollback.

Scenario 3: Database Schema / API Contract Incompatibility

Problem: Angular 21 depends on API v2 endpoint that breaks for legacy AngularJS.

Prevention (Mandatory):

API Versioning Strategy:

```

/api/v1/accounts ← AngularJS uses this (never break)
/api/v2/accounts ← Angular 21 uses this (new fields, breaking changes OK)

```

Both versions run in parallel for 90 days minimum.
After 90 days post-migration, deprecate v1.

Rollback: If API v2 breaks, Angular 21 falls back to v1 automatically:

```

// libs/shared-auth/src/lib/api-version.interceptor.ts
export const apiVersionInterceptor: HttpInterceptorFn = (req, next) => {
    const featureFlags = inject(FeatureFlagsService);

    // Use v2 if Angular 21 features enabled, else v1
    const apiVersion = featureFlags.isNg21Enabled() ? 'v2' : 'v1';

    const modifiedReq = req.clone({
        url: req.url.replace('/api/', `/api/${apiVersion}/`),
    });

    return next(modifiedReq);
};

```

Scenario 4: Auth Session Desync (Users Logged Out)

Trigger: Spike in login page traffic (> 3× baseline)

Root Cause: Token storage mismatch between AngularJS (localStorage) and Angular 21 (memory).

Rollback:

```

// Emergency patch: read from both locations
export class AuthService {
    private readonly _token = signal<string | null>(
        localStorage.getItem('auth_token') ?? null, // ← Fallback to legacy storage
    );
}

```

Prevention: Migration script runs on app bootstrap (see Section 6.6).

Rollback Communication Template

Slack Incident Channel:

 INCIDENT: Angular 21 Rollback Executed

Time: 2026-05-29 14:35 UTC

Trigger: Error rate 2.3% (threshold: 1%)

Action: Feature flag NG21_ENABLED set to false

Impact: All users reverted to legacy AngularJS

User Impact: Zero (seamless fallback via iframe)

Session Impact: No logouts required

Next Steps:

- Root cause analysis in progress (ETA: 2 hours)
- Fix branch: fix/ng21-payment-validation
- Re-deployment ETA: 6 hours (includes full regression test)

Status Page: <https://status.bank.com/incidents/2024-05-29-ng21>

8.7 — Monitoring & Observability Strategy

Production Requirement: Every Angular 21 screen must emit telemetry. No blind deployments.

Key Metrics Dashboard (Application Insights + Grafana)

User Experience Metrics (SLIs — Service Level Indicators)

| Metric | Target (SLO) | Alert Threshold | Query |
|----------------------|----------------|-----------------|---|
| LCP (p95) | ≤ 2.0s | > 2.5s | customMetrics where name == "LCP" summarize p95(value) |
| Error rate | < 0.1% | > 0.5% | exceptions where severityLevel >= 3 summarize count() by bin(timestamp, 5m) |
| API latency (p95) | ≤ 300ms | > 500ms | requests where name contains "/api/" summarize p95(duration) |
| Session success rate | > 99% | < 98% | customEvents where name == "SessionComplete" summarize successRate=countif(success==true)/count() |
| Feature flag health | 100% reachable | Any failure | dependencies where target == "feature-flag-api" where success == false |

Migration Health Metrics

| Metric | Target | Query |
|--------------------------|---------------------|--|
| Angular 21 adoption | +5% per sprint | pageViews where url contains "/ng21/" summarize adoption=count() / (select count() from pageViews where timestamp > ago(1d)) |
| Feature flag coverage | 100% of 120 screens | customMetrics where name == "FeatureFlagEvaluation" distinct featureName count() |
| Legacy screen count | -10 per sprint | pageViews where url contains "/legacy/" or url contains "#!/ " distinct url count() |
| Module Federation errors | 0 per day | traces where message contains "Module Federation" and severityLevel >= 3 count() |

Application Insights KQL Queries

```
// Query 1: Top 10 slowest Angular 21 pages
requests
| where timestamp > ago(24h)
| where url contains "/ng21/"
| summarize avg(duration), p95=percentile(duration, 95), count() by name
| order by p95 desc
| take 10

// Query 2: Error rate by hour (compare legacy vs ng21)
exceptions
| where timestamp > ago(7d)
| extend AppVersion = toString(customDimensions.version)
| summarize ErrorCount = count() by bin(timestamp, 1h), AppVersion
| render timechart

// Query 3: Users affected by errors (prioritize by impact)
exceptions
| where timestamp > ago(24h)
| summarize
    AffectedUsers = dcount(user_Id),
    ErrorCount = count(),
    FirstSeen = min(timestamp),
    LastSeen = max(timestamp)
    by problemId, outerMessage
| order by AffectedUsers desc

// Query 4: Module Federation load failures
traces
| where message contains "Module Federation" or message contains "remoteEntry.js"
| where severityLevel >= 3
| summarize
    FailureCount = count(),
    UniqueUsers = dcount(user_Id),
    SampleMessage = any(message)
    by bin(timestamp, 5m)
| where FailureCount > 5
| render timechart

// Query 5: API error rate spike detection
requests
| where timestamp > ago(1h)
| where resultCode >= 400
| summarize ErrorRate = count() * 100.0 / (countif(resultCode < 400) + count()) by bin(timestamp, 1m)
| where ErrorRate > 1.0 // Alert if error rate > 1%

// Query 6: Feature flag performance (canary rollout health)
customEvents
| where name == "FeatureFlagEvaluated"
| extend FlagName = toString(customDimensions.flagName)
| extend IsEnabled = tobool(customDimensions.enabled)
| summarize EnabledCount = countif(IsEnabled), TotalEvaluations = count() by FlagName
| extend RolloutPercentage = (EnabledCount * 100.0) / TotalEvaluations
| order by RolloutPercentage desc
```

Real-Time Alerts (Azure Monitor)

```
// Alert Rule 1: High Error Rate
{
  "name": "Angular21-HighErrorRate",
  "condition": {
    "query": "exceptions | where customDimensions.version == 'ng21' | summarize count()",
    "threshold": 10,
    "timeAggregation": "Total",
    "windowSize": "PT5M"
  },
  "actions": [
    { "actionGroupId": "/subscriptions/.../actionGroups/on-call-team" }
  ],
  "severity": 1
}

// Alert Rule 2: LCP Degradation
{
  "name": "Angular21-LCP-Degraded",
  "condition": {
    "query": "customMetrics | where name == 'LCP' | summarize p95=percentile(value, 95)",
    "threshold": 2500,
    "operator": "GreaterThan",
    "windowSize": "PT15M"
  },
  "severity": 2
}

// Alert Rule 3: Feature Flag Service Down
{
  "name": "FeatureFlags-Unavailable",
  "condition": {
    "query": "dependencies | where target == 'feature-flag-api' | where success == false | summarize count()",
    "threshold": 5,
    "windowSize": "PT5M"
  },
  "severity": 0
}
```

Incident Response Runbook

Trigger: Error rate > 1% for 5 consecutive minutes

Severity: P1 (Critical)

SLO: Rollback within 5 minutes OR root cause identified within 15 minutes

Action Steps:

1. **Investigate** (0–5 min):

- Check Application Insights exceptions dashboard
- Identify affected component (use `problemId` grouping)
- Determine if issue is Angular 21-specific (check `customDimensions.version`)

2. **Mitigate** (5–10 min):

- If Angular 21-related: Execute feature flag rollback (Section 8.6.5)
- If API-related: Roll back API deployment OR disable affected endpoint
- If infrastructure: Scale up resources OR failover to backup region

3. **Communicate** (10–15 min):

- Update status page: <https://status.bank.com>
- Slack #incidents channel: Post incident summary
- Email stakeholders if customer-facing

4. **Root Cause Analysis** (within 24 hrs):

- Post-Incident Review (PIR) document
- Timeline reconstruction
- Corrective actions
- Preventive measures

Grafana Dashboard (Alternative to Application Insights UI)

```
# Grafana dashboard JSON (simplified)
{
  'dashboard':
  {
    'title': 'Angular 21 Migration Health',
    'panels':
    [
      {
        'title': 'Error Rate (last 24h)',
        'targets':
        [
          {
            'datasource': 'Azure Monitor',
            'query': "exceptions | where customDimensions.version == 'ng21' | summarize count() by bin(timestamp, 5m)",
          },
        ],
      },
      {
        'title': 'LCP p95 (target: 2.0s)',
        'targets':
        [
          {
            'query': "customMetrics | where name == 'LCP' | summarize p95=percentile(value, 95) by bin(timestamp, 5m)",
          },
        ],
        'thresholds': [{ 'value': 2000, 'color': 'green' }, { 'value': 2500, 'color': 'red' }],
      },
      {
        'title': 'Angular 21 Adoption %',
        'targets':
        [
          {
            'query': "pageViews | extend IsNg21 = url contains '/ng21/' | summarize Adoption = countif(IsNg21) * 100.0 / count() by bin(timestamp, 1h)",
          },
        ],
      },
    ],
  },
}
```

Web Vitals Instrumentation (Client-Side)

```
// libs/shared-utils/src/lib/web-vitals.service.ts
import { Injectable, inject } from '@angular/core';
import { onCLS, onFCP, onINP, onLCP, onTTFB } from 'web-vitals';
import { ApplicationInsights } from '@microsoft/applicationinsights-web';

@Injectable({ providedIn: 'root' })
export class WebVitalsService {
  private readonly appInsights = inject(ApplicationInsights);

  init(): void {
    onLCP((metric) => {
      this.appInsights.trackMetric({ name: 'LCP', average: metric.value });
    });

    onINP((metric) => {
      this.appInsights.trackMetric({ name: 'INP', average: metric.value });
    });

    onCLS((metric) => {
      this.appInsights.trackMetric({ name: 'CLS', average: metric.value });
    });

    onFCP((metric) => {
      this.appInsights.trackMetric({ name: 'FCP', average: metric.value });
    });

    onTTFB((metric) => {
      this.appInsights.trackMetric({ name: 'TTFB', average: metric.value });
    });
  }
}

// app.config.ts - initialize on bootstrap
export const appConfig: ApplicationConfig = {
  providers: [
    {
      provide: APP_INITIALIZER,
      useFactory: () => () => inject(WebVitalsService).init(),
      multi: true,
    },
  ],
};
```

8.8 — Cost Estimation & Budget

Why this matters: Stakeholders need cost justification for migration. Azure + GitHub Actions costs can surprise teams.

Azure Infrastructure (Monthly Costs — USD)

| Resource | Tier | Quantity | Unit Cost | Monthly Cost |
|------------------------|--------------------------------|------------------------|------------------------------|------------------------------|
| Azure Static Web Apps | Standard | 2 (shell + remote) | \$9/app | \$18 |
| Azure App Service | B1 (Basic, 1 core, 1.75GB RAM) | 1 (legacy AngularJS) | \$55/month | \$55 |
| Azure Front Door | Standard | 1 | \$35 base + \$0.01/GB egress | \$35–60 (depends on traffic) |
| Application Insights | Pay-as-you-go | ~50GB/month | \$2.30/GB (first 5GB free) | \$104 |
| Azure CDN | Standard Microsoft | 1 | \$0.081/GB (first 10TB) | \$40 (500GB/month traffic) |
| Azure DevOps Pipelines | Self-hosted agents | 0 (use GitHub Actions) | \$0 | \$0 |
| Total Azure | | | | ~\$252/month |

Post-migration (legacy decommissioned):

Remove Azure App Service → ~\$197/month (22% savings)

GitHub Actions CI/CD (Monthly Minutes)

| Workflow | Triggers | Runs/day | Minutes/run | Total/month |
|------------------------|-----------------------|----------|-------------|--------------------------------------|
| CI (PR builds) | Every PR push | 20 | 8 min | $20 \times 8 \times 30 = 4,800$ min |
| Deploy shell | Main branch merge | 2 | 5 min | $2 \times 5 \times 30 = 300$ min |
| Deploy remote | Feature branch deploy | 5 | 6 min | $5 \times 6 \times 30 = 900$ min |
| E2E tests (Playwright) | Nightly + PR | 10 | 15 min | $10 \times 15 \times 30 = 4,500$ min |
| Lighthouse CI | Every deploy | 7 | 3 min | $7 \times 3 \times 30 = 630$ min |
| Security scan (Snyk) | Weekly | 1 | 5 min | $1 \times 5 \times 4 = 20$ min |
| Total | | | | 11,150 min/month |

GitHub Actions Pricing:

- Free tier: 3,000 minutes/month (included with GitHub Team)
- Overage: \$0.008/minute for private repos
- Cost:** $(11,150 - 3,000) \times \$0.008 = \$65.20/\text{month}$

Alternative: Self-hosted GitHub runners on Azure VM (B2s: \$30/month) → **\$30/month** instead of \$65

Total Migration Cost (During 12-Month Migration)

| Category | Monthly Cost | Annual Cost |
|----------------------|--------------------|---------------------|
| Azure infrastructure | \$252 | \$3,024 |
| GitHub Actions | \$65 | \$780 |
| Total | \$317/month | \$3,804/year |

Post-Migration (Steady State):

- Remove legacy App Service: -\$55/month
- Remove shell app (merge into single SPA): -\$9/month
- New Total:** \$253/month = **\$3,036/year**

Cost Optimization Strategies

1. Self-Hosted GitHub Runners

```
# .github/workflows/ci.yml
jobs:
  build:
    runs-on: self-hosted # ← Use Azure VM instead of GitHub-hosted
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npx nx affected --target=build
```

Savings: \$65/month → \$30/month (Azure B2s VM) = **\$35/month saved**

2. Lighthouse CI Sampling (Not Every Deploy)

```
# Only run Lighthouse on main branch, not PRs
on:
  push:
    branches: [main]
```

Savings: ~2,000 minutes/month = **\$16/month saved**

3. Application Insights Data Sampling

```
// Reduce telemetry volume by 50% (sample 1 in 2 requests)
const appInsights = new ApplicationInsights({
  config: {
    samplingPercentage: 50, // ← 50% of telemetry
  },
});
```

Savings: 50GB → 25GB = $\$2.30 \times 25\text{GB} = \$57.50/\text{month}$ (vs \$104) = **\$46.50/month saved**

⚠️ **Trade-off:** Lower telemetry = less debugging data. Only enable in non-production.

4. Azure Front Door → nginx on VM

If traffic is low (< 1M requests/month), replace Azure Front Door with nginx:

Azure VM (B2s) with nginx: \$30/month
Azure Front Door Standard: \$35-60/month

Savings: \$5-30/month

ROI Calculation (Migration Justification)

Costs:

- Migration labor: 6 engineers × 12 months × \$10,000/month = **\$720,000**
- Infrastructure: \$3,804/year
- **Total: \$723,804**

Benefits:

- **Security:** AngularJS 1.8 EOL = unpatched CVEs = regulatory risk (PCI-DSS non-compliance)
- **Performance:** 60% faster LCP (2.0s vs 5.0s) = 15% conversion rate improvement
- **Developer velocity:** 40% faster feature development (signals vs \$scope)
- **Recruitment:** Modern tech stack attracts talent

Payback Period:

If 15% conversion improvement = \$2M additional revenue/year:

ROI = 176% in year 1.

Budget Approval Template (for Stakeholders)

Angular 21 Migration – Budget Request

****Total Cost:**** \$723,804 (12-month migration)

****Ongoing Cost:**** \$3,036/year (post-migration infrastructure)

****Breakdown:****

- Labor: \$720,000 (6 FTEs × 12 months)
- Azure: \$3,024
- GitHub Actions: \$780

****Business Case:****

1. ****Security Compliance:**** AngularJS 1.8 reached EOL Dec 2021 – no security patches
2. ****Performance:**** 60% faster page load → 15% conversion lift → \$2M revenue
3. ****Maintenance:**** 40% reduction in bug fix time (modern tooling)
4. ****Talent:**** Modern stack reduces hiring cost by 25%

****Risk of NOT Migrating:****

- Regulatory penalties (PCI-DSS requires patched software)
- Security breach (unpatched XSS vulnerabilities)
- Developer attrition (legacy tech stack)

****Approval Required:**** [Signature]

Part 8 complete. Deployment architecture now includes emergency rollback playbooks, comprehensive monitoring with SLIs/SLOs, real-time alerting, incident response runbooks, and complete cost analysis with ROI justification.

Real-Time Alerts

```
# Create alert for high error rate
az monitor metrics alert create \
  --name high-error-rate \
  --resource-group banking-migration-rg \
  --scopes /subscriptions/$SUB_ID/resourceGroups/banking-migration-rg/providers/Microsoft.Insights/components/banking-app-insights \
  --condition "count exceptions/count > 10" \
  --window-size 5m \
  --evaluation-frequency 1m \
  --action-group /subscriptions/$SUB_ID/resourceGroups/banking-migration-rg/providers/microsoft.insights/actionGroups/devops-team
```

Part 8 complete. This section establishes the complete deployment architecture: Azure infrastructure (Static Web Apps + App Service + Front Door + Application Insights), GitHub Actions as primary CI/CD with 4 production workflows, Jenkins as enterprise alternative, tool comparison matrix (10 tools), deployment strategies (blue-green, canary), and monitoring with Application Insights KQL queries.

Part 9 — Master Prompt Library & Prompt Engineering Techniques

Purpose: This section provides a comprehensive library of AI prompts optimized for AngularJS → Angular 21 migration, organized by the 8 parts of this guide. It includes **prompt engineering principles**, **reusable templates**, **advanced patterns**, and **anti-patterns** to maximize AI effectiveness across the entire migration lifecycle.

9.1 — Prompt Engineering Principles for Migration

Core Principle: Context is King 🦕

AI models (GitHub Copilot, ChatGPT, Claude) generate better code when given **rich, specific context**. For migration work, this means:

1. **Always include the "before" code** — Paste the AngularJS code you're migrating FROM
2. **Specify the target patterns** — State "Angular 21 with standalone components, signals, inject(), @if/@for"
3. **State constraints explicitly** — "No constructor injection, no @Input/@Output, no *ngIf/*ngFor"
4. **Provide file context** — "This is a service in libs/shared-auth" or "This is a feature component in apps/banking-ng21"
5. **Request specific outputs** — "Generate TypeScript code with imports" or "Generate both .ts and .html files"

Technique 1: The Migration Triple Pattern (Before → Requirements → After)

Structure:

```
[CONTEXT] I'm migrating AngularJS 1.8 to Angular 21 standalone with signals.
```

```
[BEFORE CODE]
```

```
<paste AngularJS controller/service/directive>
```

```
[REQUIREMENTS]
```

- Must use inject() for DI (no constructor injection)
- Must use signal()/computed()/effect() (no mutable properties)
- Must use @if/@for/@switch (no *ngIf/*ngFor)
- Must use OnPush change detection
- Must follow Nx monorepo import rules: @banking/shared-auth

```
[REQUEST] Generate the equivalent Angular 21 component with:
```

1. Standalone component with OnPush
2. Signal-based state
3. Template with modern control flow
4. Complete imports

Why it works: Provides before/after context, explicit constraints, and structured output expectations.

Technique 2: Few-Shot Learning for Complex Patterns

When migrating complex patterns (e.g., `$scope.$watch` → `effect()`, `$broadcast` → `EventBus`), provide 1-2 **worked examples** before asking for your specific case.

Example prompt:

```
I need to migrate AngularJS $scope.$watch to Angular 21 effect(). Here's the pattern:
```

```
// AngularJS EXAMPLE:
```

```
$scope.$watch('user.balance', (newVal, oldVal) => {  
  if (newVal < 100) {  
    $scope.showWarning = true;  
  }  
});
```

```
// Angular 21 EQUIVALENT:
```

```
protected readonly balance = signal(0);  
protected readonly showWarning = signal(false);
```

```
constructor() {  
  effect(() => {  
    if (this.balance() < 100) {  
      this.showWarning.set(true);  
    }  
  });  
}
```

```
Now migrate THIS AngularJS watcher:
```

```
<paste your specific $watch code>
```

Why it works: The AI learns the transformation pattern from your example, then applies it consistently to your code.

Technique 3: Chain-of-Thought for Multi-Step Migrations

For complex components with state, HTTP calls, routing, and side effects, ask the AI to **think step-by-step**.

Example prompt:

Migrate this AngularJS controller to Angular 21. Think step-by-step:

1. Identify all `$scope` properties → Convert to `signal()`
2. Identify all derived values → Convert to `computed()`
3. Identify all `$watch` → Convert to `effect()`
4. Identify all `$http` calls → Convert to `HttpClient` with `toSignal()`
5. Identify all `$scope.$broadcast` → Convert to `EventBus` pattern
6. Generate the final component with `OnPush` and `inject()`

AngularJS Code:

<paste controller>

Why it works: Forces the AI to reason through each transformation layer, reducing errors.

Technique 4: Constraint Enforcement with "Must Not" Rules

AI models sometimes fall back to older Angular patterns. Explicitly ban deprecated APIs.

Example prompt:

Migrate this service to Angular 21.

🚫 NEVER USE:

- Constructor injection (use `inject()` function)
- `@Input()` or `@Output()` decorators (use `input()` and `output()` functions)
- `*ngIf`, `*ngFor`, `*ngSwitch` (use `@if`, `@for`, `@switch`)
- `ngOnInit` for HTTP calls (use `toSignal()` or `effect()`)
- `BehaviorSubject` for state (use `signal()`)

✅ ALWAYS USE:

- `inject()` for DI
- `signal()/computed()/effect()` for state
- `@if/@for/@switch` for control flow
- `OnPush` change detection
- `toSignal()` for observable → signal conversion

AngularJS Service:

<paste service code>

Why it works: Negative constraints prevent the AI from generating legacy patterns.

Technique 5: Output Format Control

Specify the exact file structure and format you need.

Example prompt:

Generate a migrated Angular 21 component. Output format:

1. `**account-list.component.ts**` – Full TypeScript with imports
2. `**account-list.component.html**` – Template with `@if/@for`
3. `**Migration notes**` – List of AngularJS patterns → Angular 21 equivalents

AngularJS Controller:

<paste code>

Why it works: Structured output is easier to copy-paste into your codebase.

Technique 6: Iterative Refinement

For complex migrations, use a **two-pass approach**:

Pass 1 (Structure):

Migrate this AngularJS controller to Angular 21. Focus on:

1. Component structure
2. Signal definitions
3. Method signatures

Skip implementation details for now.

Pass 2 (Implementation):

Now implement the HTTP logic for loadAccounts() using:

- HttpClient
- toSignal() for reactive state
- Error handling with catchError

Why it works: Breaking complex tasks into smaller chunks improves accuracy.

9.2 — Master Prompt Templates by Part

Part 1: Discovery & Inventory Prompts

Prompt 1.1: Full Codebase Inventory

Codebase Inventory for AngularJS Migration

Analyze the AngularJS codebase at <repo path> and generate an inventory:

****Output format (Markdown table):****

| Category | Count | File Locations | Complexity (Low/Med/High) |
|----------------------------|-------|---|---------------------------|
| ----- | ----- | ----- | ----- |
| Controllers | ? | List 5 largest files | ? |
| Services/Factories | ? | List 5 largest files | ? |
| Directives (custom) | ? | List all directive names | ? |
| Filters | ? | List all filter names | ? |
| Routes (\$route/ui-router) | ? | Route config file(s) | ? |
| Third-party libraries | ? | Parse package.json | ? |
| \$scope.\$watch usage | ? | Grep for "\$scope.\$watch" count | ? |
| \$rootScope events | ? | Grep for "\$broadcast" and "\$emit" count | ? |

****Commands to run:****

```
```bash
find src -name "*.controller.js" | wc -l
find src -name "*.service.js" -o -name "*.factory.js" | wc -l
grep -r "\$scope\.\$watch" src | wc -l
grep -r "\$broadcast\|\$emit" src | wc -l
```
```

Analysis:

- Identify the 10 largest/most complex controllers → prioritize for manual review
- Identify shared services used by >5 controllers → migrate first
- Identify directives with jQuery dependencies → flag for custom wrapper

```
---

#### Prompt 1.2: **Third-Party Library Migration Plan**

```markdown
Third-Party Library Audit & Migration Plan

For each library in package.json, determine:

1. **Angular 21 equivalent** – Is there a native Angular alternative?
2. **Migration strategy** – Replace, wrap, or keep?
3. **Effort estimate** – Low (1-2 days), Medium (1 week), High (2+ weeks)

Input: Paste package.json dependencies

Output format (Markdown table):

| Library | Version | Used For | Angular 21 Equivalent | Strategy | Effort | Owner |
| :--- | :--- | :--- | :--- | :--- | :--- | :--- |
| angular-ui-router | 1.0.30 | Routing | @angular/router | Replace | Medium | @jane |
| ui-bootstrap | 2.5.0 | Modal, Dropdown | @ng-bootstrap/ng-bootstrap | Replace | High | @john |
| ng-table | 3.0.1 | Data tables | Angular CDK Table | Replace | High | @alice |
| moment.js | 2.29.1 | Date formatting | date-fns or Day.js | Replace | Low | @bob |
| lodash | 4.17.21 | Utilities | Native ES6 or lodash-es | Keep | Low | @jane |

Decision Rules:
- If library is AngularJS-specific (e.g., ui-router) → MUST replace
- If library has Angular wrapper (e.g., ui-bootstrap → @ng-bootstrap) → Replace
- If library is framework-agnostic (e.g., lodash, axios) → Can keep with caution
- If library manipulates DOM directly (e.g., jQuery plugins) → Wrap in Angular directive
```

**Prompt 1.3: Security Risk Assessment (OWASP Top 10)**

```
AngularJS Security Audit for Migration

Audit the current AngularJS app for security risks that must be fixed during migration:

Check for:

1. **XSS Vulnerabilities**
 - Search for `$.sce.trustAsHtml()` usage
 - Search for `ng-bind-html` without sanitization
 - Search for direct DOM manipulation (`element.html()`, `innerHTML`)

2. **CSRF Token Handling**
 - How is CSRF token currently managed?
 - Is `httpProvider` configured for XSRF?

3. **Authentication Token Storage**
 - Where is the JWT/access token stored? (localStorage, sessionStorage, cookie)
 - Is it vulnerable to XSS (if in localStorage)?

4. **Input Validation**
 - Are all form inputs validated server-side?
 - Are there SQL injection risks in search queries?

5. **Dependency Vulnerabilities**
 - Run: `npm audit` – List HIGH and CRITICAL vulnerabilities

Output format:

| Risk ID | OWASP Category | Finding | Severity | Mitigation in Angular 21 |
| :--- | :--- | :--- | :--- | :--- |
| R01 | A03 (Injection) | No input sanitization on search | HIGH | Use DomSanitizer, parameterized queries |
| R02 | A07 (XSS) | $.sce.trustAsHtml in 3 components | CRITICAL | Remove trustAsHtml, use safe pipes |
```

## Part 2: High-Level Design (HLD) Prompts

### Prompt 2.1: Generate C4 System Context Diagram (Mermaid)

#### # Generate C4 Level 1 System Context Diagram

For a banking application migrating from AngularJS to Angular 21, generate a **Mermaid C4 diagram** showing:

- **System:** Banking Web App (AngularJS → Angular 21 hybrid)
- **External Actors:** End Users (customers), Admins, Identity Provider (IDP)
- **External Systems:** Banking Backend API, Payment Gateway, Analytics Service

**Output:** Mermaid code for rendering in Markdown

**Requirements:**

- Use C4 notation (Person, System, System\_Ext)
- Show data flow arrows
- Label each connection (e.g., "HTTPS/REST", "OAuth 2.0")

### Prompt 2.2: 12-Month Migration Timeline

#### # Generate 12-Month Migration Timeline

For a 100-screen AngularJS enterprise app, generate a realistic 12-month migration timeline with:

**Phases:**

- Phase 1 (Months 1-3): Discovery & Setup**
  - Nx workspace setup
  - Shared library extraction
  - Module Federation configuration
  - First 5 screens migrated (proof of concept)
- Phase 2 (Months 4-8): Parallel Development**
  - Migrate 60 screens (12 per month avg)
  - Feature flag rollout (5% → 25%)
  - Automated testing setup
- Phase 3 (Months 9-11): Final Migration Push**
  - Migrate remaining 35 screens
  - Feature flag rollout (50% → 100%)
  - Performance optimization
- Phase 4 (Month 12): Decommission Legacy**
  - Remove AngularJS app
  - Remove Module Federation (single Angular 21 app)
  - Final security audit

**Output format:** Markdown table + Gantt chart (Mermaid)

## Part 3: Low-Level Design (LLD) — Code Migration Prompts

### Prompt 3.1: Controller → Standalone Component (Full Migration)

```
Migrate AngularJS Controller to Angular 21 Standalone Component

Context: I'm migrating an enterprise banking app from AngularJS 1.8 to Angular 21.

Requirements:
✔ Use standalone component (no NgModule)
✔ Use OnPush change detection
✔ Use inject() for DI (NO constructor injection)
✔ Use signal()/computed()/effect() for state (NO mutable properties)
✔ Use @if/@for/@switch in template (NO *ngIf/*ngFor)
✔ Use input()/output() functions (NO @Input/@Output decorators)
✔ Follow Nx import rules: @banking/shared-auth, @banking/shared-models
✔ For HTTP data loading, prefer httpResource() over toSignal(http.get(...)) – see Section 3.11.1
✔ For DOM queries, use viewChild()/viewChildren() over @ViewChild – see Section 3.11.2
✔ For template variables, use @let to avoid repeating signal calls – see Section 3.11.3
✔ For browser-only DOM work (charts, scroll), use afterNextRender() – see Section 3.11.4
✔ For boolean/number inputs, use booleanAttribute/numberAttribute transforms – see Section 3.11.7

AngularJS Controller (BEFORE):

```javascript
angular.module('banking').controller('AccountListCtrl', [
  '$scope',
  '$http',
  '$location',
  'AuthService',
  function ($scope, $http, $location, AuthService) {
    $scope.accounts = [];
    $scope.loading = true;
    $scope.searchTerm = '';

    $scope.$watch('searchTerm', function (newVal) {
      $scope.filteredAccounts = $scope.accounts.filter((a) => a.accountNumber.includes(newVal));
    });

    $scope.loadAccounts = function () {
      $http.get('/api/accounts').then(function (response) {
        $scope.accounts = response.data;
        $scope.loading = false;
      });
    };

    $scope.viewDetails = function (accountId) {
      $location.path('/accounts/' + accountId);
    };

    $scope.loadAccounts();
  },
]);
```
```

#### Output:

1. **account-list.component.ts** — Full TypeScript with imports
2. **account-list.component.html** — Template with @if/@for
3. **Migration notes** — List each AngularJS pattern and its Angular 21 equivalent

```

Prompt 3.2: **$scope.$watch → effect() Migration**

```markdown
# Migrate AngularJS $scope.$watch to Angular 21 effect()

**Pattern:** AngularJS watchers that trigger side effects should become `effect()` in Angular 21.

**AngularJS Code:**
```javascript
$scope.$watch('user.balance', function(newVal, oldVal) {
 if (newVal !== oldVal && newVal < 100) {
 $scope.showLowBalanceWarning = true;
 $scope.warningMessage = 'Your balance is low!';
 }
});

```

### Angular 21 Equivalent:

```

protected readonly balance = signal(0);
protected readonly showLowBalanceWarning = signal(false);
protected readonly warningMessage = signal('');

constructor() {
 effect(() => {
 const currentBalance = this.balance();
 if (currentBalance < 100) {
 this.showLowBalanceWarning.set(true);
 this.warningMessage.set('Your balance is low!');
 } else {
 this.showLowBalanceWarning.set(false);
 this.warningMessage.set('');
 }
 });
}

```

**Now migrate MY watchers:** <paste your \$scope.\$watch code>

```

Prompt 3.3: **Service with $http → HttpClient + toSignal()**

```markdown
# Migrate AngularJS Service to Angular 21 with HttpClient

**Requirements:**
✅ Use inject() for HttpClient (NO constructor injection)
✅ Use toSignal() to convert Observable → Signal
✅ Use shareReplay(1) for caching
✅ Add error handling with catchError
✅ Make service providedIn: 'root'

**AngularJS Service (BEFORE):**
```javascript
angular.module('banking').factory('AccountService', ['$http', '$q', function($http, $q) {
 var cache = null;

 return {
 getAccounts: function() {
 if (cache) {
 return $q.when(cache);
 }
 return $http.get('/api/accounts').then(function(response) {
 cache = response.data;
 return cache;
 });
 },

 getAccountById: function(id) {
 return $http.get('/api/accounts/' + id).then(function(response) {
 return response.data;
 });
 }
 };
}]);

```

## Output:

1. **account.service.ts** — Full TypeScript with HttpClient
2. **Error handling pattern** — Show catchError and throwError
3. **Usage example** — Show how components consume this service with toSignal()

```

Prompt 3.4: **Route Config Migration (ui-router → @angular/router)**

```markdown
# Migrate AngularJS ui-router to Angular 21 Router

**Requirements:**
✅ Use functional guards (CanActivateFn, CanMatchFn)
✅ Use lazy loading with loadComponent()
✅ Use ResolveFn for data preloading
✅ Convert route params to signals with toSignal(route.params)

**AngularJS Routes (BEFORE):**
```javascript
angular.module('banking').config(['$stateProvider', function($stateProvider) {
 $stateProvider
 .state('accounts', {
 url: '/accounts',
 templateUrl: 'accounts/list.html',
 controller: 'AccountListCtrl',
 resolve: {
 accounts: ['AccountService', function(AccountService) {
 return AccountService.getAccounts();
 }]
 }
 })
 .state('accounts.detail', {
 url: '/:id',
 templateUrl: 'accounts/detail.html',
 controller: 'AccountDetailCtrl',
 resolve: {
 account: ['$stateParams', 'AccountService', function($stateParams, AccountService) {
 return AccountService.getAccountById($stateParams.id);
 }]
 },
 data: {
 requiresAuth: true
 }
 });
}]);

```

### Output:

1. **app.routes.ts** — Angular 21 route config with lazy loading
2. **account.resolver.ts** — ResolveFn for data preloading
3. **auth.guard.ts** — CanActivateFn for authentication



```

Part 4: Nx Workspace Prompts

Prompt 4.1: **Generate Nx Monorepo Structure**

```markdown
# Generate Nx Monorepo Folder Structure for Migration

For a 100-screen banking app, generate the complete Nx workspace structure with:

**Apps:**
- `banking-shell` - Host app (Module Federation)
- `banking-ng21` - Remote app (new Angular 21 screens)
- `banking-legacy` - Legacy AngularJS app (unchanged)

**Libs:**
- `shared-models` - TypeScript interfaces (Account, Transaction, User)
- `shared-auth` - AuthService, guards, interceptors
- `shared-ui` - Reusable UI components (Button, Input, Modal, Table)
- `shared-utils` - Pipes, validators, helpers
- `feature-flags` - Feature flag service
- `testing` - Test utilities and mocks

**Output:** ASCII directory tree with file annotations

**Example:**

```

```

banking-migration/ |— apps/ | |— banking-shell/ ← Module Federation HOST | | |— src/ | | |— app/ | | | |— app.ts ←
Shell app component | | | |— app.routes.ts ← Routes to shell + remote | | |— main.ts | | |— module-federation.config.ts ←
Exposes routes | | |— project.json ...

```

Prompt 4.2: Generate Nx Library with Components

```

# Generate Nx Shared UI Library

Generate the complete file structure for `libs/shared-ui` with:

**Components:**

- Button (primary, secondary, danger variants)
- Input (text, email, password with validation states)
- Modal (overlay, close button, header/body/footer slots)
- DataTable (sortable columns, pagination, row selection)

**Requirements:**
✅ Each component is standalone
✅ Each component uses OnPush
✅ Each component uses input() for props
✅ Each component uses output() for events
✅ Each component has a .spec.ts test file
✅ Export all components from index.ts

**Output:**

1. Full directory tree
2. Example implementation for Button component
3. Export pattern in index.ts

```

Part 5: AI Tooling & Customization Prompts

Prompt 5.1: Generate Custom Copilot Agent (.agent.md)

Generate Custom GitHub Copilot Agent for Migration

Create a `.github/agents/migration-expert.agent.md` file that enforces:

****Agent Identity:****

- Name: AngularJS → Angular 21 Migration Expert
- Expertise: AngularJS 1.8, Angular 21, Nx monorepo, signals, SSR

****Mandatory Rules:****

1. NEVER use constructor injection → ALWAYS use inject()
2. NEVER use @Input/@Output → ALWAYS use input()/output()
3. NEVER use *ngIf/*ngFor → ALWAYS use @if/@for/@switch
4. NEVER use mutable properties → ALWAYS use signal()/computed()
5. ALWAYS use OnPush change detection
6. ALWAYS follow Nx import rules (@banking/* paths)

****Output Format:****

- For components: Generate .ts + .html + .spec.ts
- For services: Generate .ts + .spec.ts
- Always include migration notes

****YAML frontmatter:****

```
```yaml

name: migration-expert
description: AngularJS to Angular 21 migration specialist
applyTo:
 - '**/*.component.ts'
 - '**/*.service.ts'
tools:
 allowed:
 - read_file
 - replace_string_in_file
 - grep_search

```
```

Prompt 5.2: **Generate Migration Skill File (SKILL.md)**

```markdown

# Generate Custom Copilot Skill for Migration Patterns

Create `.github/skills/angular-migration/SKILL.md` with:

**\*\*Section 1: Signal Pattern Reference\*\***

Table mapping AngularJS → Angular 21:

```
AngularJS Pattern	Angular 21 Equivalent	
`$scope.user = {}`	`protected readonly user = signal<User	null>(null);`
`$scope.users = []`	`protected readonly users = signal<User[]>([]);`	
`$scope.fullName = $scope.firstName + ' ' + $scope.lastName`	`protected readonly fullName = computed(() => this.firstName() + ' ' + this.lastName());`	
`$scope.$watch('balance', fn)`	`effect(() => { const b = this.balance(); /* side effect */ });`	
`$http.get('/api/users')`	`private http = inject(HttpClient); users$ = this.http.get<User[]>('/api/users'); users = toSignal(users$, { initialValue: [] });`	
```

**\*\*Section 2: Component Anatomy Template\*\***

Show the standard structure for every migrated component:

```typescript

```
import { Component, inject, signal, computed, effect } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { toSignal } from '@angular/core/rxjs-interop';
```

```
@Component({
  selector: 'app-example',
  standalone: true,
  imports: [],
  templateUrl: './example.component.html',
  changeDetection: ChangeDetectionStrategy.OnPush
})
```

```
export class ExampleComponent {
  // DI
  private readonly http = inject(HttpClient);
  private readonly router = inject(Router);

  // Signals (state)
  protected readonly items = signal<Item[]>([]);
  protected readonly loading = signal(false);
  protected readonly searchTerm = signal('');

  // Computed (derived state)
  protected readonly filteredItems = computed(() => {
    const term = this.searchTerm().toLowerCase();
    return this.items().filter(i => i.name.toLowerCase().includes(term));
  });
```

```
  // Effects (side effects)
  constructor() {
    effect(() => {
      console.log('Items changed:', this.items().length);
    });
  }
```

```
  // Methods
  protected loadItems(): void {
    this.loading.set(true);
    this.http.get<Item[]>('/api/items').subscribe({
      next: (data) => {
        this.items.set(data);
        this.loading.set(false);
      },
      error: (err) => {
        console.error(err);
        this.loading.set(false);
      }
    });
  }
}
```

Part 6: Architecture Decision Prompts

Prompt 6.1: **State Management Decision Framework**

```markdown

# Help Me Choose: Signals vs NgRx for This Screen

##### **Screen Context:**

- Name: Account Dashboard
- Data sources: 3 HTTP endpoints (accounts, transactions, balance)
- User actions: Filter transactions, export PDF, transfer money
- Shared state: No (only used in this screen)
- Async complexity: Medium (1 endpoint depends on another)

##### **Decision Criteria:**

Answer these questions:

1. Is state shared across 3+ components? (Yes/No)
2. Are there complex async flows (e.g., optimistic updates, retry logic)? (Yes/No)
3. Is state persisted to localStorage? (Yes/No)
4. Do you need time-travel debugging? (Yes/No)
5. Is the team experienced with NgRx? (Yes/No)

##### **My Answers:**

1. No – Only used in AccountDashboardComponent
2. No – Simple HTTP GET calls
3. No
4. No
5. No

##### **Based on my answers, recommend:**

- Pattern A: Component-level signals only
- Pattern B: Service-level signals (shared across 2-5 components)
- Pattern C: NgRx Signal Store (complex async with rxMethod)
- Pattern D: Full NgRx with Effects (global state, time-travel)

##### **Output:**

1. Recommended pattern
2. Code example for this screen
3. Justification (why this pattern fits)

## Prompt 6.2: Security Token Storage Decision

# Choose JWT Storage Strategy for Angular 21 App

**\*\*Context:\*\***

- SPA with Angular 21 + SSR
- Backend: REST API with JWT authentication
- Threat model: Must protect against XSS and CSRF

**\*\*Options:\*\***

| Storage                       | XSS-Safe? | CSRF-Safe?         | SSR-Safe? | Logout Multi-Tab?  | Recommended?      |
|-------------------------------|-----------|--------------------|-----------|--------------------|-------------------|
| localStorage                  | ✗ No      | ✓ Yes              | ✗ No      | ✗ No               | ✗ No              |
| sessionStorage                | ✗ No      | ✓ Yes              | ✗ No      | ✗ No               | ✗ No              |
| HttpOnly Cookie               | ✓ Yes     | ⚠ Needs XSRF token | ✓ Yes     | ✓ Yes              | ✓ Yes (with XSRF) |
| In-Memory (Signal)            | ✓ Yes     | ✓ Yes              | ✗ No      | ⚠ BroadcastChannel | ✓ Yes (SPA only)  |
| BFF Pattern (HttpOnly + OIDC) | ✓ Yes     | ✓ Yes              | ✓ Yes     | ✓ Yes              | ✓ Best            |

**\*\*My Requirements:\*\***

- SSR: Yes
- Multi-tab logout: Yes
- Team has backend control: Yes

**\*\*Based on my requirements, recommend:\*\***

1. The best storage strategy
2. Full AuthService implementation
3. Angular provideHttpClient configuration
4. Nginx/backend configuration

## Part 7: Testing Prompts

### Prompt 7.1: Generate Vitest Unit Test from Component

# Generate Vitest Unit Test for Angular 21 Component

**\*\*Component to Test:\*\***

```
``typescript
import { Component, inject, signal, computed } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { toSignal } from '@angular/core/rxjs-interop';
import { Account } from '@banking/shared-models';

@Component({
 selector: 'app-account-list',
 standalone: true,
 templateUrl: './account-list.component.html',
 changeDetection: ChangeDetectionStrategy.OnPush,
})
export class AccountListComponent {
 private readonly http = inject(HttpClient);

 protected readonly accounts$ = this.http.get<Account[]>('/api/accounts');
 protected readonly accounts = toSignal(this.accounts$, { initialValue: [] });
 protected readonly searchTerm = signal('');

 protected readonly filteredAccounts = computed(() => {
 const term = this.searchTerm().toLowerCase();
 return this.accounts().filter((a) => a.accountNumber.toLowerCase().includes(term));
 });

 protected updateSearch(term: string): void {
 this.searchTerm.set(term);
 }
}
```

**Test Requirements:** ☒ Use Vitest with @angular/core/testing ☒ Mock HttpClient with HttpTestingController ☒ Test signal reactivity (search updates filteredAccounts) ☒ Test computed signal (filtering logic) ☒ Test HTTP call on component init ☒ Test error handling ☒ Aim for 80%+ branch coverage

### Output:

1. **account-list.component.spec.ts** — Complete test file
2. **Test coverage report** — What's covered?

```

Prompt 7.2: **Generate Playwright E2E Test from User Story**

```markdown
# Generate Playwright E2E Test from User Story

**User Story:**

As a customer,
I want to create a payment from my account,
So that I can transfer money to another account.

**Acceptance Criteria:**
1. User must be logged in
2. User selects "From Account" from dropdown
3. User enters "To Account" number
4. User enters amount (must be > 0 and ≤ account balance)
5. User clicks "Submit Payment"
6. System shows success message
7. User is redirected to payment confirmation page

**Technical Details:**
- Start URL: `/payments/new`
- Auth: Use `beforeEach` to log in as `test@example.com`
- Mock API: Intercept POST `/api/payments` → return 201

**Output:**
1. **payment-flow.spec.ts** – Playwright test with Page Object Model
2. **payment-form.page.ts** – Page Object with locators + methods
3. **Test assertions** – Check success message, URL change
```

Prompt 7.3: Generate Accessibility Test with axe-core

```
# Generate Accessibility Test for Login Screen

**Screen:** Login page at `/login`

**Requirements:**
☒ Use Playwright + @axe-core/playwright
☒ Check WCAG 2.1 Level AA compliance
☒ Test keyboard navigation (Tab through form)
☒ Test screen reader announcements (aria-label, aria-live)
☒ Fail the test if ANY critical violations found

**Output:**

1. **login-accessibility.spec.ts** – Playwright test with AxeBuilder
2. **Expected results** – 0 critical violations
3. **Common fixes** – If violations found, suggest remediation
```

Part 8: Deployment & Infrastructure Prompts

Prompt 8.1: Generate GitHub Actions CI/CD Pipeline

```
# Generate GitHub Actions Workflow for Nx Monorepo

**Requirements:**

**Workflow 1: CI (on PR and main push)**

- Checkout code
- Setup Node.js 20
- Install dependencies (npm ci)
- Lint: `nx affected --target=lint --base=origin/main`
- Test: `nx affected --target=test --base=origin/main --coverage`
- Build: `nx affected --target=build --base=origin/main --configuration=production`
- E2E: `nx affected --target=e2e --base=origin/main`
- Upload coverage to Codecov
- Fail if coverage < 80%

**Workflow 2: Deploy Shell (on main push to apps/banking-shell/**)\**

- Build shell: `nx build banking-shell --configuration=production`
- Deploy to Azure Static Web Apps
- Run smoke test: `curl -f https://banking.company.com || exit 1`
- Send Slack notification on success/failure

**Workflow 3: PR Preview (on PR opened/synchronize)**

- Build affected apps
- Deploy to Azure Static Web Apps preview environment
- Comment preview URL on PR

**Output:**

1. `.github/workflows/ci.yml`
2. `.github/workflows/deploy-shell.yml`
3. `.github/workflows/preview.yml`
```

Prompt 8.2: Generate Azure Infrastructure as Code

```
# Generate Azure CLI Commands for Migration Infrastructure

**Infrastructure:**

1. Azure Static Web Apps for `banking-shell` (host)
2. Azure Static Web Apps for `banking-ng21` (remote)
3. Azure App Service for `banking-legacy` (AngularJS Node.js app)
4. Azure Front Door for unified routing
5. Application Insights for monitoring

**Requirements:**

- Resource group: `banking-migration-rg`
- Location: `eastus`
- Static Web Apps: Free tier
- App Service: B1 tier (Basic)
- Front Door: Standard tier

**Output:**

1. setup-azure.sh – Bash script with all `az` commands
2. azure-config.json – Configuration file for automation
3. teardown-azure.sh – Script to delete all resources
```

Prompt 8.3: Generate Canary Deployment Strategy

```
# Generate Canary Deployment Service for Angular 21

**Requirements:**

Implement a `CanaryService` that:

1. Reads canary percentage from server config (e.g., 5%, 10%, 25%, 50%, 100%)
2. Determines if current user is in canary group (deterministic based on user ID hash)
3. Provides `isInCanary()` signal for components to check
4. Logs canary status to Application Insights

**Usage Example:**

```typescript
export const routes: Routes = [
 {
 path: 'accounts',
 canMatch: [canaryGuard('accounts')],
 loadComponent: () => import('./account-list-ng21.component'),
 },
 {
 path: 'accounts',
 loadComponent: () => import('./account-list-legacy.component'),
 },
];
```
```

Output:

1. **canary.service.ts** — Full implementation with signals
2. **canary.guard.ts** — CanMatchFn that checks canary status
3. **Rollout schedule** — Day 1: 5%, Day 2: 10%, Day 3: 25%, Day 4: 50%, Day 5: 100%

```
---

## 9.3 – Advanced Prompt Patterns

### Pattern 1: **Diff-Based Prompting**

When you want to see only the changes between AngularJS and Angular 21:

```markdown
Show me a DIFF between AngularJS and Angular 21 for this controller.

Output format:
```diff
- $scope.users = [];
+ protected readonly users = signal<User[]>([]);

- $scope.loadUsers = function() {
-   $http.get('/api/users').then(function(response) {
-     $scope.users = response.data;
-   });
- };
+ protected loadUsers(): void {
+   this.http.get<User[]>('/api/users').subscribe({
+     next: (data) => this.users.set(data)
+   });
+ }
```

AngularJS Controller:

Why it works: Diff format highlights exactly what changed, making it easier to review.

Pattern 2: Table-Driven Transformation

For batch migrations (e.g., converting 10 filters → pipes):

```markdown

Convert these AngularJS filters to Angular 21 pipes.

Input format (Markdown table):

| AngularJS Filter | Purpose              | Example Usage                        |
|------------------|----------------------|--------------------------------------|
| uppercase        | Convert to uppercase | {{ name   uppercase }}               |
| currency         | Format as USD        | {{ price   currency }}               |
| dateFormat       | Format date          | {{ date   dateFormat:'MM/DD/YYYY' }} |

Output format:

- Pipes to create** (list file names)
- Implementation** (code for each pipe)
- Migration notes** (AngularJS → Angular 21 mapping)

**Why it works:** Batch processing is faster than one-by-one migrations.

## Pattern 3: Test-First Prompting

Generate the **test first**, then the implementation:

### # Step 1: Generate Test

Generate a Vitest unit test for AccountService with these test cases:

- getAccounts() returns array of accounts
- getAccountById() returns single account
- createAccount() POSTs new account and returns result
- HTTP errors are handled gracefully

DO NOT generate the service implementation yet.

### # Step 2: Generate Implementation

Now generate account.service.ts that makes all these tests pass.

**Why it works:** Test-driven approach ensures your migration is testable.

## 9.4 — AI Tool Selection Matrix

### When to Use Which AI Tool?

| Task                                 | GitHub Copilot | ChatGPT/Claude | GitHub Copilot Chat | Cursor     |
|--------------------------------------|----------------|----------------|---------------------|------------|
| Inline code completion               | ✅ Best         | ❌ No           | ❌ No                | ✅ Good     |
| Refactor existing function           | ✅ Good         | ✅ Best         | ✅ Best              | ✅ Best     |
| Generate full component from scratch | ⚠️ Limited     | ✅ Best         | ✅ Best              | ✅ Best     |
| Explain AngularJS code               | ❌ No           | ✅ Best         | ✅ Good              | ✅ Good     |
| Multi-file refactoring               | ❌ No           | ⚠️ Manual      | ✅ Good              | ✅ Best     |
| Generate tests                       | ✅ Good         | ✅ Best         | ✅ Best              | ✅ Best     |
| Architecture decisions               | ❌ No           | ✅ Best         | ✅ Good              | ⚠️ Limited |
| Batch migrations (10+ files)         | ❌ No           | ⚠️ Manual      | ⚠️ Limited          | ✅ Best     |

#### Recommendations:

1. **GitHub Copilot (inline):** Use for autocompleting signal definitions, imports, method signatures
2. **GitHub Copilot Chat:** Use for single-file migrations (controller → component)
3. **ChatGPT/Claude:** Use for architecture Q&A, decision frameworks, documentation
4. **Cursor:** Use for multi-file refactoring (e.g., rename service across 20 files)

## 9.5 — Common Pitfalls & Solutions

### Pitfall 1: ❌ AI Generates Deprecated Angular Patterns

**Problem:** AI generates `*ngIf`, `@Input()`, constructor injection.

**Solution:** Add explicit constraints to EVERY prompt:

#### 🚫 NEVER USE:

- Constructor injection → Use `inject()`
- `@Input/@Output` → Use `input()/output()`
- `*ngIf/*ngFor` → Use `@if/@for`

#### ✅ ALWAYS USE:

- `inject()` for DI
- `signal()/computed()/effect()` for state
- `@if/@for/@switch` for control flow

### Pitfall 2: ❌ AI Hallucinates Non-Existent APIs

**Problem:** AI invents methods like `signal.update()` (doesn't exist — use `.set()` or `.update((prev) => ...)`).

**Solution:** Ask AI to cite documentation:

Migrate this component to Angular 21 signals.

Before generating code, reference the official Angular 21 signal API:

- signal(initialValue)
- computed(() => derivation)
- effect(() => sideEffect)
- toSignal(observable\$, options)

Do NOT invent methods that don't exist.

### Pitfall 3: ❌ Prompt Too Vague → Generic Output

#### Bad Prompt:

Migrate this controller to Angular 21.  
<paste code>

#### Good Prompt:

Migrate this AngularJS controller to Angular 21 standalone component with:

- OnPush change detection
- Signal-based state
- toSignal() for HTTP → signal conversion
- @if/@for in template
- Nx import paths: @banking/shared-models

AngularJS Controller:  
<paste code>

Output:

1. .ts file with imports
2. .html template
3. Migration notes

**Difference:** Specificity drives quality.

### Pitfall 4: ❌ AI Doesn't Preserve Business Logic

**Problem:** AI simplifies complex validation logic during migration.

**Solution:** Explicitly state "PRESERVE ALL BUSINESS LOGIC":

Migrate this service to Angular 21.

⚠️ CRITICAL: This service contains complex business logic for loan approval.  
DO NOT simplify or remove any validation rules.  
PRESERVE ALL if/else branches exactly as-is.

AngularJS Service:  
<paste code with complex logic>

### Pitfall 5: ❌ Batch Prompt Generates Inconsistent Code

**Problem:** Asking AI to migrate 10 files at once results in inconsistent patterns.

**Solution:** Migrate one file as a reference, then use it as a template:

### # Step 1: Migrate Reference Component

Migrate AccountListComponent as the reference implementation.

### # Step 2: Use as Template

Now migrate these 9 components using AccountListComponent as the pattern:

1. TransactionListComponent
2. PaymentListComponent
3. LoanListComponent
- ...

Ensure ALL components follow the SAME structure:

- inject() for DI
- signal()/computed() for state
- OnPush change detection
- @if/@for in templates

## 9.6 — Prompt Library Quick Reference

### Quick Copy-Paste Prompts

#### 1 Fast Controller Migration

Migrate this AngularJS controller to Angular 21 standalone component.  
Use: OnPush, inject(), signal(), @if/@for, input()/output().  
No constructor injection. No @Input/@Output. No \*ngIf/\*ngFor.

<paste AngularJS code>

#### 2 Fast Service Migration

Migrate this AngularJS service to Angular 21 @Injectable.  
Use: inject(HttpClient), toSignal(), shareReplay(1), catchError.  
No constructor injection.

<paste AngularJS code>

#### 3 Fast Test Generation

Generate Vitest unit test for this Angular 21 component.  
Use: TestBed, HttpTestingController, signal testing, 80% coverage.

<paste component code>

#### 4 Fast Route Migration

Migrate this ui-router config to Angular 21 Router.  
Use: lazy loading (loadComponent), CanActivateFn guards, ResolveFn.

<paste route config>

#### 5 Fast GitHub Actions Workflow

Generate GitHub Actions workflow for Nx monorepo.  
Jobs: lint, test (with coverage), build, e2e.  
Use: nx affected --base=origin/main.  
Fail if coverage < 80%.

**Part 9 complete.** This section provides a comprehensive AI prompt library organized by all 8 parts of the migration guide, plus advanced prompt engineering techniques (context-setting, few-shot learning, chain-of-thought, constraint enforcement, iterative refinement), AI tool selection matrix (Copilot vs ChatGPT vs Cursor), common pitfalls with solutions (deprecated patterns, hallucinations, vague prompts, lost business logic, inconsistent batch migrations), and quick-reference copy-paste prompts for fast migrations.

---

 **ALL 9 PARTS COMPLETE** — The AngularJS to Angular 21 Enterprise Migration Guide is now complete with **comprehensive AI prompt engineering techniques** covering discovery, architecture, code transformation, testing, deployment, and advanced prompt patterns for maximum AI effectiveness throughout the migration lifecycle.